Swiss Post Voting System

System Specification

Swiss Post*

Version 1.6.1

Abstract

The Swiss Post Voting System is a return code-based remote online voting system that provides individual verifiability, universal verifiability, and vote secrecy. This document provides a detailed specification of the cryptographic protocol—from the configuration phase to the voting phase to the counting phase—and includes pseudocode representations of most algorithms. The document specifies the protocol's higher-level algorithms as well as some of the underlying building blocks.

* Copyright 2026 Swiss Post Ltd. - Based on original work by Scytl Secure Electronic Voting S.A. (succeeded by Scytl Election Technologies S.L.U.), modified by Swiss Post. Scytl Election Technologies S.L.U. is not responsible for the information contained in this document.

Copyright 2026 Swiss Post Ltd.

Disclaimer

E-Voting Community Program Material - please follow our Code of Conduct describing what you can expect from us, the Coordinated Vulnerability Disclosure Policy, and the contributing guidelines.

Revision chart

Version	Description	Author	Reviewer	Date
0.9.5	Initial published version	OE	XM, JS, TH	2021-05-04
0.9.6	See change log for version 0.9.6	OE	XM, JS, TH	2021-06-25
0.9.7	See change log for version 0.9.7	OE	XM, JS, TH	2021-10-15
0.9.8	See change log for version 0.9.8	OE	XM, JS, TH	2022-02-17
0.9.9	See change log for version 0.9.9	HR, OE	XM, JS, TH	2022-04-18
1.0.0	See change log for version 1.0.0	HR, OE, TH	XM, JS	2022-06-24
1.1.0	See change log for version 1.1.0	HR, OE, TH	XM, JS	2022-10-03
1.1.1	See change log for version 1.1.1	HR, OE, TH	XM, JS	2022-10-31
1.2.0	See change log for version 1.2.0	AH, OE	XM, JS, TH	2022-12-09
1.3.0	See change log for version 1.3.0	AH, OE	XM, JS, TH	2023-04-19
1.3.1	See change log for version 1.3.1	AH, OE	XM, JS, TH	2023-06-15
1.3.2	See change log for version 1.3.2	AH, OE	XM, JS, CK	2023-10-23
1.4.0	See change log for version 1.4.0	AH, CC, CK, OE	XM, JS	2024-02-14
1.4.1	See change log for version 1.4.1	AH, CC, CK, OE	XM, JS	2024-06-17
1.4.1.1	See change log for version 1.4.1.1	CC, OE	CK, PO	2024-07-30
1.4.2	See change log for version 1.4.2	AH, CC, CK, OE	XM, JS	2025-05-16
1.5.0	See change log for version 1.5.0	AH, CC, CK, OE	XM, JS	2025-06-20
1.5.1	See change log for version 1.5.1	AH, CC, CK, OE	XM, JS	2025-08-15
1.5.2	See change log for version 1.5.2	AH, CC, CK, OE	XM, JS	2025-10-10
1.5.3	See change log for version 1.5.3	AH, CC, CK	OE	2026-05-15
1.6.0	See change log for version 1.6.0	AH, CC, CK	OE	2026-07-03
1.6.1	See change log for version 1.6.1	AH, CC, CK	CS, OE	2026-08-07

Copyright 2026 Swiss Post Ltd.

Contents

Copyright 2026 Swiss Post Ltd.

Copyright 2026 Swiss Post Ltd.

Copyright 2026 Swiss Post Ltd.

Symbols

$\mathbb{A}_{10}$	Decimal numbers
$\mathbb{A}_{Base16}$	Base16 (Hex) alphabet [15]
$\mathbb{A}_{Base32}$	Base32 alphabet, including the padding character = [15]
$\mathbb{A}_{Base64}$	Base64 alphabet, including the padding character = [15]
$\mathbb{A}_{UCS}$	Alphabet of the Universal Coded Character Set (UCS) according to ISO/IEC10646
$\mathbb{A}_{\mathsf{latin}}$	Extended Latin alphabet, as described in the crypto primitives specification
$\mathbb{A}_{\mathsf{u32}}$	User-friendly alphabet for codes, as described in the crypto primitives specification
$\mathcal{T}_1^{50}$	Domain of a string token according to the XML Schema datatype [22]: strings without tabs, carriage returns, or line feeds, with collapsed whitespace (no leading/trailing spaces and no multiple internal spaces)
$\mathcal{D}_{\mathrm{v}}$	Actual voting option domain, as defined in Section 3.4.6
$\mathcal{D}_{\tau}$	Correctness information domain, as defined in Section 3.4.6
$\mathcal{D}_{\mathsf{EA}}$	Extended authentication factor domain, $= \mathcal{D}_{\mathsf{date}} \cup \mathcal{D}_{\mathsf{year}} \cup \mathcal{D}_{\mathsf{eGov}}$
$\mathcal{D}_{\mathsf{date}}$	Local birth date domain (`YYYY-MM-DD`, ISO 8601)
$\mathcal{D}_{\mathsf{year}}$	Birth year domain (`YYYY`)
$\mathcal{D}_{\mathsf{eGov}}$	eGovernment token domain $(\mathbb{A}_{Base16})^{1_{\mathrm{ID}}}$
$\mathbf{ag}$	Verification cards specific vector of abstention groups
$\hat{\mathbf{a}}_{\mathtt{id}}$	Voter's selected abstentions
`ACC`	Abstention Choice Return Code
`aux`	Auxiliary string
$\mathcal{B}$	Set of possible values for a byte
$\mathcal{B}^*$	Set of byte arrays of arbitrary length
$\mathbb{B}^n$	Set of bit arrays of length n
`bb`	Ballot box ID
`BCK`	Ballot Casting Key
`CC`	Short Choice Return Codes
$\mathbf{c}_{\mathsf{ck}}$	Encrypted confirmation keys
$\mathbf{c}_{\mathsf{Dec}}$	List of partially decrypted ciphertexts
$\mathbf{c}_{\mathsf{expCK}}$	Exponentiated, encrypted, hashed Confirmation Keys
$\mathbf{c}_{\mathsf{expPCC}}$	Exponentiated, encrypted, hashed partial Choice Return Codes
$\mathbf{c}_{\mathsf{mix}}$	List of shuffled ciphertexts
$\mathbf{c}_{\mathsf{pCC}}$	Encrypted, hashed partial Choice Return Codes
$\mathbf{c}_{\mathsf{pC}}$	Encrypted pre-Choice Return Codes
`CK`	Confirmation Key
CCM	Mixing control components
CCR	Return Codes control components

Copyright 2026 Swiss Post Ltd.

$\mathbf{d}$	Exponentiated γ elements of the encrypted Partial Choice Return Codes: $(d_0, \ldots, d_{\psi-1})$
DoI	Domains of influence defining the voting rights of a verification card set
δ	Number of write-in options + 1 for a specific verification card set
$\delta_{\mathtt{max}}$	Maximum number of write-in options + 1 across all verification card sets
$\delta_{\mathtt{sup}}$	Maximum supported number of write-in options + 1
$\mathbf{e}$	Vector of elections in a verification card set
Λ	Electoral Model Context for a verification card set
$\mathtt{EA}$	Extended authentication factor
$\mathtt{E1}$	Encrypted vote
$\widetilde{\mathtt{E1}}$	Exponentiated encrypted vote
$\mathtt{E2}$	Encrypted partial Choice Return Codes
$\widetilde{\mathtt{E2}}$	Multiplied, encrypted partial Choice Return Codes
$\mathtt{ee}$	Election event id
g	Generator of the encryption group
γ	Gamma - the ElGamal ciphertext's first element
$\mathbb{G}_q$	Group of quadratic residues modulo p of size q
$\mathtt{hAuth}$	Base authentication challenge
$\mathtt{hCK}$	Hashed, squared Confirmation Key
$\mathtt{hpCC}$	Hashed, squared partial Choice Return Codes
η	Number of presentation groups for a verification card set
κ	Number of abstention groups for a verification card set
$\mathtt{id}$	Voter index $\{0, \ldots, \mathtt{N_E} - 1\}$
j	Index over control components $\{1, 2, 3, 4\}$
$\mathtt{lCC}$	Long Choice Return Codes
$\mathtt{lVCC}$	Long Vote Cast Return Code
$\mathtt{l_{ACC}}$	Character length of Abstention Choice Return Codes
$\mathtt{l_{BCK}}$	Character length of ballot casting keys
$\mathtt{l_{CC}}$	Character length of Choice Return Codes
$\mathtt{l_{ID}}$	Character length of unique identifiers: 32
$\mathtt{l_{HB64}}$	Character length of the Base64 encoded hash function output
$\mathtt{l_{VCKS}}$	Character length of the Base64 encoded verification card keystore: 572
$\mathtt{l_{KD}}$	Output byte length of the key derivation function: 32
$\mathtt{l_{SVK}}$	Character length of Start Voting Keys
$\mathtt{l_{VCC}}$	Character length of Vote Cast Return Codes
$\mathtt{l_w}$	Maximum number of characters in a write-in field: 400
$\mathsf{L}_{\mathtt{lVCC}}$	Long Vote Cast Return Codes allow list
$\mathsf{L}_{\mathtt{pCC}}$	Partial Choice Return Codes allow list
L_{votes}	List of decrypted votes
$L_{\mathsf{decodedVotes}}$	List of decoded votes
L_{writeIns}	List of decoded write-ins

Copyright 2026 Swiss Post Ltd.

m	Message representing the vote
$\mathbf{m}$	List of plaintext votes
$\mathbb{N}$	Set of positive integer numbers including 0
$\mathbb{N}^+$	Set of strictly positive integer numbers
$\mathtt{N_{bb}}$	Number of ballot boxes of an election event
$\mathtt{N_S}$	Number of sent votes in a specific ballot box
$\mathtt{N_C}$	Number of confirmed votes in a specific ballot box
$\hat{\mathtt{N}}_\mathtt{C}$	Number of mixed votes including trivial encryptions
$\mathtt{N_E}$	Number of eligible voters of a specific verification card set
$\mathbf{n}$	Verification card set specific vector of voting options
n	Number of voting options for a given verification card set
$n_{\mathtt{max}}$	Maximum number of voting options across all verification card sets
$n_{\mathtt{sup}}$	Maximum supported number of voting options
ϕ	Phi - the ElGamal ciphertext's second element
$\boldsymbol{\psi}$	Verification card set specific vector of allowed selections
ψ	Allowed number of selections for a given verification card set
$\psi_{\mathtt{max}}$	Maximum number of selections across all verification card sets
$\psi_{\mathtt{sup}}$	Maximum supported number of selections
$\boldsymbol{\pi}_{\mathsf{decPCC},j}$	Proof of correct decryption of the partial Choice Return Codes
$\pi_{\mathtt{Exp}}$	Proof of correct exponentiation
$\pi_{\mathtt{EqEnc}}$	Proof of equality of encryptions (plaintext equality)
$\mathtt{pCC}$	Partial Choice Return Codes
$\mathtt{pC}$	pre-Choice Return Codes
$\mathtt{pVCC}$	pre-Vote Cast Return Codes
$\mathbf{pg}$	Verification card set specific vector of presentation groups
$\tilde{\mathbf{p}}$	Encoded voting options $(\tilde{\mathrm{p}}_0, \ldots, \tilde{\mathrm{p}}_{n-1}),\ \tilde{\mathrm{p}}_k \in (\mathbb{G}_q \cap \mathbb{P}) \setminus g$
$\tilde{\mathbf{p}}_\mathrm{w}$	Encoded voting options corresponding to the choice of a write-in $(\tilde{\mathrm{p}}_{\mathrm{w},0}, \ldots, \tilde{\mathrm{p}}_{\mathrm{w},\delta-2}), \tilde{\mathrm{p}}_{\mathrm{w},k} \in (\mathbb{G}_q \cap \mathbb{P}) \setminus g$
$\hat{\mathbf{p}}$	Selected encoded voting options $(\hat{\mathrm{p}}_0, \ldots, \hat{\mathrm{p}}_{\psi-1}),\ \hat{\mathrm{p}}_k \in (\mathbb{G}_q \cap \mathbb{P}) \setminus g$
$\mathbb{P}$	Set of prime numbers
$\mathbf{p}$	Vector of small prime group members
p	Encryption group modulus
q	Encryption group cardinality $\mathsf{s.t.}\ p = 2q + 1$
$\hat{\mathbf{s}}$	Selected write-in candidates
t	Number of elections for a verification card set
$\mathtt{SVK}$	Start Voting Key
$\boldsymbol{\sigma}$	Semantic information $(\sigma_0, \ldots, \sigma_{n-1}),\ \sigma_k \in {\mathbb{A}_{UCS}}^*$
$\boldsymbol{\tau}$	Correctness information $(\tau_0, \ldots, \tau_{n-1}),\ \tau_k \in \mathcal{D}_\tau$
$\hat{\boldsymbol{\tau}}$	Blank correctness information $(\hat{\tau}_0, \ldots, \hat{\tau}_{\psi-1}), \hat{\tau}_k \in \mathcal{D}_\tau$
$\tilde{\mathbf{v}}$	Actual voting options $(\mathrm{v}_0, \ldots, \mathrm{v}_{n-1}),\ \mathrm{v}_k \in \mathcal{D}_\mathrm{v}$

Copyright 2026 Swiss Post Ltd.

$\hat{\mathbf{v}}_{\mathtt{id}}$	Selected actual voting options $(\hat{\mathrm{v}}_0, \ldots, \hat{\mathrm{v}}_{\psi-1}), \hat{\mathrm{v}}_k \in \mathcal{D}_{\mathrm{v}}$
$\mathtt{vc}$	Verification card
$\mathbf{vc}$	Vector of verification card IDs of an election event
$\mathtt{VCks}$	Verification card keystore
$\mathtt{vcs}$	Verification card set ID
$\mathbf{vcs}$	Vector of verification card set IDs of an election event ordered lexicographically
$\mathtt{VCC}$	Short Vote Cast Return Code
$\mathtt{vcd}$	Voting card
$\chi_{\mathtt{id}}$	Counting circle mapping of voter ID
$\lvert x \rvert$	Bit length of the number x
$\mathbf{w}_{\mathtt{id}}$	Voter's encoded write-ins $(\mathrm{w}_{\mathtt{id},0}, \ldots, \mathrm{w}_{\mathtt{id},\delta-2})$
$\mathbb{Z}_q$	Group of integers modulo q
$\top$	Truth value true or successful termination
$\bot$	Truth value false or unsuccessful termination

Copyright 2026 Swiss Post Ltd.

1 Introduction

Switzerland has a longstanding tradition of direct democracy, allowing Swiss citizens to vote approximately four times a year on elections and referendums. In recent years, voter turnout hovered below 40 percent [11].
The vast majority of voters in Switzerland fill out their paper ballots at home and send them back to the municipality by postal mail, usually days or weeks ahead of the actual election date. Remote online voting (referred to as e-voting in this document) would provide voters with some advantages. First, it would guarantee the timely arrival of return envelopes at the municipality (especially for Swiss citizens living abroad). Second, it would improve accessibility for people with disabilities. Third, it would eliminate the possibility of an invalid ballot when inadvertently filling out the ballot incorrectly.
In the past, multiple cantons offered e-voting to a part of their electorate. Many voters would welcome the option to vote online - provided the e-voting system protects the integrity and privacy of their vote [12].
State-of-the-art e-voting systems alleviate the practical concerns of mail-in voting and, at the same time, provide a high level of security. Above all, they must display three properties [18]:

- Individual verifiability: allow a voter to convince herself that the system correctly registered her vote
- Universal verifiability: allow an auditor to check that the election outcome corresponds to the registered votes
- Vote secrecy: do not reveal a voter's vote to anyone

Following these principles, the Federal Chancellery defined stringent requirements for e-voting systems. The Ordinance on Electronic Voting (VEleS - Verordnung über die elektronische Stimmabgabe) and its technical annex (VEleS annex) [7] describes these requirements.
Swiss democracy deserves an e-voting system with excellent security properties. Swiss Post is thankful to all security researchers for their contributions and the opportunity to improve the system's security guarantees. We look forward to actively engaging with academic experts and the hacker community to maximize public scrutiny of the Swiss Post Voting System.

Copyright 2026 Swiss Post Ltd.

1.1 Two-Round Return Code Scheme

The Swiss Post Voting System is a return code scheme: every voter receives a printed code sheet prior to the election. Figure 1 shows an example code sheet, which contains the following types of codes (the codes are freshly generated for every voter and every election event):

- A Start Voting Key to start the voting process
- A Choice Return Code for each voting option
- A Ballot Casting Key to confirm the vote
- A Vote Cast Return Code

In certain Swiss elections, a distinction is made between casting a blank vote and not participating in the election or popular vote at all. In that case, the code sheet also contains the following:

- An Abstention Choice Return Code for each abstention group. An abstention group can correspond to one or multiple elections or popular votes.

The distinction between blank votes and abstentions can affect the outcome. In particular, blank votes are sometimes taken into account when calculating the absolute majority, whereas abstentions are not considered in that calculation. In the traditional paper-based process, abstention corresponds to not placing one or more ballot papers into the return envelope.

Copyright 2026 Swiss Post Ltd.

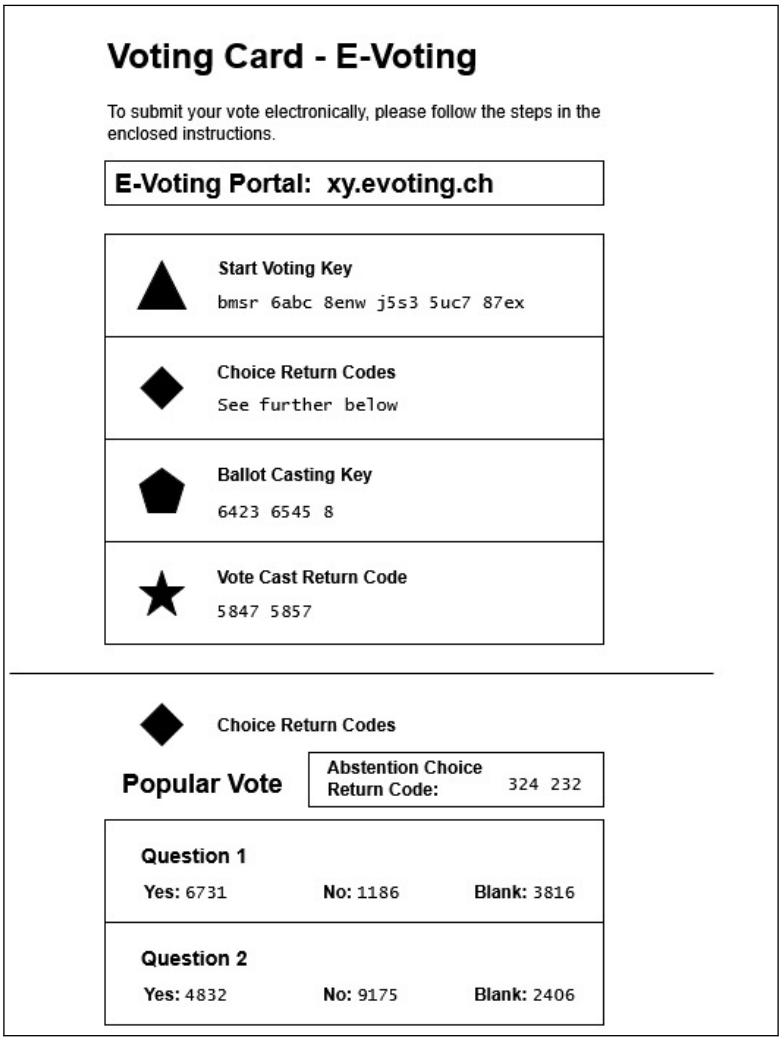
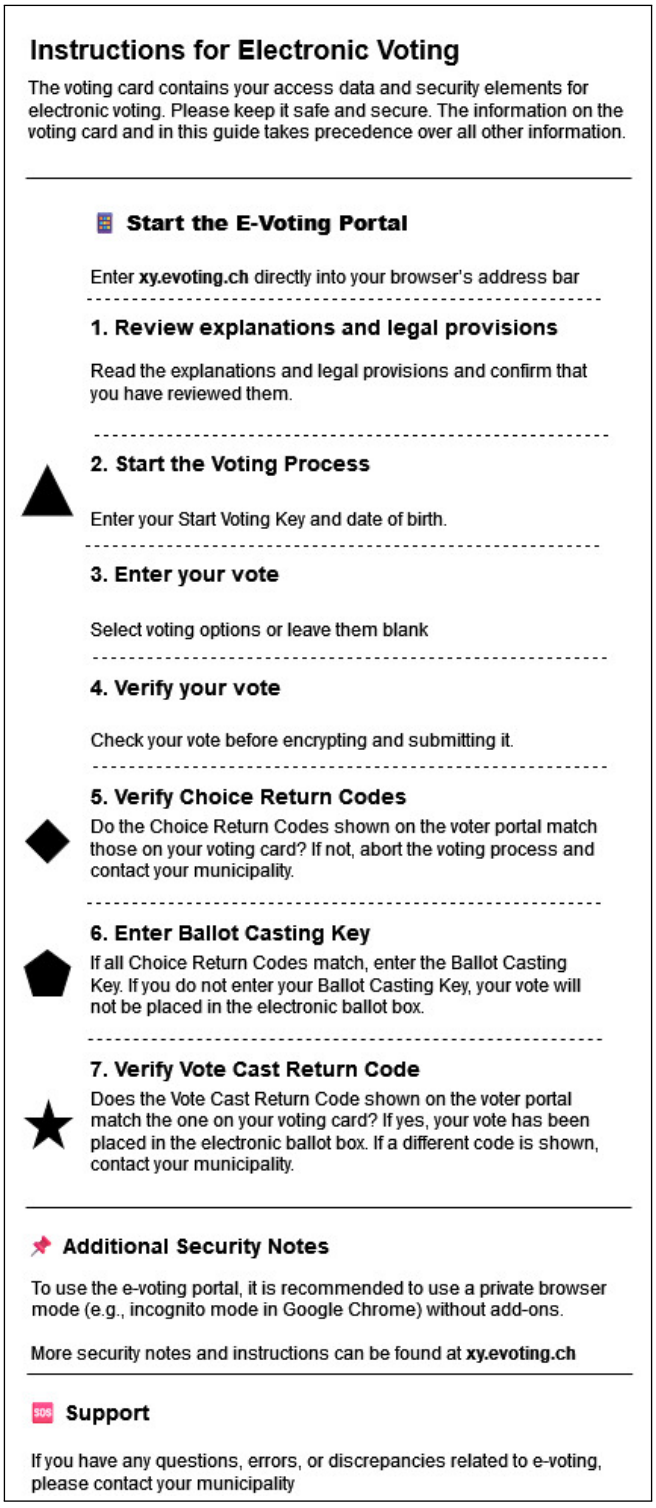


Fig. 1: Example of printed code sheet sent to the voters. Codes are freshly generated per voter and election event.

Moreover, the voter receives instructions by postal mail that help ensure the voting process is followed correctly and allow the detection of any attempts by a malicious voting portal to mislead the voter into deviating from the expected procedure. Figure 2 shows example voting instructions.

Copyright 2026 Swiss Post Ltd.



Instructions for Electronic Voting

The voting card contains your access data and security elements for electronic voting. Please keep it safe and secure. The information on the voting card and in this guide takes precedence over all other information.

Start the E-Voting Portal

Enter **xy.evoting.ch** directly into your browser's address bar

1. Review explanations and legal provisions

Read the explanations and legal provisions and confirm that you have reviewed them.

2. Start the Voting Process

Enter your Start Voting Key and date of birth.

3. Enter your vote

Select voting options or leave them blank

4. Verify your vote

Check your vote before encrypting and submitting it.

5. Verify Choice Return Codes

Do the Choice Return Codes shown on the voter portal match those on your voting card? If not, abort the voting process and contact your municipality.

6. Enter Ballot Casting Key

If all Choice Return Codes match, enter the Ballot Casting Key. If you do not enter your Ballot Casting Key, your vote will not be placed in the electronic ballot box.

7. Verify Vote Cast Return Code

Does the Vote Cast Return Code shown on the voter portal match the one on your voting card? If yes, your vote has been placed in the electronic ballot box. If a different code is shown, contact your municipality.

Additional Security Notes

To use the e-voting portal, it is recommended to use a private browser mode (e.g., incognito mode in Google Chrome) without add-ons.

More security notes and instructions can be found at **xy.evoting.ch**

Support

If you have any questions, errors, or discrepancies related to e-voting, please contact your municipality

Fig. 2: Example of printed instructions sent to the voters.

Copyright 2026 Swiss Post Ltd.

Figure 3 highlights the two rounds of the voting process from the voter's perspective.

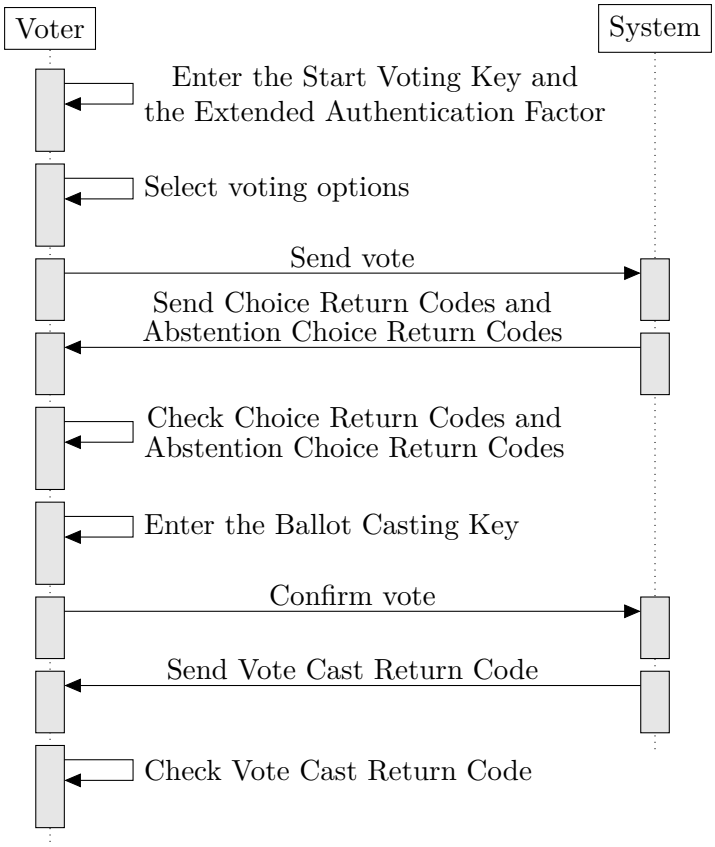


Fig. 3: The voting process in a two-round return code scheme. In the implementation, an authentication step precedes the voter's voting options selection.

First, the voter enters the Start Voting Key to authenticate herself and selects the candidates she wants to vote for (or answers questions in the case of a referendum). In turn, the system responds with a Choice Return Code for each selected voting option. The voter checks that the Choice Return Codes match the ones printed on the voting card—otherwise, the voter aborts the process and alerts the election authorities.
If the voter abstains for an election, the voter receives a single Abstention Choice Return Code, even if the abstention group allows to make multiple selections. Displaying a single Abstention Choice Return Code reduces the cognitive burden for the voter and makes semantically more sense: the voter made a single abstention decision for the whole group, not one decision per voting option.
If the Choice Return Codes and/or the Abstention Choice Return Codes match, the submitted vote corresponds to the voter's intention, and the voter enters the Ballot Casting Key. Finally, the system acknowledges a successful confirmation by sending back the Vote Cast Return Code.
If the voter aborts the process after sending or confirming the vote, she can resume the process (potentially on a different voting device) by logging in again.
Figure 4 shows the process after the voter sent the vote in a previous session (but did not confirm it), while figure 5 shows the process after the voter confirmed the vote earlier on.

Copyright 2026 Swiss Post Ltd.

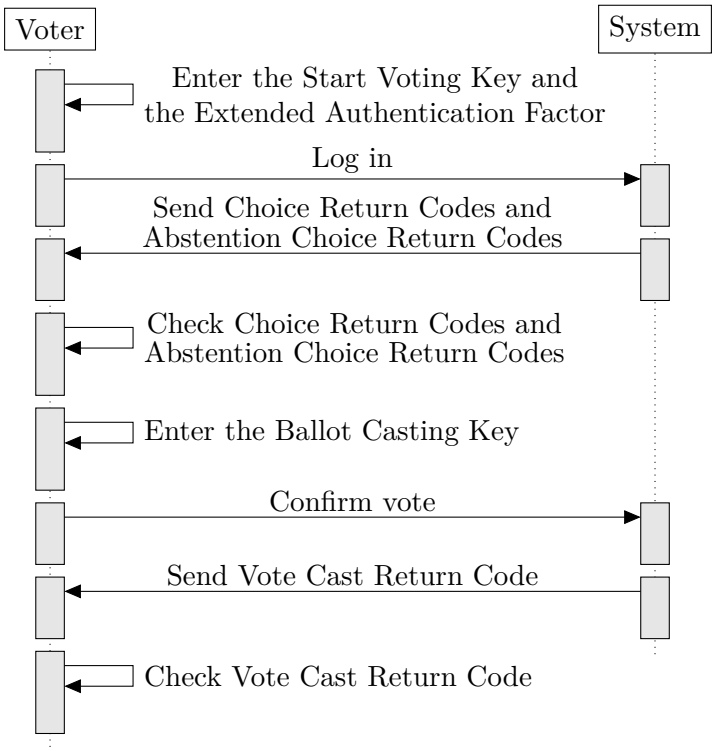


Fig. 4: The voting process when the voter aborted voting in an earlier session after having sent the vote. In the earlier session, the voter did not yet confirm the vote. The Choice Return Codes and Abstention Choice Return Codes returned by the system correspond to the selected voting options in the earlier session. Please note that at this stage, the voter can no longer change the selections. However, the vote is not yet considered final, and the voter might still decide to use a different voting channel.

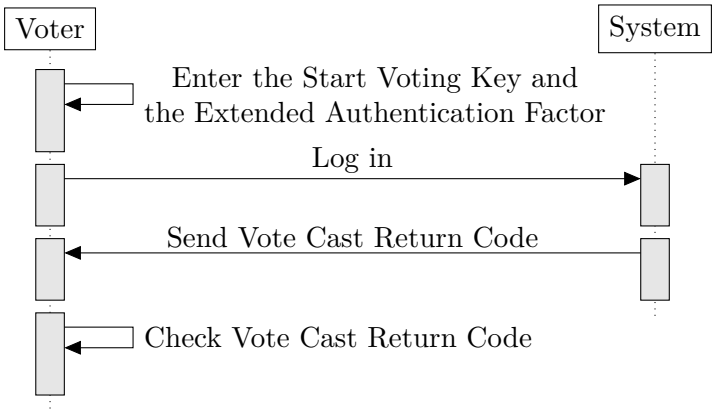


Fig. 5: The voting process when the voter aborted voting in an earlier session after vote confirmation. The voter can no longer change the selections and the vote is considered final. The voter may repeat this process on multiple devices until she convinces herself that the system returns the correct Vote Cast Return Code.

Copyright 2026 Swiss Post Ltd.

1.2 Security Objectives

The computational proof [20] details the Swiss Post Voting System's threat model and formally proves the following security properties:

- Individual verifiability
- Universal verifiability
- Vote secrecy

Individual verifiability allows the voter to check that the server correctly registered her vote and contains the voter's choices. Faithfully executing the two-round return code scheme (see section 1.1) guarantees to the voter that the system's trustworthy part registered the intended vote. Conversely, this implies that even a powerful adversary, controlling the voting client and most of the server infrastructure, cannot alter or drop a vote without being detected by a diligent voter or auditor.
Universal verifiability allows voters or auditors to check that the system correctly counted all confirmed votes. Independent auditors verify every step in the counting process—from the registration of the encrypted voting options to the decryption and tallying process—without compromising vote secrecy. Advanced cryptographic techniques such as non-interactive zero-knowledge proofs and verifiable mix-nets allow the Swiss Post Voting System to have the best of both worlds: the election result is correct beyond doubt, and every single vote remains secret.
Vote secrecy preserves the privacy of the voter. By extension, vote secrecy ensures that no component learns the election results before the final decryption step. The Swiss Post Voting System protects vote secrecy by encrypting votes end-to-end and splitting the decryption key among multiple entities.

Copyright 2026 Swiss Post Ltd.

1.3 Limitations

No cryptographic protocol is unconditionally secure. The protocol of the Swiss Post Voting System bases its security objectives on a few assumptions. Even though these assumptions comply with the Federal Chancellery's Ordinance on Electronic Voting [7], we will highlight some of them for the sake of clarity and transparency.

- First, we are defending our security properties against an adversary that is polynomially bounded.
- An efficient, fault-tolerant quantum computer could break some of the mathematical assumptions underpinning the Swiss Post Voting System.
- Quantum-resistant e-voting protocols exist, but they are relatively recent, and their security properties are less well understood.
- We consider quantum-resistance an area for future improvement of the protocol.
- Second, to provide vote secrecy, the Swiss Post Voting System assumes that the attacker is not controlling the voting client.
- While we guarantee individual verifiability even if the voting client is malicious, we cannot use cryptography to prevent an attacker from spying on the voter's choices on malicious clients.
- However, the Swiss Post Voting System prevents the server from breaking vote secrecy if at least one control component is honest.
- Third, the Swiss Post Voting System assumes a trustworthy setup and printing component.
- The setup and printing component generate keys and codes in conjunction with the control components and sends the voters' voting cards.
- By its nature, the setup and printing component handle sensitive data, and the process for generating and sending voting cards must meet rigorous requirements. In general, e-voting systems aim to distribute trust and detect any attack or malfunction if at least one component works correctly. However, we believe that there are certain limits for distributing the functionality of the setup and printing component: one cannot expect the voter to combine different code sheets by hand, and the costs of printing multiple code sheets in independent printing facilities would currently be prohibitive. For now, we propose to implement a return code scheme with a single setup and printing component, which must put organizational and technical means into place to prevent the leaking of the secret codes.

Copyright 2026 Swiss Post Ltd.

1.4 Conventions

We use the following conventions throughout the document.

- Display algorithms in font without serif and **vectors** in **boldface**.
- Explicit domains and ranges of context, input, and output variables. We assume that the implementation ensures the correct domain of the input and context elements. E.g. this means that when an input has the expected form $\mathbf{x} = (x_0, \ldots, x_{n-1}) \in (\mathbb{G}_q)^n$, the implementation checks that the input elements have the correct form: That $\mathbf{x}$ has exactly n elements and each one is a group member, for instance by calculating that the Jacobi Symbol of each element of $\mathbf{x}$ equals 1.
- Use the terms public key and secret key (instead of private key) and abbreviate them with pk and sk.
- Use 0-based indexing.
- Flatten lists when defined over multiple lines of pseudocode. Therefore, the expression

  $$list \leftarrow (a, b)$$

  $$list \leftarrow (list, c, d)$$

  results in a list without any nested structures:

  $$list = (a, b, c, d)$$

- Use nested structures when defined over one line of pseudocode. Therefore, the expression

  $$list_1 \leftarrow (a, b)$$

  $$list_2 \leftarrow (c, d)$$

  $$list_3 \leftarrow (list_1, list_2)$$

  results in a list with nested structures:

  $$list_3 = ((a, b), (c, d))$$

- Explicitly include the structure of empty lists in the algorithms.

  $$() \neq (())$$

  Hence, we also represent empty lists in nested structures.

- Use the operator $||$ to indicate that we append an element to a list:

  $$list \leftarrow (a, b)$$

  $$list \leftarrow list \,||\, c$$

  results in

  $$list = (a, b, c)$$

Copyright 2026 Swiss Post Ltd.

- Slightly abuse the $\in$-notation to check whether an element is part of a list or not:

$$list \leftarrow (a, b)$$

$$a \in list = \top$$

$$c \in list = \bot$$

- Define sets for ranges, e.g. for $i \in [0, n)$. This expression indicates that we include the lower bound but exclude the upper bound, i.e. $0 \leq i < n$

- For a key-value map L consisting of pairs $(\texttt{key}, \texttt{value})$, we define the retrieval of the $\texttt{value} \in \mathcal{B}$ corresponding to a $\texttt{key} \in \mathcal{A}$ as following:

$$\mathsf{L} : \mathcal{A} \rightarrow \mathcal{B}$$

$$\mathsf{L}(\texttt{key}) = \texttt{value}$$

  If the map has more than two entries, we still use the same notation while making it clear which of the entries should be retrieved from the map.

Copyright 2026 Swiss Post Ltd.

1.5 Context, State, and Input Variables

The specification distinguishes *context* and *input* variables as well as *stateful lists and maps*.

1.5.1 Context Variables

In cryptographic protocols, a context variable acts as a common reference for participants during algorithm execution. The trustworthiness of these variables is derived from their deterministic nature, mutual acceptance among participants, and the capacity for verification against external public data or information from prior protocol stages. As a result, context variables are trusted more than input variables. In algorithms, input variables undergo validation against these context variables. Typically, context variables remain invariant across multiple algorithm invocations. Furthermore, when an algorithm calls another, the callee inherits the context variables from the caller without the need for explicitly passing context variables as parameters. Context variables encompass the following parameters:

- Group parameters
- Election event context identifiers and component indices
- Election event parameters
- Public keys (under certain conditions)
- Static parameters

Group parameters: These parameters can be deterministically derived or validated by protocol participants using a seed (see section 3.2). Consequently, a participant either initially establishes the group parameters independently or obtains them from a trustworthy source.

Election event context identifiers and component indices: Algorithms are executed with reference to a specific entity within the election event context, as indicated in table 11. Protocol participants in the Swiss Post Voting System have a common view of the election event context (see Section 3.4). They achieve this common view by receiving the election event context from a trustworthy source, cross-checking it with publicly available information, or integrating it in the challenges of the zero-knowledge proofs (see the algorithms 3.14 GetHashElectionEventContext, 3.15 GetHashContext and 3.18 GetHashExtractedElectionEvent). This common understanding enables each participant to deduce the corresponding identifiers of other entities within the election event. For example, one might derive the verification card set ID from a verification card ID, or the election event ID from a verification card ID. Therefore, the election event context identifiers typically originate from the component's internal view, thereby constituting a context for the algorithm. If the context of an algorithm requires an entire vector of verification card IDs, the protocol participants must verify that the vector contains the correct verification card IDs in the expected order by matching it with information from earlier protocol steps.
We omit the election event context identifiers in the specification of an algorithm if they are not utilized during the algorithm's operation.
Furthermore, it is imperative for the implementation to validate the election event context. Certain algorithms are restricted to specific protocol phases or conditions. For instance,

Copyright 2026 Swiss Post Ltd.

re-executing an algorithm from the configuration phase is prohibited once that phase has concluded.
Several algorithms are executed by multiple control components, each possessing a unique index. We assume that each control component knows its respective index, thus we classify the index as a context variable.

Election event parameters: Section 3.4 elaborates on the electoral model, which sets the parameters for various elections and votes. These parameters, such as the maximum number of selections or the total number of candidates, can typically be verified using publicly available information. Moreover, the association of voting options with prime numbers and the proper structure of a submitted vote are also verifiable, as further discussed in Section 3.4.7. Therefore, we consider the parameters of the election event as context variables.

Public keys (under certain conditions): The Swiss Post Voting System distributes the generation of certain public keys and ensures their correct generation with zero-knowledge proofs (see section 3.6.5). When the protocol ensures that a trustworthy component participated in the generation of a public key and the proofs of correct key generation were previously verified, this public key can be treated as a context variable for the execution of a given algorithm. On the other hand, if the public key does not fulfill these conditions, it must be classified as an input variable, requiring extensive verification. Specifically, secret keys and their corresponding key pairs are always classified as input variables, as the private nature of secret keys precludes their inclusion in a possibly shared context.

Static parameters: The section Symbols defines the majority of constants and frequently utilized parameters. Should an algorithm necessitate a particular static parameter, it will be categorized as a context variable, given that this parameter can be extrapolated from established standards or cryptographic best practices.

1.5.2 Input Variables

Any variable not classified as a context variable is deemed an input variable. These input variables, potentially originating from unreliable sources, necessitate comprehensive, multi-layered validation. It is critical, especially when an algorithm employs a variable as both context and input, to prioritize the context variable as the authoritative source. For instance, in scenarios where the context incorporates a variable n and the input includes a list of length n, the implementation must verify that the list's actual size corresponds to the context variable n.
Components maintaining an internal view are assumed to store the outputs of their algorithms in this internal view. If an input variable corresponds to an output previously produced by an algorithm of the same component, it must either be retrieved directly from the component's internal view or validated against it.

1.5.3 Stateful Lists and Maps

Certain algorithms must be executed only once, or only a specified number of times, or must not be executed before another algorithm. Consequently, maintaining and updating state across multiple algorithm invocations is critical. Broadly, algorithms modifying stateful lists

Copyright 2026 Swiss Post Ltd.

and maps lack idempotency, as repeated executions with identical context and input variables may yield different results.
The implementation must establish a robust synchronization mechanism to reliably manage updates and retrievals involving stateful lists and maps.

Copyright 2026 Swiss Post Ltd.

2 System Overview

The following sections explain the different actors of the Swiss Post Voting System. The computational proof of the cryptographic protocol elaborates on the threat models for the different security objectives [20].

2.1 Voters

The voters authenticate to the voting server, select voting options, check the short Choice Return Codes (and/or Abstention Choice Return Codes) and confirm their vote.
The voters receive the following secret codes by postal mail (see section 1.1).

Description	Abbreviation
Start Voting Key	$\texttt{SVK}$
Vector of short Choice Return Codes	$\mathbf{CC}$
Abstention Choice Return Codes	$\mathbf{ACC}$
Ballot Casting Key	$\texttt{BCK}$
Vote Cast Return Code	$\texttt{VCC}$

Tab. 1: Overview of the Voter's codes

2.2 Voting Client

The voting client authenticates to the voting server, encrypts the vote and sends it to the voting server and control components. The voting client works with the following key material.

Abbreviation	Key	Domain
$\texttt{K}_{\texttt{id}}, \texttt{k}_{\texttt{id}}$	Verification card key pair	$\texttt{K}_{\texttt{id}} \in \mathbb{G}_q, \texttt{k}_{\texttt{id}} \in \mathbb{Z}_q$
$\texttt{EL}_{\texttt{pk}}$	Election public key	$\texttt{EL}_{\texttt{pk}} \in \mathbb{G}_q^{\delta_{\texttt{max}}}$
$\mathbf{pk}_{\text{CCR}}$	Choice Return Codes encryption public key	$\mathbf{pk}_{\text{CCR}} \in \mathbb{G}_q^{\psi_{\texttt{max}}}$

Tab. 2: List of the Voting Client's keys

2.3 Voting Server

The voting server relays messages to the control components and extracts the short Choice Return Codes and the Vote Cast Return Code.

Copyright 2026 Swiss Post Ltd.

2.4 Control Components

The control components generate the return codes, shuffle the encrypted votes, and decrypt them at the end of the election event. Conceptually, we distinguish two control component functionalities:

Return Codes control components CCR: compute the Choice Return Codes and the Vote Cast Return Code—in interaction with the setup component (configuration phase) and the voting server (voting phase).

Mixing control components CCM: mix and partially decrypt ciphertexts containing the encrypted votes.

In the specification, we refer to each functionality separately, even though the control components are a single entity combining the CCR and CCM functionalities.

Abbreviation	Key	Domain
$\mathtt{EL}_{\mathtt{pk},j}, \mathtt{EL}_{\mathtt{sk},j}$	CCM_j election key pair	$\mathtt{EL}_{\mathtt{pk},j} \in \mathbb{G}_q^{\delta_{\max}}, \mathtt{EL}_{\mathtt{sk},j} \in \mathbb{Z}_q^{\delta_{\max}}$
$\overline{\mathtt{EL}}_{\mathtt{pk}}$	Remaining election public key	$\overline{\mathtt{EL}}_{\mathtt{pk}} \in \mathbb{G}_q^{\delta_{\max}}$
$\mathtt{K'}_j, \mathtt{k'}_j$	$\mathtt{CCR_j}$ Return Codes Generation key pair	$\mathtt{K'}_j \in \mathbb{G}_q, \mathtt{k'}_j \in \mathbb{Z}_q$
$\mathtt{K}_{j,\mathtt{id}}, \mathtt{k}_{j,\mathtt{id}}$	Voter Choice Return Code Generation key	$\mathtt{K}_{j,\mathtt{id}} \in \mathbb{G}_q, \mathtt{k}_{j,\mathtt{id}} \in \mathbb{Z}_q$
$\mathtt{Kc}_{j,\mathtt{id}}, \mathtt{kc}_{j,\mathtt{id}}$	Voter Vote Cast Return Code Generation key	$\mathtt{Kc}_{j,\mathtt{id}} \in \mathbb{G}_q, \mathtt{kc}_{j,\mathtt{id}} \in \mathbb{Z}_q$
$\mathbf{pk}_{\text{CCR}_j}, \mathbf{sk}_{\text{CCR}_j}$	CCR_j Choice Return Codes encryption keys	$\mathbf{pk}_{\text{CCR}_j} \in \mathbb{G}_q^{\psi_{\max}}, \mathbf{sk}_{\text{CCR}_j} \in \mathbb{Z}_q^{\psi_{\max}}$

Tab. 3: List of the Control Component's keys

2.5 Electoral Board

The cantons set up an electoral board comprising citizens or party representatives to observe the orderly processing of an election event. Each member of the electoral board possesses a password, the combination of which derives the electoral board secret key $\mathtt{EB}_{\mathtt{sk}}$ used for the final decryption of votes. Thereby, we leverage the electoral board as an additional operational safeguard enforcing the four-eyes principle for the final decryption of the votes. The electoral board interacts with the setup component during the configuration phase and with the tally control component during the tally phase.

2.6 Setup Component

The setup component combines the control component's contributions and generates the codes. The setup component is active only during the configuration phase and its software runs in a controlled, offline environment on the canton's premises. Table 4 summarizes the setup component's keys which consist of an ElGamal key pair for encrypting data during the configuration phase.

Abbreviation	Key	Domain
$\mathsf{pk}_{\mathtt{setup}}, \mathsf{sk}_{\mathtt{setup}}$	Setup encryption key pair	$\mathsf{pk}_{\mathtt{setup}} \in \mathbb{G}_q^{n_{\max}}, \mathsf{sk}_{\mathtt{setup}} \in \mathbb{Z}_q^{n_{\max}}$

Tab. 4: List of the Setup Component's keys

Copyright 2026 Swiss Post Ltd.

2.7 Printing Component

The printing component prints the code sheets and sends them to the voter. All operations in the setup and printing component are subject to strict four-eyes principles and are executed on hardened laptops with special access rights.

2.8 Tally Control Component

The tally control component derives the secret key for the final decryption of the votes from the electoral board members' passwords. Moreover, the Tally Control Component handles the tasks required for the output of the cryptographic protocol: it factorizes the decrypted votes, decodes them to the corresponding voters' selections, and tallies the results into the expected format. Those tasks are fully deterministic and verifiable. The tally control component runs in a controlled, offline environment on the canton's premises.

Abbreviation	Key	Domain
$\mathtt{EB}_{\mathtt{pk}}, \mathtt{EB}_{\mathtt{sk}}$	Electoral board key pair	$\mathtt{EB}_{\mathtt{pk}} \in \mathbb{G}_q^{\delta_{\max}}, \mathtt{EB}_{\mathtt{sk}} \in \mathbb{Z}_q^{\delta_{\max}}$

Tab. 5: List of the Tally Control Component's keys

2.9 Auditors

Auditors verify that the parties faithfully executed their operations within the Swiss Post Voting System. To this end, they employ software as a technical aid: the verifier. We require that at least one trustworthy auditor/verifier checks an election event. The process to select auditors and develop a verifier is outside of the scope of this document.
The verifier does *not* generate keys or participate in calculating the secret codes: we limit the verifier's role to running various verifications.
A verifier specification [21] details the verifier's algorithms.

2.10 Dispute Resolver

The dispute resolver intervenes when control components fail to agree on the list of confirmed votes, a prerequisite for initiating the tally process. As an independent entity, the dispute resolver reconciles these discrepancies by executing the dispute resolution protocol outlined in section 6.2. This process occurs in a controlled, offline environment on cantonal premises.

Copyright 2026 Swiss Post Ltd.

3 Preliminaries

3.1 Basic Data Types

The crypto primitives specification details how we represent, convert, and operate on basic data types such as bytes, integers, strings, and arrays.

3.2 Cryptographic Parameters and System Security Level

We refer the reader to the crypto primitives specification for a detailed description of the cryptographic parameters and the associated security level.
The setup component generates the election event encryption parameters with algorithm 3.1. To prevent modulo overflow when encoding votes, the algorithm ensures that the product of the largest possible combination of primes is smaller than p.
The input `seed` to algorithm 3.1 is restricted to be in the format CT_YYYYMMDD_XYnm, where:

- CT: Two uppercase letters for the cantonal abbreviation.
- YYYYMMDD: Date of the election event, e.g. 20231022.
- XYnm: TT/TP/PP, followed by the ascending sequence number, e.g. TT01. The abbreviations have the following meaning:
  - Test event on test environment (TT)
  - Test event on production environment (TP)
  - Productive event on production environment (PP)

Algorithm 3.1 GetElectionEventEncryptionParameters

Context:

Maximum supported number of voting options $n_{\mathtt{sup}} \in \mathbb{N}^+$ ▷ See section 3.4.5
Maximum supported number of selections $\psi_{\mathtt{sup}} \in \mathbb{N}^+$ ▷ $\psi_{\mathtt{sup}} < n_{\mathtt{sup}}$, see table 10

Input:

$\mathtt{seed} \in {\mathbb{A}_{UCS}}^{16}$ ▷ The name of the election event in the format specified above

Operation: ▷ For all algorithms see the crypto primitives specification

1: $(p, q, g) \leftarrow \mathsf{GetEncryptionParameters}(\mathtt{seed})$
2: $(p_0, \ldots, p_{n_{\mathtt{sup}}-1}) \leftarrow \mathsf{GetSmallPrimeGroupMembers}(p, q, g, n_{\mathtt{sup}})$
3: **if** $\prod_{i=(n_{\mathtt{sup}}-\psi_{\mathtt{sup}})}^{n_{\mathtt{sup}}-1} \mathrm{p}_i \geq p$ **then**
   return $\perp$
   ▷ The product of the $\psi_{\mathtt{sup}}$ largest prime numbers cannot be equal to or larger than p
4: **end if**

Output:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Small primes $\mathbf{p} = (p_0, \ldots, p_{n_{\mathtt{sup}}-1}),\ p_i \in (\mathbb{G}_q \cap \mathbb{P}) \setminus g, (p_i < p_{i+1})\ \forall\ i \in [0, n_{\mathtt{sup}} - 1)$

Copyright 2026 Swiss Post Ltd.

3.3 Keys and Codes of the Voting Card

The following table describes the elements of the voting card received by the voters (see section 1.1).

Description	Abbr.	Char length	Alphabet	Alphabet Size
Start Voting Key	SVK	$\mathtt{l}_{\mathtt{SVK}} = 24$	$\mathbb{A}_{\mathtt{u32}}$	$\lvert\mathbb{A}_{\mathtt{u32}}\rvert = 32$
Short Choice Return Code	CC	$\mathtt{l}_{\mathtt{CC}} = 4$	$\mathbb{A}_{10}$	$\lvert\mathbb{A}_{10}\rvert = 10$
Abstention Choice Return Code	ACC	$\mathtt{l}_{\mathtt{ACC}} = 6$	$\mathbb{A}_{10}$	$\lvert\mathbb{A}_{10}\rvert = 10$
Ballot Casting Key	BCK	$\mathtt{l}_{\mathtt{BCK}} = 9$	$\mathbb{A}_{10}$	$\lvert\mathbb{A}_{10}\rvert = 10$
Vote Cast Return Code	VCC	$\mathtt{l}_{\mathtt{VCC}} = 8$	$\mathbb{A}_{10}$	$\lvert\mathbb{A}_{10}\rvert = 10$

Tab. 6: Overview of the Voter's codes. The alphabet $\mathbb{A}_{\mathtt{u32}}$ used for the Start Voting Key is described in the crypto primitives specification.

We visually distinguish the short Choice Return Codes from the Abstention Choice Return Codes by their length.
For the authentication of the voter we require a 128-bit security strength, which is considered secure against brute-force attacks. To keep a good balance between usability and security, we use a Start Voting Key with a length of 24 characters in a Base32 alphabet (see table 6), and then apply the Argon2 key derivation function, described in the crypto primitives specification. The length of the Start Voting Key provides up to

$$e = \lfloor \mathtt{l}_{\mathtt{SVK}} \cdot \log_2(\lvert\mathbb{A}_{\mathtt{u32}}\rvert) \rfloor = 120 \text{ bits}$$

of entropy. To make brute-force attacks against this space computationally infeasible, we apply the Argon2id key derivation function. As discussed by Percival [17], this raises the cost of offline guessing attacks substantially, compensating for the small entropy shortfall in practice. Argon2id makes exhaustive key search over 120 bits practically infeasible and provides effective resistance comparable to 128-bit security in practice. See also RFC 9106 [3, section 7] for security considerations for Argon2id.
In the crypto primitives specification we define different Argon2id profiles. Each time Argon2id is used, we specify which of these profiles is used.
In contrast to the SVK, the other codes on the voting card, CC, BCK, and VCC, are not required to meet the same entropy standards, as they are not used as cryptographic keys. For usability, they are chosen as short numeric codes. See also the computational proof of the cryptographic protocol [20, section 19].
The codes CC, ACC and VCC serve to provide individual verifiability to voters. To protect against guessing attacks, their length ensure that the probability of an attacker correctly guessing a valid code remains below 0.1%, in accordance with the requirements of the Federal Chancellery's Ordinance [7].
The BCK is entered by the voter to confirm the casting of the ballot, and can only be used within the same session in which the correct corresponding SVK was used. To prevent brute-force attacks, the number of allowed attempts is limited to five per verification card. This restriction makes systematic guessing attacks ineffective.

Copyright 2026 Swiss Post Ltd.

3.4 Electoral Model

3.4.1 Election Data Model

The Swiss Post Voting System supports the electoral model outlined in the eCH-standards. An eCH standard is a guideline developed by the eCH association to ensure interoperability and quality in Swiss e-government projects and processes. The eCH-0155 standard [10], in combination with the eCH-0045 standard on electoral registers [9], defines the data format for attributes related to votes and elections at all political levels in Switzerland.
We identify each *election event* with the election event ID $\texttt{ee}$.
Following a similar approach taken by CHVote [13], an election event has exactly $t_{\texttt{total}} \geq 1$ simultaneous, independent ψ_j-out-of-n_j elections

$$\mathbf{e}_{\texttt{total}} = (e_0, \ldots, e_{t_{\texttt{total}}-1})$$

where

$$e_j = j \qquad \forall j \in [0, t_{\texttt{total}})$$

The voter selects for an election e_j exactly ψ_j voting options out of n_j possible ($\psi_j \leq n_j$).
Across all $t_{\texttt{total}}$ elections, $\psi_{\texttt{total}} = \sum_{j=0}^{t_{\texttt{total}}-1} \psi_j$ represents the cumulative number of selections.

$$\boldsymbol{\psi}_{\texttt{total}} = (\psi_0, \ldots, \psi_{t_{\texttt{total}}-1})$$

Across all $t_{\texttt{total}}$ elections, $n_{\texttt{total}} = \sum_{j=0}^{t_{\texttt{total}}-1} n_j$ signifies the aggregate number of voting options.

$$\mathbf{n}_{\texttt{total}} = (n_0, \ldots, n_{t_{\texttt{total}}-1})$$

Furthermore, every election has a *domain of influence* $\texttt{DoI}_j$, which is the political area for which voting rights apply in a specific vote or election. A domain of influence can exist at federal, cantonal, district, regional or municipal level.

$$\mathbf{DoI}'_{\texttt{total}} = (\texttt{DoI}'_0, \ldots, \texttt{DoI}'_{t_{\texttt{total}}-1})$$

where $\texttt{DoI}'_j \in [0, t_{\texttt{total}})$ and $\texttt{DoI}'_0 = 0$ and

$$(\texttt{DoI}'_{j+1} - \texttt{DoI}'_j) \in \{0, 1\} \qquad \forall j \in [0, t_{\texttt{total}} - 1)$$

We assign an identifier $\texttt{DoI}_j \in \mathcal{T}_1^{50}$ to each domain of influence.

$$\mathbf{DoI}_{\texttt{total}} = (\texttt{DoI}_0, \ldots, \texttt{DoI}_{t_{\texttt{total}}-1})$$

Moreover, elections are part of a *presentation group*: a configuration concept that groups votes or elections which are presented together on the ballot paper.
An election event contains $\eta_{\texttt{total}} \geq 1$ presentation groups. We formalize presentation groups as a vector of size $t_{\texttt{total}}$.

$$\mathbf{pg}_{\texttt{total}} = (\texttt{pg}_0, \ldots, \texttt{pg}_{t_{\texttt{total}}-1})$$

where $\texttt{pg}_j \in [0, t_{\texttt{total}})$ and $\texttt{pg}_0 = 0$ and

$$(\texttt{pg}_{j+1} - \texttt{pg}_j) \in \{0, 1\} \qquad \forall j \in [0, t_{\texttt{total}} - 1)$$

$\texttt{pg}_j$ is the presentation group to which election e_j belongs.

Copyright 2026 Swiss Post Ltd.

If the presentation group allows abstention (see Section 3.9), the voter either participates in all items in the presentation group or abstains from the group as a whole. A presentation group that allows abstention is called an *abstention group*.
An election event contains $\kappa_{\texttt{total}} \geq 0$ abstention groups.

$$\mathbf{ag}_{\texttt{total}} = (\mathsf{ag}_0, \ldots, \mathsf{ag}_{\eta_{\texttt{total}}-1})$$

where $\mathsf{ag}_j \in \mathbb{Z}_2$ indicates whether a presentation group allows abstention.

Copyright 2026 Swiss Post Ltd.

3.4.2 Election Types and Parameters

Table 7 highlights examples of the number of selections ψ_j and the number of voting options n_j for common elections.

Election Type	**Number of Candidates**	**Accumulation**	**Blank Voting Options**	**Abstention Voting Options**	**Write-in Voting Options**	ψ_j	n_j
Majoritarian election with 5 candidates and 2 seats without write-ins, abstention allowed	5	1	2	2	0	2	9
Majoritarian election with 5 candidates and 2 seats with write-ins, no abstention allowed	5	1	2	0	2	2	9
Proportional election (list part) with 12 lists, abstention allowed	12	1	1	1	0	1	14
Proportional election (candidates part) with 180 candidates, 30 seats, and accumulation 2, abstention allowed	180	2	30	30	0	30	420
Popular Vote Question, no abstention allowed	2	1	1	0	0	1	3
Popular Vote Question, abstention allowed	2	1	1	1	0	1	4

Tab. 7: Example parameters of the electoral model.

Broadly, we distinguish three types of elections:

Majoritarian election. An election where voters vote for candidates based on the majority system, in which seats are allocated to the candidates with the most votes or more than half of the votes. In majoritarian elections, there are usually no party lists and ψ_j corresponds to the number of candidate positions the voter can select. A majoritarian election may allow write-in candidates, see Section 3.8.

Proportional election. An election in which seats are allocated among parties in proportion to the votes cast for them. In proportional elections, voters vote for lists *and* candidates, and we break down the election into two distinct sub-elections: one for the list selection with $\psi_j = 1$ and another for the candidate selection, where ψ_j corresponds to the number of candidate positions the voter can select. In the candidate selection, a voter may be allowed to select the same candidate multiple times, which is called *accumulation*.

Copyright 2026 Swiss Post Ltd.

Popular vote question. A proposal to the voters (e.g. an initiative or a referendum) that voters can accept or reject. We model it as a special case of a majoritarian election with two candidates (YES, NO) and $\psi_j = 1$.

The total number of voting options n_j for a given election is determined by the following parameters:

- *Number of candidates:* Depending on the type of election, this corresponds to the number of lists, candidates, or answers.
- *Accumulation:* Specifies how many times a candidate can appear on a ballot paper (1, 2, or 3). If the accumulation is set to 2 or 3, the system assigns 2 or 3 distinct voting options to the candidate, each with a different Choice Return Code (see Section 1.1).
- *Blank voting options:* Every election contains exactly ψ_j blank voting options, ensuring an equal count of blank options and selections.
- *Abstention voting options:* If the election belongs to an abstention group, there are ψ_j additional abstention voting options. Otherwise, there are none.
- *Write-in voting options:* Write-in voting options allow a voter to select a candidate who is not officially registered. If the election permits write-ins, there are ψ_j write-in voting options; otherwise, there are none.

Hence, the number of voting options n_j for election e_j is:

$$n_j = n_{c,j} \cdot \alpha_j + \psi_j + ag_j \cdot \psi_j + \omega_j \cdot \psi_j = n_{c,j} \cdot \alpha_j + (1 + ag_j + \omega_j) \cdot \psi_j$$

where $n_{c,j}$ is the number of candidates, $\alpha_j \in \{1, 2, 3\}$ is the accumulation, $ag_j \in \{0, 1\}$ indicates whether the election belongs to an abstention group, $\omega_j \in \{0, 1\}$ indicates whether write-ins are allowed, and ψ_j is the number of selections for election e_j.

Copyright 2026 Swiss Post Ltd.

3.4.3 Electorate, Verification Card Sets and Ballot Boxes

We assume a list of $\mathtt{N}_{\mathtt{E}_{\mathtt{total}}}$ voters, each identified by a unique identifier $\mathtt{id} \in [0, \mathtt{N}_{\mathtt{E}_{\mathtt{total}}})$.
We assign each voter $\mathtt{id}$ a verification card $\mathtt{vc}_{\mathtt{id}}$ and a voting card $\mathtt{vcd}_{\mathtt{id}}$. The distinction between *verification card* and *voting card* is merely for technical reasons, and in this document we always refer to the verification card $\mathtt{vc}_{\mathtt{id}}$. Moreover, the setup component and the voting client derive a voter's identifier $\mathtt{credentialID}_{\mathtt{id}}$ from the Start Voting Key $\mathtt{SVK}_{\mathtt{id}}$ (see Section 3.7).
Every voter has voting rights for certain domains of influence and each election is assigned to a specific domain of influence (see Section 3.4.1). Certain voters might have voting rights for multiple domains of influence, while others might have voting rights for only the federal or cantonal level.
We formalize this using a function VoterDoI that maps a voter identifier to the subset of domains of influence for which the voter has voting rights in this particular election event:

$$\mathsf{VoterDoI}(\mathtt{id}, \mathbf{DoI}_{\mathtt{total}}) = \mathbf{DoI}_{\mathtt{id}} = (\mathtt{DoI}_{i_0}, \ldots, \mathtt{DoI}_{i_{k-1}})$$

where

$$0 \leq i_0 < \cdots < i_{k-1} < t_{\mathtt{total}}$$

and $k \leq t_{\mathtt{total}}$ denotes the number of unique domains of influence for which voter $\mathtt{id}$ has voting rights in election event $\mathtt{ee}$. Consequently, $\mathbf{DoI}_{\mathtt{id}}$ is the subsequence of $\mathbf{DoI}_{\mathtt{total}}$ that preserves the order of the domains of influence.
Note that the function only returns the domains of influence that are relevant for the current election event, i.e. those contained in $\mathbf{DoI}_{\mathtt{total}}$. A voter might hold voting rights for additional domains of influence for which no election takes place in this election event. Those domains of influence are not included in the output of VoterDoI. Moreover, we exclude voters where $\mathbf{DoI}_i$ is empty, as they do not have voting rights for any of the elections in this election event.
Further, we define a function VoterCC that maps a voter identifier to a counting circle description that identifies in which counting circle the vote is going to be counted.

$$\mathsf{VoterCC}(\mathtt{id}, \mathbf{DoI}_{\mathtt{total}}) = \chi_{\mathtt{id}}$$

$\chi_{\mathtt{id}}$ designates the counting circle for voter $\mathtt{id}$ in the current election event. Usually, the counting circle corresponds to the voter's municipality, but it can also contain a more complex mapping where different domains of influence are counted in different counting circles.
Applying VoterDoI and VoterCC to all $\mathtt{N}_{\mathtt{E}_{\mathtt{total}}}$ voters divides the electorate into subsets of voters who all have the same voting rights and counting circle assignment for this particular election event. We call each such subset a *verification card set* and assign a *ballot box* to each verification card set, i.e. all voters in a verification card set submit their votes to the same ballot box.

$$\mathbf{vcs}' = (\mathtt{vcs}'_0, \ldots, \mathtt{vcs}'_{\mathtt{N}_{\mathtt{bb}}-1})$$

where $\mathtt{N}_{\mathtt{bb}}$ is the number of ballot boxes of an election event and $\mathtt{vcs}'_j \in [0, \mathtt{N}_{\mathtt{bb}})$.
We assign each verification card set a specific identifier $\mathtt{vcs}_j$, which leads to the following lexicographically sorted vector.

$$\mathbf{vcs} = (\mathtt{vcs}_0, \ldots, \mathtt{vcs}_{\mathtt{N}_{\mathtt{bb}}-1})$$

Copyright 2026 Swiss Post Ltd.

where $\texttt{vcs}_j \in \mathcal{T}_1^{50}$.
Formally, two voters $\texttt{id}_i$ and $\texttt{id}_k$ belong to the same verification card set if:

$$\mathbf{DoI}_{\texttt{id}_i} = \mathbf{DoI}_{\texttt{id}_k} \wedge \chi_{\texttt{id}_i} = \chi_{\texttt{id}_k}$$

Each verification card set $\texttt{vcs}'_i$ encompasses $\texttt{N}_{\texttt{E},\texttt{vcs}'_\texttt{i}}$ eligible voters.

$$\texttt{N}_{\texttt{E}_{\texttt{total}}} = \sum_{\texttt{vcs}'=0}^{\texttt{N}_{\texttt{bb}}-1} \texttt{N}_{\texttt{E},\texttt{vcs}'}$$

The election authorities also define *test* voters that are always assigned to distinct test counting circles and hence lead to specific *test* verification card sets and ballot boxes. These test voters are used to validate the correct configuration of the election event prior to the actual voting and tally phase.
During the voting phase, voters send and confirm their votes in the ballot box assigned to their verification card set. After the voting phase, $\texttt{N}_{\texttt{C}_{\texttt{bb}}}$ voters of a specific verification card set confirmed their vote and thus cast their ballot into the corresponding ballot box.
A ballot box may contain unconfirmed votes. Since we only tally confirmed votes, the GetMixnetInitialCiphertexts algorithm (see section 6.1.1) omits unconfirmed votes. For any verification card set and corresponding ballot box, we have $\texttt{N}_{\texttt{C}_{\texttt{bb}}} \leq \texttt{N}_{\texttt{E},\texttt{vcs}'}$.
Figure 6 summarizes the election event context.

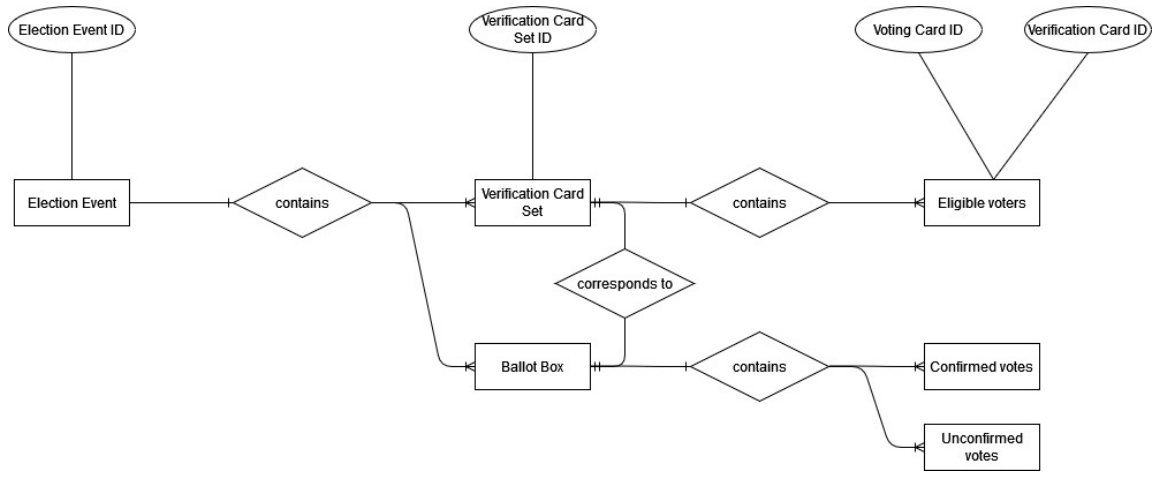


Fig. 6: Overview of the election event context. Every election event has multiple verification card sets. Each verification card set has a corresponding ballot box. A verification card set has $\texttt{N}_\texttt{E}$ eligible voters; a ballot box contains $\texttt{N}_\texttt{C}$ confirmed votes.

Furthermore, we assume that the election authorities define the verification card sets and ballot boxes in a way that guarantees vote secrecy; if a ballot box contains only one vote, this voter's privacy is trivially broken.
Unless stated otherwise, whenever the variables are used in a context restricted to a single verification card set, we omit the verification card set subscript for readability. Thus, $\texttt{vcs}$ denotes the verification card set identifier, $\mathbf{DoI}$ its associated domains of influence vector, $\texttt{N}_\texttt{E}$ the number of eligible voters in that verification card set, and $\texttt{N}_\texttt{C}$ the number of confirmed votes in the corresponding ballot box.

Copyright 2026 Swiss Post Ltd.

3.4.4 Verification Card Set Specific Parameters and Functions

For every verification card set $\texttt{vcs}'$, we assume functions that derive the verification card set specific electoral model from the global electoral model introduced in Section 3.4.1 and that canonically reindex the entries of the associated vectors. The reindexing maps the distinct values occurring in a vector to $\{0, \dots, k-1\}$, preserving equality relations and the order of first occurrence. For example, $(0, 1, 3)$ is reindexed as $(0, 1, 2)$ and $(2, 2, 2)$ is reindexed as $(0, 0, 0)$.
In particular, these functions derive the following vectors:

$$\mathbf{e}_{\texttt{vcs}'} = (e_0, \dots, e_{t_{\texttt{vcs}'}-1})$$

where $t_{\texttt{vcs}'}$ is the number of elections in verification card set $\texttt{vcs}'$ and $e_j = j \in [0, t_{\texttt{vcs}'})$.
Similarly, we derive the verification card set specific vector of number of selections and number of voting options:

$$\boldsymbol{\psi}_{\texttt{vcs}'} = (\psi_0, \dots, \psi_{t_{\texttt{vcs}'}-1})$$

$$\mathbf{n}_{\texttt{vcs}'} = (n_0, \dots, n_{t_{\texttt{vcs}'}-1})$$

where $\psi_{\texttt{vcs}'} = \sum_{j=0}^{t_{\texttt{vcs}'}-1} \psi_j$ is the cumulative number of selection of the verification card set and $n_{\texttt{vcs}'} = \sum_{j=0}^{t_{\texttt{vcs}'}-1} n_j$ the cumulative number of voting options.
The verification card specific presentation and abstention groups contain the groups for which the voter has voting rights in at least one of the elections contained in these groups.

$$\mathbf{pg}_{\texttt{vcs}'} = (\texttt{pg}_0, \dots, \texttt{pg}_{t_{\texttt{vcs}'}-1})$$

where $\eta_{\texttt{vcs}'}$ is the number of presentation groups in the verification card set and $\texttt{pg}_j \in [0, \eta_{\texttt{vcs}'})$.
Similarly, we derive the verification card set specific vector of abstention groups:

$$\mathbf{ag}_{\texttt{vcs}'} = (\texttt{ag}_0, \dots, \texttt{ag}_{\eta_{\texttt{vcs}'}-1})$$

where $\texttt{ag}_j \in \mathbb{Z}_2$ and the verification card set contains $\kappa_{\texttt{vcs}'}$ abstention groups.
To improve readability, whenever a variable is used in a verification card set specific context, we omit the verification card set subscript. Therefore, the variables $\mathbf{e}$, t, $\boldsymbol{\psi}$, ψ, $\mathbf{n}$, n, η, κ, $\mathbf{pg}$ and $\mathbf{ag}$ designate the verification card set specific variables.
We combine the most relevant verification card set specific parameters into the Electoral Model Context Λ, where we omit the verification card set specific subscript as well. The Electoral Model Context Λ contains the parameters that cannot be easily derived from the pTable, see Section 3.4.7. Table 8 summarizes the parameters contained in the Electoral Model Context Λ for a verification card set.

Parameter	Description	Constraints
$\boldsymbol{\psi} = (\psi_0, \dots, \psi_{t-1})$	Number of selections	$\psi_j \in \mathbb{N}^+$
$\mathbf{DoI} = (\texttt{DoI}_0, \dots, \texttt{DoI}_{k-1})$	Domains of Influence	$\texttt{DoI}_j \in \mathcal{T}_1^{50}, k \leq t$
$\mathbf{pg} = (\texttt{pg}_0, \dots, \texttt{pg}_{t-1})$	Presentation groups	$\texttt{pg}_j \in [0, \eta)$
$\mathbf{ag} = (\texttt{ag}_0, \dots, \texttt{ag}_{\eta-1})$	Abstention groups	$\texttt{ag}_j \in \mathbb{Z}_2, \eta \leq t$

Tab. 8: Electoral Model Context Λ with verification card set specific values.

Copyright 2026 Swiss Post Ltd.

We formally define the electoral model context as a tuple of the vectors defined above:

$$\Lambda = (\boldsymbol{\psi}, \mathbf{DoI}, \mathbf{pg}, \mathbf{ag})$$

Further, we define a helper function $\mathsf{indices}$ that returns the interval of the global indices of the selections that belong to a presentation group pg_i:

$$\mathsf{indices}(i, \mathbf{pg}, \boldsymbol{\psi}) = [\Psi_i, \Psi_{i+1}) \tag{1}$$

where

$$\Psi_i = \sum_{h \in [0,i)} \sum_{\substack{j \in [0,t) \\ \mathsf{pg}_j = h}} \psi_j$$

Example. Consider the case in which a verification card set has 5 elections, each one of these having a fixed number of selectable voting options. Moreover, each election, is part of one out of three presentation groups.
By using the notation introduced above, we have

$$\begin{aligned} \mathbf{pg} &= (0, 0, 1, 2, 2) \\ \mathbf{e} &= (0, 1, 2, 3, 4) \\ \psi &= (2, 1, 4, 1, 1) \end{aligned}$$

Therefore, we can compute

$$\mathsf{indices}(0, \mathbf{pg}, \boldsymbol{\psi}) = [\Psi_0, \Psi_1) = [0, \sum_{j=0}^{1} \psi_j) = [0, 3)$$

Similarly, we can compute

$$\begin{aligned} \mathsf{indices}(1, \mathbf{pg}, \boldsymbol{\psi}) &= [3, 7) \\ \mathsf{indices}(2, \mathbf{pg}, \boldsymbol{\psi}) &= [7, 9) \end{aligned}$$

Copyright 2026 Swiss Post Ltd.

3.4.5 Election Parameters

Table 9 defines the symbols for the election event parameters configured for a specific verification card set.

Parameters	Actual value (configured per verification card set)
Number of voting options	n
Selectable voting options	ψ
Number of write-in options	$\delta - 1$

Tab. 9: Verification card set specific parameters.

Hence, a voter must select exactly ψ out of n voting options. We derive the verification card set specific values n, ψ, and δ from the primes mapping table pTable (see section 3.4.7). Alongside the specific parameters for each verification card set, the algorithm GenSetupData (see section 4.1.1) derives these global parameters for the entire *election event*. Table 10 shows these parameters as well as the maximum value supported by the system for these settings, as per our analysis of federal, cantonal, and municipal elections.

Parameters	Actual value (global per election event)	Maximum supported value
Maximum number of voting options	$n_{\mathtt{max}}$	$n_{\mathtt{sup}} = 5000$
Maximum number of selections	$\psi_{\mathtt{max}}$	$\psi_{\mathtt{sup}} = 150$
Maximum number of write-ins	$\delta_{\mathtt{max}} - 1$	$\delta_{\mathtt{sup}} - 1 = 30$

Tab. 10: Election event specific parameters.

Algorithm 4.14 generates the election public key $\mathtt{EL}_{\mathtt{pk}}$ with exactly $\delta_{\mathtt{max}}$ elements.

Copyright 2026 Swiss Post Ltd.

3.4.6 Encoding of Voting Options

We define the actual voting option v_i as the concatenation of multiple identifiers as follows:

- Answer: question identifier | answer identifier
- Lists: election identifier | list identifier
- Candidates: election identifier | candidate identifier | candidate accumulation
- Blank candidate position: election identifier | blank candidate position identifier
- Write-in position: election identifier | write-in position identifier
- Abstention position: presentation group identifier | abstention option identifier

Each of the identifiers above is represented with a string from $\mathcal{T}_1^{50}$. The candidate accumulation is a positive integer converted to a string using IntegerToString. The domain of an actual voting option v_i is $\mathcal{D}_\mathrm{v} = \{x_1 \parallel x_2 \mid x_1, x_2 \in \mathcal{T}_1^{50}\} \cup \{x_1 \parallel x_2 \parallel x_3 \mid x_1, x_2, x_3 \in \mathcal{T}_1^{50}\}$.
We denote the actual voting options as $\tilde{\mathbf{v}} \leftarrow (\mathrm{v}_0, \ldots, \mathrm{v}_{n-1}),\ \mathrm{v}_i \in \mathcal{D}_\mathrm{v}$.
The voting options are encoded as prime numbers $\tilde{\mathbf{p}} \leftarrow (\tilde{\mathrm{p}}_0, \ldots, \tilde{\mathrm{p}}_{n-1}),\ \tilde{\mathrm{p}}_i \in (\mathbb{G}_q \cap \mathbb{P}) \setminus g$ (maintaining the order between both vectors).
Algorithm 3.1 GetElectionEventEncryptionParameters details the generation of small primes used to encode voting options. The mapping of voting options to prime numbers is *bijective*: each voting option maps to a distinct prime number and vice versa.
Moreover, we denote the semantic information corresponding to each voting option as a vector of strings $\boldsymbol{\sigma} \leftarrow (\sigma_0, \ldots, \sigma_{n-1}),\ \sigma_i \in {\mathbb{A}_{UCS}}^*$. We assume that the election event configuration contains German, French, Italian, and Rumantsch. We construct the voting option's semantic information σ_i in the following way:

- Non-blank answer: "`NON_BLANK`" | question (de, fr, it, rm) | answer (de, fr, it, rm)
- Blank answer: "`BLANK`" | question (de, fr, it, rm) | answer (de, fr, it, rm)
- Non-blank list: "`NON_BLANK`" | list description (de, fr, it, rm)
- Blank list: "`BLANK`" | list description (de, fr, it, rm)
- Candidate: "`NON_BLANK`" | family name | call name | date/year of birth
- Blank candidate position: "`BLANK`" | EMPTY_CANDIDATE_POSITION-[position number]
- Write-in position: "`WRITE_IN`" | WRITE_IN_POSITION-[position number]
- Abstention position: "`ABSTENTION`" | PRESENTATION_GROUP_POSITION-[position number] | ABSTENTION_POSITION-[position number]

Abstention options follow an all-or-nothing semantics at the level of an abstention group. Such a group is counted as an abstention only if the voter selects *all* abstention options belonging to it.
If the voter selects only a strict subset of the group, the vote remains *valid*, but contains an *invalid encoding* of voting options. A trustworthy voting client never creates such an encoding,

Copyright 2026 Swiss Post Ltd.

but a malicious one might. Therefore, the tally control component must process the invalid encoding according to predefined rules.
Specifically, the tally control component processes the invalid encoding as described in algorithm 6.13 and counts the incomplete set of abstention options as blank voting options. This approach is similar to the one proposed by CHVote [13, Ch. 2.2.4].
This semantics preserves individual verifiability. The voter receives the correct Abstention Choice Return Code only when the full abstention group was selected, as described in Section 3.9.1. Hence, the return code semantics match the tally semantics: only a full abstention group produces the Abstention Choice Return Code required for individual verifiability.
Finally, every voting option belongs either to a question, to a list selection of an election, or to a candidate selection of an election. Since this information is very important to assess whether a vote is *correct*, i.e. whether it conforms to a predefined way of filling out the ballot, we assign to each actual voting option v_i a correctness information τ_i. We denote the correctness information corresponding to each voting option as a vector of strings
$\boldsymbol{\tau} \leftarrow (\tau_0, \ldots, \tau_{n-1}),\ \tau_i \in \mathcal{D}_\tau$.

Copyright 2026 Swiss Post Ltd.

We construct the voting option's correctness information τ_i in the following way:

- If v_i corresponds to an answer of a referendum-style question (including abstention), use the question identifier as τ_i.
- If v_i corresponds to a list in an election (or to the empty or abstention list), use the prefix "L" concatenated with the pipe character (|) and the election identifier as τ_i.
- If v_i corresponds to a candidate, empty, write-in or abstention position in an election, use the prefix "C" concatenated with the pipe character (|) and the election identifier as τ_i.

The domain of a correctness information τ_i is $\mathcal{D}_\tau = \mathcal{T}_1^{50} \cup \{x_1 \parallel x_2 \mid x_1 \in \{\text{"L"}, \text{"C"}\}, x_2 \in \mathcal{T}_1^{50}\}$.

3.4.7 Primes Mapping Table

We define a primes mapping table pTable, conceptually, as the combination of $\tilde{\mathbf{v}}$, $\tilde{\mathbf{p}}$, $\boldsymbol{\sigma}$, and $\boldsymbol{\tau}$ and represented as an ordered list of tuples, *i.e.* $((\mathrm{v}_0, \tilde{\mathrm{p}}_0, \sigma_0, \tau_0), \ldots, (\mathrm{v}_{n-1}, \tilde{\mathrm{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1}))$. The primes mapping table must sequentially organize voting options as they appear in the election event configuration. We first order the voting options by votePosition and electionGroupPosition. Then, we sort within the election groups and votes. Within an election group, the voting options are sorted by electionPosition. Within a single election, the voting options are sorted in the following way:

1. If the election has lists, the list entries, sorted by listOrderOfPrecedence.
2. If the election has lists, the empty list entry.
3. If the election is within an abstention group, the abstention list entry.
4. The candidate entries. If the election has lists, then in order of appearance, otherwise sorted by candidatePosition.
5. The blank position entries, in order of appearance.
6. If the election has write-ins, the write-in position entries, in order of appearance.
7. If the election is within an abstention group, the abstention position entries.

Within a single vote, the voting options are sorted by ballotPosition. Within a standard ballot the voting options are sorted by answerPosition. Within a variant ballot the standard and tie-break questions are sorted by questionPosition, and within each question the voting options are sorted by answerPosition. If the popular vote is within an abstention group, an abstention option is added to every question.
The setup component generates the primes mapping table pTable in the algorithm 4.1 GenSetupData and sends it to the auditors (see figure 7) and to the Tally control component (see figure 8). The cryptographic protocol ensures that all participants have the same view of pTable by linking it to the Verification Card Keystore $\mathtt{VCks}_{\mathtt{id}}$ and the voting client's zero-knowledge proofs. In particular, the following algorithms ensure that all protocol participants have a consistent view of pTable:

- The setup component includes the contents of pTable in the authenticated symmetric encryption in 4.12 GenCredDat.

Copyright 2026 Swiss Post Ltd.

- The voting client includes the contents of pTable in the authenticated symmetric decryption in 5.3 GetKey and in the zero-knowledge proofs in 5.4 CreateVote.
- The control components include the contents of pTable when verifying the voting client's zero-knowledge proofs in 5.5 VerifyBallotCCR.
- The control components include the contents of pTable when generating a zero-knowledge proof in 5.6 PartialDecryptPCC and verifying each other's proofs in 5.7 DecryptPCC.
- The tally control component includes the contents of pTable when verifying the voting client's zero-knowledge proofs in 6.9 VerifyVotingClientProofs.
- The auditors include the contents of pTable when verifying the voting client's zero-knowledge proofs in VerifyTally.

The following algorithms operate on the primes mapping table pTable. As a special case, algorithms 3.2, 3.3, and 3.5 can be called with an empty list argument to retrieve all corresponding elements of the pTable.

Algorithm 3.2 GetEncodedVotingOptions

Context:

Primes mapping table $\mathsf{pTable} \in (\mathcal{D}_{\mathrm{v}} \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times {\mathbb{A}_{UCS}}^* \times \mathcal{D}_\tau)^n$ ▷ pTable is of the form $((\mathrm{v}_0, \tilde{\mathrm{p}}_0, \sigma_0, \tau_0), \ldots, (\mathrm{v}_{n-1}, \tilde{\mathrm{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1}))$

Input:

Actual voting options $(\mathrm{v}'_0, \ldots, \mathrm{v}'_{m'-1}) \in (\mathcal{D}_{\mathrm{v}})^{m'}$

Require: $0 \leq m' \leq n$

Require: $\mathrm{v}'_i \in \mathsf{pTable}\ \forall\ i \in \{0, \ldots, m'-1\}$

Require: $\mathrm{v}'_i \neq \mathrm{v}'_k, \forall i, k \in \{0, \ldots, m'-1\} \wedge i \neq k$ ▷ All actual voting options must be distinct

Operation:

1: **if** $m' = 0$ **then**
2: $m \leftarrow n$
3: $(\tilde{\mathrm{p}}'_0, \ldots, \tilde{\mathrm{p}}'_{m-1}) \leftarrow (\tilde{\mathrm{p}}_0, \ldots, \tilde{\mathrm{p}}_{m-1})$
4: **else**
5: $m \leftarrow m'$
6: **for** $i \in [0, m)$ **do**
7: $\tilde{\mathrm{p}}'_i \leftarrow \mathsf{pTable}(\mathrm{v}'_i)$ ▷ Retrieve the encoded voting option from pTable
8: **end for**
9: **end if**

Output:

Encoded voting options $(\tilde{\mathrm{p}}'_0, \ldots, \tilde{\mathrm{p}}'_{m-1}) \in ((\mathbb{G}_q \cap \mathbb{P}) \setminus g)^m$

Copyright 2026 Swiss Post Ltd.

Algorithm 3.3 GetActualVotingOptions

Context:

Primes mapping table $\mathsf{pTable} \in (\mathcal{D}_{\mathrm{v}} \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times {\mathbb{A}_{UCS}}^* \times \mathcal{D}_\tau)^n$ ▷ pTable is of the form $((\mathrm{v}_0, \tilde{\mathrm{p}}_0, \sigma_0, \tau_0), \ldots, (\mathrm{v}_{n-1}, \tilde{\mathrm{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1}))$

Input:

Encoded voting options $(\tilde{\mathrm{p}}'_0, \ldots, \tilde{\mathrm{p}}'_{m'-1}) \in ((\mathbb{G}_q \cap \mathbb{P}) \setminus g)^{m'}$

Require: $0 \leq m' \leq n$

Require: $\tilde{\mathrm{p}}'_i \in \mathsf{pTable}\ \forall\ i \in \{0, \ldots, m'-1\}$

Require: $\tilde{\mathrm{p}}'_i \neq \tilde{\mathrm{p}}'_k, \forall i, k \in \{0, \ldots, m'-1\} \wedge i \neq k$ ▷ All encoded voting options must be distinct

Operation:

1: **if** $m' = 0$ **then**
2: $m \leftarrow n$
3: $(\mathrm{v}'_0, \ldots, \mathrm{v}'_{m-1}) \leftarrow (\mathrm{v}_0, \ldots, \mathrm{v}_{m-1})$
4: **else**
5: $m \leftarrow m'$
6: **for** $i \in [0, m)$ **do**
7: $\mathrm{v}'_i \leftarrow \mathsf{pTable}(\tilde{\mathrm{p}}'_i)$ ▷ Retrieve the actual voting option from pTable
8: **end for**
9: **end if**

Output:

Actual voting options $(\mathrm{v}'_0, \ldots, \mathrm{v}'_{m-1}) \in (\mathcal{D}_{\mathrm{v}})^m$

Algorithm 3.4 GetSemanticInformation

Context:

Primes mapping table $\mathsf{pTable} \in (\mathcal{D}_{\mathrm{v}} \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times {\mathbb{A}_{UCS}}^* \times \mathcal{D}_\tau)^n$ ▷ pTable is of the form $((\mathrm{v}_0, \tilde{\mathrm{p}}_0, \sigma_0, \tau_0), \ldots, (\mathrm{v}_{n-1}, \tilde{\mathrm{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1}))$

Operation:

1: **return** $(\sigma_0, \ldots, \sigma_{n-1})$

Output:

Semantic information $(\sigma_0, \ldots, \sigma_{n-1}) \in ({\mathbb{A}_{UCS}}^*)^n$

Copyright 2026 Swiss Post Ltd.

Algorithm 3.5 GetCorrectnessInformation

Context:

Primes mapping table $\mathsf{pTable} \in (\mathcal{D}_{\mathrm{v}} \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times {\mathbb{A}_{UCS}}^* \times \mathcal{D}_\tau)^n$ ▷ pTable is of the form $((\mathrm{v}_0, \tilde{\mathrm{p}}_0, \sigma_0, \tau_0), \ldots, (\mathrm{v}_{n-1}, \tilde{\mathrm{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1}))$

Input:

Actual voting options $(\mathrm{v}'_0, \ldots, \mathrm{v}'_{m'-1}) \in (\mathcal{D}_{\mathrm{v}})^{m'}$

Require: $0 \leq m' \leq n$

Require: $\mathrm{v}'_i \in \mathsf{pTable}\ \forall\ i \in \{0, \ldots, m'-1\}$

Require: $\mathrm{v}'_i \neq \mathrm{v}'_k, \forall i, k \in \{0, \ldots, m'-1\} \wedge i \neq k$ ▷ All actual voting options must be distinct

Operation:

1: **if** $m' = 0$ **then**
2: $m \leftarrow n$
3: $(\tau'_0, \ldots, \tau'_{m-1}) \leftarrow (\tau_0, \ldots, \tau_{m-1})$
4: **else**
5: $m \leftarrow m'$
6: **for** $i \in [0, m)$ **do**
7: $\tau'_i \leftarrow \mathsf{pTable}(\mathrm{v}'_i)$ ▷ Retrieve the correctness information from pTable
8: **end for**
9: **end if**

Output:

Correctness information $(\tau'_0, \ldots, \tau'_{m-1}) \in (\mathcal{D}_\tau)^m$

Copyright 2026 Swiss Post Ltd.

Furthermore, we define additional algorithms that allow us to leverage the structured information contained in the voting options' semantic information.
First, the algorithm GetBlankCorrectnessInformation returns the vector of correctness information corresponding to the blank voting options. Since our electoral model (see section 3.4.1) ensures that there are as many blank voting options as selections, the resulting vector $\hat{\boldsymbol{\tau}}$ is of size ψ. Section 3.4.8 explains how we leverage this information to ensure that a vote contains a valid combination of voting options.

Algorithm 3.6 GetBlankCorrectnessInformation

Context:

Primes mapping table pTable $\in (\mathcal{D}_{\mathrm{v}} \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times {\mathbb{A}_{UCS}}^* \times \mathcal{D}_\tau)^n$ ▷ pTable is of the form $((\mathrm{v}_0, \tilde{\mathrm{p}}_0, \sigma_0, \tau_0), \ldots, (\mathrm{v}_{n-1}, \tilde{\mathrm{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1}))$

Operation:

1: $(\mathrm{v}_{\mathrm{b},0}, \ldots, \mathrm{v}_{\mathrm{b},\psi-1}) \leftarrow$ GetBlankActualVotingOptions() ▷ See algorithm 3.7.
2: $(\hat{\tau}_0, \ldots, \hat{\tau}_{\psi-1}) \leftarrow$ GetCorrectnessInformation$(\mathrm{v}_{\mathrm{b},0}, \ldots, \mathrm{v}_{\mathrm{b},\psi-1})$ ▷ See algorithm 3.5.

Output:

Blank correctness information $(\hat{\tau}_0, \ldots, \hat{\tau}_{\psi-1}) \in (\mathcal{D}_\tau)^\psi$

The algorithm GetBlankActualVotingOptions returns the vector of blank actual voting options.

Algorithm 3.7 GetBlankActualVotingOptions

Context:

Primes mapping table pTable $\in (\mathcal{D}_{\mathrm{v}} \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times {\mathbb{A}_{UCS}}^* \times \mathcal{D}_\tau)^n$ ▷ pTable is of the form $((\mathrm{v}_0, \tilde{\mathrm{p}}_0, \sigma_0, \tau_0), \ldots, (\mathrm{v}_{n-1}, \tilde{\mathrm{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1}))$

Operation:

1: $k \leftarrow 0$
2: **for** $i \in [0, n)$ **do**
3: **if** σ_i starts with "BLANK" **then**
4: $\mathrm{v}_{\mathrm{b},k} \leftarrow \mathrm{v}_i$ ▷ Retrieve the actual voting option v_i from pTable
5: $k \leftarrow k + 1$
6: **end if**
7: **end for**
8: $\mathbf{v}_{\mathrm{b}} \leftarrow (\mathrm{v}_{\mathrm{b},0}, \ldots, \mathrm{v}_{\mathrm{b},\psi-1})$

Output:

Blank actual voting options $(\mathrm{v}_{\mathrm{b},0}, \ldots, \mathrm{v}_{\mathrm{b},\psi-1}) \in (\mathcal{D}_{\mathrm{v}})^\psi$

Copyright 2026 Swiss Post Ltd.

Second, the algorithm GetWriteInEncodedVotingOptions returns the vector of encoded voting options corresponding to the write-in voting options. The resulting vector is of size $\delta - 1$.

Algorithm 3.8 GetWriteInEncodedVotingOptions

Context:

Primes mapping table $\mathsf{pTable} \in (\mathcal{D}_{\mathrm{v}} \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times {\mathbb{A}_{UCS}}^* \times \mathcal{D}_\tau)^n$ ▷ pTable is of the form $((\mathrm{v}_0, \tilde{\mathrm{p}}_0, \sigma_0, \tau_0), \ldots, (\mathrm{v}_{n-1}, \tilde{\mathrm{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1}))$

Operation:

1: $k \leftarrow 0$
2: **for** $i \in [0, n)$ **do**
3: **if** σ_i starts with "`WRITE_IN`" **then**
4: $\tilde{\mathrm{p}}_{\mathrm{w},k} \leftarrow \mathsf{pTable}(\sigma_i)$ ▷ Retrieve the encoded voting option $\tilde{\mathrm{p}}_i$ from pTable
5: $k \leftarrow k + 1$
6: **end if**
7: **end for**
8: **return** $(\tilde{\mathrm{p}}_{\mathrm{w},0}, \ldots, \tilde{\mathrm{p}}_{\mathrm{w},\delta-2})$ ▷ The resulting vector has the size $\delta - 1 \geq 0$

Output:

Encoded write-in voting options $(\tilde{\mathrm{p}}_{\mathrm{w},0}, \ldots, \tilde{\mathrm{p}}_{\mathrm{w},\delta-2}) \in ((\mathbb{G}_q \cap \mathbb{P}) \setminus g)^{\delta-1}$

Copyright 2026 Swiss Post Ltd.

Third, we define the algorithm GetAbstentionActualVotingOptions that returns the actual voting options corresponding to the abstention voting options of a specific presentation group. And the algorithm GetPresentationGroup that returns the presentation group corresponding to the given actual voting option if it is an abstention voting option, -1 otherwise.

Algorithm 3.9 GetAbstentionActualVotingOptions

Context:

Primes mapping table $\mathsf{pTable} \in (\mathcal{D}_\mathrm{v} \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times {\mathbb{A}_{UCS}}^* \times \mathcal{D}_\tau)^n$ ▷ pTable is of the form $((\mathrm{v}_0, \tilde{\mathrm{p}}_0, \sigma_0, \tau_0), \ldots, (\mathrm{v}_{n-1}, \tilde{\mathrm{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1}))$

Input:

Presentation group $\mathsf{pg} \in [0, \eta)$

Operation:

1: $k \leftarrow 0$
2: **for** $i \in [0, n)$ **do**
3: **if** GetPresentationGroup$(\mathrm{v}_i) = \mathsf{pg}$ **then** ▷ See algorithm 3.10.
4: $\mathrm{v}_{\mathrm{a},k} \leftarrow \mathrm{v}_i$
5: $k \leftarrow k + 1$
6: **end if**
7: **end for**

Output:

Abstention actual voting options $(\mathrm{v}_{\mathrm{a},0}, \ldots, \mathrm{v}_{\mathrm{a},k-1}) \in (\mathcal{D}_\mathrm{v})^k$

Algorithm 3.10 GetPresentationGroup

Context:

Primes mapping table $\mathsf{pTable} \in (\mathcal{D}_\mathrm{v} \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times {\mathbb{A}_{UCS}}^* \times \mathcal{D}_\tau)^n$ ▷ pTable is of the form $((\mathrm{v}_0, \tilde{\mathrm{p}}_0, \sigma_0, \tau_0), \ldots, (\mathrm{v}_{n-1}, \tilde{\mathrm{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1}))$

Input:

Actual voting option $\mathrm{v}'_i \in \mathcal{D}_\mathrm{v}$

Operation:

1: $\sigma'_i \leftarrow \mathsf{pTable}(\mathrm{v}'_i)$ ▷ Retrieve the semantic information σ'_i from pTable
2: **if** σ'_i does not start with "`ABSTENTION`" **then**
return -1
3: **else**
4: $\mathsf{pg} \leftarrow \sigma'_i$[presentation group position number] ▷ See section 3.4.6
5: **end if**

Output:

Presentation group $\mathsf{pg} \in [0, \eta)$ if the voting option is an abstention, -1 otherwise.

Copyright 2026 Swiss Post Ltd.

Moreover, we define two auxiliary algorithms GetPsi and GetDelta calculating the number of selectable voting options ψ and the number of write-in options + 1: δ.

Algorithm 3.11 GetPsi

Context:

Primes mapping table $\mathsf{pTable} \in (\mathcal{D}_{\mathrm{v}} \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times {\mathbb{A}_{UCS}}^* \times \mathcal{D}_\tau)^n$ ▷ pTable is of the form $((\mathrm{v}_0, \tilde{\mathrm{p}}_0, \sigma_0, \tau_0), \ldots, (\mathrm{v}_{n-1}, \tilde{\mathrm{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1}))$

Operation:

1: $(\hat{\tau}_0, \ldots, \hat{\tau}_{\psi-1}) \leftarrow$ GetBlankCorrectnessInformation() ▷ See algorithm 3.6
2: **return** ψ

Output:

The number of selectable voting options: $\psi \in [1, \psi_{\mathtt{sup}}]$

Algorithm 3.12 GetDelta

Context:

Primes mapping table $\mathsf{pTable} \in (\mathcal{D}_{\mathrm{v}} \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times {\mathbb{A}_{UCS}}^* \times \mathcal{D}_\tau)^n$ ▷ pTable is of the form $((\mathrm{v}_0, \tilde{\mathrm{p}}_0, \sigma_0, \tau_0), \ldots, (\mathrm{v}_{n-1}, \tilde{\mathrm{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1}))$

Operation:

1: $(\tilde{\mathrm{p}}_{\mathrm{w},0}, \ldots, \tilde{\mathrm{p}}_{\mathrm{w},\delta-2}) \leftarrow$ GetWriteInEncodedVotingOptions() ▷ See algorithm 3.8
2: **return** δ

Output:

The number of allowed write-ins + 1: $\delta \in [1, \delta_{\mathtt{sup}}]$

Copyright 2026 Swiss Post Ltd.

3.4.8 Valid Combinations of Voting Options

The Federal Chancellery's Ordinance [7] requires that an e-voting system must not accept invalid votes.

> [VEleS Annex 10]: Conformity check and storing finalised votes:
> - Votes not cast in conformity with the system are not stored in the electronic ballot box.

The Ordinance's annex further elaborates on the definition of *votes cast in conformity with the system*.

> [VEleS Annex 2.1]: *vote cast in conformity with the system* is a vote:
> - that complies with a predetermined way of completing a ballot paper in a vote or election
> - [...].

In the electoral model outlined in Section 3.4.1, the essence is that a vote must encompass ψ selections and adhere to a valid combination of voting options. Specifically, this means each vote must include the appropriate number of selections for each election or question. For instance, if the ballot has two questions—each with the corresponding voting options YES / NO / EMPTY—we expect the voter to select exactly *one* voting option for the first question and *one* for the second question. Conversely, if a voter were to select an invalid combination, for instance, YES and NO to the same question, her vote would be considered invalid.
To prevent invalid combinations of voting options, we utilize the principle detailed in section 3.4.1, which specifies an equivalence between the number of blank voting options and selections. Moreover, we can leverage the fact that each component of the Swiss Post Voting System knows the primes mapping table pTable. The primes mapping table pTable associates each voting option with a correctness information τ_i; resulting in a vector $\boldsymbol{\tau} = (\tau_0, \ldots, \tau_{n-1})$. The configuration of the election event also defines how many voting options from which vote or election can be selected and in which order; resulting in a vector of correct selections $\hat{\boldsymbol{\tau}} = (\hat{\tau}_0, \ldots, \hat{\tau}_{\psi-1})$. We can derive the vector $\hat{\boldsymbol{\tau}}$ by invoking the algorithm GetBlankCorrectnessInformation on the primes mapping table pTable.
We illustrate the aforementioned concept with a simple example. Imagine the following election event configuration:

- One question with three voting options (YES / NO / EMPTY) - question identifier = "$\texttt{question}_1$",
- One question with three voting options (YES / NO / EMPTY) - question identifier = "$\texttt{question}_2$",
- An election (without lists) for three seats and five candidates - election identifier = "$\texttt{election}_1$".

Such an election event configuration indicates that the total number of selectable voting options ψ is 5 (one for the first question, one for the second, and three for the election), and if no accumulation of candidates is allowed, the total number of voting options n is 14 (three for

Copyright 2026 Swiss Post Ltd.

the first question, three for the second, and eight for the election, including three blank seat selections). Invoking the algorithm GetBlankCorrectnessInformation for this election event configuration returns the following vector
(“$\texttt{question}_1$”, “$\texttt{question}_2$”, “$\texttt{C|election}_1$”, “$\texttt{C|election}_1$”, “$\texttt{C|election}_1$”).
The Swiss Post Voting System checks that a vote contains a valid combination of voting options at several stages within the protocol:

- Within the voting client, during the creation of the vote, as outlined in algorithm 5.4 CreateVote.
- Within the control components during the execution of the SendVote protocol, specifically through algorithm 5.8 CreateLCCShare. Here, the control components incorporate the correctness information $\hat{\boldsymbol{\tau}}$ into the evaluation of the partial Choice Return Codes allow list $\mathsf{L}_{\mathsf{pCC}}$, initially generated by the setup component through algorithm 4.2 GenVerDat.
- Within both the verifier and the tally control component, following the decryption of votes, as detailed in algorithm 6.12 ProcessPlaintexts.

While our electoral model treats each election as independent, ensuring valid combinations of voting options presents challenges, particularly in proportional elections where voters select both a party list and candidates. In the case of the national council election, a vote for a party list without at least one non-blank candidate selection is deemed invalid. However, the use of correctness information $\hat{\boldsymbol{\tau}}$ does not address this issue, as invoking GetBlankCorrectnessInformation for a party list without candidate selections would still yield the expected correctness vector, failing to identify the vote as invalid.
To mitigate this, the Swiss Post Voting System currently employs user interface controls within the voting client to prevent the submission of such invalid combinations. Additionally, during the generation of the eCH-0222 XML, as discussed in Section 6.3.5, any invalid vote is filtered out to ensure it is not counted.
Future versions of the Swiss Post Voting System will improve the detection and prevention of this type of invalid combination as part of the ongoing improvements outlined in Measure A.26 of the Federal Chancellery’s catalogue of measures [6].

Copyright 2026 Swiss Post Ltd.

3.4.9 Multiplying and Factorizing Voting Options

The voter $\texttt{id}$ selects ψ unique encoded voting options $\hat{\mathbf{p}}_{\texttt{id}} = (\hat{\mathrm{p}}_{\texttt{id},0}, \ldots, \hat{\mathrm{p}}_{\texttt{id},\psi-1})$. The ψ-element vector of selected, encoded voting options $\hat{\mathbf{p}}_{\texttt{id}}$ is a subset of the n-element vector of encoded voting options $\tilde{\mathbf{p}}$ ($\hat{\mathbf{p}}_{\texttt{id}} \subset \tilde{\mathbf{p}}$). We verify in algorithm 3.1 that the product of any subset of $\psi_{\texttt{sup}}$ primes (the system's maximum supported number of selections - see section 3.4.5) is smaller than the group modulus p to ensure that all voting combinations are uniquely represented. To avoid encrypting multiple messages, we multiply the encoded voting options prior to encryption (see the algorithm 5.4 CreateVote):

$$\rho = \prod_{i=0}^{\psi-1} \hat{\mathrm{p}}_{\texttt{id},i} \in \mathbb{G}_q$$

Conversely, we factorize the encoded voting options after decryption (see the algorithm 6.12 ProcessPlaintexts):

$$\hat{\mathbf{p}}_{\texttt{id}} = \mathsf{Factorize}(\rho)$$

Factorizing the message is efficient since the primes are small and the set of primes is known. Since every voting option is selected at most once and each voter must select ψ voting options, we expect *exactly* ψ distinct prime factors.

Algorithm 3.13 Factorize

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Encoded voting options $\tilde{\mathbf{p}} = (\tilde{\mathrm{p}}_0, \ldots, \tilde{\mathrm{p}}_{n-1}) \in ((\mathbb{G}_q \cap \mathbb{P}) \setminus g)^n$ ▷ Can be derived from pTable using algorithm 3.2
Number of allowed selections for this specific ballot box $\psi \in [1, \psi_{\texttt{sup}}]$ ▷ Can be derived from pTable using algorithm 3.11

Input:

$x \in \mathbb{G}_q$ ▷ The number to be factorized

Operation:

1: $i \leftarrow 0$
2: **for** $k \in [0, n)$ **do**
3: **if** $\tilde{\mathrm{p}}_k \mid x$ **then**
4: $\hat{\mathrm{p}}_i \leftarrow \tilde{\mathrm{p}}_k$
5: $i \leftarrow i + 1$
6: **end if**
7: **end for**
8: **if** $i \neq \psi \vee \prod_{j=0}^{\psi-1} \hat{\mathrm{p}}_j \neq x$ **then**
return $\perp$ ▷ The factorization is only successful if the number of prime factors is ψ and their product equals x.
9: **end if**

Output:

$\hat{\mathbf{p}} = (\hat{\mathrm{p}}_0, \ldots, \hat{\mathrm{p}}_{\psi-1}) \in ((\mathbb{G}_q \cap \mathbb{P}) \setminus g)^{\psi}$ ▷ The list of prime factors

---

Copyright 2026 Swiss Post Ltd.

3.5 Execution Context of Algorithms

Each algorithm executes within a given context; sometimes, the protocol runs an algorithm per election event, per verification card set, per ballot box, or actual vote. Table 11 summarizes the algorithms' execution context.

Algorithm	Execution context	Reference
GenSetupData	per election event	4.1
GenVerDat	per verification card set	4.2
GetVoterAuthenticationData	per verification card set	4.3
GenKeysCCR	per election event	4.4
GenEncLongCodeShares	per verification card set	4.5
CombineEncLongCodeShares	per verification card set	4.6
GenCMTable	per verification card set	4.7
VerifyCCRExponentiationProofs	per verification card set	4.8
GenVerCardSetKeys	per election event	4.11
GenCredDat	per verification card set	4.12
SetupTallyCCM	per election event	4.13
SetupTallyEB	per election event	4.14
Voting phase algorithms	per sent/confirmed vote	section 5
MixOnline algorithms	per ballot box	section 6.1
Dispute resolution algorithms	per election event	section 6.2
VerifyVotingClientProofs	per ballot box	6.9
VerifyMixDecOffline	per ballot box	6.10
MixDecOffline	per ballot box	6.11
ProcessPlaintexts	per ballot box	6.12
CreateECH0222	per election event	section 6.3.5
RequestProofNonParticipation	per verification card	6.14

Tab. 11: Overview of the algorithms' context. The protocol executes some configuration phase algorithms globally per election event and some per verification card set. The voting phase algorithms run for every sent or confirmed vote. Most tally phase algorithms are executed per ballot box. In contrast, the dispute resolution process and the generation of the tally file are performed globally for the entire election event. The proof of non-participation executes individually for a specific verification card.

Copyright 2026 Swiss Post Ltd.

3.6 Agreement Algorithms

3.6.1 Agreement on the Global Election Event Context

At specific stages of the cryptographic protocol, the Swiss Post Voting System requires the protocol parties to agree on the election event context and the applicable electoral model (see Section 3.4).
Therefore, we define an algorithm GetHashElectionEventContext that hashes the global election event context. This algorithm is used in the configuration phase by the Setup Component in algorithm SetupTallyEB and by the Control Components in algorithm SetupTallyCCM, ensuring consensus of the Election Event Context between these components.
The start and finish times of the election event and of the ballot boxes are restricted to be in local date time format (YYYY-MM-DDTHH:MM:SS) without time zone as defined by the ISO-8601 calendar system. The time zone used will always be the local time zone in Switzerland, therefore no indication of timezone is necessary.

Copyright 2026 Swiss Post Ltd.

Algorithm 3.14 GetHashElectionEventContext

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Election Event alias $\mathtt{eeAlias} \in \mathcal{T}_1^{50}$
Election Event description $\mathtt{eeDesc} \in \mathbb{A}_{UCS}{}^*$
Verification Card Set Contexts ▷ See below for the contents
Election Event start time $t_{\text{s,ee}} \in \mathbb{A}_{UCS}{}^{19}$ ▷ Start and finish time in the format stated above
Election Event finish time $t_{\text{f,ee}} \in \mathbb{A}_{UCS}{}^{19}$
Maximum number of voting options $n_{\mathtt{max}} \in [1, n_{\mathtt{sup}}]$
Maximum number of selections $\psi_{\mathtt{max}} \in [1, \psi_{\mathtt{sup}}]$
Maximum number of write-ins + 1: $\delta_{\mathtt{max}} \in [1, \delta_{\mathtt{sup}}]$

Verification Card Set Contexts: For each of the $\mathtt{N_{bb}}$ verification card sets $\mathtt{vcs} \in \mathbf{vcs}$ ▷ Given in lexicographic order

Verification card set ID $\mathtt{vcs} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Verification card set alias $\mathtt{vcsAlias} \in \mathcal{T}_1^{50}$
Verification card set description $\mathtt{vcsDesc} \in \mathbb{A}_{UCS}{}^*$
Ballot box ID $\mathtt{bb} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Ballot box start time $t_{\text{s,bb}} \in \mathbb{A}_{UCS}{}^{19}$ ▷ Start and finish time in the format stated above
Ballot box finish time $t_{\text{f,bb}} \in \mathbb{A}_{UCS}{}^{19}$
Test ballot box boolean $\mathtt{testBallotBox} \in$ {"true", "false"}
Number of eligible voters for this verification card set $\mathtt{N_E} \in \mathbb{N}^+$
Grace period $\mathtt{gracePeriod} \in [0, 3600]$ ▷ The grace period is given in seconds
Primes mapping table $\mathsf{pTable} \in (\mathcal{D}_{\text{v}} \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times \mathbb{A}_{UCS}{}^* \times \mathcal{D}_\tau)^n$ ▷ pTable is of the form $((\text{v}_0, \tilde{\text{p}}_0, \sigma_0, \tau_0), \ldots, (\text{v}_{n-1}, \tilde{\text{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1}))$
Electoral Model Context $\Lambda = (\boldsymbol{\psi}, \mathbf{DoI}, \mathbf{pg}, \mathbf{ag}) \in \left((\mathbb{N}^+)^t \times (\mathcal{T}_1^{50})^k \times [0, \eta)^t \times \mathbb{Z}_2^\eta\right)$ ▷ See Section 3.4.4

Operation: ▷ We use nested structures in this algorithm

1: **for** $\mathtt{vcs} \in \mathbf{vcs}$ **do**
2: $h_{\mathtt{pTable,j}} \leftarrow \left(\left((\text{v}_0, \tilde{\text{p}}_0, \sigma_0, \tau_0), \ldots, (\text{v}_{n-1}, \tilde{\text{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1})\right)\right)$
3: $h_{\Lambda,\mathtt{j}} \leftarrow \left((\psi_0, \ldots, \psi_{t-1}), (\mathtt{DoI}_0, \ldots, \mathtt{DoI}_{k-1}), (\mathtt{pg}_0, \ldots, \mathtt{pg}_{t-1}), (\mathtt{ag}_0, \ldots, \mathtt{ag}_{\eta-1})\right)$
4: $h_{\mathtt{vcs,j}} \leftarrow (\mathtt{vcs}, \mathtt{vcsAlias}, \mathtt{vcsDesc}, \mathtt{bb}, t_{\text{s,bb}}, t_{\text{f,bb}}, \mathtt{testBallotBox}, \mathtt{N_E}, \mathtt{gracePeriod}, h_{\mathtt{pTable,j}}, h_{\Lambda,\mathtt{j}})$
5: **end for**
6: $h_{\mathtt{vcs}} \leftarrow (h_{\mathtt{vcs},0}, h_{\mathtt{vcs},1}, \ldots, h_{\mathtt{vcs},\mathtt{N_{bb}}-1})$
7: $h \leftarrow ((p, q, g), \mathtt{ee}, \mathtt{eeAlias}, \mathtt{eeDesc}, h_{\mathtt{vcs}}, t_{\text{s,ee}}, t_{\text{f,ee}}, n_{\mathtt{max}}, \psi_{\mathtt{max}}, \delta_{\mathtt{max}})$
8: $d \leftarrow \mathsf{Base64Encode}(\mathsf{RecursiveHash}(h))$ ▷ See crypto primitives specification

Output:

The digest $d \in \mathbb{A}_{Base64}{}^{\mathtt{l_{HB64}}}$ ▷ $\mathtt{l_{HB64}}$ is the character length of the Base64 encoded hash output

Test values for the algorithm 3.14 are provided in get-hash-election-event-context.json.

Copyright 2026 Swiss Post Ltd.

3.6.2 Agreement on the Verification Card Set-Specific Data

At specific stages of the cryptographic protocol, the Swiss Post Voting System requires the protocol parties to agree on the verification card set-specific context, which includes elements of the election event context as well as certain cryptographic data and public keys.
Therefore, we define an algorithm GetHashContext that hashes the cryptographic parameters (Section 3.2), the verification card set specific context (Section 3.4.3), the primes mapping table pTable as well as the election public key $\mathtt{EL_{pk}}$ and the Choice Return Codes encryption public key $\mathbf{pk}_{\mathrm{CCR}}$.
This algorithm is used in the configuration phase by the Setup Component and in the voting phase by the Voting Client and the Control Components.

Algorithm 3.15 GetHashContext

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Verification card set ID $\mathtt{vcs} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Primes mapping table $\mathsf{pTable} \in (\mathcal{D}_{\mathrm{v}} \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times {\mathbb{A}_{UCS}}^* \times \mathcal{D}_\tau)^n$ ▷ pTable is of the form $((\mathrm{v}_0, \tilde{\mathrm{p}}_0, \sigma_0, \tau_0), \ldots, (\mathrm{v}_{n-1}, \tilde{\mathrm{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1}))$
Electoral Model Context $\Lambda = (\boldsymbol{\psi}, \mathbf{DoI}, \mathbf{pg}, \mathbf{ag}) \in \left((\mathbb{N}^+)^t \times (\mathcal{T}_1^{50})^k \times [0, \eta)^t \times \mathbb{Z}_2^\eta\right)$
Election public key $\mathtt{EL_{pk}} = (\mathtt{EL_{pk},0}, \ldots, \mathtt{EL_{pk},}\delta_{\max}{}_{-1}) \in \mathbb{G}_q^{\delta_{\max}}$
Choice Return Codes encryption public key $\mathbf{pk}_{\mathrm{CCR}} = (\mathtt{pk_{CCR},0}, \ldots, \mathtt{pk_{CCR},}\psi_{\max}{}_{-1}) \in \mathbb{G}_q^{\psi_{\max}}$

Operation: ▷ We flatten the lists in this algorithm

1: $h \leftarrow ()$
2: $h \leftarrow (h, \text{“}\mathtt{EncryptionParameters}\text{”}, p, q, g)$
3: $h \leftarrow (h, \text{“}\mathtt{ElectionEventContext}\text{”}, \mathtt{ee}, \mathtt{vcs})$
4: $h \leftarrow (h, \text{“}\mathtt{ActualVotingOptions}\text{”}, \mathsf{GetActualVotingOptions}(()))$ ▷ See 3.3
5: $h \leftarrow (h, \text{“}\mathtt{EncodedVotingOptions}\text{”}, \mathsf{GetEncodedVotingOptions}(()))$ ▷ See 3.2
6: $h \leftarrow (h, \text{“}\mathtt{SemanticInformation}\text{”}, \mathsf{GetSemanticInformation}())$ ▷ See 3.4
7: $h \leftarrow (h, \text{“}\mathtt{CorrectnessInformation}\text{”}, \mathsf{GetCorrectnessInformation}(()))$ ▷ See 3.5
8: $h \leftarrow (h, \text{“}\mathtt{NumberOfSelections}\text{”}, \psi_0, \ldots, \psi_{t-1})$
9: $h \leftarrow (h, \text{“}\mathtt{DomainsOfInfluence}\text{”}, \mathtt{DoI}_0, \ldots, \mathtt{DoI}_{k-1})$
10: $h \leftarrow (h, \text{“}\mathtt{PresentationGroups}\text{”}, \mathtt{pg}_0, \ldots, \mathtt{pg}_{t-1})$
11: $h \leftarrow (h, \text{“}\mathtt{AbstentionGroups}\text{”}, \mathtt{ag}_0, \ldots, \mathtt{ag}_{\eta-1})$
12: $h \leftarrow (h, \text{“}\mathtt{ELpk}\text{”}, \mathtt{EL_{pk},0}, \ldots, \mathtt{EL_{pk},}\delta_{\max}{}_{-1})$
13: $h \leftarrow (h, \text{“}\mathtt{pkCCR}\text{”}, \mathtt{pk_{CCR},0}, \ldots, \mathtt{pk_{CCR},}\psi_{\max}{}_{-1})$
14: $d \leftarrow \mathsf{Base64Encode}(\mathsf{RecursiveHash}(h))$ ▷ See crypto primitives specification

Output:

The digest $d \in {\mathbb{A}_{Base64}}^{\mathtt{l_{HB64}}}$ ▷ $\mathtt{l_{HB64}}$ is the character length of the Base64 encoded hash output

Test values for the algorithm 3.15 are provided in get-hash-context.json.

Copyright 2026 Swiss Post Ltd.

3.6.3 Agreement on the Entire Election Event-Specific Data

At specific stages of the cryptographic protocol, the Swiss Post Voting System requires the protocol parties to agree on the entire election event-specific data, including both the election event context (Section 3.6.1) and the verification card set-specific data (Section 3.6.2).
Therefore, we define an algorithm GetHashExtractedElectionEvent that hashes the entire election event-specific data. GetHashExtractedElectionEvent is used by the Control Components in the voting phase in algorithms PartialDecryptPCC and DecryptPCC.
To facilitate the algorithm GetHashExtractedElectionEvent, we define the algorithm ExtractElectionEvent, which extracts all relevant information required to represent the entirety of the election event, with helper algorithm ExtractVerificationCardSet.

Copyright 2026 Swiss Post Ltd.

We assume the existence of functions ExtractElectionEventContext, ExtractElectionPublicKey, and ExtractChoiceReturnCodesEncryptionPublicKey that retrieve the necessary values from internal view.

Algorithm 3.16 ExtractElectionEvent

Context:

Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$

Operation: ▷ All extraction algorithms retrieve the necessary values from internal view

1: electionEventContext ← ExtractElectionEventContext(ee)
2: hContext ← GetHashElectionEventContext() ▷ See algorithm 3.14
3: $\mathtt{EL_{pk}}$ ← ExtractElectionPublicKey(ee)
4: $\mathbf{pk}_{\mathrm{CCR}}$ ← ExtractChoiceReturnCodesEncryptionPublicKey(ee)
5: **for** $i \in [0, \mathtt{N_{bb}})$ **do**
6: $\quad \mathtt{evcs}_i$ ← ExtractVerificationCardSet() ▷ See algorithm 3.17
7: **end for**
8: **evcs** ← $(\mathtt{evcs}_0, \ldots, \mathtt{evcs}_{\mathtt{N_{bb}}-1})$
9: **evcs** ← Order(**evcs**) ▷ Lexicographic ordering by the verification card set id.

Output:

Extracted election event $\mathtt{eee} = (\mathtt{hContext}, (p, q, g), \mathtt{ee}, \mathbf{evcs})$
▷ $\mathtt{evcs} = (\mathtt{h}, \mathtt{vcs}, \mathsf{L}_{\mathtt{pCC}}, \mathsf{L}_{\mathtt{lVCC}})$

We assume the existence of functions ExtractPartialChoiceReturnCodesAllowList and ExtractLongVoteCastReturnCodesAllowList that retrieve the necessary values from internal view.

Algorithm 3.17 ExtractVerificationCardSet

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Verification card set ID $\mathtt{vcs} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Primes mapping table pTable $\in (\mathcal{D}_{\mathrm{v}} \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times {\mathbb{A}_{UCS}}^* \times \mathcal{D}_\tau)^n$ ▷ pTable is of the form $((\mathrm{v}_0, \tilde{\mathrm{p}}_0, \sigma_0, \tau_0), \ldots, (\mathrm{v}_{n-1}, \tilde{\mathrm{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1}))$
Electoral Model Context $\Lambda = (\boldsymbol{\psi}, \mathbf{DoI}, \mathbf{pg}, \mathbf{ag}) \in \left((\mathbb{N}^+)^t \times (\mathcal{T}_1^{50})^k \times [0, \eta)^t \times \mathbb{Z}_2^\eta\right)$ ▷ See Section 3.4.4
Election public key $\mathtt{EL_{pk}} = (\mathtt{EL_{pk},0}, \ldots, \mathtt{EL_{pk},}\delta_{\mathtt{max}}-1) \in \mathbb{G}_q^{\delta_{\mathtt{max}}}$
Choice Return Codes encryption public key $\mathbf{pk}_{\mathrm{CCR}} = (\mathtt{pk}_{\mathrm{CCR},0}, \ldots, \mathtt{pk}_{\mathrm{CCR},}\psi_{\mathtt{max}}-1) \in \mathbb{G}_q^{\psi_{\mathtt{max}}}$

Operation: ▷ All extraction algorithms retrieve the necessary values from internal view

1: h ← GetHashContext() ▷ See algorithm 3.15
2: $\mathsf{L}_{\mathtt{pCC}}$ ← ExtractPartialChoiceReturnCodesAllowList(vcs)
3: $\mathsf{L}_{\mathtt{lVCC}}$ ← ExtractLongVoteCastReturnCodesAllowList(vcs)

Output:

Extracted verification card set $\mathtt{evcs} = (\mathtt{h}, \mathtt{vcs}, \mathsf{L}_{\mathtt{pCC}}, \mathsf{L}_{\mathtt{lVCC}})$

Copyright 2026 Swiss Post Ltd.

Finally, we define the algorithm GetHashExtractedElectionEvent that hashes the extracted election event $\texttt{eee} = (\texttt{hContext}, (p, q, g), \texttt{ee}, \textbf{evcs})$.

Algorithm 3.18 GetHashExtractedElectionEvent

Context:

Extracted election event $\texttt{eee} = (\texttt{hContext}, (p, q, g), \texttt{ee}, \textbf{evcs})$

▷ $\texttt{evcs} = (\texttt{h}, \texttt{vcs}, \mathsf{L}_{\texttt{pCC}}, \mathsf{L}_{\texttt{lVCC}})$

Operation: ▷ We use nested structures in this algorithm

1: **for** $i \in [0, \mathtt{N_{bb}})$ **do**
2: $\quad h_{\texttt{evcs},i} \leftarrow (\texttt{h}, \texttt{vcs}, \mathsf{L}_{\texttt{pCC}}, \mathsf{L}_{\texttt{lVCC}})$
3: **end for**
4: $h_{\texttt{evcs}} \leftarrow (h_{\texttt{evcs},0}, \ldots, h_{\texttt{evcs},\mathtt{N_{bb}}-1})$
5: $h \leftarrow (\texttt{hContext}, (p, q, g), \texttt{ee}, h_{\texttt{evcs}})$
6: $d \leftarrow \mathsf{Base64Encode}(\mathsf{RecursiveHash}(h))$ ▷ See crypto primitives specification

Output:

The digest $d \in \mathbb{A}_{Base64}{}^{\mathtt{l_{HB64}}}$ ▷ $\mathtt{l_{HB64}}$ is the character length of the Base64 encoded hash output

Copyright 2026 Swiss Post Ltd.

3.6.4 Agreement on the Sent Votes from the Voting Phase

We describe the control components' ExtractVerificationCards algorithm. This algorithm is only invoked in the dispute resolution process (see section 6.2). As a *precondition*, the ExtractVerificationCards algorithm can only be invoked after the election event period ended. We assume the existence of functions ExtractVerificationCardSetId, ExtractEncryptedVote, and ExtractHashedLVCCShares that retrieve the necessary values from internal view.

Algorithm 3.19 ExtractVerificationCards

Context:

Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$

Stateful Lists and Maps:

$\mathtt{CCR_j}$'s list of sent voting cards $\mathsf{L}_{\mathtt{sentVotes},j}$

$\mathtt{CCR_j}$'s list of confirmed voting cards $\mathsf{L}_{\mathtt{confirmedVotes},j}$

Operation: ▷ All extraction algorithms retrieve the necessary values from internal view
▷ We use nested structures in this algorithm

1: $i \leftarrow 0$
2: **for** $\mathtt{vc_{id}} \in \mathsf{L}_{\mathtt{sentVotes},j}$ **do**
3: $\quad \mathtt{vcs} \leftarrow \mathsf{ExtractVerificationCardSetId}(\mathtt{vc_{id}})$
4: $\quad \mathtt{E1} \leftarrow \mathsf{ExtractEncryptedVote}(\mathtt{vc_{id}})$
5: $\quad$ **if** $\mathtt{vc_{id}} \in \mathsf{L}_{\mathtt{confirmedVotes},j}$ **then**
6: $\quad\quad (\mathtt{hlVCC}_{1,\mathtt{id}}, \mathtt{hlVCC}_{2,\mathtt{id}}, \mathtt{hlVCC}_{3,\mathtt{id}}, \mathtt{hlVCC}_{4,\mathtt{id}}) \leftarrow \mathsf{ExtractHashedLVCCShares}(\mathtt{vc_{id}})$
7: $\quad\quad \mathtt{evc}_i \leftarrow (\mathtt{vc_{id}}, \mathtt{vcs}, \mathtt{E1}, (\mathtt{hlVCC}_{1,\mathtt{id}}, \mathtt{hlVCC}_{2,\mathtt{id}}, \mathtt{hlVCC}_{3,\mathtt{id}}, \mathtt{hlVCC}_{4,\mathtt{id}}))$
8: $\quad$ **else**
9: $\quad\quad \mathtt{evc}_i \leftarrow (\mathtt{vc_{id}}, \mathtt{vcs}, \mathtt{E1}, ())$
10: $\quad$ **end if**
11: $\quad i \leftarrow i + 1$
12: **end for**
13: $\mathbf{evc} \leftarrow (\mathtt{evc}_0, \ldots, \mathtt{evc}_{\mathtt{N_s}-1})$
14: $\mathbf{evc} \leftarrow \mathsf{Order}(\mathbf{evc})$ ▷ Lexicographic ordering by the verification card id.

Output:

Extracted verification cards $\mathbf{evc} = (\mathtt{evc}_0, \ldots, \mathtt{evc}_{\mathtt{N_s}-1})$

Copyright 2026 Swiss Post Ltd.

3.6.5 Proof of Correct Key Generation

In the algorithms GenKeysCCR, SetupTallyCCM, and SetupTallyEB the control components and the setup component generate shares of ElGamal encryption key pairs, subsequently distributing the public keys among the other protocol participants. Bernhard et al. showed that this approach might be vulnerable to an attack [2]: a malicious party k could delay its key pair generation until after receiving the public keys from honest participants. This party could then generate a key pair (pk_k, sk_k) and send a rogue public key $pk'_k = \frac{pk_k}{\prod_{i \neq k} pk_i}$ to the other participants. The resulting combined public key $pk = \prod_{i \neq k} pk_i \cdot pk'_k$ would then correspond to the attacker's public key pk_k.
To prevent this attack, each participant creates a Schnorr proof of knowledge of their secret key matching the distributed public key.
Since multiple protocol participants verify these proofs throughout the protocol, the following section specifies the algorithms for verifying the Schnorr proofs of knowledge for correct key generation.
If the public keys are available from multiple sources—for instance from a message and from an internal view—one must check their consistency when invoking the algorithm VerifyKeyGenerationSchnorrProofs.

Copyright 2026 Swiss Post Ltd.

Algorithm 3.20 VerifyKeyGenerationSchnorrProofs

Context:

The Election Event Context ▷ See Section 3.4 ▷ Including $p, q, g, \mathtt{ee}, \psi_{\mathtt{max}}, \delta_{\mathtt{max}}$

Input:

CCR Choice Return Codes encryption public keys $\mathbf{pk}_{\mathrm{CCR}} = (\mathbf{pk}_{\mathrm{CCR}_1}, \mathbf{pk}_{\mathrm{CCR}_2}, \mathbf{pk}_{\mathrm{CCR}_3}, \mathbf{pk}_{\mathrm{CCR}_4}) \in (\mathbb{G}_q^{\psi_{\mathtt{max}}})^4$

CCR Schnorr proofs of knowledge $\boldsymbol{\pi}_{\mathsf{pkCCR}} = (\boldsymbol{\pi}_{\mathsf{pkCCR},1}, \boldsymbol{\pi}_{\mathsf{pkCCR},2}, \boldsymbol{\pi}_{\mathsf{pkCCR},3}, \boldsymbol{\pi}_{\mathsf{pkCCR},4}) \in \left((\mathbb{Z}_q \times \mathbb{Z}_q)^{\psi_{\mathtt{max}}}\right)^4$

CCM election public keys $\mathbf{EL}_{\mathtt{pk}} = (\mathtt{EL}_{\mathtt{pk},1}, \mathtt{EL}_{\mathtt{pk},2}, \mathtt{EL}_{\mathtt{pk},3}, \mathtt{EL}_{\mathtt{pk},4}) \in \left(\mathbb{G}_q^{\delta_{\mathtt{max}}}\right)^4$

CCM Schnorr proofs of knowledge $\boldsymbol{\pi}_{\mathsf{ELpk}} = (\boldsymbol{\pi}_{\mathsf{ELpk},1}, \boldsymbol{\pi}_{\mathsf{ELpk},2}, \boldsymbol{\pi}_{\mathsf{ELpk},3}, \boldsymbol{\pi}_{\mathsf{ELpk},4}) \in \left((\mathbb{Z}_q \times \mathbb{Z}_q)^{\delta_{\mathtt{max}}}\right)^4$

Electoral board public key $\mathtt{EB}_{\mathtt{pk}} \in \mathbb{G}_q^{\delta_{\mathtt{max}}}$

Electoral board Schnorr proofs of knowledge $\boldsymbol{\pi}_{\mathsf{EB}} \in (\mathbb{Z}_q \times \mathbb{Z}_q)^{\delta_{\mathtt{max}}}$

Operation:

1: $\mathtt{hContext} \leftarrow$ GetHashElectionEventContext() ▷ See algorithm 3.14
2: $\mathsf{VerifSchnorrCCR} \leftarrow$ VerifyCCSchnorrProofs$(\mathbf{pk}_{\mathrm{CCR}}, \boldsymbol{\pi}_{\mathsf{pkCCR}}, (\mathtt{ee}, \text{“}\mathtt{GenKeysCCR}\text{”}))$ ▷ See algorithm 3.21
3: $\mathsf{VerifSchnorrCCM} \leftarrow$ VerifyCCSchnorrProofs$(\mathbf{EL}_{\mathtt{pk}}, \boldsymbol{\pi}_{\mathsf{ELpk}}, (\mathtt{hContext}, \text{“}\mathtt{SetupTallyCCM}\text{”}))$
4: $\mathsf{VerifSchnorrEB} \leftarrow$ VerifyCCSchnorrProofs$((\mathtt{EB}_{\mathtt{pk}}), (\boldsymbol{\pi}_{\mathsf{EB}}), (\mathtt{hContext}, \text{“}\mathtt{SetupTallyEB}\text{”}))$
5: **if** $\mathsf{VerifSchnorrCCR} \wedge \mathsf{VerifSchnorrCCM} \wedge \mathsf{VerifSchnorrEB}$ **then**
6: **return** $\top$
7: **else**
8: **return** $\bot$
9: **end if**

Output:

The result of the verification: $\top$ if the verification is successful, $\bot$ otherwise.

Copyright 2026 Swiss Post Ltd.

Algorithm 3.21 VerifyCCSchnorrProofs

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Number of components to verify $\mathtt{N_V} \in \{1, 4\}$
Number of elements of the public key $\mathtt{N_{pk}} \in \mathbb{N}^+$

Input:

Vector of public keys $\mathbf{pk}_{\mathrm{CC}} \in (\mathbb{G}_q^{\mathtt{N_{pk}}})^{\mathtt{N_V}}$
Vector of Schnorr proofs of knowledge $\boldsymbol{\pi}_{\mathsf{pkCC}} \in \left((\mathbb{Z}_q \times \mathbb{Z}_q)^{\mathtt{N_{pk}}}\right)^{\mathtt{N_V}}$
The additional information $\mathbf{i}_{\mathsf{aux}} \in (\mathbb{A}_{UCS}{}^*)^s, s \in \mathbb{N}$

Operation: ▷ For all algorithms see the crypto primitives specification ▷ We flatten the lists in this algorithm

1: **for** $j \in [1, \mathtt{N_V}]$ **do**
2: $\quad \mathbf{i}_{\mathtt{aux,j}} \leftarrow (\mathbf{i}_{\mathsf{aux}}, \mathsf{IntegerToString}(j))$
3: $\quad$ **for** $i \in [0, \mathtt{N_{pk}})$ **do**
4: $\quad\quad \mathbf{i}'_{\mathtt{aux,j}} \leftarrow \mathbf{i}_{\mathtt{aux,j}} \,||\, \mathsf{IntegerToString}(i)$
5: $\quad\quad \mathsf{VerifSchnorrCC}_{j,i} \leftarrow \mathsf{VerifySchnorr}(\pi_{\mathsf{pkCC},j,i}, \mathtt{pk}_{CC_{j,i}}, \mathbf{i}'_{\mathtt{aux,j}})$
6: $\quad$ **end for**
7: **end for**
8: **if** $\mathsf{VerifSchnorrCC}_{j,i}\ \forall\ j, i$ **then**
9: $\quad$ **return** $\top$
10: **else**
11: $\quad$ **return** $\bot$
12: **end if**

Output:

The result of the verification: $\top$ if the verification is successful, $\bot$ otherwise.

Copyright 2026 Swiss Post Ltd.

3.7 Voter Authentication

Authentication of the voter to the voting server requires the derivation of a unique voter identifier and a base authentication challenge by both the setup component and the voting client. To this end, the DeriveCredentialId algorithm generates fresh identifiers for each election event by instantiating Argon2id with a "salt" containing the election event ID and a hard-coded string. Although it is recommended that Argon2 uses a unique salt for each password (see [3]), in our case, we use the same salt for all voters participating in a given election event.
For all Argon2id invocations related to voter authentication and verification-card key derivation, the system uses the LESS_MEMORY profile defined in the crypto primitives specification. This profile is chosen because these algorithms are executed on voter devices, which may have limited memory and because the voting server must verify authentication challenges without allocating excessive memory per request. A higher memory profile would increase the resource cost of each verification attempt and would make memory-exhaustion attacks against the voting server more effective.
Using the same salt and the LESS_MEMORY profile does not compromise security since the setup component generates a unique high-entropy password for each voter, the Start Voting Key $\texttt{SVK}_{\texttt{id}}$, which is used by the voting client to invoke Argon2. Furthermore, when the algorithm 3.22 is first invoked, the voting client has not yet received any unique information about the voter that could be used as a salt for the Argon2 algorithm.

Algorithm 3.22 DeriveCredentialId

Context:

Election event ID $\texttt{ee} \in (\mathbb{A}_{Base16})^{l_{\text{ID}}}$

Input:

Start Voting Key $\texttt{SVK}_{\texttt{id}} \in (\mathbb{A}_{\texttt{u32}})^{l_{\text{SVK}}}$

Operation: ▷ For all algorithms see the crypto primitives specification

1: $\texttt{salt} \leftarrow \text{CutToBitLength}(\text{RecursiveHash}(\texttt{ee}, \text{“}\texttt{credentialId}\text{”}), 128)$
2: $\texttt{bcredentialID}_{\texttt{id}} \leftarrow \text{GetArgon2id}(\text{StringToByteArray}(\texttt{SVK}_{\texttt{id}}), \texttt{salt})$ ▷ Use the Argon2 profile for less memory, as defined in the crypto primitives specification
3: $\texttt{credentialID}_{\texttt{id}} \leftarrow \text{Base16Encode}(\text{CutToBitLength}(\texttt{bcredentialID}_{\texttt{id}}, 128))$

Output:

$\texttt{credentialID}_{\texttt{id}} \in (\mathbb{A}_{Base16})^{l_{\text{ID}}}$

Copyright 2026 Swiss Post Ltd.

Furthermore, both the setup component and the voting client need to generate a base authentication challenge using their shared secrets: the Start Voting Key and the extended authentication factor. Algorithm 3.23 is used to create this base authentication challenge. The extended authentication factor is explained in 4.1.3.
We use the same salt per election event for all voters for the same reasons as in algorithm 3.22.

Algorithm 3.23 DeriveBaseAuthenticationChallenge

Context:

Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$

Input:

Start Voting Key $\mathtt{SVK_{id}} \in (\mathbb{A}_{\mathtt{u32}})^{\mathtt{l_{SVK}}}$

Extended authentication factor $\mathtt{EA_{id}} \in \mathcal{D}_{\mathsf{EA}}$

Operation: ▷ For all algorithms see the crypto primitives specification

1: $\mathtt{salt_{auth}} \leftarrow \mathsf{CutToBitLength}(\mathsf{RecursiveHash}(\mathtt{ee}, \text{“}\mathtt{hAuth}\text{”}), 128)$
2: $\mathtt{k} \leftarrow \Big(\mathsf{StringToByteArray}(\mathtt{EA_{id}}) \mid\mid \mathsf{StringToByteArray}(\text{”}\mathtt{Auth}\text{”}) \mid\mid \mathsf{StringToByteArray}(\mathtt{SVK_{id}})\Big)$
3: $\mathtt{bhAuth_{id}} \leftarrow \mathsf{GetArgon2id}(\mathtt{k}, \mathtt{salt_{auth}})$ ▷ Use the Argon2 profile for less memory, as defined in the crypto primitives specification
4: $\mathtt{hAuth_{id}} \leftarrow \mathsf{Base64Encode}(\mathtt{bhAuth_{id}})$

Output:

Base authentication challenge $\mathtt{hAuth_{id}} \in (\mathbb{A}_{Base64})^{\mathtt{l_{HB64}}}$

Copyright 2026 Swiss Post Ltd.

3.8 Write-Ins

Certain elections in Switzerland allow voters to write a candidate's name in a form instead of selecting a predefined candidate (so-called write-in candidates). Usually, only a tiny fraction of voters select write-in candidates.
For write-in candidates, the Federal Chancellery's Ordinance waives the requirement of individual verifiability [7].

> [VEleS art. 16 para. 2]: The Federal Chancellery may in exceptional cases exempt a canton from meeting individual requirements, provided:
> - a. the requirements that have not been met are indicated in the application;
> - b. reasonable justification is brought forward for allowing an exemption; and
> - c. the canton describes any alternative measures and justifies in reference to the risk assessment why it regards the risks as sufficiently low.

The explanatory report elaborates on that exemption [8].

> [Explanatory Report, Sec 4.2.1]: Para. 2: In exceptional cases, a canton may be exempted from meeting individual requirements. This option is subject to the three conditions set out on letters a-c. In particular, there must be a clear justification for making an exception. An exception might be: in an election run using the first-past-the-post system, the requirement of individual verifiability can be waived if the vote is cast by entering a name in blank text field ('write-in votes')

Moreover, the explanatory report clarifies that write-in votes are always *cast in conformity with the system*.

> [Explanatory Report, Sec 4.2.1]: Let. o No 1: In elections run using the first-past-the-post system, blank text fields ('write-in votes') are always considered to have been completed in conformity with the system.

Copyright 2026 Swiss Post Ltd.

The Swiss Post Voting System supports the usage of write-in candidates along predefined candidates. However, the security guarantees regarding individual verifiability apply only to predefined voting options.
Using multi-recipient ElGamal encryption, the system encodes and encrypts write-in candidates as separate messages along with the predefined candidates (which are still encoded as the product of primes and linked to a specific Choice Return Code using NIZK proofs).
The following two algorithms are mainly affected by the presence of write-in candidates:

- 5.4 CreateVote
- 6.12 ProcessPlaintexts

The voter must choose to enter a write-in candidate for a given position and enters a write-in from a given alphabet. The voting client will encode the write-in to a group element and encrypts the encoded write-in as part of the multi-recipient ElGamal ciphertext.
Upon decryption of the ciphertext votes in the Tally control component, the ProcessPlaintexts decodes the ciphertext elements back to the write-in contents.

3.8.1 Alphabet used for Write-Ins

For the write-ins, we use the alphabet $\mathbb{A}_{\mathsf{latin}}$ described in the crypto primitives specification. The size of this alphabet is $|\mathbb{A}_{\mathsf{latin}}| = a$. The symbol #, which is the rank 0 character in the alphabet $\mathbb{A}_{\mathsf{latin}}$, is by the system used as a separator and must therefore be prevented within the write-in fields. A trustworthy voting client will prevent usage of # within the write-in fields, as seen in algorithm 5.4.

3.8.2 Encoding Write-Ins

Encoding a write-in field uses algorithms 3.24 and 3.25:

Algorithm 3.24 WriteInToQuadraticResidue: Map a character string to a value $\in \mathbb{G}_q$

Context:
Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Input:
A character string $s \in \mathbb{A}_{\mathsf{latin}}{}^*$ ▷ See section 3.8.1
Require: $a^{|s|} < q$ ▷ where a is the size of $\mathbb{A}_{\mathsf{latin}}$ and $|s|$ is the character length of s
Require: $|s| > 0$
Require: s does not start with the rank 0 character (leading 0s are lost by the encoding)

Operation:
$x \leftarrow \mathsf{WriteInToInteger}(s)$ ▷ See algorithm 3.25
$y \leftarrow x^2 \mod p$

Output:
$y \in \mathbb{G}_q$
Test values for the algorithm 3.24 are provided in write-in-to-quadratic-residue.json.

Copyright 2026 Swiss Post Ltd.

Algorithm 3.25 WriteInToInteger: Map a character string to an integer value

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$

Input:

A character string $s \in \mathbb{A}_{\mathsf{latin}}{}^*$ ▷ See section 3.8.1

Require: $a^{|s|} < q$ ▷ where a is the size of $\mathbb{A}_{\mathsf{latin}}$ and $|s|$ is the character length of s
Require: $|s| > 0$
Require: s does not start with the rank 0 character (leading 0s are lost by the encoding)

Operation:

1: $x \leftarrow 0$
2: **for** $i \in [0, |s|)$ **do** ▷ $|s|$ is the character length of s
3: $c \leftarrow$ the character at the i^{th} position of s
4: $b \leftarrow$ the rank of character c within $\mathbb{A}_{\mathsf{latin}}$ ▷ See section 3.8.1
5: $x \leftarrow xa + b$ ▷ Leading characters of rank 0 would be ignored.
6: **end for**

Output:

$x \in \mathbb{Z}_q \backslash \{0\}$ ▷ Since $|s| > 0$ and $a^{|s|} < q$

Test values for the algorithm 3.25 are provided in write-in-to-integer.json.

Copyright 2026 Swiss Post Ltd.

3.8.3 Decoding Write-Ins

Decoding a write-in field uses algorithms 3.26 and 3.27:

Algorithm 3.26 QuadraticResidueToWriteIn: Map a quadratic residue to a write-in string

Input:

The quadratic residue $y \in \mathbb{G}_q$ ▷ By contract, we assume that y is in the correct group.

Operation:

1: $x \leftarrow y^{(p+1)/4} \mod p$ ▷ Thus $x^2 \equiv y^{(p+1)/2} \equiv y^{(2q+2)/2} \equiv y^q \cdot y \equiv y \pmod{p}$
2: **if** $x > q$ **then** ▷ We want the smallest of the two roots of y
3: $x \leftarrow p - x$
4: **end if**
5: $s \leftarrow \mathsf{IntegerToWriteIn}(x)$ ▷ See algorithm 3.27

Output:

The string $s \in \mathbb{A}_{\mathsf{latin}}{}^*$ ▷ See section 3.8.1

Test values for the algorithm 3.26 are provided in qr-to-write-in.json.

While Algorithm 3.25 WriteInToInteger only outputs elements of $\mathbb{Z}_q \backslash \{0\}$, Algorithm 3.27 IntegerToWriteIn is defined on any positive integer to accept more general inputs.

Algorithm 3.27 IntegerToWriteIn: Map a positive integer to a write-in string

Input:

The encoding $x \in \mathbb{N}^+$

Operation:

1: $s \leftarrow$ ""
2: **while** $x > 0$ **do**
3: $b \leftarrow x \mod a$
4: $c \leftarrow$ the character at rank b within alphabet $\mathbb{A}_{\mathsf{latin}}$ ▷ See section 3.8.1
5: $s \leftarrow c||s$ ▷ String concatenation
6: $x \leftarrow \frac{(x-b)}{a}$
7: **end while**

Output:

The string $s \in \mathbb{A}_{\mathsf{latin}}{}^*$ ▷ See section 3.8.1

Test values for the algorithm 3.27 are provided in integer-to-write-in.json.

Copyright 2026 Swiss Post Ltd.

3.8.4 Creating a Vote with Write-Ins

For example, let us assume that we have an election for three seats, where write-ins are allowed. Hence δ is 4 (number of write-in positions + 1). We assume that the voter makes the following choice:

- Position 1: Write-in Candidate "`Jane Doe`"
- Position 2: Pre-Defined Candidate "`Richard Doe`"
- Position 3: Write-in Candidate "`John Doe`"

Here, the voter selected 2 out of 3 possible write-in options. In general, we can say that the voter selects k out of $(\delta - 1)$ write-in options.
In case the voter selected a write-in candidate, the system is going to display to the voter a Choice Return Code indicating that the write-in field was selected.
The algorithm EncodeWriteIns converts the write-in entries into a vector of $\mathbb{G}_q$ elements. Since the voter might select less than $(\delta - 1)$ write-ins, EncodeWriteIns encodes the write-in contents first and then fills the remaining (unused) write-in positions with dummy values. The algorithm EncodeWriteIns requires that the input elements are ordered carefully: the selected write-in candidates must be arranged according to the order of the elections as defined by the election event configuration. This specific order is crucial for accurately attributing write-ins during the tally, particularly when multiple elections permit write-in candidates.

Algorithm 3.28 EncodeWriteIns

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Number of allowed write-ins + 1 for this specific ballot box $\delta \in [1, \delta_{\text{sup}}]$ ▷ Can be derived from pTable using algorithm 3.12

Input:

The vector of selected write-ins $\hat{\mathbf{s}} = (\hat{s}_0, \dots, \hat{s}_{k-1}) \in (\mathbb{A}_{\text{latin}}{}^*)^k$ ▷ See section 3.8.1 ▷ the k write-in contents ordered in the way described in 3.8.4

Require: $k \leq \delta - 1$ ▷ $\delta = 1$, if the ballot box does not have any write-in candidates.

Operation:

1: **for** $i \in [0, k)$ **do**
2: $\quad w_i \leftarrow \text{WriteInToQuadraticResidue}(\hat{s}_i)$ ▷ See Algorithm 3.24
3: **end for**
4: **for** $i \in [k, (\delta - 1))$ **do**
5: $\quad w_i \leftarrow 1$ ▷ Fill the remaining position with dummy values
6: **end for**

Output:

Encoded write-ins $\mathbf{w} = (w_0, \dots, w_{\delta-2}) \in \left((\mathbb{G}_q \cap \mathbb{P}) \setminus g\right)^{\delta-1}$

Test values for the algorithm 3.28 are provided in encode-write-ins.json.

Copyright 2026 Swiss Post Ltd.

3.8.5 Processing Write-Ins in the Tally Phase

Upon decryption, the ProcessPlaintexts algorithm decodes the write-ins. The ProcessPlaintexts algorithm does *not* decode more write-ins than the number of selected write-in options.
We define a method IsWriteInOption that determines if a voting option corresponds to a write-in.

Algorithm 3.29 IsWriteInOption

Context:

Encoded write-in voting options $\tilde{\mathbf{p}}_{\mathrm{w}} = (\tilde{\mathrm{p}}_{\mathrm{w},0}, \ldots, \tilde{\mathrm{p}}_{\mathrm{w},\delta-2}) \in \left((\mathbb{G}_q \cap \mathbb{P}) \setminus g\right)^{\delta-1}$ ▷ Can be derived from pTable using algorithm 3.8

Input:

Encoded voting option $\tilde{\mathrm{p}}_i \in (\mathbb{G}_q \cap \mathbb{P}) \setminus g$ ▷ By contract, we assume that $\tilde{\mathrm{p}}_i$ is in the correct group.

Operation:

1: **if** $\tilde{\mathrm{p}}_i \in \tilde{\mathbf{p}}_{\mathrm{w}}$ **then**
   return $\top$
2: **else**
   return $\bot$
3: **end if**

Output:

$\top$ if the voting option is a write-in, $\bot$ otherwise.

Test values for the algorithm 3.29 are provided in is-write-in-option.json.

To decode write-ins, we thus define an algorithm DecodeWriteIns that takes as input a list of encoded voting options and a list of encoded write-ins. Note that DecodeWriteIns returns an empty list if the input list of encoded write-ins is empty. Moreover, the eCH-0222 XML (see section 6.3.5) imposes certain restrictions on the character length of the write-in contents. The voter portal restricts the character length of the write-in to a much smaller value. However, a malicious voting client could potentially encode a write-in of larger character length.
Therefore, the algorithm 3.30 truncates the write-in values to prevent the creation of an invalid eCH-0222 XML.

Copyright 2026 Swiss Post Ltd.

Algorithm 3.30 DecodeWriteIns

Context:

Encoded write-in voting options $\tilde{\mathbf{p}}_{\mathrm{w}} = (\tilde{\mathrm{p}}_{\mathrm{w},0}, \ldots, \tilde{\mathrm{p}}_{\mathrm{w},\delta-2}) \in \left((\mathbb{G}_q \cap \mathbb{P}) \setminus g\right)^{\delta-1}$ ▷ Can be derived from pTable using algorithm 3.8

Number of allowed selections for this specific ballot box $\psi \in [1, \psi_{\mathtt{sup}}]$ ▷ Can be derived from pTable using algorithm 3.11

Number of allowed write-ins + 1 for this specific ballot box $\delta \in [1, \delta_{\mathtt{sup}}]$ ▷ Can be derived from pTable using algorithm 3.12

Input: ▷ By contract, we assume that the input elements have the correct mathematical group.

Selected encoded voting options $\hat{\mathbf{p}} = (\hat{\mathrm{p}}_0, \ldots, \hat{\mathrm{p}}_{\psi-1}) \in \left((\mathbb{G}_q \cap \mathbb{P}) \setminus g\right)^{\psi}$

List of encoded write-ins $\mathbf{w} = (\mathrm{w}_0, \ldots, \mathrm{w}_{\delta-2}) \in {\mathbb{G}_q}^{(\delta-1)}$

Operation:

1: $\hat{\mathbf{s}} \leftarrow ()$
2: **if** $\delta = 1$ **then** ▷ The election event has no write-ins
3: **return** $\hat{\mathbf{s}}$
4: **end if**
5: $k \leftarrow 0$
6: **for** $i \in [0, \psi)$ **do**
7: **if** IsWriteInOption$(\hat{\mathrm{p}}_i)$ **then** ▷ See Algorithm 3.29
8: $\hat{\mathrm{s}}_k \leftarrow$ Truncate(QuadraticResidueToWriteIn$(\mathrm{w}_k), \mathtt{l_w})$ ▷ See Algorithm 3.26 and crypto primitives specification
9: $\hat{\mathbf{s}} \leftarrow (\hat{\mathbf{s}}, \hat{\mathrm{s}}_k)$
10: $k \leftarrow k + 1$
11: **end if**
12: **end for**

Output:

The vector of decoded write-ins $\hat{\mathbf{s}} \in ({\mathbb{A}_{\mathsf{latin}}}^*)^k$ ▷ See section 3.8.1 ▷ k is the number of selected write-in candidates

Test values for the algorithm 3.30 are provided in decode-write-ins.json.

Given the example from section 3.8.4, we would end up with the following results:

- Write-in 1: Truncate(QuadraticResidueToWriteIn$(\mathrm{w}_{\mathtt{id},0}), \mathtt{l_w})$
- Write-in 2: Truncate(QuadraticResidueToWriteIn$(\mathrm{w}_{\mathtt{id},1}), \mathtt{l_w})$
- Write-in 3: Nothing to decode, since only two write-in voting options were selected.

Copyright 2026 Swiss Post Ltd.

3.9 Abstention

3.9.1 Processing Abstention Choice Return Codes

From the voter's perspective, the system returns one Abstention Choice Return Code $\texttt{ACC}$ for each abstention group from which the voter abstains (see Section 1.1).
Internally, however, the electoral model encodes an abstention group by one abstention voting option per selection (see Section 3.4.2). Hence, an abstention group may correspond to several encoded voting options.
The voting client therefore derives the voter-facing Abstention Choice Return Codes locally. For each abstention group selected by the voter, it identifies the corresponding short Choice Return Codes, merges them into a single string, and appends a voter-specific suffix. The suffix ensures that the resulting Abstention Choice Return Code has length $\texttt{l}_{\texttt{ACC}}$, whereas ordinary Choice Return Codes have length $\texttt{l}_{\texttt{CC}}$ (see Section 3.3).
The Setup Component precomputes the corresponding Abstention Choice Return Code $\texttt{ACC}$ for each voter and prints it on the code sheet (see Section 1.1).
During the voting phase, the voting client receives from the voter the selected abstentions

$$\hat{\mathbf{a}}_{\texttt{id}} = (\hat{a}_{\texttt{id},0}, \ldots, \hat{a}_{\texttt{id},\eta-1}) \in \mathbb{Z}_2^{\eta},$$

For each presentation group i, $\hat{a}_{\texttt{id},i} = 1$ means that the voter abstains from presentation group i, whereas $\hat{a}_{\texttt{id},i} = 0$ means that the voter does not abstain from that presentation group.
After receiving the short Choice Return Codes $\mathbf{CC}$ from the voting server, the voting client invokes ProcessAbstentionChoiceReturnCodes (see Algorithm 3.31).
For presentation groups for which the voter did not abstain, the voting client displays the individual short Choice Return Codes as ordinary Choice Return Codes.

Copyright 2026 Swiss Post Ltd.

Algorithm 3.31 ProcessAbstentionChoiceReturnCodes

Context:

Number of selections vector $\boldsymbol{\psi} = (\psi_0, \dots, \psi_{t-1}) \in (\mathbb{N}^+)^t$
Presentation groups $\mathbf{pg} = (\mathtt{pg}_0, \dots, \mathtt{pg}_{t-1}) \in [0, \eta)^t$
Abstention groups $\mathbf{ag} = (\mathtt{ag}_0, \dots, \mathtt{ag}_{\eta-1}) \in \mathbb{Z}_2^\eta$

Input:

Short Choice Return Codes $\mathbf{CC} = (\mathtt{CC}_0, \dots, \mathtt{CC}_{\psi-1}) \in ((\mathbb{A}_{10})^{\mathtt{l_{CC}}})^\psi$
Selected abstentions $\hat{\mathbf{a}}_{\mathtt{id}} = (\hat{a}_{\mathtt{id},0}, \dots, \hat{a}_{\mathtt{id},\eta-1}) \in \mathbb{Z}_2^\eta$

Require: $\psi = \sum_{i=0}^{t-1} \psi_i$
Require: $\forall i \in [0, \eta) : \hat{a}_{\mathtt{id},i} \leq \mathtt{ag}_i$

Operation:

1: $\mathbf{ACC} \leftarrow ()$
2: **for** $i \in [0, \eta)$ **do**
3: **if** $\hat{a}_{\mathtt{id},i} = 1$ **then**
4: $\mathbf{pg}_{\text{idx}} \leftarrow \mathsf{indices}(i, \mathbf{pg}, \boldsymbol{\psi})$ ▷ See Equation (1)
5: $\mathbf{pg}_{\text{CC}} \leftarrow (\mathtt{CC}_s)_{s \in \mathbf{pg}_{\text{idx}}}$
6: $\mathtt{mACC} \leftarrow \mathsf{GetMergedString}(\mathbf{pg}_{\text{CC}}, \mathbb{A}_{10})$ ▷ See crypto primitives specification
7: $\xi_i \leftarrow \mathsf{IntegerToString}(i + 10)$ ▷ See crypto primitives specification
8: $\mathtt{ACC}_i \leftarrow \mathtt{mACC} \,||\, \xi_i$
9: $\mathbf{ACC} \leftarrow \mathbf{ACC} \,||\, \mathtt{ACC}_i$
10: **end if**
11: **end for**

Output:

Abstention Choice Return Codes $\mathbf{ACC} = (\mathtt{ACC}_0, \dots, \mathtt{ACC}_{m-1})$, where $m \leq \kappa$ and each $\mathtt{ACC}_i \in (\mathbb{A}_{10})^{\mathtt{l_{ACC}}}$

Copyright 2026 Swiss Post Ltd.

3.9.2 Encrypting the Abstention Vector

The voting client gets the selected abstentions $\hat{\mathbf{a}}_{\mathtt{id}}$ from the voter and encrypts them with algorithm 3.32, producing $\mathtt{ct}_{\mathbf{a},\mathtt{id}}$. Algorithm 5.4 CreateVote outputs $\mathtt{ct}_{\mathbf{a},\mathtt{id}}$ as part of the SendVote phase and the voting server stores it.
Upon a relogin (see Section 5.2.8), the voting client retrieves $\mathtt{ct}_{\mathbf{a},\mathtt{id}}$ from the voting server and decrypts it with Algorithm 3.33, recovering the selected abstentions $\hat{\mathbf{a}}_{\mathtt{id}}$.

Algorithm 3.32 GenEncryptedAbstentionVector

Context:
Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{1_{\mathtt{ID}}}$
Verification card set ID $\mathtt{vcs} \in (\mathbb{A}_{Base16})^{1_{\mathtt{ID}}}$
Verification card ID $\mathtt{vc_{id}} \in (\mathbb{A}_{Base16})^{1_{\mathtt{ID}}}$
Primes mapping table $\mathsf{pTable} \in \left(\mathcal{D}_{\mathrm{v}} \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times {\mathbb{A}_{UCS}}^* \times \mathcal{D}_\tau\right)^n$ ▷ pTable is of the form $((\mathrm{v}_0, \tilde{\mathrm{p}}_0, \sigma_0, \tau_0), \ldots, (\mathrm{v}_{n-1}, \tilde{\mathrm{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1}))$
Electoral Model Context $\Lambda = (\psi, \mathbf{DoI}, \mathbf{pg}, \mathbf{ag}) \in \left((\mathbb{N}^+)^t \times (\mathcal{T}_1^{50})^k \times [0, \eta)^t \times \mathbb{Z}_2^\eta\right)$ ▷ See Section 3.4.4
Election public key $\mathtt{EL_{pk}}(\mathtt{EL}_{\mathtt{pk},0}, \ldots, \mathtt{EL}_{\mathtt{pk},\delta_{\max}-1}) \in \mathbb{G}_q^{\delta_{\max}}$
Choice Return Codes encryption public key $\mathbf{pk}_{\mathrm{CCR}} = (\mathbf{pk}_{\mathrm{CCR},0}, \ldots, \mathbf{pk}_{\mathrm{CCR},\psi_{\max}-1}) \in \mathbb{G}_q^{\psi_{\max}}$
Input:
Selected abstentions $\hat{\mathbf{a}}_{\mathtt{id}} = (\hat{a}_{\mathtt{id},0}, \ldots, \hat{a}_{\mathtt{id},\eta-1}) \in \mathbb{Z}_2^\eta$
Verification card secret key $\mathtt{k_{id}} \in \mathbb{Z}_q$
Require: $\forall i \in [0, \eta) : \hat{a}_{\mathtt{id},i} \leq \mathtt{ag}_i$

Operation: ▷ For all algorithms see the crypto primitives specification
1: $\mathbf{i}_{\mathrm{aux}} \leftarrow$ ("AbstentionVector", GetHashContext()) ▷ See algorithm 3.15
2: $\mathtt{ct}_{\mathbf{a},\mathtt{id},\mathtt{salt}} \leftarrow$ RandomBytes(16) ▷ We use a 16-byte salt in analogy with Argon2id
3: $\mathtt{sk} \leftarrow$ RecursiveHash("AbstentionVector", ee, vcs, $\mathtt{vc_{id}}$, $\mathtt{ct}_{\mathbf{a},\mathtt{id},\mathtt{salt}}$, $\mathtt{k_{id}}$)
4: $\mathtt{b} \leftarrow$ IntegerToFixedLengthByteArray$(\hat{a}_{\mathtt{id},0}, 1)$ || $\cdots$ || IntegerToFixedLengthByteArray$(\hat{a}_{\mathtt{id},\eta-1}, 1)$
5: $(\mathtt{ct}_{\mathbf{a},\mathtt{id},\mathtt{ciphertext}}, \mathtt{ct}_{\mathbf{a},\mathtt{id},\mathtt{nonce}}) \leftarrow$ GenCiphertextSymmetric(sk, b, $\mathbf{i}_{\mathrm{aux}}$)
6: $\mathtt{ct}_{\mathbf{a},\mathtt{id}} \leftarrow$ Base64Encode($\mathtt{ct}_{\mathbf{a},\mathtt{id},\mathtt{ciphertext}}$ || $\mathtt{ct}_{\mathbf{a},\mathtt{id},\mathtt{nonce}}$ || $\mathtt{ct}_{\mathbf{a},\mathtt{id},\mathtt{salt}}$)

Output:
Encrypted abstention vector $\mathtt{ct}_{\mathbf{a},\mathtt{id}} \in {\mathbb{A}_{Base64}}^*$

Copyright 2026 Swiss Post Ltd.

Algorithm 3.33 GetDecryptedAbstentionVector

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Verification card set ID $\mathtt{vcs} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Verification card ID $\mathtt{vc_{id}} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Primes mapping table $\mathsf{pTable} \in (\mathcal{D}_{\mathrm{v}} \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times {\mathbb{A}_{UCS}}^* \times \mathcal{D}_\tau)^n$ ▷ pTable is of the form $((\mathrm{v}_0, \tilde{\mathrm{p}}_0, \sigma_0, \tau_0), \ldots, (\mathrm{v}_{n-1}, \tilde{\mathrm{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1}))$
Electoral Model Context $\Lambda = (\boldsymbol{\psi}, \mathbf{DoI}, \mathbf{pg}, \mathbf{ag}) \in \left((\mathbb{N}^+)^t \times (\mathcal{T}_1^{50})^k \times [0, \eta)^t \times \mathbb{Z}_2^\eta\right)$
Election public key $\mathtt{EL_{pk}} = (\mathtt{EL_{pk},0}, \ldots, \mathtt{EL_{pk},\delta_{max}-1}) \in \mathbb{G}_q^{\delta_{\mathtt{max}}}$
Choice Return Codes encryption public key $\mathbf{pk}_{\mathrm{CCR}} = (\mathtt{pk_{CCR},0}, \ldots, \mathtt{pk_{CCR},\psi_{max}-1}) \in \mathbb{G}_q^{\psi_{\mathtt{max}}}$

Input:

Encrypted abstention vector $\mathtt{ct_{a,id}} \in {\mathbb{A}_{Base64}}^*$
Verification card secret key $\mathtt{k_{id}} \in \mathbb{Z}_q$

Operation: ▷ For all algorithms see the crypto primitives specification

1: $\mathbf{i}_{\mathtt{aux}} \leftarrow$ ("AbstentionVector", GetHashContext()) ▷ See algorithm 3.15
2: $(\mathtt{ct_{a,id,ciphertext}}, \mathtt{ct_{a,id,nonce}}, \mathtt{ct_{a,id,salt}}) \leftarrow \mathsf{GetSymmetricCiphertextParts}(\mathtt{ct_{a,id}})$
3: $\mathtt{sk} \leftarrow \mathsf{RecursiveHash}(\text{"}\mathtt{AbstentionVector}\text{"}, \mathtt{ee}, \mathtt{vcs}, \mathtt{vc_{id}}, \mathtt{ct_{a,id,salt}}, \mathtt{k_{id}})$
4: $\mathtt{pt} \leftarrow \mathsf{GetPlaintextSymmetric}(\mathtt{sk}, \mathtt{ct_{a,id,ciphertext}}, \mathtt{ct_{a,id,nonce}}, \mathbf{i}_{\mathtt{aux}})$
5: **for** $i \in [0, \eta)$ **do**
6: $\quad \hat{a}_{\mathtt{id},i} \leftarrow \mathsf{ByteArrayToInteger}(\mathtt{pt}[i : i + 1))$
7: **end for**
8: $\hat{\mathbf{a}}_{\mathtt{id}} \leftarrow (\hat{a}_{\mathtt{id},0}, \ldots, \hat{a}_{\mathtt{id},\eta-1})$

Output:

Selected abstentions $\hat{\mathbf{a}}_{\mathtt{id}} = (\hat{a}_{\mathtt{id},0}, \ldots, \hat{a}_{\mathtt{id},\eta-1})$

Copyright 2026 Swiss Post Ltd.

4 Configuration Phase

The configuration phase consists of two sub-protocols: SetupVoting and SetupTally. Table 12 highlights the algorithms involved.

Algorithm	Actor	Reference
SetupVoting		Figure 7
GenSetupData	Setup Component	4.1
GenVerDat	Setup Component	4.2
GetVoterAuthenticationData	Setup Component	4.3
GenKeysCCR	Control Component (CCR)	4.4
GenEncLongCodeShares	Control Component (CCR)	4.5
CombineEncLongCodeShares	Setup Component	4.6
GenCMTable	Setup Component	4.7
VerifyCCRExponentiationProofs	Setup Component	4.8
GenVerCardSetKeys	Setup Component	4.11
GenCredDat	Setup Component	4.12
SetupTally		Figure 8
SetupTallyCCM	Control Component (CCM)	4.13
SetupTallyEB	Setup Component	4.14
VerifyConfigPhase	Auditors	Verifier Specification [21]

Tab. 12: Overview of the algorithms in the configuration phase.

Section 4.3 elaborates on the data flows and agreement mechanisms at the end of the configuration phase, as well as the transition of the election event's configuration to the voting phase.
The control components ensure that the algorithms listed in table 12 can only be executed during the configuration phase, and not after it has ended. The setup component is only used in the configuration phase, and remains unused and offline throughout the voting and tally phases.

Copyright 2026 Swiss Post Ltd.

4.1 SetupVoting

Figure 7 depicts the SetupVoting sub-protocol.

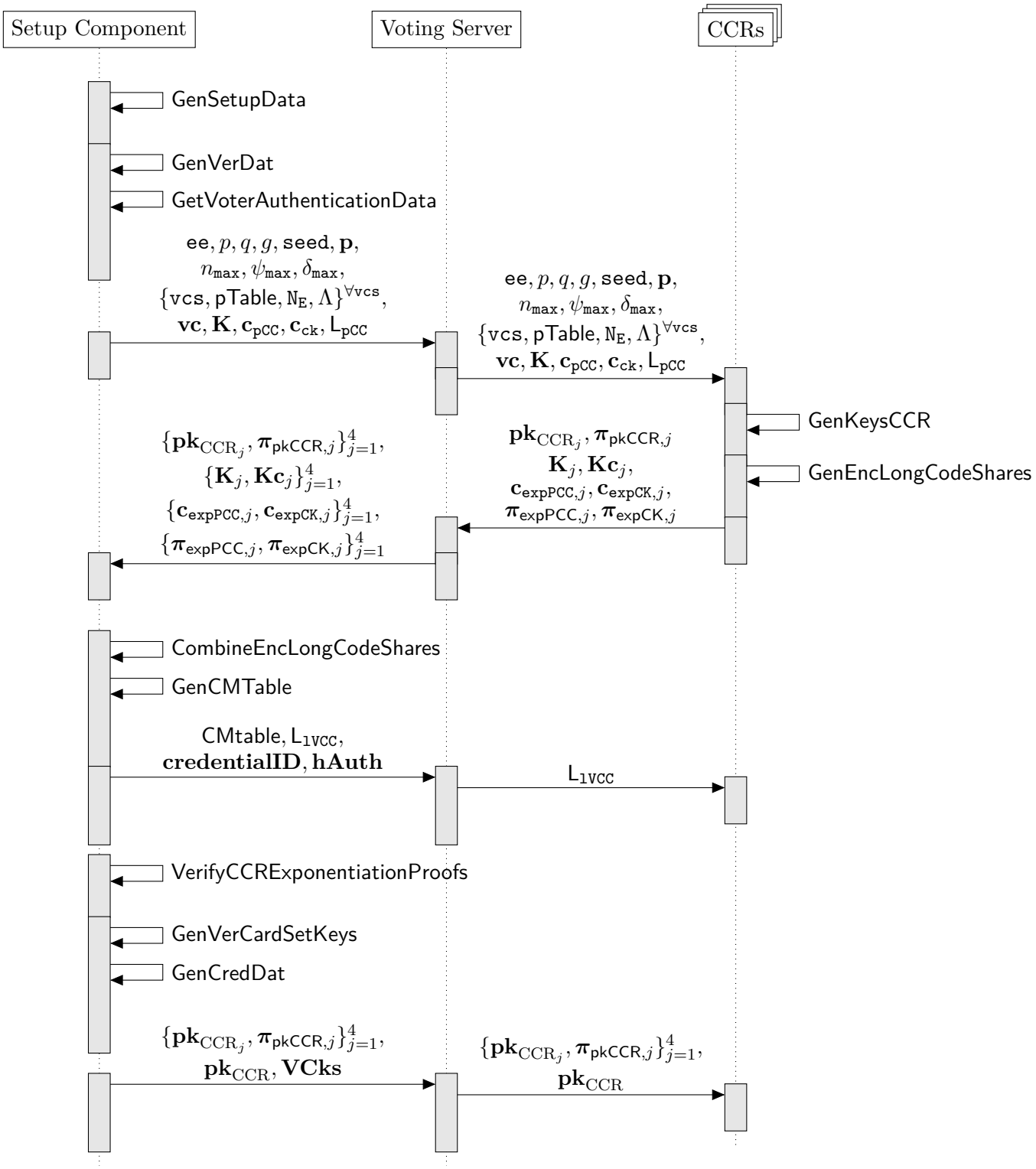


Fig. 7: Sequence diagram of the SetupVoting protocol.

Copyright 2026 Swiss Post Ltd.

4.1.1 GenSetupData

The setup component generates the election event context, the primes mapping table pTable and a key pair to encrypt the partial Choice Return Codes $\mathbf{pCC}_{\mathtt{id}}$ during the configuration phase. GenSetupData ensures that the same actual voting option maps to the same prime number in all verification card sets. This is achieved by defining the initially empty key-value map $\mathsf{p}_{\mathsf{map}}$ which is augmented during the GenSetupData algorithm with the key-value pairs $(\mathrm{v}_i, \tilde{\mathrm{p}}_i)$, where the actual voting option is the key and the prime is the corresponding value.

Copyright 2026 Swiss Post Ltd.

Algorithm 4.1 GenSetupData

Context:

Vector of verification card set IDs $\mathbf{vcs} = (\mathtt{vcs}_0, \dots, \mathtt{vcs}_{\mathtt{N_{bb}}-1}) \in \left((\mathbb{A}_{Base16})^{1_{\mathtt{ID}}}\right)^{\mathtt{N_{bb}}}$

For each of the $\mathtt{N_{bb}}$ verification card sets $\mathtt{vcs} \in \mathbf{vcs}$ (see Section 3.4.3)

Actual voting options $\tilde{\mathbf{v}} = (\mathrm{v}_0, \dots, \mathrm{v}_{n-1}),\ \mathrm{v}_i \in \mathcal{D}_{\mathrm{v}}$ ▷ The actual voting options of this specific verification card set in the order defined by the election event configuration
Semantic information $\boldsymbol{\sigma} = (\sigma_0, \dots, \sigma_{n-1}),\ \sigma_i \in {\mathbb{A}_{UCS}}^*$ ▷ Matching order between actual voting options and semantic information
Correctness information $\boldsymbol{\tau} = (\tau_0, \dots, \tau_{n-1}),\ \tau_i \in \mathcal{D}_\tau$ ▷ Matching order between actual voting options and correctness information

Maximum supported number of voting options $n_{\mathtt{sup}} \in \mathbb{N}^+$ ▷ See section 3.4.5
Maximum supported number of selections $\psi_{\mathtt{sup}} \in \mathbb{N}^+$
Maximum supported number of write-ins + 1: $\delta_{\mathtt{sup}} \in \mathbb{N}^+$

Input:

Encryption parameter's seed: $\mathtt{seed} \in {\mathbb{A}_{UCS}}^{16}$ ▷ The name of the election event in the format specified in section 3.2

Require: $1 \leq n \leq n_{\mathtt{sup}}, \forall \mathtt{vcs} \in \mathbf{vcs}$

Operation:

$(p, q, g, (p_0, \dots, p_{n_{\mathtt{sup}}-1})) \leftarrow$ GetElectionEventEncryptionParameters(seed) ▷ See algorithm 3.1
$\mathsf{p_{map}} \leftarrow ()$
$k \leftarrow 0$
$n_{\mathtt{max}} \leftarrow 0$
$\psi_{\mathtt{max}} \leftarrow 0$
$\delta_{\mathtt{max}} \leftarrow 0$
for $\mathtt{vcs} \in \mathbf{vcs}$ **do** ▷ Iterating through **vcs** in lexicographic order
  for $i \in [0, n)$ **do**
    if $\mathrm{v}_i \in \mathsf{p_{map}}$ **then** ▷ Look up if the value v_i exists as a key in $\mathsf{p_{map}}$
      $\tilde{\mathrm{p}}_i \leftarrow \mathsf{p_{map}}(\mathrm{v}_i)$ ▷ Retrieve the value from the key-value map
    else
      if $k \geq n_{\mathtt{sup}}$ **then**
        return $\perp$ ▷ The amount of distinct voting options across all verification card sets must not exceed $n_{\mathtt{sup}}$
      end if
      $\tilde{\mathrm{p}}_i \leftarrow p_k$
      $\mathsf{p_{map}} \leftarrow \mathsf{p_{map}} \,||\, (\mathrm{v}_i, \tilde{\mathrm{p}}_i)$
      $k \leftarrow k + 1$
    end if
  end for
  $\mathsf{pTable_{vcs}} \leftarrow ((\mathrm{v}_0, \tilde{\mathrm{p}}_0, \sigma_0, \tau_0), \dots, (\mathrm{v}_{n-1}, \tilde{\mathrm{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1}))$
  $\psi \leftarrow$ GetPsi() ▷ See algorithm 3.11
  $\delta \leftarrow$ GetDelta() ▷ See algorithm 3.12
  if $(\delta - 1 > \psi)$ **then**
    return $\perp$ ▷ The number of write-ins of a verification card set must not exceed the number of selections
  end if
  $n_{\mathtt{max}} \leftarrow \max(n_{\mathtt{max}}, n)$
  $\psi_{\mathtt{max}} \leftarrow \max(\psi_{\mathtt{max}}, \psi)$
  $\delta_{\mathtt{max}} \leftarrow \max(\delta_{\mathtt{max}}, \delta)$
end for
if $(\psi_{\mathtt{max}} > \psi_{\mathtt{sup}}) \vee (\delta_{\mathtt{max}} > \delta_{\mathtt{sup}})$ **then**
  return $\perp$ ▷ The maximum number of selections or write-ins must not exceed the supported values
end if
pTable $\leftarrow (\mathsf{pTable_{vcs_0}}, \dots, \mathsf{pTable_{vcs_{N_{bb}-1}}})$ ▷ Each verification card set has its pTable
$(\mathsf{pk_{setup}}, \mathsf{sk_{setup}}) \leftarrow$ GenKeyPair$(p, q, g, n_{\mathtt{max}})$ ▷ See crypto primitives specification

---

Output:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Small primes $\mathbf{p} = (p_0, \dots, p_{n_{\mathtt{sup}}-1}),\ p_i \in (\mathbb{G}_q \cap \mathbb{P}) \setminus g, (p_i < p_{i+1})\ \forall\, i \in [0, n_{\mathtt{sup}} - 1)$
Maximum number of voting options $n_{\mathtt{max}} \in [1, n_{\mathtt{sup}}]$
Maximum number of selections $\psi_{\mathtt{max}} \in [1, \psi_{\mathtt{sup}}]$
Maximum number of write-ins + 1: $\delta_{\mathtt{max}} \in [1, \delta_{\mathtt{sup}}]$
Prime mapping tables for each verification card set **pTable**
Setup public key $\mathsf{pk_{setup}} \in \mathbb{G}_q^{n_{\mathtt{max}}}$
Setup secret key $\mathsf{sk_{setup}} \in \mathbb{Z}_q^{n_{\mathtt{max}}}$

Copyright 2026 Swiss Post Ltd.

4.1.2 GenVerDat

The setup component runs the GenVerDat (Generate Verification Data) algorithm to initialize the control components' computation of the return codes.

Algorithm 4.2 GenVerDat

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Number of eligible voters for this verification card set $\mathtt{N_E} \in \mathbb{N}^+$
Primes mapping table $\mathsf{pTable} \in (\mathcal{D}_\mathrm{v} \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times {\mathbb{A}_{UCS}}^* \times \mathcal{D}_\tau)^n$ ▷ pTable is of the form $((\mathrm{v}_0, \tilde{\mathrm{p}}_0, \sigma_0, \tau_0), \ldots, (\mathrm{v}_{n-1}, \tilde{\mathrm{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1}))$
Maximum number of voting options $n_{\mathtt{max}} \in [1, n_{\mathtt{sup}}]$

Input:

Setup public key $\mathsf{pk}_{\mathtt{setup}} \in \mathbb{G}_q^{n_{\mathtt{max}}}$

Operation: ▷ For all algorithms see the crypto primitives specification

$\mathsf{L}_{\mathtt{pCC}} \leftarrow ()$
$\tilde{\mathbf{p}} = (\tilde{\mathrm{p}}_0, \ldots, \tilde{\mathrm{p}}_{n-1}) \leftarrow \mathsf{GetEncodedVotingOptions}())$ ▷ See Algorithm 3.2
$\boldsymbol{\tau} = (\tau_0, \ldots, \tau_{n-1}) \leftarrow \mathsf{GetCorrectnessInformation}())$ ▷ See Algorithm 3.5
for $\mathtt{id} \in [0, \mathtt{N_E})$ **do**
  $\mathtt{vc_{id}} \leftarrow \mathsf{GenRandomString}(\mathtt{l_{ID}}, \mathbb{A}_{Base16})$
  $\mathtt{SVK_{id}} \leftarrow \mathsf{GenRandomString}(\mathtt{l_{SVK}}, \mathbb{A}_{\mathtt{u32}})$
  $(\mathtt{K_{id}}, \mathtt{k_{id}}) \leftarrow \mathsf{GenKeyPair}(p, q, g, 1)$
  for $k \in [0, n)$ **do**
    $\mathtt{pCC}_{\mathtt{id},k} \leftarrow \tilde{\mathrm{p}}_k^{\mathtt{k_{id}}} \bmod p$
    $\mathtt{hpCC}_{\mathtt{id},k} \leftarrow \mathsf{HashAndSquare}(\mathtt{pCC}_{\mathtt{id},k})$
    $\mathtt{lpCC}_{\mathtt{id},k} \leftarrow \mathsf{RecursiveHash}(\mathtt{hpCC}_{\mathtt{id},k}, \mathtt{vc_{id}}, \mathtt{ee}, \tau_k)$
    $\mathsf{L}_{\mathtt{pCC}} \leftarrow \mathsf{L}_{\mathtt{pCC}} \,||\, \mathsf{Base64Encode}(\mathtt{lpCC}_{\mathtt{id},k})$
  end for
  $\mathbf{hpCC}_{\mathtt{id}} \leftarrow (\mathtt{hpCC}_{\mathtt{id},0}, \ldots, \mathtt{hpCC}_{\mathtt{id},n-1})$
  $\mathsf{c}_{\mathtt{pCC,id}} \leftarrow \mathsf{GetCiphertext}(\mathbf{hpCC}_{\mathtt{id}}, \mathsf{GenRandomInteger}(q), \mathsf{pk}_{\mathtt{setup}})$
  do
    $\mathtt{BCK_{id}} \leftarrow \mathsf{GenUniqueDecimalStrings}(\mathtt{l_{BCK}}, 1)$ ▷ Assign first element of list to $\mathtt{BCK_{id}}$
  while $\mathsf{StringToInteger}(\mathtt{BCK_{id}}) = 0$
  $\mathtt{hBCK_{id}} \leftarrow \mathsf{HashAndSquare}(\mathsf{StringToInteger}(\mathtt{BCK_{id}}))$
  $\mathtt{CK_{id}} \leftarrow \mathtt{hBCK_{id}}^{\mathtt{k_{id}}} \bmod p$
  $\mathtt{hCK_{id}} \leftarrow \mathsf{HashAndSquare}(\mathtt{CK_{id}})$
  $\mathsf{c}_{\mathtt{ck,id}} \leftarrow \mathsf{GetCiphertext}(\mathtt{hCK_{id}}, \mathsf{GenRandomInteger}(q), \mathsf{pk}_{\mathtt{setup}})$
end for
$\mathsf{L}_{\mathtt{pCC}} \leftarrow \mathsf{Order}(\mathsf{L}_{\mathtt{pCC}})$ ▷ Lexicographic ordering of the list ensures the unlinkability to the order of insertion.

Output:

Vector of verification card IDs $\mathbf{vc} = (\mathtt{vc}_0, \ldots, \mathtt{vc}_{\mathtt{N_E}-1}) \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}} \times \mathtt{N_E}}$
Vector of Start Voting Keys $\mathbf{SVK} = (\mathtt{SVK}_0, \ldots, \mathtt{SVK}_{\mathtt{N_E}-1}) \in ((\mathbb{A}_{\mathtt{u32}})^{\mathtt{l_{SVK}}})^{\mathtt{N_E}}$
Vector of verification card public keys $\mathbf{K} = (\mathtt{K}_0, \ldots, \mathtt{K}_{\mathtt{N_E}-1}) \in {\mathbb{G}_q}^{\mathtt{N_E}}$
Vector of verification card secret keys $\mathbf{k} = (\mathtt{k}_0, \ldots, \mathtt{k}_{\mathtt{N_E}-1}) \in {\mathbb{Z}_q}^{\mathtt{N_E}}$
Partial Choice Return Codes allow list $\mathsf{L}_{\mathtt{pCC}} \in ({\mathbb{A}_{Base64}}^{\mathtt{l_{HB64}}})^{n \cdot \mathtt{N_E}}$ ▷ Order of elements uncorrelated to order of insertion
Vector of ballot casting keys $\mathbf{BCK} = (\mathtt{BCK}_0, \ldots, \mathtt{BCK}_{\mathtt{N_E}-1}) \in ((\mathbb{A}_{10})^{\mathtt{l_{BCK}}})^{\mathtt{N_E}}$
Vector of encrypted, hashed partial Choice Return Codes $\mathbf{c}_{\mathtt{pCC}} = (\mathsf{c}_{\mathtt{pCC},0}, \ldots, \mathsf{c}_{\mathtt{pCC},\mathtt{N_E}-1}) \in ({\mathbb{G}_q}^{n+1})^{\mathtt{N_E}}$
Vector of encrypted, hashed Confirmation Keys $\mathbf{c}_{\mathtt{ck}} = (\mathsf{c}_{\mathtt{ck},0}, \ldots, \mathsf{c}_{\mathtt{ck},\mathtt{N_E}-1}) \in ({\mathbb{G}_q}^2)^{\mathtt{N_E}}$

Copyright 2026 Swiss Post Ltd.

4.1.3 GetVoterAuthenticationData

The setup component runs the GetVoterAuthenticationData algorithm to generate the data for the voter authentication protocol between the voting client and the voting server.
The cantons provide an extended authentication factor to the Swiss Post Voting System. Switzerland is a highly federal state, and the cantons vary considerably in how e-voting is integrated into their administrative processes and in what voter data is available at the cantonal or communal level. As a result, the cantons currently use three types of extended authentication factors: the voter's date of birth, the voter's year of birth, or an eGovernment token.
The date of birth and year of birth are static voter attributes tied to the voter's identity. They are straightforward to integrate because most cantons already maintain this data in their electoral rolls.
The eGovernment token provides the strongest security among the three options: it is a high-entropy secret, unique per election event and voter, and not derivable from publicly available information. In the voting phase, and prior to the voter authentication protocol between the voting client and the voting server, the voter is redirected to the cantonal eGovernment portal. After successful authentication, the eGovernment token is issued to the voting client. However, issuing the eGovernment token to the voting client requires a strong integration into a cantonal eGovernment portal, an infrastructure that is not available or feasible for all cantons. This is why certain cantons still rely on the date of birth or year of birth as their extended authentication factor.
Although the date or year of birth offers limited protection due to low entropy and potential exposure through social media, it nonetheless enhances authentication.

- The date or year of birth serves as a minor psychological hurdle for individuals attempting voter impersonation.
- The voter and cantons are the primary holders of this data. As such, the operators of the voting server, control components or printing component must exert considerable effort to learn the extended authentication factors from a large voter base.
- Its inclusion maintains consistency with previous online voting systems in Switzerland, which also utilized this extended authentication factor.

Hence, despite its limitations, using the date of birth or year of birth as extended authentication factor has some advantages compared to not having one at all.

Copyright 2026 Swiss Post Ltd.

Algorithm 4.3 GetVoterAuthenticationData

Context:

Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$

Number of eligible voters for this verification card set $\mathtt{N_E} \in \mathbb{N}^+$

Input:

Vector of Start Voting Keys $\mathbf{SVK} = (\mathtt{SVK}_0, \ldots, \mathtt{SVK}_{\mathtt{N_E}-1}) \in ((\mathbb{A}_{\mathtt{u32}})^{\mathtt{l_{SVK}}})^{\mathtt{N_E}}$

Vector of extended authentication factors $\mathbf{EA} = (\mathtt{EA}_0, \ldots, \mathtt{EA}_{\mathtt{N_E}-1}) \in (\mathcal{D}_{\mathtt{EA}})^{\mathtt{N_E}}$

Operation:

1: **for** $\mathtt{id} \in [0, \mathtt{N_E})$ **do**
2: $\quad \mathtt{credentialID_{id}} \leftarrow \mathsf{DeriveCredentialId}(\mathtt{SVK_{id}})$ ▷ See algorithm 3.22
3: $\quad \mathtt{hAuth_{id}} \leftarrow \mathsf{DeriveBaseAuthenticationChallenge}(\mathtt{SVK_{id}}, \mathtt{EA_{id}})$ ▷ See algorithm 3.23
4: **end for**
5: $\mathbf{credentialID} \leftarrow (\mathtt{credentialID}_0 \ldots, \mathtt{credentialID}_{\mathtt{N_E}-1})$
6: $\mathbf{hAuth} \leftarrow (\mathtt{hAuth}_0 \ldots, \mathtt{hAuth}_{\mathtt{N_E}-1})$

Output:

Vector of derived voter identifiers $\mathbf{credentialID} \in \left((\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}\right)^{\mathtt{N_E}}$

Vector of base authentication challenges $\mathbf{hAuth} \in \left((\mathbb{A}_{Base64})^{\mathtt{l_{HB64}}}\right)^{\mathtt{N_E}}$

Copyright 2026 Swiss Post Ltd.

After running the algorithms 4.1, 4.2 and 4.3, the setup component sends the following information to the Return Codes control components CCR.

- The list of verification card IDs $\mathbf{vc}$
- The vector of verification card public keys $\mathbf{K}$
- The vector of encrypted, hashed, squared partial Choice Return Codes $\mathbf{c}_{\mathtt{pCC}}$
- The list of encrypted, hashed, squared confirmation keys $\mathbf{c}_{\mathtt{ck}}$
- The partial Choice Return Codes allow list $\mathsf{L}_{\mathtt{pCC}}$

The amount of data increases with the number of voting options n and the number of eligible voters $\mathtt{N_E}$ of the verification card set; therefore, the implementation should divide the data into chunks to keep the size manageable.

4.1.4 GenKeysCCR

The Return Codes control components CCR generate the CCR_j Choice Return Codes encryption key pair $\mathbf{pk}_{\text{CCR}_j}, \mathbf{sk}_{\text{CCR}_j}$ to encrypt the partial Choice Return Codes $\mathbf{pCC}_{\mathtt{id}}$ during the voting phase and the CCR_j Return Codes Generation secret key $\mathtt{k'}_j$.

Algorithm 4.4 GenKeysCCR

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
The CCR's index $j \in [1, 4]$
Election event ID $\mathsf{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Maximum number of selections $\psi_{\mathtt{max}} \in [1, \psi_{\mathtt{sup}}]$

Operation: ▷ For all algorithms see the crypto primitives specification

1: $(\mathbf{pk}_{\text{CCR}_j}, \mathbf{sk}_{\text{CCR}_j}) \leftarrow \mathsf{GenKeyPair}(p, q, g, \psi_{\mathtt{max}})$
2: $\mathbf{i}_{\mathtt{aux}} \leftarrow (\mathsf{ee}, \text{“}\mathtt{GenKeysCCR}\text{”}, \mathsf{IntegerToString}(j))$
3: **for** $i \in [0, \psi_{\mathtt{max}})$ **do**
4: $\quad \mathbf{i}'_{\mathtt{aux}} \leftarrow \mathbf{i}_{\mathtt{aux}} \,||\, \mathsf{IntegerToString}(i)$
5: $\quad \pi_{\mathsf{pkCCR},j,i} \leftarrow \mathsf{GenSchnorrProof}(\mathtt{sk}_{\text{CCR}_{j,i}}, \mathtt{pk}_{\text{CCR}_{j,i}}, \mathbf{i}'_{\mathtt{aux}})$
6: **end for**
7: $\boldsymbol{\pi}_{\mathsf{pkCCR},j} \leftarrow (\pi_{\mathsf{pkCCR},j,0}, \ldots, \pi_{\mathsf{pkCCR},j,\psi_{\mathtt{max}}-1})$
8: $\mathtt{k'}_j \leftarrow \mathsf{GenRandomInteger}(q)$

Output:

CCR_j Choice Return Codes encryption public key $\mathbf{pk}_{\text{CCR}_j} = (\mathtt{pk}_{\text{CCR}_{j,0}}, \ldots, \mathtt{pk}_{\text{CCR}_{j,\psi_{\mathtt{max}}-1}}) \in \mathbb{G}_q^{\psi_{\mathtt{max}}}$
CCR_j Choice Return Codes encryption secret key $\mathbf{sk}_{\text{CCR}_j} = (\mathtt{sk}_{\text{CCR}_{j,0}}, \ldots, \mathtt{sk}_{\text{CCR}_{j,\psi_{\mathtt{max}}-1}}) \in \mathbb{Z}_q^{\psi_{\mathtt{max}}}$
CCR_j Return Codes Generation secret key $\mathtt{k'}_j \in \mathbb{Z}_q$
CCR_j Schnorr proofs of knowledge $\boldsymbol{\pi}_{\mathsf{pkCCR},j} = (\pi_{\mathsf{pkCCR},j,0}, \ldots, \pi_{\mathsf{pkCCR},j,\psi_{\mathtt{max}}-1}) \in (\mathbb{Z}_q \times \mathbb{Z}_q)^{\psi_{\mathtt{max}}}$

Copyright 2026 Swiss Post Ltd.

4.1.5 GenEncLongCodeShares

Each Return Codes control component CCR_j runs the GenEncLongCodeShares (Generate Encrypted Long Return Code Shares) to create shares of the long return codes. The main idea behind this algorithm is to derive a secret key per voter and to exponentiate the setup component's input. GenEncLongCodeShares is analogous to the voting phase's algorithms CreateLCCShare (See algorithm 5.8) and CreateLVCCShare (algorithm 5.11).
The Return Codes control component CCR_j derives two different keys from the CCR_j Return Codes Generation secret key $\mathtt{k}'_j$:

- Voter Choice Return Code Generation key pair $\mathtt{K}_{j,\mathtt{id}}, \mathtt{k}_{j,\mathtt{id}}$
- Voter Vote Cast Return Code Generation key pair $\mathtt{Kc}_{j,\mathtt{id}}, \mathtt{kc}_{j,\mathtt{id}}$

Using different keys for generating the control component's shares of the long Choice Return Codes and the long Vote Cast Return Codes prevents an attacker from abusing some confirmation attempts as a way to glean Choice Return Codes.
The control components store the list of processed voting cards $\mathsf{L}_{\mathtt{genVC},j}$, which is initialized to the empty list before the first execution of the algorithm.
Moreover, the control components initialize the following lists for all processed voting cards:

- list of voting cards $\mathsf{L}_{\mathtt{decPCC},j}$ for which the control components decrypted the partial Choice Return Codes $\mathbf{pCC}_{\mathtt{id}}$,
- list of voting cards $\mathsf{L}_{\mathtt{sentVotes},j}$ for which the control components already generated long Choice Return Code shares,
- list of confirmed votes $\mathsf{L}_{\mathtt{confirmedVotes},j}$,
- key-value map of number of confirmation attempts per voting card $\mathsf{L}_{\mathtt{confirmationAttempts},j}$; initially with all 0-values.

At the end of the SetupTallyEB algorithm of the SetupTally protocol, the setup component sends the public keys to the control components.
The control components verify that the election event context is consistent with the information processed in GenEncLongCodeShares. In particular, the control components check the following:

- The number of verification cards for each verification card set corresponds to the number of verification cards processed in GenEncLongCodeShares.
- The correctness of the setup and other control components' key generation by executing the VerifyKeyGenerationSchnorrProofs algorithm.

Copyright 2026 Swiss Post Ltd.

Algorithm 4.5 GenEncLongCodeShares

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
The CCR's index $j \in [1, 4]$
Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Verification card set ID $\mathtt{vcs} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Vector of verification card IDs $\mathbf{vc} = (\mathtt{vc}_0, \ldots, \mathtt{vc}_{\mathtt{N_E}-1}) \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}} \times \mathtt{N_E}}$
Number of voting options for this verification card set $n \in [1, n_{\mathsf{sup}}]$ ▷ Can be derived from pTable

Stateful Lists and Maps:

List of generated voting cards $\mathsf{L}_{\mathtt{genVC},j}$

Input:

CCR_j Return Codes Generation secret key $\mathtt{k'}_j \in \mathbb{Z}_q$
Vector of encrypted, hashed partial Choice Return Codes $\mathbf{c}_{\mathtt{pCC}} = (\mathtt{c}_{\mathtt{pCC},0}, \ldots, \mathtt{c}_{\mathtt{pCC},\mathtt{N_E}-1}) \in ({\mathbb{G}_q}^{n+1})^{\mathtt{N_E}}$
Vector of encrypted, hashed Confirmation Keys $\mathbf{c}_{\mathtt{ck}} = (\mathtt{c}_{\mathtt{ck},0}, \ldots, \mathtt{c}_{\mathtt{ck},\mathtt{N_E}-1}) \in ({\mathbb{G}_q}^{2})^{\mathtt{N_E}}$

Require: $\mathtt{vc_{id}} \notin \mathsf{L}_{\mathtt{genVC},j}\ \forall\ \mathtt{vc_{id}} \in \mathbf{vc}$

Operation: ▷ For all algorithms see the crypto primitives specification

1: $\mathtt{PRK} \leftarrow \mathsf{IntegerToByteArray}(\mathtt{k'}_j)$
2: **for** $\mathtt{id} \in [0, \mathtt{N_E})$ **do**
3: $\quad \mathtt{info} \leftarrow (\text{“}\mathtt{VoterChoiceReturnCodeGeneration}\text{”}, \mathtt{ee}, \mathtt{vcs}, \mathtt{vc_{id}})$
4: $\quad \mathtt{k}_{j,\mathtt{id}} \leftarrow \mathsf{KDFToZq}(\mathtt{PRK}, \mathtt{info}, q)$
5: $\quad \mathtt{K}_{j,\mathtt{id}} \leftarrow g^{\mathtt{k}_{j,\mathtt{id}}} \bmod p$
6: $\quad \mathtt{info_{CK}} \leftarrow (\text{“}\mathtt{VoterVoteCastReturnCodeGeneration}\text{”}, \mathtt{ee}, \mathtt{vcs}, \mathtt{vc_{id}})$
7: $\quad \mathtt{kc}_{j,\mathtt{id}} \leftarrow \mathsf{KDFToZq}(\mathtt{PRK}, \mathtt{info_{CK}}, q)$
8: $\quad \mathtt{Kc}_{j,\mathtt{id}} \leftarrow g^{\mathtt{kc}_{j,\mathtt{id}}} \bmod p$
9: $\quad \mathtt{c}_{\mathtt{expPCC},j,\mathtt{id}} \leftarrow \mathsf{GetCiphertextExponentiation}(\mathtt{c}_{\mathtt{pCC},\mathtt{id}}, \mathtt{k}_{j,\mathtt{id}})$
10: $\quad \mathbf{i}_{\mathtt{aux}} \leftarrow (\mathtt{ee}, \mathtt{vc_{id}}, \text{“}\mathtt{GenEncLongCodeShares}\text{”}, \mathsf{IntegerToString}(j))$
11: $\quad \pi_{\mathtt{expPCC},j,\mathtt{id}} \leftarrow \mathsf{GenExponentiationProof}((g, \mathtt{c}_{\mathtt{pCC},\mathtt{id}}), \mathtt{k}_{j,\mathtt{id}}, (\mathtt{K}_{j,\mathtt{id}}, \mathtt{c}_{\mathtt{expPCC},j,\mathtt{id}}), \mathbf{i}_{\mathtt{aux}})$
12: $\quad \mathtt{c}_{\mathtt{expCK},j,\mathtt{id}} \leftarrow \mathsf{GetCiphertextExponentiation}(\mathtt{c}_{\mathtt{ck},\mathtt{id}}, \mathtt{kc}_{j,\mathtt{id}})$
13: $\quad \pi_{\mathtt{expCK},j,\mathtt{id}} \leftarrow \mathsf{GenExponentiationProof}((g, \mathtt{c}_{\mathtt{ck},\mathtt{id}}), \mathtt{kc}_{j,\mathtt{id}}, (\mathtt{Kc}_{j,\mathtt{id}}, \mathtt{c}_{\mathtt{expCK},j,\mathtt{id}}), \mathbf{i}_{\mathtt{aux}})$
14: $\quad \mathsf{L}_{\mathtt{genVC},j} \leftarrow \mathsf{L}_{\mathtt{genVC},j} \,||\, \mathtt{vc_{id}}$
15: **end for**

Output:

Vector of Voter Choice Return Code Generation public keys $\mathbf{K}_j = (\mathtt{K}_{j,0}, \ldots, \mathtt{K}_{j,\mathtt{N_E}-1}) \in {\mathbb{G}_q}^{\mathtt{N_E}}$
Vector of Voter Vote Cast Return Code Generation public keys $\mathbf{Kc}_j = (\mathtt{Kc}_{j,0}, \ldots, \mathtt{Kc}_{j,\mathtt{N_E}-1}) \in {\mathbb{G}_q}^{\mathtt{N_E}}$
Vector of exponentiated, encrypted, hashed partial Choice Return Codes $\mathbf{c}_{\mathtt{expPCC},j} = (\mathtt{c}_{\mathtt{expPCC},j,0}, \ldots, \mathtt{c}_{\mathtt{expPCC},j,\mathtt{N_E}-1}) \in ({\mathbb{G}_q}^{n+1})^{\mathtt{N_E}}$
Proofs of correct exponentiation of the partial Choice Return Codes $\boldsymbol{\pi}_{\mathtt{expPCC},j} = (\pi_{\mathtt{expPCC},j,0}, \ldots, \pi_{\mathtt{expPCC},j,\mathtt{N_E}-1}) \in (\mathbb{Z}_q \times \mathbb{Z}_q)^{\mathtt{N_E}}$
Vector of exponentiated, encrypted, hashed Confirmation Keys $\mathbf{c}_{\mathtt{expCK},j} = (\mathtt{c}_{\mathtt{expCK},j,0}, \ldots, \mathtt{c}_{\mathtt{expCK},j,\mathtt{N_E}-1}) \in ({\mathbb{G}_q}^{2})^{\mathtt{N_E}}$
Proofs of correct exponentiation of the Confirmation Keys $\boldsymbol{\pi}_{\mathtt{expCK},j} = (\pi_{\mathtt{expCK},j,0}, \ldots, \pi_{\mathtt{expCK},j,\mathtt{N_E}-1}) \in (\mathbb{Z}_q \times \mathbb{Z}_q)^{\mathtt{N_E}}$

Subsequently, the control components send their encrypted return code shares back to the setup component. Upon reception, the setup component checks the consistency of the received

Copyright 2026 Swiss Post Ltd.

information, notably that the number of encrypted return code shares corresponds to the correct number of verification cards.
The verification of the control components' exponentiation proofs is deliberately placed after the generation of the Return Codes in the GenCMTable algorithm, allowing the canton to operationally parallelize the zero-knowledge proof verification with other operations, since verifying all exponentiation proofs for every voter and every control component is time-consuming.
The subsequent steps of the configuration phase, in particular GenVerCardSetKeys and GenCredDat, can only proceed if the verification in VerifyCCRExponentiationProofs (see section 4.1.8) completes successfully. If any zero-knowledge proof verification fails, the setup component aborts. In that case, the canton shall launch a forensic investigation to determine the cause of the failure and restart the setup of the election event from scratch.

Copyright 2026 Swiss Post Ltd.

4.1.6 CombineEncLongCodeShares

The setup component combines the control components' encrypted long return code shares. Moreover, the setup component generates the long Vote Cast Return Codes allow list $\mathsf{L}_{\mathtt{lVCC}}$ that is used during the voting phase in the algorithm 5.12.

Algorithm 4.6 CombineEncLongCodeShares

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Election event ID $\mathsf{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Verification card set ID $\mathtt{vcs} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Vector of verification card IDs $\mathbf{vc} = (\mathtt{vc}_0, \ldots, \mathtt{vc}_{\mathtt{N_E}-1}) \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}} \times \mathtt{N_E}}$
Number of voting options for this verification card set $n \in [1, n_{\mathtt{max}}]$ ▷ Can be derived from pTable
Maximum number of voting options $n_{\mathtt{max}} \in [1, n_{\mathtt{sup}}]$

Input:

Setup secret key $\mathsf{sk}_{\mathtt{setup}} \in \mathbb{Z}_q^{n_{\mathtt{max}}}$
Vectors of exponentiated, encrypted, hashed partial Choice Return Codes $\mathbf{C}_{\mathtt{expPCC}} = (\mathbf{c}_{\mathtt{expPCC},1}, \ldots, \mathbf{c}_{\mathtt{expPCC},4}) \in \left((\mathbb{G}_q{}^{n+1})^{\mathtt{N_E}}\right)^4$
Vectors of exponentiated, encrypted, hashed Confirmation Keys $\mathbf{C}_{\mathtt{expCK}} = (\mathbf{c}_{\mathtt{expCK},1}, \ldots, \mathbf{c}_{\mathtt{expCK},4}) \in \left((\mathbb{G}_q{}^{2})^{\mathtt{N_E}}\right)^4$

Operation: ▷ For all algorithms see the crypto primitives specification

1: $\mathsf{L}_{\mathtt{lVCC}} \leftarrow ()$
2: **for** $\mathtt{id} \in [0, \mathtt{N_E})$ **do**
3: $\quad \mathsf{c}_{\mathtt{pC,id}} \leftarrow (1, \{1\}^n)$ ▷ Neutral element of ciphertext multiplication
4: $\quad$ **for** $j \in [1, 4]$ **do**
5: $\quad\quad \mathsf{c}_{\mathtt{pC,id}} \leftarrow \mathsf{GetCiphertextProduct}(\mathsf{c}_{\mathtt{pC,id}}, \mathsf{c}_{\mathtt{expPCC},j,\mathtt{id}})$
6: $\quad\quad \mathtt{lVCC}_{j,\mathtt{id}} \leftarrow \mathsf{GetMessage}(\mathsf{c}_{\mathtt{expCK},j,\mathtt{id}}, \mathsf{sk}_{\mathtt{setup}})$
7: $\quad\quad \mathbf{i}_{\mathtt{aux},1} \leftarrow (\text{“}\mathtt{CreateLVCCShare}\text{”}, \mathsf{ee}, \mathtt{vcs}, \mathtt{vc}_{\mathtt{id}}, \mathsf{IntegerToString}(j))$
8: $\quad\quad \mathtt{hlVCC}_{j,\mathtt{id}} \leftarrow \mathsf{Base64Encode}(\mathsf{RecursiveHash}(\mathbf{i}_{\mathtt{aux},1}, \mathtt{lVCC}_{j,\mathtt{id}}))$
9: $\quad$ **end for**
10: $\quad \mathtt{pVCC}_{\mathtt{id}} \leftarrow \prod_{j=1}^{4} \mathtt{lVCC}_{j,\mathtt{id}} \bmod p$
11: $\quad \mathbf{i}_{\mathtt{aux},2} \leftarrow (\text{“}\mathtt{VerifyLVCCHash}\text{”}, \mathsf{ee}, \mathtt{vcs}, \mathtt{vc}_{\mathtt{id}})$
12: $\quad \mathtt{hhlVCC}_{\mathtt{id}} \leftarrow \mathsf{Base64Encode}(\mathsf{RecursiveHash}(\mathbf{i}_{\mathtt{aux},2}, \mathtt{hlVCC}_{1,\mathtt{id}}, \mathtt{hlVCC}_{2,\mathtt{id}}, \mathtt{hlVCC}_{3,\mathtt{id}}, \mathtt{hlVCC}_{4,\mathtt{id}}))$
13: $\quad \mathsf{L}_{\mathtt{lVCC}} \leftarrow \mathsf{L}_{\mathtt{lVCC}} \,||\, \mathtt{hhlVCC}_{\mathtt{id}}$
14: **end for**

Output:

Vector of encrypted pre-Choice Return Codes $\mathbf{c}_{\mathtt{pC}} = (\mathsf{c}_{\mathtt{pC},0}, \ldots, \mathsf{c}_{\mathtt{pC},\mathtt{N_E}-1}) \in (\mathbb{G}_q{}^{n+1})^{\mathtt{N_E}}$
Pre-Vote Cast Return Codes $\mathbf{pVCC} = (\mathtt{pVCC}_0, \ldots, \mathtt{pVCC}_{\mathtt{N_E}-1}) \in (\mathbb{G}_q)^{\mathtt{N_E}}$
Long Vote Cast Return Codes allow list $\mathsf{L}_{\mathtt{lVCC}} \in (\mathbb{A}_{Base64}{}^{\mathtt{l_{HB64}}})^{\mathtt{N_E}}$

Copyright 2026 Swiss Post Ltd.

4.1.7 GenCMTable

Subsequently, the setup component generates the Return Codes Mapping table CMtable that allows the control components and the voting server to retrieve the short Choice Return Codes $\mathbf{CC}_{\mathtt{id}}$. At the end of the algorithm, we order the Return Codes Mapping table CMtable by the hash of the long Return Codes to break the correlation between the voting option and the order of insertion.
Moreover, the setup component generates Abstention Choice Return Codes (see Section 3.9.1) for each abstention group.
The generated `CMtable` is a key-value map with keys $\mathtt{key} \in {\mathbb{A}_{Base64}}^{\mathtt{l_{HB64}}}$ and corresponding values $\mathtt{value} \in {\mathbb{A}_{Base64}}^{*}$

Copyright 2026 Swiss Post Ltd.

Algorithm 4.7 GenCMTable

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Vector of verification card IDs $\mathbf{vc} = (\mathtt{vc}_0, \ldots, \mathtt{vc}_{\mathtt{N_E}-1}) \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}} \times \mathtt{N_E}}$
Primes mapping table $\mathsf{pTable} \in (\mathcal{D}_\mathrm{v} \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times {\mathbb{A}_{UCS}}^* \times \mathcal{D}_\tau)^n$ ▷ pTable is of the form $((\mathrm{v}_0, \tilde{\mathrm{p}}_0, \sigma_0, \tau_0), \ldots, (\mathrm{v}_{n-1}, \tilde{\mathrm{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1}))$
Maximum number of voting options $n_{\mathtt{max}} \in [1, n_{\mathtt{sup}}]$
Presentation groups $\mathbf{pg} = (\mathtt{pg}_0, \ldots, \mathtt{pg}_{t-1}) \in [0, \eta)^t$
Abstention groups $\mathbf{ag} = (\mathtt{ag}_0, \ldots, \mathtt{ag}_{\eta-1}) \in \mathbb{Z}_2^\eta$

Input:

Setup secret key $\mathsf{sk_{setup}} \in \mathbb{Z}_q^{n_{\mathtt{max}}}$
Vector of encrypted pre-Choice Return Codes $\mathbf{c_{pC}} = (\mathsf{c_{pC},0}, \ldots, \mathsf{c_{pC},N_E-1}) \in ({\mathbb{G}_q}^{n+1})^{\mathtt{N_E}}$
Pre-Vote Cast Return Codes $\mathbf{pVCC} = (\mathtt{pVCC}_0, \ldots, \mathtt{pVCC}_{\mathtt{N_E}-1}) \in (\mathbb{G}_q)^{\mathtt{N_E}}$

Operation: ▷ For all algorithms see the crypto primitives specification

1: $\mathsf{CMtable} \leftarrow ()$
2: **for** $\mathtt{id} \in [0, \mathtt{N_E})$ **do**
3: $\quad \mathbf{CC}_{\mathtt{id}} \leftarrow \mathsf{GenUniqueDecimalStrings}(\mathtt{l_{CC}}, n)$
4: $\quad \mathbf{pC}_{\mathtt{id}} \leftarrow \mathsf{GetMessage}(\mathsf{c_{pC,id}}, \mathsf{sk_{setup}})$
5: $\quad$ **for** $i \in [0, \eta)$ **do**
6: $\quad\quad \mathbf{CC}_{\mathtt{id,abstention},i} \leftarrow ()$
7: $\quad$ **end for**
8: $\quad$ **for** $k \in [0, n)$ **do**
9: $\quad\quad \mathtt{lCC}_{\mathtt{id},k} \leftarrow \mathsf{RecursiveHash}(\mathsf{pC}_{\mathtt{id},k}, \mathtt{vc}_{\mathtt{id}}, \mathtt{ee}, \tau_k)$
10: $\quad\quad \mathtt{skcc}_{\mathtt{id},k} \leftarrow \mathsf{KDF}(\mathtt{lCC}_{\mathtt{id},k}, (), \mathtt{l_{KD}})$
11: $\quad\quad (\mathtt{ctCC}_{\mathtt{id},k,\mathtt{ciphertext}}, \mathtt{ctCC}_{\mathtt{id},k,\mathtt{nonce}}) \leftarrow \mathsf{GenCiphertextSymmetric}(\mathtt{skcc}_{\mathtt{id},k}, \mathsf{StringToByteArray}(\mathtt{CC}_{\mathtt{id},k}), ())$
12: $\quad\quad \mathsf{CMtable} \leftarrow \mathsf{CMtable} \,||\, (\mathsf{Base64Encode}(\mathsf{RecursiveHash}(\mathtt{lCC}_{\mathtt{id},k})), \mathsf{Base64Encode}(\mathtt{ctCC}_{\mathtt{id},k,\mathtt{ciphertext}} || \mathtt{ctCC}_{\mathtt{id},k,\mathtt{nonce}}))$
13: $\quad\quad \mathtt{pg} \leftarrow \mathsf{GetPresentationGroup}(\mathrm{v}_k)$ ▷ See algorithm 3.10
14: $\quad\quad$ **if** $\mathtt{pg} \neq -1$ **then**
15: $\quad\quad\quad \mathbf{CC}_{\mathtt{id,abstention,pg}} \leftarrow \mathbf{CC}_{\mathtt{id,abstention,pg}} \,||\, \mathtt{CC}_{\mathtt{id},k}$
16: $\quad\quad$ **end if**
17: $\quad$ **end for**
18: $\quad \mathtt{lVCC}_{\mathtt{id}} \leftarrow \mathsf{RecursiveHash}(\mathtt{pVCC}_{\mathtt{id}}, \mathtt{vc}_{\mathtt{id}}, \mathtt{ee})$
19: $\quad \mathtt{VCC}_{\mathtt{id}} \leftarrow \mathsf{GenUniqueDecimalStrings}(\mathtt{l_{VCC}}, 1)$ ▷ Assign first element of list to $\mathtt{VCC}_{\mathtt{id}}$
20: $\quad \mathtt{skvcc}_{\mathtt{id}} \leftarrow \mathsf{KDF}(\mathtt{lVCC}_{\mathtt{id}}, (), \mathtt{l_{KD}})$
21: $\quad (\mathtt{ctVCC}_{\mathtt{id,ciphertext}}, \mathtt{ctVCC}_{\mathtt{id,nonce}}) \leftarrow \mathsf{GenCiphertextSymmetric}(\mathtt{skvcc}_{\mathtt{id}}, \mathsf{StringToByteArray}(\mathtt{VCC}_{\mathtt{id}}), ())$
22: $\quad \mathsf{CMtable} \leftarrow \mathsf{CMtable} \,||\, (\mathsf{Base64Encode}(\mathsf{RecursiveHash}(\mathtt{lVCC}_{\mathtt{id}})), \mathsf{Base64Encode}(\mathtt{ctVCC}_{\mathtt{id,ciphertext}} || \mathtt{ctVCC}_{\mathtt{id,nonce}}))$
23: $\quad \mathbf{ACC}_{\mathtt{id}} \leftarrow ()$
24: $\quad$ **for** $i \in [0, \eta)$ **do**
25: $\quad\quad$ **if** $\mathtt{ag}_i = 1$ **then**
26: $\quad\quad\quad \mathtt{mACC} \leftarrow \mathsf{GetMergedString}(\mathbf{CC}_{\mathtt{id,abstention},i}, \mathbb{A}_{10})$
27: $\quad\quad\quad \xi_i \leftarrow \mathsf{IntegerToString}(i + 10)$
28: $\quad\quad\quad \mathtt{ACC}_i \leftarrow \mathtt{mACC} \,||\, \xi_i$
29: $\quad\quad\quad \mathbf{ACC}_{\mathtt{id}} \leftarrow \mathbf{ACC}_{\mathtt{id}} \,||\, \mathtt{ACC}_i$
30: $\quad\quad$ **end if**
31: $\quad$ **end for**
32: **end for**
33: $\mathsf{Order}(\mathsf{CMtable}, 1)$ ▷ Lexicographic ordering by the table's first column (containing the encoded hash of the long Return Codes) ensures the unlinkability to the order of insertion.

Output:

Return Codes Mapping table $\mathsf{CMtable} \in ({\mathbb{A}_{Base64}}^{\mathtt{l_{HB64}}} \times {\mathbb{A}_{Base64}}^*)^{\mathtt{N_E} \cdot (n+1)}$
Vector of short Choice Return Codes $(\mathbf{CC}_0, \ldots, \mathbf{CC}_{\mathtt{N_E}-1}) \in (({\mathbb{A}_{10}}^{\mathtt{l_{CC}}})^n)^{\mathtt{N_E}}$
Vector of short Vote Cast Return Codes $(\mathtt{VCC}_0, \ldots, \mathtt{VCC}_{\mathtt{N_E}-1}) \in ({\mathbb{A}_{10}}^{\mathtt{l_{VCC}}})^{\mathtt{N_E}}$
Abstention Choice Return Codes $(\mathbf{ACC}_0, \ldots, \mathbf{ACC}_{\mathtt{N_E}-1}) \in (({\mathbb{A}_{10}}^{\mathtt{l_{ACC}}})^\kappa)^{\mathtt{N_E}}$

Copyright 2026 Swiss Post Ltd.

4.1.8 VerifyCCRExponentiationProofs

The setup component verifies the control components' exponentiation proofs.

Algorithm 4.8 VerifyCCRExponentiationProofs

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Vector of verification card IDs $\mathbf{vc} = (\mathtt{vc}_0, \ldots, \mathtt{vc}_{\mathtt{N_E}-1}) \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}} \times \mathtt{N_E}}$
Number of voting options for this verification card set $n \in [1, n_{\mathtt{max}}]$ ▷ Can be derived from pTable
Maximum number of voting options $n_{\mathtt{max}} \in [1, n_{\mathtt{sup}}]$

Input:

Vector of encrypted, hashed partial Choice Return Codes $\mathbf{c}_{\mathtt{pCC}} = (\mathtt{c}_{\mathtt{pCC},0}, \ldots, \mathtt{c}_{\mathtt{pCC},\mathtt{N_E}-1}) \in ({\mathbb{G}_q}^{n+1})^{\mathtt{N_E}}$
Vector of encrypted, hashed Confirmation Keys $\mathbf{c}_{\mathtt{ck}} = (\mathtt{c}_{\mathtt{ck},0}, \ldots, \mathtt{c}_{\mathtt{ck},\mathtt{N_E}-1}) \in ({\mathbb{G}_q}^{2})^{\mathtt{N_E}}$
Control components' vectors of Voter Choice Return Code Generation public keys $(\mathbf{K}_1, \mathbf{K}_2, \mathbf{K}_3, \mathbf{K}_4) \in ({\mathbb{G}_q}^{\mathtt{N_E}})^4$
Control components' vectors of Voter Vote Cast Return Code Generation public keys $(\mathbf{Kc}_1, \mathbf{Kc}_2, \mathbf{Kc}_3, \mathbf{Kc}_4) \in ({\mathbb{G}_q}^{\mathtt{N_E}})^4$
Vectors of exponentiated, encrypted, hashed partial Choice Return Codes $\mathbf{C}_{\mathtt{expPCC}} = (\mathbf{c}_{\mathtt{expPCC},1}, \ldots, \mathbf{c}_{\mathtt{expPCC},4}) \in \left((\mathbb{G}_q^{n+1})^{\mathtt{N_E}}\right)^4$
Control components' proofs of correct exponentiation of the partial Choice Return Codes $(\boldsymbol{\pi}_{\mathtt{expPCC},1}, \boldsymbol{\pi}_{\mathtt{expPCC},2}, \boldsymbol{\pi}_{\mathtt{expPCC},3}, \boldsymbol{\pi}_{\mathtt{expPCC},4}) \in \left((\mathbb{Z}_q \times \mathbb{Z}_q)^{\mathtt{N_E}}\right)^4$
Vectors of exponentiated, encrypted, hashed Confirmation Keys $\mathbf{C}_{\mathtt{expCK}} = (\mathbf{c}_{\mathtt{expCK},1}, \ldots, \mathbf{c}_{\mathtt{expCK},4}) \in \left(({\mathbb{G}_q}^{2})^{\mathtt{N_E}}\right)^4$
Control components' proofs of correct exponentiation of the Confirmation Keys $(\boldsymbol{\pi}_{\mathtt{expCK},1}, \boldsymbol{\pi}_{\mathtt{expCK},2}, \boldsymbol{\pi}_{\mathtt{expCK},3}, \boldsymbol{\pi}_{\mathtt{expCK},4}) \in \left((\mathbb{Z}_q \times \mathbb{Z}_q)^{\mathtt{N_E}}\right)^4$

Operation:

1: **for** $\mathtt{id} \in [0, \mathtt{N_E})$ **do**
2: $\quad \mathbf{K}_{\mathtt{CC},\mathtt{id}} \leftarrow (\mathtt{K}_{1,\mathtt{id}}, \ldots, \mathtt{K}_{4,\mathtt{id}})$
3: $\quad \mathbf{c}_{\mathtt{expPCC},\mathtt{CC},\mathtt{id}} \leftarrow (\mathtt{c}_{\mathtt{expPCC},1,\mathtt{id}}, \ldots, \mathtt{c}_{\mathtt{expPCC},4,\mathtt{id}})$
4: $\quad \boldsymbol{\pi}_{\mathtt{expPCC},\mathtt{CC},\mathtt{id}} \leftarrow (\pi_{\mathtt{expPCC},1,\mathtt{id}}, \ldots, \pi_{\mathtt{expPCC},4,\mathtt{id}})$
5: $\quad \mathtt{piExpPccVerif}_{\mathtt{id}} \leftarrow$ VerifyEncryptedPCCExponentiationProofs$(\mathtt{c}_{\mathtt{pCC},\mathtt{id}}, \mathbf{K}_{\mathtt{CC},\mathtt{id}}, \mathbf{c}_{\mathtt{expPCC},\mathtt{CC},\mathtt{id}}, \boldsymbol{\pi}_{\mathtt{expPCC},\mathtt{CC},\mathtt{id}})$ ▷ See algorithm 4.9
6: $\quad \mathbf{Kc}_{\mathtt{CC},\mathtt{id}} \leftarrow (\mathtt{Kc}_{1,\mathtt{id}}, \ldots, \mathtt{Kc}_{4,\mathtt{id}})$
7: $\quad \mathbf{c}_{\mathtt{expCK},\mathtt{CC},\mathtt{id}} \leftarrow (\mathtt{c}_{\mathtt{expCK},1,\mathtt{id}}, \ldots, \mathtt{c}_{\mathtt{expCK},4,\mathtt{id}})$
8: $\quad \boldsymbol{\pi}_{\mathtt{expCK},\mathtt{CC},\mathtt{id}} \leftarrow (\pi_{\mathtt{expCK},1,\mathtt{id}}, \ldots, \pi_{\mathtt{expCK},4,\mathtt{id}})$
9: $\quad \mathtt{piExpCkVerif}_{\mathtt{id}} \leftarrow$ VerifyEncryptedCKExponentiationProofs$(\mathtt{c}_{\mathtt{ck},\mathtt{id}}, \mathbf{Kc}_{\mathtt{CC},\mathtt{id}}, \mathbf{c}_{\mathtt{expCK},\mathtt{CC},\mathtt{id}}, \boldsymbol{\pi}_{\mathtt{expCK},\mathtt{CC},\mathtt{id}})$ ▷ See algorithm 4.10
10: $\quad$ **if** $\neg \mathtt{piExpPccVerif}_{\mathtt{id}} \vee \neg \mathtt{piExpCkVerif}_{\mathtt{id}}$ **then**
11: $\quad\quad$ **return** $\bot$
12: $\quad$ **end if**
13: **end for**

Output:

The result of the verification: $\top$ if the verification is successful for *all* verification cards, $\bot$ otherwise.

If any of the verifications in algorithm 4.8 (VerifyCCRExponentiationProofs) fails, the whole algorithm fails, the setup component aborts, and the subsequent steps of the configuration phase are not executed. The verifications in Algorithm 4.8 rely on the following algorithms.

Copyright 2026 Swiss Post Ltd.

Algorithm 4.9 VerifyEncryptedPCCExponentiationProofs

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Verification card ID $\mathtt{vc_{id}} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Number of voting options for this verification card set $n \in [1, n_{\mathtt{sup}}]$

Input:

Encrypted, hashed partial Choice Return Codes $\mathtt{c_{pCC,id}} \in {\mathbb{G}_q}^{n+1}$
Control components' Voter Choice Return Code Generation public keys $(\mathtt{K_{1,id}}, \ldots, \mathtt{K_{4,id}}) \in {\mathbb{G}_q}^4$
Control components' exponentiated, encrypted, hashed partial Choice Return Codes $(\mathtt{c_{expPCC,1,id}}, \ldots, \mathtt{c_{expPCC,4,id}}) \in ({\mathbb{G}_q}^{n+1})^4$
Control components' proofs of correct exponentiation $(\pi_{\mathtt{expPCC,1,id}}, \ldots, \pi_{\mathtt{expPCC,4,id}}) \in (\mathbb{Z}_q \times \mathbb{Z}_q)^4$

Operation:

1: $\mathbf{g} \leftarrow (g, \mathtt{c_{pCC,id}})$ ▷ The bases shared by all exponentiation proofs
2: **for** $j \in [1, 4]$ **do**
3: $\quad \mathbf{y} \leftarrow (\mathtt{K}_{j,\mathtt{id}}, \mathtt{c_{expPCC},}_{j,\mathtt{id}})$ ▷ The exponentiations
4: $\quad \mathbf{i}_{\mathtt{aux}} \leftarrow (\mathtt{ee}, \mathtt{vc_{id}}, \text{"}\mathtt{GenEncLongCodeShares}\text{"}, \mathsf{IntegerToString}(j))$ ▷ See crypto primitives specification
5: $\quad \mathsf{exponentiationVerif}_j \leftarrow \mathsf{VerifyExponentiation}(\mathbf{g}, \mathbf{y}, \pi_{\mathtt{expPCC},j,\mathtt{id}}, \mathbf{i}_{\mathtt{aux}})$ ▷ See crypto primitives specification
6: **end for**
7: **if** $\mathsf{exponentiationVerif}_j \ \forall \ j$ **then**
8: $\quad$ **return** $\top$
9: **else**
10: $\quad$ **return** $\bot$
11: **end if**

Output:

The result of the verification: $\top$ if the verification is successful for *this specific* voter, $\bot$ otherwise.

Copyright 2026 Swiss Post Ltd.

Algorithm 4.10 VerifyEncryptedCKExponentiationProofs

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Verification card ID $\mathtt{vc_{id}} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$

Input:

Encrypted, hashed Confirmation Key $\mathtt{c_{ck,id}} \in {\mathbb{G}_q}^2$
Control components' Voter Vote Cast Return Code Generation public keys $(\mathtt{Kc}_{1,\mathtt{id}}, \ldots, \mathtt{Kc}_{4,\mathtt{id}}) \in {\mathbb{G}_q}^4$
Control components' exponentiated, encrypted, hashed Confirmation Key $(\mathtt{c}_{\mathtt{expCK},1,\mathtt{id}}, \ldots, \mathtt{c}_{\mathtt{expCK},4,\mathtt{id}}) \in ({\mathbb{G}_q}^2)^4$
Control components' proofs of correct exponentiation $(\pi_{\mathtt{expCK},1,\mathtt{id}}, \ldots, \pi_{\mathtt{expCK},4,\mathtt{id}}) \in (\mathbb{Z}_q \times \mathbb{Z}_q)^4$

Operation:

1: $\mathbf{g} \leftarrow (g, \mathtt{c_{ck,id}})$ ▷ The bases shared by all exponentiation proofs
2: **for** $j \in [1, 4]$ **do**
3: $\quad \mathbf{y} \leftarrow (\mathtt{Kc}_{j,\mathtt{id}}, \mathtt{c}_{\mathtt{expCK},j,\mathtt{id}})$ ▷ The exponentiations
4: $\quad \mathbf{i}_{\mathtt{aux}} \leftarrow (\mathtt{ee}, \mathtt{vc_{id}}, \text{"}\mathtt{GenEncLongCodeShares}\text{"}, \mathsf{IntegerToString}(j))$ ▷ See crypto primitives specification
5: $\quad \mathsf{exponentiationVerif}_j \leftarrow \mathsf{VerifyExponentiation}(\mathbf{g}, \mathbf{y}, \pi_{\mathtt{expCK},j,\mathtt{id}}, \mathbf{i}_{\mathtt{aux}})$ ▷ See crypto primitives specification
6: **end for**
7: **if** $\mathsf{exponentiationVerif}_j \ \forall\ j$ **then**
8: $\quad$ **return** $\top$
9: **else**
10: $\quad$ **return** $\bot$
11: **end if**

Output:

The result of the verification: $\top$ if the verification is successful for *this specific* voter, $\bot$ otherwise.

Copyright 2026 Swiss Post Ltd.

4.1.9 GenVerCardSetKeys

The setup component generates the verification card set keys by combining the CCR Choice Return Codes encryption public keys.

Algorithm 4.11 GenVerCardSetKeys

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Election event ID $\mathsf{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Maximum number of selections $\psi_{\mathtt{max}} \in [1, \psi_{\mathtt{sup}}]$

Input:

CCR Choice Return Codes encryption public keys $\mathbf{pk}_{\text{CCR}} = ((\mathtt{pk}_{\text{CCR}_{1,0}}, \ldots, \mathtt{pk}_{\text{CCR}_{1,\psi_{\mathtt{max}}-1}}), \ldots, (\mathtt{pk}_{\text{CCR}_{4,0}}, \ldots, \mathtt{pk}_{\text{CCR}_{4,\psi_{\mathtt{max}}-1}})) \in (\mathbb{G}_q^{\psi_{\mathtt{max}}})^4$
CCR Schnorr proofs of knowledge $\boldsymbol{\pi}_{\mathsf{pkCCR}} = (\boldsymbol{\pi}_{\mathsf{pkCCR},1}, \boldsymbol{\pi}_{\mathsf{pkCCR},2}, \boldsymbol{\pi}_{\mathsf{pkCCR},3}, \boldsymbol{\pi}_{\mathsf{pkCCR},4}) \in \Big((\mathbb{Z}_q \times \mathbb{Z}_q)^{\psi_{\mathtt{max}}}\Big)^4$

Operation:

1: $\mathsf{VerifSch} \leftarrow \mathsf{VerifyCCSchnorrProofs}(\mathbf{pk}_{\text{CCR}}, \boldsymbol{\pi}_{\mathsf{pkCCR}}, (\mathsf{ee}, \text{“}\mathtt{GenKeysCCR}\text{”}))$ ▷ See algorithm 3.21
2: **if** $\neg\mathsf{VerifSch}$ **then**
3: **return** $\perp$ ▷ If any proof fails, the algorithm aborts and returns nothing
4: **end if**
5: $\mathbf{pk}_{\text{CCR}} \leftarrow \mathsf{CombinePublicKeys}\Big((\mathbf{pk}_{\text{CCR}_1}, \mathbf{pk}_{\text{CCR}_2}, \mathbf{pk}_{\text{CCR}_3}, \mathbf{pk}_{\text{CCR}_4})\Big)$ ▷ See crypto primitives specification

Output:

Choice Return Codes encryption public key $\mathbf{pk}_{\text{CCR}} \in \mathbb{G}_q^{\psi_{\mathtt{max}}}$

Copyright 2026 Swiss Post Ltd.

4.1.10 GenCredDat

The setup component generates the voter's credential data in the algorithm GenCredDat.

Algorithm 4.12 GenCredDat

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Verification card set ID $\mathtt{vcs} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Vector of verification card IDs $\mathbf{vc} = (\mathtt{vc}_0, \ldots, \mathtt{vc}_{\mathtt{N_E}-1}) \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}} \times \mathtt{N_E}}$
Primes mapping table $\mathsf{pTable} \in (\mathcal{D}_{\mathrm{v}} \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times {\mathbb{A}_{UCS}}^* \times \mathcal{D}_{\tau})^n$ ▷ pTable is of the form $((\mathrm{v}_0, \tilde{\mathrm{p}}_0, \sigma_0, \tau_0), \ldots, (\mathrm{v}_{n-1}, \tilde{\mathrm{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1}))$
Electoral Model Context $\Lambda = (\boldsymbol{\psi}, \mathbf{DoI}, \mathbf{pg}, \mathbf{ag}) \in \left((\mathbb{N}^+)^t \times (\mathcal{T}_1^{50})^k \times [0, \eta)^t \times \mathbb{Z}_2^{\eta}\right)$
Election public key $\mathtt{EL_{pk}} = (\mathtt{EL_{pk},0}, \ldots, \mathtt{EL_{pk},}\delta_{\mathtt{max}}-1) \in \mathbb{G}_q^{\delta_{\mathtt{max}}}$
Choice Return Codes encryption public key $\mathbf{pk}_{\mathrm{CCR}} = (\mathtt{pk}_{\mathrm{CCR},0}, \ldots, \mathtt{pk}_{\mathrm{CCR},\psi_{\mathtt{max}}-1}) \in \mathbb{G}_q^{\psi_{\mathtt{max}}}$

Input:

Vector of verification card secret keys $\mathbf{k} = (\mathtt{k}_0, \ldots, \mathtt{k}_{\mathtt{N_E}-1}) \in {\mathbb{Z}_q}^{\mathtt{N_E}}$
Vector of Start Voting Keys $\mathbf{SVK} = (\mathtt{SVK}_0, \ldots, \mathtt{SVK}_{\mathtt{N_E}-1}) \in ((\mathbb{A}_{\mathtt{u32}})^{\mathtt{l_{SVK}}})^{\mathtt{N_E}}$

Operation: ▷ For all algorithms see the crypto primitives specification

1: $\mathbf{i}_{\mathtt{aux}} \leftarrow$ ("GetKey", GetHashContext()) ▷ See algorithm 3.15
2: **for** $\mathtt{id} \in [0, \mathtt{N_E})$ **do**
3: $(\mathtt{dSVK_{id}}, \mathtt{VCks_{id,salt}}) \leftarrow$ GenArgon2id(StringToByteArray($\mathtt{SVK_{id}}$)) ▷ Use the Argon2 profile for less memory, as defined in the crypto primitives specification
4: $\mathtt{KSkey_{id}} \leftarrow$ RecursiveHash("VerificationCardKeystore", ee, vcs, $\mathtt{vc_{id}}$, $\mathtt{dSVK_{id}}$)
5: $\mathtt{k_{id,bytes}} \leftarrow$ IntegerToFixedLengthByteArray($\mathtt{k_{id}}$, $\left\lceil \frac{|q|}{8} \right\rceil$)
6: $(\mathtt{VCks_{id,ciphertext}}, \mathtt{VCks_{id,nonce}}) \leftarrow$ GenCiphertextSymmetric($\mathtt{KSkey_{id}}, \mathtt{k_{id,bytes}}, \mathbf{i}_{\mathtt{aux}}$)
7: $\mathtt{VCks_{id}} \leftarrow$ Base64Encode($\mathtt{VCks_{id,ciphertext}}||\mathtt{VCks_{id,nonce}}||\mathtt{VCks_{id,salt}}$)
8: **end for**

Output:

Vector of verification card keystores $\mathbf{VCks} = (\mathtt{VCks}_0, \ldots, \mathtt{VCks}_{\mathtt{N_E}-1}) \in (\mathbb{A}_{Base64}{}^{\mathtt{l_{VCKS}}})^{\mathtt{N_E}}$

Copyright 2026 Swiss Post Ltd.

4.2 SetupTally

The SetupTally protocol sets up the key to encrypt the actual vote. SetupTally consists of the SetupTallyCCM and SetupTallyEB algorithm. The SetupTally algorithms integrate the election event context in the challenges of the zero-knowledge proofs, enabling the other control components and the auditors' verifier to confirm that the control components have a consistent view of the election event context (see section 3.6.5). Figure 8 shows the relevant algorithms and data flows.

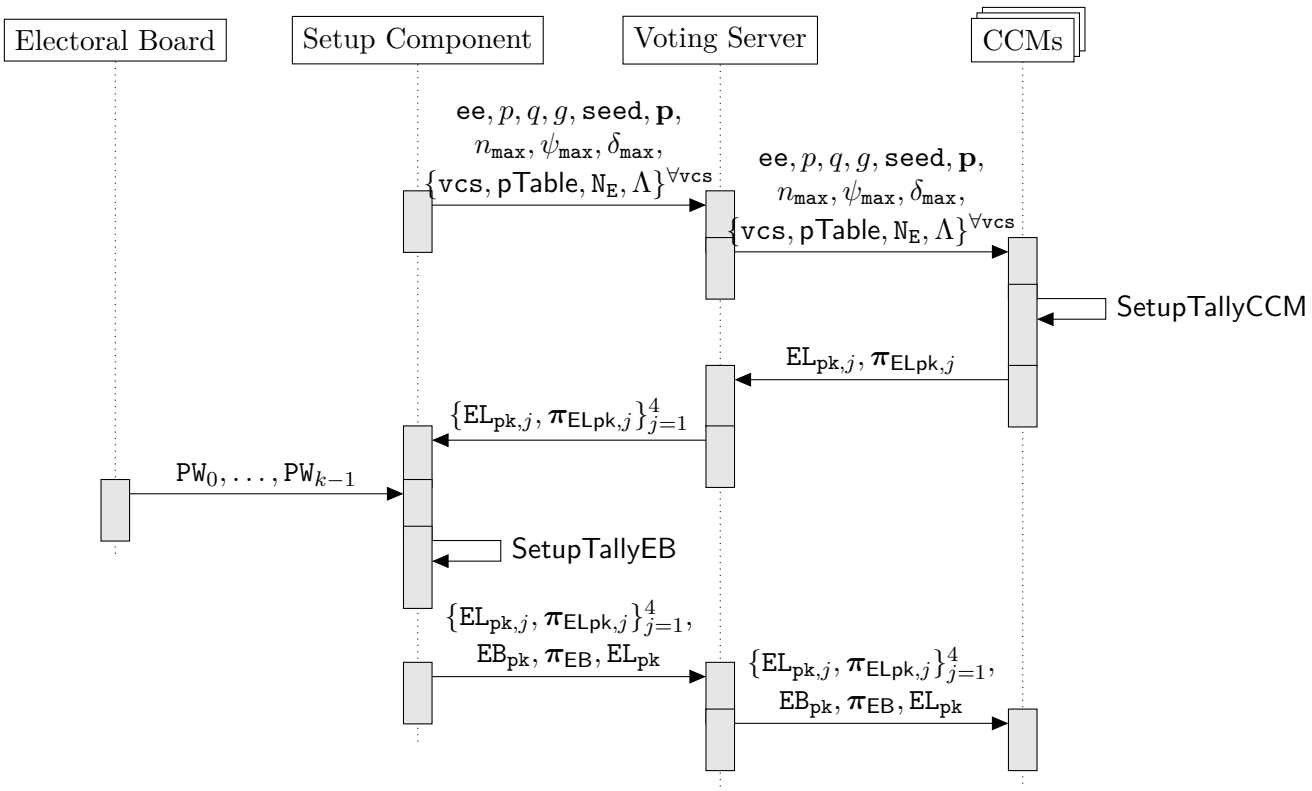


Fig. 8: Sequence diagram of the SetupTally protocol.

Copyright 2026 Swiss Post Ltd.

4.2.1 SetupTallyCCM

Each Mixing control component CCM generates a key pair and proves knowledge of the secret key.

Algorithm 4.13 SetupTallyCCM

Context:

CCM's index $j \in [1, 4]$
The Election Event Context ▷ See Section 3.4 ▷ Including $p, q, g, \delta_{\mathtt{max}}$

Operation: ▷ For all algorithms see the crypto primitives specification

1: $\mathtt{hContext} \leftarrow \mathsf{GetHashElectionEventContext}()$ ▷ See algorithm 3.14
2: $(\mathtt{EL}_{\mathtt{pk},j}, \mathtt{EL}_{\mathtt{sk},j}) \leftarrow \mathsf{GenKeyPair}(p, q, g, \delta_{\mathtt{max}})$
3: $\mathbf{i}_{\mathtt{aux}} \leftarrow (\mathtt{hContext}, \text{“}\mathtt{SetupTallyCCM}\text{”}, \mathsf{IntegerToString}(j))$
4: **for** $i \in [0, \delta_{\mathtt{max}})$ **do**
5: $\quad \mathbf{i}'_{\mathtt{aux}} \leftarrow \mathbf{i}_{\mathtt{aux}} \,||\, \mathsf{IntegerToString}(i)$
6: $\quad \pi_{\mathsf{ELpk},j,i} \leftarrow \mathsf{GenSchnorrProof}(\mathtt{EL}_{\mathtt{sk},j,i}, \mathtt{EL}_{\mathtt{pk},j,i}, \mathbf{i}'_{\mathtt{aux}})$
7: **end for**
8: $\boldsymbol{\pi}_{\mathsf{ELpk},j} \leftarrow (\pi_{\mathsf{ELpk},j,0}, \ldots, \pi_{\mathsf{ELpk},j,\delta_{\mathtt{max}}-1})$

Output:

CCM$_j$ election public key $\mathtt{EL}_{\mathtt{pk},j} \in \mathbb{G}_q^{\delta_{\mathtt{max}}}$
CCM$_j$ election secret key $\mathtt{EL}_{\mathtt{sk},j} \in \mathbb{Z}_q^{\delta_{\mathtt{max}}}$
CCM$_j$ Schnorr proofs of knowledge $\boldsymbol{\pi}_{\mathsf{ELpk},j} \in (\mathbb{Z}_q \times \mathbb{Z}_q)^{\delta_{\mathtt{max}}}$

Copyright 2026 Swiss Post Ltd.

4.2.2 SetupTallyEB

The setup component interacts with the electoral board (see section 2.5) and derives the electoral board key pair $(\mathtt{EB}_{\mathtt{pk}}, \mathtt{EB}_{\mathtt{sk}})$ from the electoral board members' password, verifies the proofs of knowledge of the CCM_j election secret key $\mathtt{EL}_{\mathtt{sk},j}$, and combines the CCM_j election public key $\mathtt{EL}_{\mathtt{pk},j}$ and electoral board public key $\mathtt{EB}_{\mathtt{pk}}$ to yield the election public key $\mathtt{EL}_{\mathtt{pk}}$. Deriving the electoral board secret key $\mathtt{EB}_{\mathtt{sk}}$ from the electoral board members' passwords provides an operational safeguard against the premature decryption of the election results. As an additional operational safeguard, the setup component sends non-invertible hashes $(\mathtt{hPW}_0, \ldots, \mathtt{hPW}_{k-1})$ of the electoral board passwords to the Tally control component, so that the Tally control component can verify each password independently before deriving the Electoral Board secret key.
The electoral board passwords must comply with a suitable password policy. For a given security strength of 128 bits, and assuming an alphabet of at least 128 characters, thus ensuring 7-bit entropy per character, a password of character length 19 is sufficient. The hashes of the passwords are generated using the Argon2id algorithm with the STANDARD profile (see crypto primitives specification).
At the end of SetupTallyEB, the setup component sends the public keys to the control components.
The control components and tally control component initialize the following lists:

- Control component's list of shuffled and decrypted ballot boxes $\mathsf{L}_{\mathtt{bb},j}$
- Tally control component's list of shuffled and decrypted ballot boxes $\mathsf{L}_{\mathtt{bb,Tally}}$

The voting server initializes the following lists:

- Key-value map of number of authentication attempts per credential ID $\mathsf{L}_{\mathtt{authAttempts}}$; initially with all 0-values.
- Key-value map of the used successful authentication challenges per credential ID $\mathsf{L}_{\mathtt{authChallenge}}$; initially as an empty list.

Copyright 2026 Swiss Post Ltd.

Algorithm 4.14 SetupTallyEB

Context:

The Election Event Context ▷ See Section 3.4 ▷ Including $p, q, g, \mathtt{ee}, \delta_{\mathtt{max}}$

Input:

CCM election public keys $\mathbf{EL}_{\mathtt{pk}} = (\mathtt{EL}_{\mathtt{pk},1}, \mathtt{EL}_{\mathtt{pk},2}, \mathtt{EL}_{\mathtt{pk},3}, \mathtt{EL}_{\mathtt{pk},4}) \in \left(\mathbb{G}_q^{\delta_{\mathtt{max}}}\right)^4$

CCM Schnorr proofs of knowledge $\boldsymbol{\pi}_{\mathtt{ELpk}} = (\boldsymbol{\pi}_{\mathtt{ELpk},1}, \boldsymbol{\pi}_{\mathtt{ELpk},2}, \boldsymbol{\pi}_{\mathtt{ELpk},3}, \boldsymbol{\pi}_{\mathtt{ELpk},4}) \in \left((\mathbb{Z}_q \times \mathbb{Z}_q)^{\delta_{\mathtt{max}}}\right)^4$

Passwords of k electoral board members $(\mathtt{PW}_0, \ldots, \mathtt{PW}_{k-1}) \in (\mathbb{A}_{UCS}{}^*)^k$ ▷ We expect the passwords to fulfill a suitable policy

Require: $k \geq 2$ ▷ We require at least 2 electoral board members

Operation: ▷ For all algorithms see the crypto primitives specification

1: $\mathtt{hContext} \leftarrow \mathsf{GetHashElectionEventContext}()$ ▷ See algorithm 3.14
2: $\mathbf{i}_{\mathtt{aux},\mathtt{CCM}} \leftarrow (\mathtt{hContext}, \text{“}\mathtt{SetupTallyCCM}\text{”})$
3: $\mathsf{VerifSch} \leftarrow \mathsf{VerifyCCSchnorrProofs}(\mathbf{EL}_{\mathtt{pk}}, \boldsymbol{\pi}_{\mathtt{ELpk}}, \mathbf{i}_{\mathtt{aux},\mathtt{CCM}})$ ▷ See algorithm 3.21
4: **if** $\neg\mathsf{VerifSch}$ **then**
5: **return** $\perp$ ▷ If any proof fails, the algorithm aborts and returns nothing
6: **end if**
7: $\mathbf{i}_{\mathtt{aux},\mathtt{EB}} \leftarrow (\mathtt{hContext}, \text{“}\mathtt{SetupTallyEB}\text{”}, \mathsf{IntegerToString}(1))$ ▷ Dummy value 1 is added for consistency with other proofs
8: **for** $i \in [0, \delta_{\mathtt{max}})$ **do**
9: $\mathtt{EB}_{\mathtt{sk},i} \leftarrow \mathsf{RecursiveHashToZq}(q, (\text{“}\mathtt{ElectoralBoardSecretKey}\text{”}, \mathtt{ee}, i, \mathtt{PW}_0, \ldots, \mathtt{PW}_{k-1}))$
10: $\mathtt{EB}_{\mathtt{pk},i} \leftarrow g^{\mathtt{EB}_{\mathtt{sk},i}} \bmod p$
11: $\mathbf{i}'_{\mathtt{aux},\mathtt{EB}} \leftarrow \mathbf{i}_{\mathtt{aux},\mathtt{EB}} \,||\, \mathsf{IntegerToString}(i)$
12: $\pi_{\mathtt{EB},i} \leftarrow \mathsf{GenSchnorrProof}(\mathtt{EB}_{\mathtt{sk},i}, \mathtt{EB}_{\mathtt{pk},i}, \mathbf{i}'_{\mathtt{aux},\mathtt{EB}})$
13: **end for**
14: $\mathtt{EB}_{\mathtt{pk}} \leftarrow (\mathtt{EB}_{\mathtt{pk},0}, \ldots, \mathtt{EB}_{\mathtt{pk},\delta_{\mathtt{max}}-1})$
15: $\boldsymbol{\pi}_{\mathtt{EB}} \leftarrow (\pi_{\mathtt{EB},0}, \ldots, \pi_{\mathtt{EB},\delta_{\mathtt{max}}-1})$
16: $\mathtt{EL}_{\mathtt{pk}} \leftarrow \mathsf{CombinePublicKeys}\Big((\mathtt{EL}_{\mathtt{pk},1}, \mathtt{EL}_{\mathtt{pk},2}, \mathtt{EL}_{\mathtt{pk},3}, \mathtt{EL}_{\mathtt{pk},4}, \mathtt{EB}_{\mathtt{pk}})\Big)$

Output:

Election public key $\mathtt{EL}_{\mathtt{pk}} = (\mathtt{EL}_{\mathtt{pk},0}, \ldots, \mathtt{EL}_{\mathtt{pk},\delta_{\mathtt{max}}-1}) \in \mathbb{G}_q^{\delta_{\mathtt{max}}}$
Electoral board public key $\mathtt{EB}_{\mathtt{pk}} = (\mathtt{EB}_{\mathtt{pk},0}, \ldots, \mathtt{EB}_{\mathtt{pk},\delta_{\mathtt{max}}-1}) \in \mathbb{G}_q^{\delta_{\mathtt{max}}}$
Electoral board Schnorr proofs of knowledge $\boldsymbol{\pi}_{\mathtt{EB}} \in (\mathbb{Z}_q \times \mathbb{Z}_q)^{\delta_{\mathtt{max}}}$

Copyright 2026 Swiss Post Ltd.

4.3 Finalize Configuration Phase

Figure 9 shows the different data flows following the SetupVoting and SetupTally protocols.

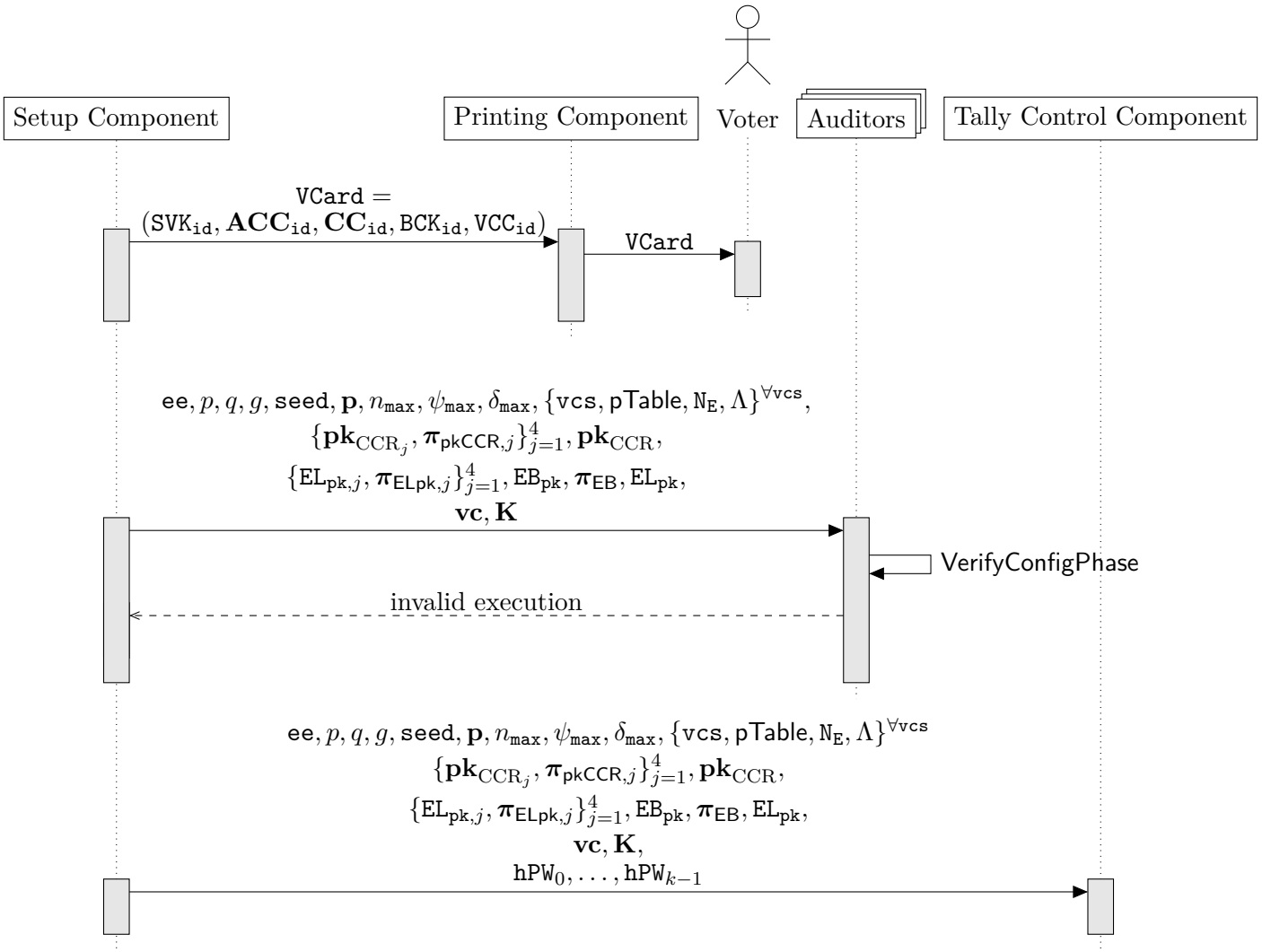


Fig. 9: Sequence diagram of the finalization of the configuration phase.

From a protocol perspective, executing VerifyConfigPhase during the configuration phase is not strictly required, as it would be sufficient to run it at the end of the tally phase. However, performing the verification earlier provides participants with assurance that the election event configuration is correct before the voting phase starts. It also allows the verification to be combined with other ceremonial tasks of the auditors, such as submitting test votes in test ballot boxes.
After the control components have received the final data flows shown in figures 7 and 8, they ensure the following:

- Each control component verifies the other components' proofs of correct key generation (see Section 3.6.5). This confirms that all parties agree on the election event context.
- The control components have received all election-event-specific data from the setup component. During the voting phase, they will explicitly agree on this data (see

Copyright 2026 Swiss Post Ltd.

Section 3.6.3). For performance reasons, the control components precompute the result of the algorithms 3.16 and 3.18 to avoid recalculating it for each vote.

- The control components block the execution of the configuration phase algorithms listed in table 12 and enable the execution of the voting phase algorithms once the start time of the ballot boxes is reached.

Copyright 2026 Swiss Post Ltd.

5 Voting Phase

The voting phase consists of three sub-protocols: AuthenticateVoter, SendVote and ConfirmVote. Table 13 shows the algorithms involved.

Algorithm	Actor	Reference
AuthenticateVoter		Figure 10
GetAuthenticationChallenge	Voting Client	5.1
VerifyAuthenticationChallenge	Voting Server	5.2
GetKey	Voting Client	5.3
SendVote		Figure 11
CreateVote	Voting Client	5.4
VerifyBallotCCR	Control Component (CCR)	5.5
PartialDecryptPCC	Control Component (CCR)	5.6
DecryptPCC	Control Component (CCR)	5.7
CreateLCCShare	Control Component (CCR)	5.8
ExtractCRC	Voting Server	5.9
ConfirmVote		Figure 13
CreateConfirmMessage	Voting Client	5.10
CreateLVCCShare	Control Component (CCR)	5.11
VerifyLVCCHash	Control Component (CCR)	5.12
ExtractVCC	Voting Server	5.13

Tab. 13: Overview of the algorithms in the voting phase.

To access the voting client application, the voter enters a URL that is printed on their voting card. This URL directs the voter to a website that contains a link to start the voting process for a specific election event. The link includes the election event ID and directs the voter to the actual voting client application.
The annex of the Federal Chancellery's Ordinance requires that the voter can check the correctness of the voting client.

> [VEleS Annex 2.7.3]: It must be ensured that no attacker can take control of user devices unnoticed by manipulating the user device software on the server. The person voting must be able to verify that the server has provided his or her user device with the correct software with the correct parameters, in particular the public key for encrypting the vote.

The Swiss Post Voting System fulfills this requirement by publishing the hashes of the voting client via an out-of-band channel. Thereby, a voter can check that her voting client is correct. A correct voting client checks the correctness of the public key in the GetKey algorithm (see section 5.1.3).

Copyright 2026 Swiss Post Ltd.

5.1 AuthenticateVoter

In the AuthenticateVoter phase, the voter enters the Start Voting Key $\texttt{SVK}_{\texttt{id}}$ and an extended authentication factor $\texttt{EA}_{\texttt{id}}$ to the voting client. (For further description of the extended authentication factor, see section 4.1.3.) The voting client derives the base authentication challenge $\texttt{hAuth}_{\texttt{id}}$ from $\texttt{SVK}_{\texttt{id}}$ and $\texttt{EA}_{\texttt{id}}$ and authenticates to the voting server.
At the end of the AuthenticateVoter phase, the voting client receives the election event context, including all relevant public keys, and extracts the verification card secret key $\texttt{k}_{\texttt{id}}$. Subsequently, the voting client displays the voting options to the voter, who selects the desired voting options prior to the SendVote phase.
Although the voting server is considered untrusted for the purpose of verifiability and voting secrecy, performing an authentication protocol between the voting client and the voting server increases the robustness of the system. The voting server acts as an intermediary layer that provides a first line of defense against malformed or malicious requests. In practice, it helps to prevent certain types of attacks, such as distributed denial-of-service attacks or attempts to drive the control components into an inconsistent state, by blocking invalid authentication attempts early. While this does not strengthen the cryptographic guarantees of verifiability or voting secrecy, it contributes to the overall operational stability and integrity of the system by shielding the control components.
Throughout the voting phase, the voting client and voting server execute the GetAuthenticationChallenge and VerifyAuthenticationChallenge algorithms at each interaction, including the SendVote and ConfirmVote phases. This ensures the freshness of the voting client's request and proves knowledge of the base authentication challenge $\texttt{hAuth}_{\texttt{id}}$.
The AuthenticateVoter phase is heavily inspired by the Time-based One-Time Password (TOTP) protocol specified in RFC6238, a widely-used authentication method with robust security proofs [16]. TOTP's timestamp component is used to prevent replay attacks. The shared secret between voting client and voting server is the base authentication challenge $\texttt{hAuth}_{\texttt{id}}$. The voting server receives the base authentication challenge $\texttt{hAuth}_{\texttt{id}}$ from the setup component while the voting client derives it from $\texttt{SVK}_{\texttt{id}}$ and $\texttt{EA}_{\texttt{id}}$ received from the voter. RFC6238 recommends that one backward time step is allowed as the network delay, and that the validator tolerates the prover being "out of sync" by a specific number of backward and forward time steps before being rejected. We allow one forward and one backward time step. However, in some aspects, we deviate from RFC6238. First, an authentication $\texttt{nonce}$ ensures that multiple authentication attempts within the same time step are possible. Second, while RFC6238 recommends a time step size of 30 seconds, we use a time step size of 300 seconds. Because of the usage of the $\texttt{nonce}$, allowing further time windows (as recommended by RFC6238) has the same result as increasing the time step size. The latter approach is preferred because of the better performance. Third, we use Argon2id [3], a state-of-the-art memory-hard key derivation function, instead of HMAC-SHA-256. This choice ensures consistency with the use of Argon2id in the voting client while keeping the authentication process secure.

Copyright 2026 Swiss Post Ltd.

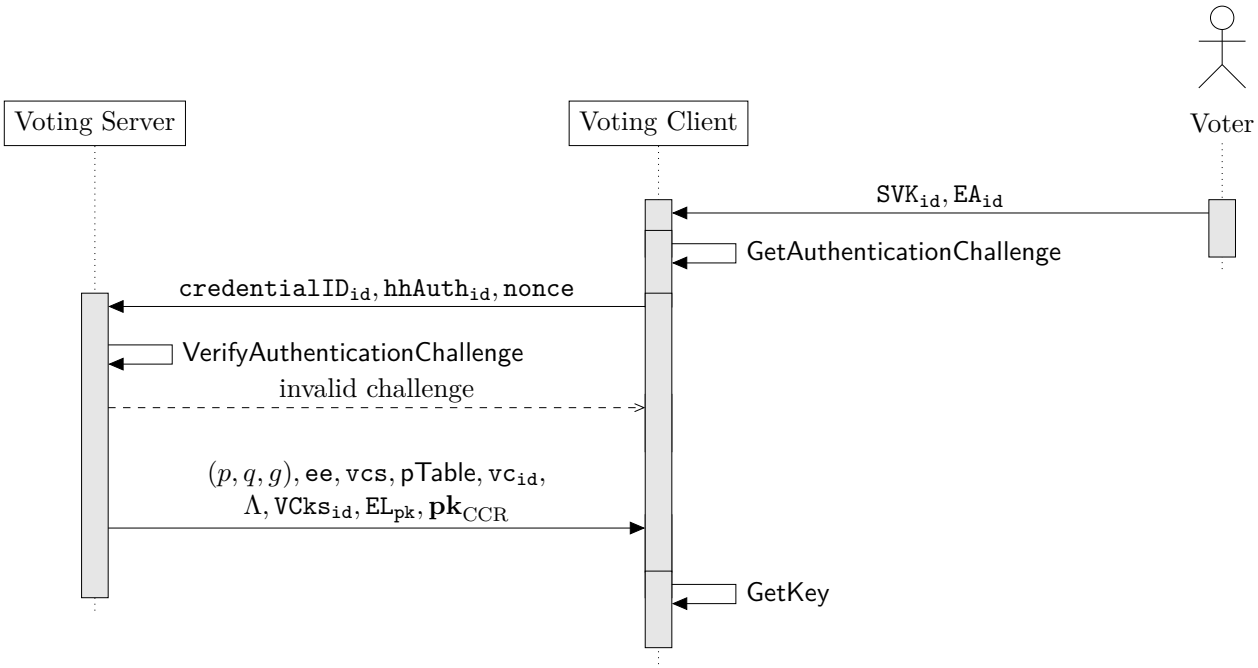


Fig. 10: Sequence diagram of the AuthenticateVoter sub-protocol

Copyright 2026 Swiss Post Ltd.

5.1.1 GetAuthenticationChallenge

The voting client executes the GetAuthenticationChallenge algorithm multiple times during the voting process. To distinguish different invocations of the GetAuthenticationChallenge algorithm, we include the corresponding authentication step (authenticateVoter, sendVote, confirmVote) in the salt.
Since authentication might happen multiple times within a given time step, the voting client generates an authentication `nonce` for every authentication attempt—allowing the voting client and voting server to distinguish different authentication attempts within the same time step. Subsequently, the voting client sends the authentication `nonce` to the voting server.

Algorithm 5.1 GetAuthenticationChallenge

Context:

Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$

Input:

Authentication step $\mathtt{authStep} \in (\text{“}\mathtt{authenticateVoter}\text{”}, \text{“}\mathtt{sendVote}\text{”}, \text{“}\mathtt{confirmVote}\text{”})$
Start Voting Key $\mathtt{SVK_{id}} \in (\mathbb{A}_{\mathtt{u32}})^{\mathtt{l_{SVK}}}$
Extended authentication factor $\mathtt{EA_{id}} \in \mathcal{D}_{\mathsf{EA}}$

Operation: ▷ For all algorithms see the crypto primitives specification

1: $\mathtt{credentialID_{id}} \leftarrow \mathsf{DeriveCredentialId}(\mathtt{SVK_{id}})$ ▷ See algorithm 3.22
2: $\mathtt{hAuth_{id}} \leftarrow \mathsf{DeriveBaseAuthenticationChallenge}(\mathtt{SVK_{id}}, \mathtt{EA_{id}})$ ▷ See algorithm 3.23
3: $\mathtt{nonce} \leftarrow \mathsf{GenRandomInteger}(2^{256})$
4: $\mathtt{TS} \leftarrow \mathsf{GetTimestamp}()$ ▷ Returns the current Unix time, measured in number of seconds passed since 01.01.1970
5: $\mathtt{T} \leftarrow \lfloor \frac{\mathtt{TS}}{300} \rfloor$
6: $\mathtt{salt_{id}} \leftarrow \mathsf{CutToBitLength}(\mathsf{RecursiveHash}(\mathtt{ee}, \mathtt{credentialID_{id}}, \text{“}\mathtt{dAuth}\text{”}, \mathtt{authStep}, \mathtt{nonce}), 128)$
7: $\mathtt{k} \leftarrow \Big(\mathsf{StringToByteArray}(\mathtt{hAuth_{id}}) \,||\, \mathsf{StringToByteArray}(\text{”}\mathtt{Auth}\text{”}) \,||\, \mathsf{IntegerToByteArray}(\mathtt{T})\Big)$
8: $\mathtt{bhhAuth_{id}} \leftarrow \mathsf{GetArgon2id}(\mathtt{k}, \mathtt{salt_{id}})$ ▷ Use the Argon2 profile for less memory, as defined in the crypto primitives specification
9: $\mathtt{hhAuth_{id}} \leftarrow \mathsf{Base64Encode}(\mathtt{bhhAuth_{id}})$

Output:

Derived voter identifier $\mathtt{credentialID_{id}} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Derived authentication challenge $\mathtt{hhAuth_{id}} \in (\mathbb{A}_{Base64})^{\mathtt{l_{HB64}}}$
Authentication $\mathtt{nonce} \in [0, 2^{256})$

Copyright 2026 Swiss Post Ltd.

5.1.2 VerifyAuthenticationChallenge

After receiving the authentication challenge from the voting client, the voting server must verify its validity. Table 14 shows the source of the context and input variables and the preliminary validations that the voting server performs before invoking the algorithm.

Information	Variable	Source	Use as	Preliminary Validation
Election event ID	$\texttt{ee}$	Voting Client	Context	Check that $\texttt{ee}$ exists in the internal view.
Derived voter identifier	$\texttt{credentialID}_{\texttt{id}}$	Voting Client	Context	Check in the internal view that $\texttt{credentialID}_{\texttt{id}}$ corresponds to a $\texttt{vc}_{\texttt{id}}$ for this $\texttt{ee}$, that the corresponding ballot box is currently open, and that the $\texttt{vc}_{\texttt{id}}$ is consistent with other information received from the voting client.
Authentication step	$\texttt{authStep}$	Voting Client	Input	Check that the authentication step is consistent with the state of the $\texttt{vc}_{\texttt{id}}$.
Derived authentication challenge	$\texttt{hhAuth}_{\texttt{id}}$	Voting Client	Input	None. Checked within algorithm 5.2.
Base authentication challenge	$\texttt{hAuth}_{\texttt{id}}$	Internal view	Input	None, since retrieved from the trusted internal view.
Authentication nonce	$\texttt{nonce}$	Voting Client	Input	None, other than the implicit domain checks.

Tab. 14: Context and input validation for VerifyAuthenticationChallenge

Moreover, the voting server must keep track of the number of authentication attempts made by the voter. The voting server maintains the following two stateful maps:

- Key-value map of number of authentication attempts per credential ID $\mathsf{L}_{\texttt{authAttempts}}$
- Key-value map of the used authentication challenges per credential ID $\mathsf{L}_{\texttt{authChallenge}}$

The stateful lists prevent a voting client from brute forcing the extended authentication factor and enforce one-time use of a successful authentication challenge as specified in RFC6238 [16, *5.2. Validation and Time-Step Size*].

Copyright 2026 Swiss Post Ltd.

Algorithm 5.2 VerifyAuthenticationChallenge

Context:

Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Derived voter identifier $\mathtt{credentialID_{id}} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$

Stateful Lists and Maps:

Key-value map of number of authentication attempts per credential ID $\mathsf{L}_{\mathtt{authAttempts}}$
Key-value map of the used successful authentication challenges per credential ID $\mathsf{L}_{\mathtt{authChallenge}}$

Input:

Authentication step $\mathtt{authStep} \in$ ("`authenticateVoter`", "`sendVote`", "`confirmVote`")
Derived authentication challenge $\mathtt{hhAuth_{id}} \in (\mathbb{A}_{Base64})^{\mathtt{l_{HB64}}}$
Base authentication challenge $\mathtt{hAuth_{id}} \in (\mathbb{A}_{Base64})^{\mathtt{l_{HB64}}}$
Authentication $\mathtt{nonce} \in [0, 2^{256})$

Operation: ▷ For all algorithms see the crypto primitives specification

1: $\mathtt{TS} \leftarrow \mathsf{GetTimestamp}()$ ▷ Returns the current Unix time
2: $\mathtt{T}_1 \leftarrow \lfloor \frac{\mathtt{TS}}{300} \rfloor$
3: $\mathtt{T}_0 \leftarrow \mathtt{T}_1 - 1$
4: $\mathtt{T}_2 \leftarrow \mathtt{T}_1 + 1$
5: $\mathtt{attempts_{id}} \leftarrow \mathsf{L}_{\mathtt{authAttempts}}(\mathtt{credentialID_{id}})$ ▷ Retrieve the value from the key-value map
6: **if** $\Big((\mathtt{attempts_{id}} \geq 5) \vee (\mathtt{hhAuth_{id}} \in \mathsf{L}_{\mathtt{authChallenge}}(\mathtt{credentialID_{id}}))\Big)$ **then**
return $\bot$ ▷ Enforces one-time usage and limits the voting client to 5 attempts
7: **end if**
8: $\mathtt{salt_{id}} \leftarrow \mathsf{CutToBitLength}(\mathsf{RecursiveHash}(\mathtt{ee}, \mathtt{credentialID_{id}}, \text{"}\mathtt{dAuth}\text{"}, \mathtt{authStep}, \mathtt{nonce}), 128)$
9: $\mathtt{bhhAuth_{id}} \leftarrow \mathsf{Base64Decode}(\mathtt{hhAuth_{id}})$
10: **for** $\mathtt{T}_i \in \{\mathtt{T}_1, \mathtt{T}_0, \mathtt{T}_2\}$ **do**
11: $\mathtt{k} \leftarrow \Big(\mathsf{StringToByteArray}(\mathtt{hAuth_{id}}) \,||\, \mathsf{StringToByteArray}(\text{"}\mathtt{Auth}\text{"}) \,||\, \mathsf{IntegerToByteArray}(\mathtt{T}_i)\Big)$
12: $\mathtt{bhhAuth'_{id,i}} \leftarrow \mathsf{GetArgon2id}(\mathtt{k}, \mathtt{salt_{id}})$ ▷ Use the Argon2 profile for less memory, as defined in the crypto primitives specification
13: **if** $(\mathtt{bhhAuth'_{id,i}} = \mathtt{bhhAuth_{id}})$ **then**
14: $\mathsf{L}_{\mathtt{authChallenge}}(\mathtt{credentialID_{id}}) \leftarrow \mathsf{L}_{\mathtt{authChallenge}}(\mathtt{credentialID_{id}}) \,||\, \mathtt{hhAuth_{id}}$ ▷ Add the authentication challenge to the list of used challenges
15: **return** $\top$
16: **end if**
17: **end for**
18: $\mathsf{L}_{\mathtt{authAttempts}}(\mathtt{credentialID_{id}}) \leftarrow \mathtt{attempts_{id}} + 1$ ▷ Set the value in the key-value map
19: **return** $\bot$

Output:

The result of the verification: $\top$ if the verification is successful, $\bot$ otherwise.

To prevent the list $\mathsf{L}_{\mathtt{authChallenge}}(\mathtt{credentialID_{id}})$ of successful authentication attempts from becoming too long, the voting server can reinitialize the list if the time step $\mathtt{T}_0$ of the current authentication attempt exceeds the time step $\mathtt{T}_{2,\mathtt{last}}$ of the last successful authentication. The voting server sends the Verification Card Keystore $\mathtt{VCks_{id}}$ to the voting client after a successful verification of the authentication challenge.

Copyright 2026 Swiss Post Ltd.

5.1.3 GetKey

In the GetKey algorithm, the voting client opens the Verification Card Keystore $\texttt{VCks}_{\texttt{id}}$ to obtain the verification card secret key $\texttt{k}_{\texttt{id}}$. The voting client receives the Start Voting Key $\texttt{SVK}_{\texttt{id}}$ from the voter and the other input and context arguments from the voting server. The voting client verifies the context data's authenticity using the authenticated encryption scheme (see the section *Symmetric Authenticated Encryption* in the crypto primitives specification) and the Start Voting Key $\texttt{SVK}_{\texttt{id}}$—a secret shared between the setup component and the voting client.

Algorithm 5.3 GetKey

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Election event ID $\texttt{ee} \in (\mathbb{A}_{Base16})^{\texttt{l}_{\texttt{ID}}}$
Verification card set ID $\texttt{vcs} \in (\mathbb{A}_{Base16})^{\texttt{l}_{\texttt{ID}}}$
Verification card ID $\texttt{vc}_{\texttt{id}} \in (\mathbb{A}_{Base16})^{\texttt{l}_{\texttt{ID}}}$
Primes mapping table $\texttt{pTable} \in (\mathcal{D}_{\text{v}} \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times {\mathbb{A}_{UCS}}^* \times \mathcal{D}_\tau)^n$ ▷ pTable is of the form $((\text{v}_0, \tilde{\text{p}}_0, \sigma_0, \tau_0), \ldots, (\text{v}_{n-1}, \tilde{\text{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1}))$
Electoral Model Context $\Lambda = (\boldsymbol{\psi}, \mathbf{DoI}, \mathbf{pg}, \mathbf{ag}) \in \left((\mathbb{N}^+)^t \times (\mathcal{T}_1^{50})^k \times [0, \eta)^t \times \mathbb{Z}_2^\eta\right)$
Election public key $\texttt{EL}_{\texttt{pk}} = (\texttt{EL}_{\texttt{pk},0}, \ldots, \texttt{EL}_{\texttt{pk},\delta_{\max}-1}) \in \mathbb{G}_q^{\delta_{\max}}$
Choice Return Codes encryption public key $\mathbf{pk}_{\text{CCR}} = (\texttt{pk}_{\text{CCR},0}, \ldots, \texttt{pk}_{\text{CCR},\psi_{\max}-1}) \in \mathbb{G}_q^{\psi_{\max}}$

Input:

Start Voting Key $\texttt{SVK}_{\texttt{id}} \in (\mathbb{A}_{\texttt{u32}})^{\texttt{l}_{\texttt{SVK}}}$
Verification Card Keystore $\texttt{VCks}_{\texttt{id}} \in {\mathbb{A}_{Base64}}^{\texttt{l}_{\texttt{VCKS}}}$

Operation: ▷ For all algorithms see the crypto primitives specification

1: $\mathbf{i}_{\texttt{aux}} \leftarrow$ ("GetKey", GetHashContext()) ▷ See algorithm 3.15
2: $(\texttt{VCks}_{\texttt{id,ciphertext}}, \texttt{VCks}_{\texttt{id,nonce}}, \texttt{VCks}_{\texttt{id,salt}}) \leftarrow$ GetSymmetricCiphertextParts($\texttt{VCks}_{\texttt{id}}$)
3: $\texttt{dSVK}_{\texttt{id}} \leftarrow$ GetArgon2id(StringToByteArray($\texttt{SVK}_{\texttt{id}}$), $\texttt{VCks}_{\texttt{id,salt}}$) ▷ Use the Argon2 profile for less memory, as defined in the crypto primitives specification
4: $\texttt{KSkey}_{\texttt{id}} \leftarrow$ RecursiveHash("VerificationCardKeystore", ee, vcs, $\texttt{vc}_{\texttt{id}}$, $\texttt{dSVK}_{\texttt{id}}$)
5: $\texttt{k}_{\texttt{id,bytes}} \leftarrow$ GetPlaintextSymmetric($\texttt{KSkey}_{\texttt{id}}$, $\texttt{VCks}_{\texttt{id,ciphertext}}$, $\texttt{VCks}_{\texttt{id,nonce}}$, $\mathbf{i}_{\texttt{aux}}$)
6: $\texttt{k}_{\texttt{id}} \leftarrow$ ByteArrayToInteger($\texttt{k}_{\texttt{id,bytes}}$)

Output:

Verification Card Secret Key $\texttt{k}_{\texttt{id}} \in \mathbb{Z}_q$

Copyright 2026 Swiss Post Ltd.

5.2 SendVote

The SendVote phase encompasses creating the vote until the voter receives the short Choice Return Codes. Figure 11 shows the relevant algorithms.
SendVote assumes that the voter already authenticated to the voting server and that the voting client knows the verification card secret key $\mathtt{k}_{\mathtt{id}}$ and validated the election event's public keys.

Copyright 2026 Swiss Post Ltd.

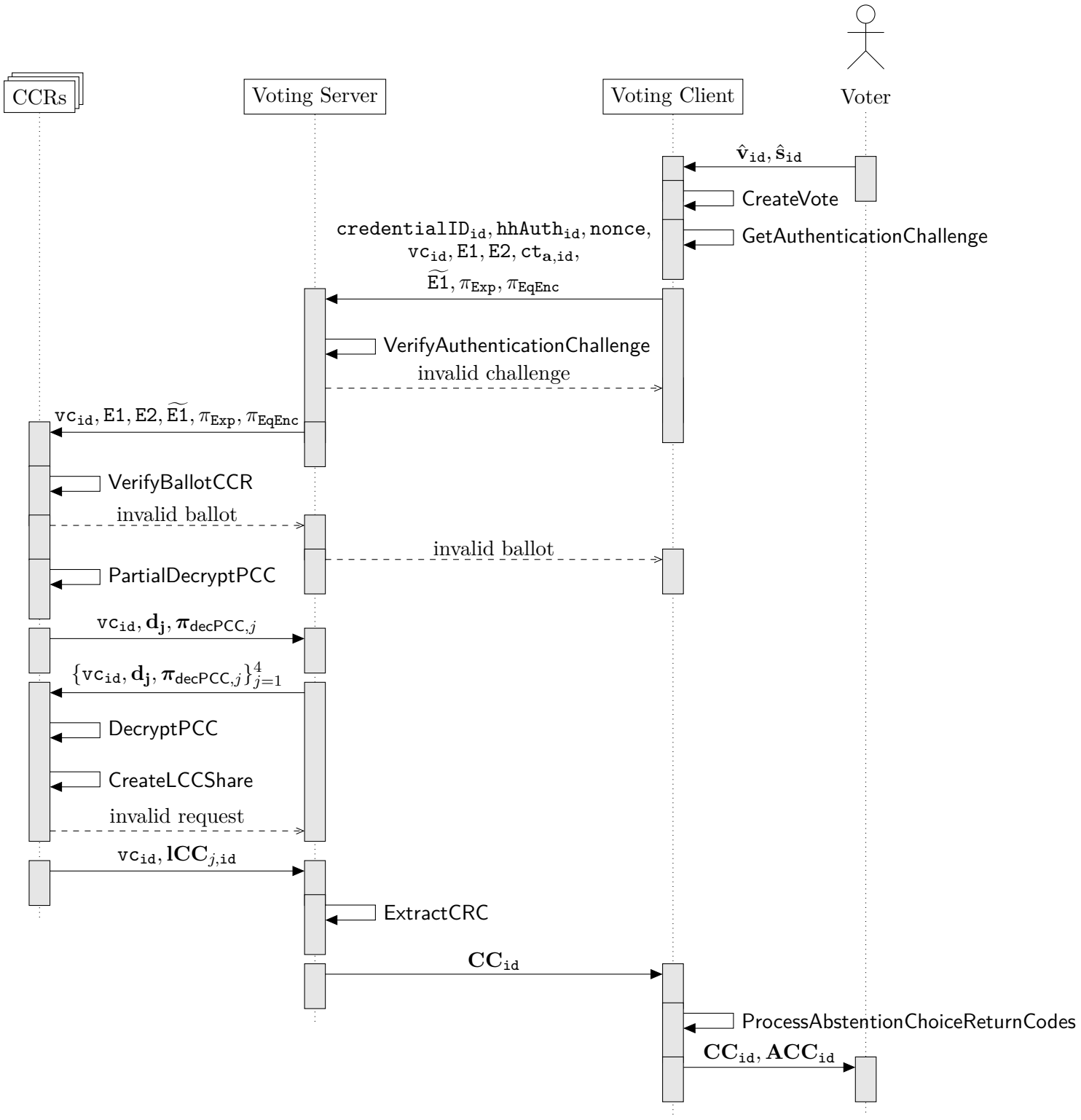


Fig. 11: Sequence diagram of the SendVote sub-protocol

Copyright 2026 Swiss Post Ltd.

5.2.1 CreateVote

The voter supplies the voting client with the following:

- a list of selected actual voting options $\hat{\mathbf{v}}_{\mathtt{id}}$, see Section 3.4.6,
- a list of selected write-ins $\hat{\mathbf{s}}_{\mathtt{id}}$, see Section 3.8.4,
- a list of selected abstentions $\hat{\mathbf{a}}_{\mathtt{id}}$, see Section 3.9.2.

In turn, the voting client creates the vote containing the encrypted voting options, the partial Choice Return Codes, the encrypted abstention vector, and the non-interactive zero-knowledge proofs linking the ElGamal ciphertexts.

The voting client generates two ElGamal ciphertexts: the first ciphertext `E1` contains the encrypted voting options and—optionally—the encrypted write-ins, the second ciphertext `E2` contains the basis for generating the short Choice Return Codes. In addition, the voting client symmetrically encrypts $\hat{\mathbf{a}}_{\mathtt{id}}$ under a key derived from the verification card secret key and the election context, producing $\mathtt{ct}_{\mathbf{a},\mathtt{id}}$, so that the voting server can persist it and return it upon relogin without learning its content. The two ElGamal ciphertexts are linked through non-interactive zero-knowledge proofs; $\mathtt{ct}_{\mathbf{a},\mathtt{id}}$ is not part of these proofs.
A Schnorr proof π_{sch}—which proves knowledge of the encryption randomness $r \in \mathbb{Z}_q$—is redundant:

- π_{exp} proves knowledge of the pre-image $k_{\mathtt{id}}$ (the verification card secret key) and that the prover raised the bases (g, E1) to the same exponent.
- π_{EqEnc} proves knowledge of the pre-images $r \cdot k_{\mathtt{id}}$ and r' and that $\widetilde{E1}$ and $\widetilde{E2}$ contain the same message (encrypted under different public keys).

If the prover knows $r \cdot k_{\mathtt{id}}$ (from the plaintext equality proof) and $k_{\mathtt{id}}$ (from the exponentiation proof), he implicitly knows r (which is the statement that the Schnorr proof claims).

Copyright 2026 Swiss Post Ltd.

Algorithm 5.4 CreateVote

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{l_{\mathtt{ID}}}$
Verification card set ID $\mathtt{vcs} \in (\mathbb{A}_{Base16})^{l_{\mathtt{ID}}}$
Verification card ID $\mathtt{vc_{id}} \in (\mathbb{A}_{Base16})^{l_{\mathtt{ID}}}$
Primes mapping table $\mathsf{pTable} \in \left(\mathcal{D}_{\mathrm{v}} \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times {\mathbb{A}_{UCS}}^* \times \mathcal{D}_\tau\right)^n$ ▷ pTable is of the form $((\mathrm{v}_0, \tilde{\mathrm{p}}_0, \sigma_0, \tau_0), \ldots, (\mathrm{v}_{n-1}, \tilde{\mathrm{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1}))$ ▷ ψ and δ can be derived from pTable using algorithms 3.11 and 3.12
Electoral Model Context $\Lambda = (\boldsymbol{\psi}, \mathbf{DoI}, \mathbf{pg}, \mathbf{ag}) \in \left((\mathbb{N}^+)^t \times (\mathcal{T}_1^{50})^k \times [0, \eta)^t \times \mathbb{Z}_2^\eta\right)$ ▷ See Section 3.4.4
Election public key $\mathtt{EL_{pk}} = (\mathtt{EL}_{\mathtt{pk},0}, \ldots, \mathtt{EL}_{\mathtt{pk},\delta_{\max}-1}) \in \mathbb{G}_q^{\delta_{\max}}$
Choice Return Codes encryption public key $\mathbf{pk}_{\mathrm{CCR}} \in \mathbb{G}_q^{\psi_{\max}}$

Input:

Selected actual voting options $\hat{\mathbf{v}}_{\mathtt{id}} = (\hat{\mathrm{v}}_0, \ldots, \hat{\mathrm{v}}_{\psi-1}) \in \left(\mathcal{D}_{\mathrm{v}}\right)^\psi$ ▷ See section 3.4
Selected write-ins $\hat{\mathbf{s}}_{\mathtt{id}} = (\hat{\mathrm{s}}_0, \ldots, \hat{\mathrm{s}}_{k-1}) \in \left((\mathbb{A}_{\mathsf{latin}} \setminus \#)^*\right)^k$ ▷ See section 3.8
Selected abstentions $\hat{\mathbf{a}}_{\mathtt{id}} = (\hat{a}_{\mathtt{id},0}, \ldots, \hat{a}_{\mathtt{id},\eta-1}) \in \mathbb{Z}_2^\eta$ ▷ See section 3.9.2
Verification card secret key $\mathtt{k_{id}} \in \mathbb{Z}_q$

Require: $\mathsf{GetBlankCorrectnessInformation}() = \mathsf{GetCorrectnessInformation}(\hat{\mathbf{v}}_{\mathtt{id}})$ ▷ See algorithms 3.5 and 3.6. The algorithm 3.5 ensures $\hat{\mathbf{v}}_{\mathtt{id}}$ is a subset of $\tilde{\mathbf{v}}$ and contains no duplicates.

Require: $\hat{\mathbf{v}}_{\mathtt{id}} \cap \mathbf{v}_{\mathrm{a}} = \{\} \vee \hat{\mathbf{v}}_{\mathtt{id}} \cap \mathbf{v}_{\mathrm{a}} = \mathbf{v}_{\mathrm{a}}, \forall \mathbf{pg}_j \in \mathbf{pg} : \mathbf{v}_{\mathrm{a}} = \mathsf{GetAbstentionActualVotingOptions}(\mathbf{pg}_j)$ ▷ See algorithm 3.9. If a voter abstains from a presentation group, the selected voting options related to the presentation group must be abstention voting options. If a voter does not abstain, none of the selected voting options of the presentation group must be abstention voting options.

Require: $k \leq \delta - 1$ ▷ $\delta = 1$, if the ballot box does not have any write-in candidates.

Require: $|\hat{\mathrm{s}}_i| < \mathtt{l_w}, \forall i \in [0, k)$ ▷ where $|\hat{\mathrm{s}}_i|$ is the character length of $\hat{\mathrm{s}}_i$

Operation: ▷ For all algorithms see the crypto primitives specification

1: $(\hat{\mathrm{p}}_0, \ldots, \hat{\mathrm{p}}_{\psi-1}) \leftarrow \mathsf{GetEncodedVotingOptions}(\hat{\mathbf{v}}_{\mathtt{id}})$ ▷ See algorithm 3.2
2: $(\mathrm{w}_{\mathtt{id},0}, \ldots, \mathrm{w}_{\mathtt{id},\delta-2}) \leftarrow \mathsf{EncodeWriteIns}(\hat{\mathbf{s}}_{\mathtt{id}})$ ▷ See algorithm 3.28
3: $\rho \leftarrow \prod_{i=0}^{\psi-1} \hat{\mathrm{p}}_i \bmod p$
4: $r \leftarrow \mathsf{GenRandomInteger}(q)$
5: $\mathtt{E1} = (\gamma_1, \phi_{1,0}, \ldots, \phi_{1,\delta-1}) \leftarrow \mathsf{GetCiphertext}((\rho, \mathrm{w}_{\mathtt{id},0}, \ldots, \mathrm{w}_{\mathtt{id},\delta-2}), r, \mathtt{EL_{pk}})$
6: **for** $i \in [0, \psi)$ **do**
7: $\quad \mathtt{pCC}_{\mathtt{id},i} \leftarrow \hat{\mathrm{p}}_i^{\mathtt{k_{id}}} \bmod p$
8: **end for**
9: $\mathbf{pCC}_{\mathtt{id}} = (\mathtt{pCC}_{\mathtt{id},0}, \ldots, \mathtt{pCC}_{\mathtt{id},\psi-1})$
10: $r' \leftarrow \mathsf{GenRandomInteger}(q)$
11: $\mathtt{E2} = (\gamma_2, \phi_{2,0}, \ldots, \phi_{2,\psi-1}) \leftarrow \mathsf{GetCiphertext}(\mathbf{pCC}_{\mathtt{id}}, r', \mathbf{pk}_{\mathrm{CCR}})$
12: $\widetilde{\mathtt{E1}} \leftarrow \mathsf{GetCiphertextExponentiation}((\gamma_1, \phi_{1,0}), \mathtt{k_{id}})$
13: $\widetilde{\mathtt{E2}} \leftarrow (\gamma_2, \prod_{i=0}^{\psi-1} \phi_{2,i} \bmod p)$
14: $\mathtt{K_{id}} \leftarrow g^{\mathtt{k_{id}}} \bmod p$
15: $\mathtt{ct}_{\mathbf{a},\mathtt{id}} \leftarrow \mathsf{GenEncryptedAbstentionVector}(\hat{\mathbf{a}}_{\mathtt{id}}, \mathtt{k_{id}})$ ▷ See algorithm 3.32
16: $\mathbf{i}_{\mathtt{aux}} \leftarrow (\text{“}\mathtt{CreateVote}\text{”}, \mathtt{vc_{id}}, \mathsf{GetHashContext}())$ ▷ See algorithm 3.15
17: $\mathbf{i}_{\mathtt{aux}} \leftarrow (\mathbf{i}_{\mathtt{aux}}, \mathsf{IntegerToString}(\gamma_1), \mathsf{IntegerToString}(\phi_{1,0}), \ldots, \mathsf{IntegerToString}(\phi_{1,\delta-1}))$
18: $\mathbf{i}_{\mathtt{aux}} \leftarrow (\mathbf{i}_{\mathtt{aux}}, \mathsf{IntegerToString}(\gamma_2), \mathsf{IntegerToString}(\phi_{2,0}), \ldots, \mathsf{IntegerToString}(\phi_{2,\psi-1}))$
19: $\pi_{\mathtt{Exp}} \leftarrow \mathsf{GenExponentiationProof}((g, \gamma_1, \phi_{1,0}), \mathtt{k_{id}}, (\mathtt{K_{id}}, \gamma_1^{\mathtt{k_{id}}}, \phi_{1,0}^{\mathtt{k_{id}}}), \mathbf{i}_{\mathtt{aux}})$
20: $\widetilde{\mathrm{pk}}_{\mathrm{CCR}} \leftarrow \prod_{i=0}^{\psi-1} \mathrm{pk}_{\mathrm{CCR},i} \bmod p$
21: $\pi_{\mathtt{EqEnc}} \leftarrow \mathsf{GenPlaintextEqualityProof}(\widetilde{\mathtt{E1}}, \widetilde{\mathtt{E2}}, \mathtt{EL}_{\mathtt{pk},0}, \widetilde{\mathrm{pk}}_{\mathrm{CCR}}, (r \cdot \mathtt{k_{id}}, r'), \mathbf{i}_{\mathtt{aux}})$

Output:

Encrypted vote $\mathtt{E1} = (\gamma_1, \phi_{1,0}, \ldots, \phi_{1,\delta-1}) \in {\mathbb{G}_q}^{\delta+1}$
Encrypted partial Choice Return Codes $\mathtt{E2} = (\gamma_2, \phi_{2,0}, \ldots, \phi_{2,\psi-1}) \in {\mathbb{G}_q}^{\psi+1}$
Exponentiated encrypted vote $\widetilde{\mathtt{E1}} \in {\mathbb{G}_q}^2$
Exponentiation proof $\pi_{\mathtt{Exp}} \in \mathbb{Z}_q \times \mathbb{Z}_q$
Plaintext equality proof $\pi_{\mathtt{EqEnc}} \in \mathbb{Z}_q \times {\mathbb{Z}_q}^2$
Encrypted abstention vector $\mathtt{ct}_{\mathbf{a},\mathtt{id}} \in {\mathbb{A}_{Base64}}^*$

Copyright 2026 Swiss Post Ltd.

5.2.2 VerifyBallotCCR

The Return Codes control components CCR check the voting client's encrypted vote, in particular the zero-knowledge proofs.

Algorithm 5.5 VerifyBallotCCR

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Verification card set ID $\mathtt{vcs} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Verification card ID $\mathtt{vc_{id}} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Primes mapping table $\mathsf{pTable} \in (\mathcal{D}_{\mathrm{v}} \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times {\mathbb{A}_{UCS}}^* \times \mathcal{D}_\tau)^n$ ▷ pTable is of the form $((\mathrm{v}_0, \tilde{\mathrm{p}}_0, \sigma_0, \tau_0), \ldots, (\mathrm{v}_{n-1}, \tilde{\mathrm{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1}))$ ▷ ψ and δ can be derived from pTable using algorithms 3.11 and 3.12
Electoral Model Context $\Lambda = (\boldsymbol{\psi}, \mathbf{DoI}, \mathbf{pg}, \mathbf{ag}) \in \left((\mathbb{N}^+)^t \times (\mathcal{T}_1^{50})^k \times [0, \eta)^t \times \mathbb{Z}_2^\eta\right)$ ▷ See Section 3.4.4
Verification card public key $\mathtt{K_{id}} \in \mathbb{G}_q$
Election public key $\mathtt{EL_{pk}} = (\mathtt{EL_{pk,0}}, \ldots, \mathtt{EL_{pk},\delta_{max}-1}) \in \mathbb{G}_q^{\delta_{\max}}$
Choice Return Codes encryption public key $\mathbf{pk}_{\mathrm{CCR}} \in \mathbb{G}_q^{\psi_{\max}}$

Input:

Encrypted vote $\mathtt{E1} = (\gamma_1, \phi_{1,0}, \ldots, \phi_{1,\delta-1}) \in {\mathbb{G}_q}^{\delta+1}$
Exponentiated encrypted vote $\widetilde{\mathtt{E1}} = (\gamma_1^{\mathtt{k_{id}}}, \phi_{1,0}^{\mathtt{k_{id}}}) \in {\mathbb{G}_q}^2$
Encrypted partial Choice Return Codes $\mathtt{E2} = (\gamma_2, \phi_{2,0}, \ldots, \phi_{2,\psi-1}) \in {\mathbb{G}_q}^{\psi+1}$
Exponentiation proof $\pi_{\mathtt{Exp}} \in \mathbb{Z}_q \times \mathbb{Z}_q$
Plaintext equality proof $\pi_{\mathtt{EqEnc}} \in {\mathbb{Z}_q \times \mathbb{Z}_q}^2$

Operation: ▷ For all algorithms see the crypto primitives specification

1: $\widetilde{\mathtt{E2}} \leftarrow (\gamma_2, \prod_{i=0}^{\psi-1} \phi_{2,i} \bmod p)$
2: $\widetilde{\mathtt{pk}}_{\mathrm{CCR}} \leftarrow \prod_{i=0}^{\psi-1} \mathtt{pk}_{\mathrm{CCR},i} \bmod p$
3: $\mathbf{i}_{\mathtt{aux}} \leftarrow$ ("CreateVote", $\mathtt{vc_{id}}$, GetHashContext()) ▷ See algorithm 3.15
4: $\mathbf{i}_{\mathtt{aux}} \leftarrow (\mathbf{i}_{\mathtt{aux}}, \mathsf{IntegerToString}(\gamma_1), \mathsf{IntegerToString}(\phi_{1,0}), \ldots, \mathsf{IntegerToString}(\phi_{1,\delta-1}))$
5: $\mathbf{i}_{\mathtt{aux}} \leftarrow (\mathbf{i}_{\mathtt{aux}}, \mathsf{IntegerToString}(\gamma_2), \mathsf{IntegerToString}(\phi_{2,0}), \ldots, \mathsf{IntegerToString}(\phi_{2,\psi-1}))$
6: VerifExp $\leftarrow$ VerifyExponentiation$((g, \gamma_1, \phi_{1,0}), (\mathtt{K_{id}}, \gamma_1^{\mathtt{k_{id}}}, \phi_{1,0}^{\mathtt{k_{id}}}), \pi_{\mathtt{Exp}}, \mathbf{i}_{\mathtt{aux}})$
7: VerifEqEnc $\leftarrow$ VerifyPlaintextEquality$(\widetilde{\mathtt{E1}}, \widetilde{\mathtt{E2}}, \mathtt{EL_{pk,0}}, \widetilde{\mathtt{pk}}_{\mathrm{CCR}}, \pi_{\mathtt{EqEnc}}, \mathbf{i}_{\mathtt{aux}})$
8: **if** VerifExp $\wedge$ VerifEqEnc **then**
    return $\top$
9: **else**
    return $\bot$
10: **end if**

Output:

The result of the verification: $\top$ if the verification is successful, $\bot$ otherwise.

The VerifyBallotCCR algorithm does *not* modify any state. However, the subsequent algorithm requires the successful execution of *all* verifications and checks.

Copyright 2026 Swiss Post Ltd.

5.2.3 PartialDecryptPCC

Upon successful execution of VerifyBallotCCR the control components remove the encryption layer from the partial Choice Return Codes $\mathbf{pCC}_{\mathtt{id}}$.
One might ask why $\mathbf{pCC}_{\mathtt{id}}$ is encrypted at all, given that both the untrustworthy voting client and voting server learn the plaintext codes in any case during normal protocol execution. Historically, the protocol included this encryption layer to protect against eavesdroppers trying to infer whether the voter chose the same voting option twice; exponentiating the same prime number to the same verification card secret key $\mathtt{k}_{\mathtt{id}}$ results in the same partial Choice Return Code. Currently, both conditions no longer hold: the current protocol defends against a much stronger adversary (controlling the voting server), and enforces that the voting client cannot select the same voting option twice.
Nevertheless, the additional encryption layer remains in the current protocol version since it is still relevant for other parts of the system. The encryption layer, and in particular the partial decryption executed by the control components, plays an important role for the agreement procedure before tallying to resolve potential discrepancies (see section 6.2). In the partial decryption step, each control component computes an exponentiation proof. This exponentiation proof acts as a commitment to the content of the encrypted vote. Before processing the encrypted vote, each control component verifies the exponentiation proofs from all other control components. Hence, providing a set of one valid exponentiation proof for each control component confirms that they all committed to process this vote. This ensures that for each voter, there is consensus on which encrypted vote to be included in the tally.

Copyright 2026 Swiss Post Ltd.

Algorithm 5.6 uses the list of voting cards $\mathsf{L}_{\mathtt{decPCC},j}$ for which the control components decrypted the partial Choice Return Codes $\mathbf{pCC}_{\mathtt{id}}$.

Algorithm 5.6 PartialDecryptPCC

Context:

The CCR's index $j \in [1, 4]$

Verification card ID $\mathtt{vc}_{\mathtt{id}} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$

Number of allowed selections for this specific ballot box $\psi \in [1, \psi_{\mathtt{sup}}]$ ▷ Can be derived from pTable using algorithm 3.11

Number of allowed write-ins + 1 for this specific ballot box $\delta \in [1, \delta_{\mathtt{sup}}]$ ▷ Can be derived from pTable using algorithm 3.12

Extracted election event $\mathtt{eee} = (\mathtt{hContext}, (p, q, g), \mathtt{ee}, \mathbf{evcs})$

Stateful Lists and Maps:

List of voting cards with decrypted partial Choice Return Codes $\mathsf{L}_{\mathtt{decPCC},j}$

Input:

Encrypted vote $\mathtt{E1} = (\gamma_1, \phi_{1,0}, \ldots, \phi_{1,\delta-1}) \in {\mathbb{G}_q}^{\delta+1}$

Exponentiated encrypted vote $\widetilde{\mathtt{E1}} = (\gamma_1^{\mathtt{k_{id}}}, \phi_{1,0}^{\mathtt{k_{id}}}) \in {\mathbb{G}_q}^{2}$

Encrypted partial Choice Return Codes $\mathtt{E2} = (\gamma_2, \phi_{2,0}, \ldots, \phi_{2,\psi-1}) \in {\mathbb{G}_q}^{\psi+1}$

CCR_j Choice Return Codes encryption secret key $(\mathtt{sk}_{\mathrm{CCR}_{j,0}}, \ldots, \mathtt{sk}_{\mathrm{CCR}_{j,\psi_{\mathtt{max}}-1}}) \in \mathbb{Z}_q^{\psi_{\mathtt{max}}}$

CCR_j Choice Return Codes encryption public key $(\mathtt{pk}_{\mathrm{CCR}_{j,0}}, \ldots, \mathtt{pk}_{\mathrm{CCR}_{j,\psi_{\mathtt{max}}-1}}) \in \mathbb{G}_q^{\psi_{\mathtt{max}}}$

Require: $\mathtt{vc}_{\mathtt{id}} \notin \mathsf{L}_{\mathtt{decPCC},j}$

Operation: ▷ For all algorithms see the crypto primitives specification

1: $\mathbf{i}_{\mathtt{aux}} \leftarrow$ ("PartialDecryptPCC", $\mathtt{vc}_{\mathtt{id}}$, GetHashExtractedElectionEvent()) ▷ See algorithm 3.18
2: $\mathbf{i}_{\mathtt{aux}} \leftarrow (\mathbf{i}_{\mathtt{aux}}, \mathsf{IntegerToString}(\gamma_1), \mathsf{IntegerToString}(\phi_{1,0}), \ldots, \mathsf{IntegerToString}(\phi_{1,\delta-1}))$
3: $\mathbf{i}_{\mathtt{aux}} \leftarrow (\mathbf{i}_{\mathtt{aux}}, \mathsf{IntegerToString}(\gamma_1^{\mathtt{k_{id}}}), \mathsf{IntegerToString}(\phi_{1,0}^{\mathtt{k_{id}}}))$
4: $\mathbf{i}_{\mathtt{aux}} \leftarrow (\mathbf{i}_{\mathtt{aux}}, \mathsf{IntegerToString}(\gamma_2), \mathsf{IntegerToString}(\phi_{2,0}), \ldots, \mathsf{IntegerToString}(\phi_{2,\psi-1}))$
5: $\mathbf{i}_{\mathtt{aux}} \leftarrow (\mathbf{i}_{\mathtt{aux}}, \mathsf{IntegerToString}(j))$
6: **for** $i \in [0, \psi)$ **do**
7: $\quad d_{j,i} \leftarrow \gamma_2^{\mathtt{sk}_{\mathrm{CCR}_{j,i}}} \bmod p$
8: $\quad \pi_{\mathsf{decPCC},j,i} \leftarrow \mathsf{GenExponentiationProof}((g, \gamma_2), \mathtt{sk}_{\mathrm{CCR}_{j,i}}, (\mathtt{pk}_{\mathrm{CCR}_{j,i}}, d_{j,i}), \mathbf{i}_{\mathtt{aux}})$
9: **end for**
10: $\mathsf{L}_{\mathtt{decPCC},j} \leftarrow \mathsf{L}_{\mathtt{decPCC},j} \,||\, \mathtt{vc}_{\mathtt{id}}$

Output:

Exponentiated γ elements $\mathbf{d_j} = (d_{j,0}, \ldots, d_{j,\psi-1}) \in {\mathbb{G}_q}^{\psi}$

Exponentiation proofs $\boldsymbol{\pi}_{\mathsf{decPCC},j} = (\pi_{\mathsf{decPCC},j,0}, \ldots, \pi_{\mathsf{decPCC},j,\psi-1}) \in (\mathbb{Z}_q \times \mathbb{Z}_q)^{\psi}$

Copyright 2026 Swiss Post Ltd.

5.2.4 DecryptPCC

Algorithm 5.7 DecryptPCC

Context:

The CCR's index $j \in [1, 4]$
The other CCR's indices $\hat{\mathbf{j}} = (\hat{j}_1, \hat{j}_2, \hat{j}_3) = (1, 2, 3, 4) \backslash j$
Verification card ID $\mathtt{vc_{id}} \in (\mathbb{A}_{Base16})^{1_{\mathtt{ID}}}$
Number of allowed selections for this specific ballot box $\psi \in [1, \psi_{\mathtt{sup}}]$ ▷ Can be derived from pTable using algorithm 3.11
Number of allowed write-ins + 1 for this specific ballot box $\delta \in [1, \delta_{\mathtt{sup}}]$ ▷ Can be derived from pTable using algorithm 3.12
Extracted election event $\mathtt{eee} = (\mathtt{hContext}, (p, q, g), \mathbf{ee}, \mathbf{evcs})$
Other CCR's Choice Return Codes encryption keys $(\mathbf{pk}_{\mathrm{CCR}_{\hat{j}_1}}, \mathbf{pk}_{\mathrm{CCR}_{\hat{j}_2}}, \mathbf{pk}_{\mathrm{CCR}_{\hat{j}_3}}) \in (\mathbb{G}_q^{\psi_{\max}})^3$

Input:

$\mathtt{CCR_j}$'s exponentiated γ elements $\mathbf{d_j} = (d_{j,0}, \ldots, d_{j,\psi-1}) \in {\mathbb{G}_q}^{\psi}$
Other CCR's exponentiated γ elements $(\mathbf{d_{\hat{j}_1}}, \mathbf{d_{\hat{j}_2}}, \mathbf{d_{\hat{j}_3}}) \in ({\mathbb{G}_q}^{\psi})^3$
Other CCR's exponentiation proofs $(\boldsymbol{\pi}_{\mathsf{decPCC},\hat{j}_1}, \boldsymbol{\pi}_{\mathsf{decPCC},\hat{j}_2}, \boldsymbol{\pi}_{\mathsf{decPCC},\hat{j}_3}) \in (\mathbb{Z}_q \times \mathbb{Z}_q)^{\psi \times 3}$
Encrypted vote $\mathtt{E1} = (\gamma_1, \phi_{1,0}, \ldots, \phi_{1,\delta-1}) \in {\mathbb{G}_q}^{\delta+1}$
Exponentiated encrypted vote $\widetilde{\mathtt{E1}} = (\gamma_1^{\mathtt{k_{id}}}, \phi_{1,0}^{\mathtt{k_{id}}}) \in {\mathbb{G}_q}^{2}$
Encrypted partial Choice Return Codes $\mathtt{E2} = (\gamma_2, \phi_{2,0}, \ldots, \phi_{2,\psi-1}) \in {\mathbb{G}_q}^{\psi+1}$

Operation: ▷ For all algorithms see the crypto primitives specification

1: **for** $k \in \hat{\mathbf{j}}$ **do**
2: $\mathbf{i}_{\mathtt{aux}} \leftarrow (\text{“}\mathtt{PartialDecryptPCC}\text{”}, \mathtt{vc_{id}}, \mathsf{GetHashExtractedElectionEvent}())$ ▷ See algorithm 3.18
3: $\mathbf{i}_{\mathtt{aux}} \leftarrow (\mathbf{i}_{\mathtt{aux}}, \mathsf{IntegerToString}(\gamma_1), \mathsf{IntegerToString}(\phi_{1,0}), \ldots, \mathsf{IntegerToString}(\phi_{1,\delta-1}))$
4: $\mathbf{i}_{\mathtt{aux}} \leftarrow (\mathbf{i}_{\mathtt{aux}}, \mathsf{IntegerToString}(\gamma_1^{\mathtt{k_{id}}}), \mathsf{IntegerToString}(\phi_{1,0}^{\mathtt{k_{id}}}))$
5: $\mathbf{i}_{\mathtt{aux}} \leftarrow (\mathbf{i}_{\mathtt{aux}}, \mathsf{IntegerToString}(\gamma_2), \mathsf{IntegerToString}(\phi_{2,0}), \ldots, \mathsf{IntegerToString}(\phi_{2,\psi-1}))$
6: $\mathbf{i}_{\mathtt{aux}} \leftarrow (\mathbf{i}_{\mathtt{aux}}, \mathsf{IntegerToString}(k))$
7: $\mathbf{pk}_{\mathrm{CCR}_k} = (\mathbf{pk}_{\mathrm{CCR}_{k,0}}, \cdots, \mathbf{pk}_{\mathrm{CCR}_{k,\psi_{\max}-1}})$
8: $\mathbf{d_k} = (d_{k,0}, \ldots, d_{k,\psi-1})$
9: $\boldsymbol{\pi}_{\mathsf{decPCC},k} = (\pi_{\mathsf{decPCC},k,0}, \ldots, \pi_{\mathsf{decPCC},k,\psi-1})$
10: **for** $i \in [0, \psi)$ **do**
11: **if** $\neg\mathsf{VerifyExponentiation}((g, \gamma_2), (\mathbf{pk}_{\mathrm{CCR}_{k,i}}, d_{k,i}), \pi_{\mathsf{decPCC},k,i}, \mathbf{i}_{\mathtt{aux}})$ **then**
12: **return** $\perp$
13: **end if**
14: **end for**
15: **end for**
16: **for** $i \in [0, \psi)$ **do**
17: $d_i \leftarrow (d_{j,i} \cdot d_{\hat{j}_1,i} \cdot d_{\hat{j}_2,i} \cdot d_{\hat{j}_3,i}) \bmod p$
18: $\mathtt{pCC_{id},}_i \leftarrow \frac{\phi_{2,i}}{d_i} \bmod p$
19: **end for**
20: $\mathbf{pCC_{id}} \leftarrow (\mathtt{pCC_{id},}_0, \ldots, \mathtt{pCC_{id},}_{\psi-1})$

Output:

21: **if** *all* control components' zero-knowledge proofs verify **then**
$\mathbf{pCC_{id}} = (\mathtt{pCC_{id},}_0, \ldots, \mathtt{pCC_{id},}_{\psi-1}) \in {\mathbb{G}_q}^{\psi}$
22: **else**
$\perp$, the verification of at least one $\pi_{\mathsf{decPCC},j,i}$ failed.
23: **end if**

Copyright 2026 Swiss Post Ltd.

5.2.5 CreateLCCShare

Upon receipt of the other control components' contributions $\mathbf{d_j}$ and $\boldsymbol{\pi}_{\mathsf{decPCC},j}$, the Return Codes control components CCR check the zero-knowledge proofs, combine the inputs, and generate long Choice Return Codes shares using the voting client's partial Choice Return Codes $\mathbf{pCC}_{\mathtt{id}}$ and the CCR_j Return Codes Generation secret key $\mathtt{k'}_j$. Each of the four control components $j \in [1, 4]$ executes this algorithm. The control components use the following lists:

- list of voting cards $\mathsf{L}_{\mathtt{decPCC},j}$ for which the control components decrypted the partial Choice Return Codes $\mathbf{pCC}_{\mathtt{id}}$,
- list of voting cards $\mathsf{L}_{\mathtt{sentVotes},j}$ for which the control components already generated long Choice Return Code shares,
- the partial Choice Return Codes allow list $\mathsf{L}_{\mathtt{pCC}}$.

Importantly, the CreateLCCShare returns *no* $\mathtt{CCR}_\mathtt{j}$'s long Choice Return Code shares $\mathtt{lCC}_{j,\mathtt{id},i}$ if one or more validations within the algorithm fail.
Moreover, the honest control component guarantees the following before generating the long Choice Return Code shares $\mathbf{lCC}_{j,\mathtt{id}}$:

- the zero-knowledge proofs ascertain that the product of partial Choice Return Codes correspond to the exponentiated encrypted vote (see algorithm VerifyBallotCCR),
- the other control components correctly decrypted the partial Choice Return Codes (see algorithm DecryptPCC),
- every partial Choice Return Code is valid since each corresponds to an entry in the partial Choice Return Codes allow list $\mathsf{L}_{\mathtt{pCC}}$,
- the combination of partial Choice Return Codes is correct.

These verifications prevent attacks against the malleability of the zero-knowledge proofs (see Haines et al. [14]) and ensure that the honest control components detect attacks against individual verifiability at the time of voting (see Gitlab issue 7).

Copyright 2026 Swiss Post Ltd.

Algorithm 5.8 CreateLCCShare

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
The CCR's index $j \in [1, 4]$
Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Verification card set ID $\mathtt{vcs} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Verification card ID $\mathtt{vc_{id}} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Blank correctness information $\hat{\boldsymbol{\tau}} = (\hat{\tau}_0, \ldots, \hat{\tau}_{\psi-1}) \in (\mathcal{D}_\tau)^\psi$ ▷ Can be derived from pTable using algorithm 3.6

Stateful Lists and Maps:

List of voting cards with decrypted partial Choice Return Codes $\mathsf{L}_{\mathtt{decPCC},j}$
$\mathtt{CCR_j}$'s list of sent voting cards $\mathsf{L}_{\mathtt{sentVotes},j}$

Input:

Partial Choice Return Codes allow list $\mathsf{L}_{\mathtt{pCC}} \in (\mathbb{A}_{Base64}{}^{\mathtt{l_{HB64}}})^{n \cdot \mathtt{N_E}}$
A vector of partial Choice Return Codes $\mathbf{pCC}_{\mathtt{id}} \in (\mathbb{G}_q)^\psi$
CCR_j Return Codes Generation secret key $\mathtt{k'}_j \in \mathbb{Z}_q$

Require: $\mathtt{pCC_{id},}_i \neq \mathtt{pCC_{id},}_k, \forall i, k \in \{0, \ldots, (\psi - 1)\} \wedge i \neq k$ ▷ All pCC must be distinct
Require: $\mathtt{vc_{id}} \in \mathsf{L}_{\mathtt{decPCC},j}$
Require: $\mathtt{vc_{id}} \notin \mathsf{L}_{\mathtt{sentVotes},j}$

Operation: ▷ For all algorithms see the crypto primitives specification

1: $\mathtt{PRK} \leftarrow \mathsf{IntegerToByteArray}(\mathtt{k'}_j)$
2: $\mathtt{info} \leftarrow (\text{“}\mathtt{VoterChoiceReturnCodeGeneration}\text{”}, \mathtt{ee}, \mathtt{vcs}, \mathtt{vc_{id}})$
3: $\mathtt{k}_{j,\mathtt{id}} \leftarrow \mathsf{KDFToZq}(\mathtt{PRK}, \mathtt{info}, q)$
4: **for** $i \in [0, \psi)$ **do**
5: $\quad \mathtt{hpCC_{id},}_i \leftarrow \mathsf{HashAndSquare}(\mathtt{pCC_{id},}_i)$
6: $\quad \mathtt{lpCC_{id},}_i \leftarrow \mathsf{RecursiveHash}(\mathtt{hpCC_{id},}_i, \mathtt{vc_{id}}, \mathtt{ee}, \hat{\tau}_i)$
7: $\quad$ **if** $\mathsf{Base64Encode}(\mathtt{lpCC_{id},}_i) \notin \mathsf{L}_{\mathtt{pCC}}$ **then**
8: $\quad\quad$ **return** $\perp$
9: $\quad$ **else**
10: $\quad\quad \mathtt{lCC}_{j,\mathtt{id},i} \leftarrow \mathtt{hpCC}_{\mathtt{id},i}^{\mathtt{k}_{j,\mathtt{id}}} \bmod p$
11: $\quad$ **end if**
12: **end for**
13: $\mathbf{lCC}_{j,\mathtt{id}} \leftarrow (\mathtt{lCC}_{j,\mathtt{id},0}, ..., \mathtt{lCC}_{j,\mathtt{id},\psi-1})$
14: $\mathsf{L}_{\mathtt{sentVotes},j} \leftarrow \mathsf{L}_{\mathtt{sentVotes},j} \,||\, \mathtt{vc_{id}}$

Output:

$\mathtt{CCR_j}$'s long Choice Return Code share $\mathbf{lCC}_{j,\mathtt{id}} = (\mathtt{lCC}_{j,\mathtt{id},0}, \ldots, \mathtt{lCC}_{j,\mathtt{id},\psi-1}) \in \mathbb{G}_q{}^\psi$

Copyright 2026 Swiss Post Ltd.

5.2.6 ExtractCRC

The voting server extracts the short Choice Return Codes $\mathbf{CC}_{\mathtt{id}}$ from the Return Codes Mapping table $\mathsf{CMtable}$. Subsequently, the voting server sends the short Choice Return Codes $\mathbf{CC}_{\mathtt{id}}$ to the voting client and the voter compares them to the one printed on the voting card.

Algorithm 5.9 ExtractCRC

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Verification card ID $\mathtt{vc_{id}} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Blank correctness information $\hat{\boldsymbol{\tau}} = (\hat{\tau}_0, \ldots, \hat{\tau}_{\psi-1}) \in (\mathcal{D}_\tau)^\psi$ ▷ Can be derived from pTable using algorithm 3.6

Input:

CCR long Choice Return Code shares $(\mathbf{lCC}_{1,\mathtt{id}}, \mathbf{lCC}_{2,\mathtt{id}}, \mathbf{lCC}_{3,\mathtt{id}}, \mathbf{lCC}_{4,\mathtt{id}}) \in (\mathbb{G}_q{}^\psi)^4$
Return Codes Mapping table $\mathsf{CMtable} \in (\mathbb{A}_{Base64}{}^{\mathtt{l_{HB64}}} \times \mathbb{A}_{Base64}{}^*)^{\mathtt{N_E}\cdot(n+1)}$

Operation: ▷ For all algorithms see the crypto primitives specification

1: **for** $i \in [0, \psi)$ **do**
2: $\quad \mathtt{pC}_{\mathtt{id},i} \leftarrow \prod_{j=1}^{4} \mathtt{lCC}_{j,\mathtt{id},i} \bmod p$
3: $\quad \mathtt{lCC}_{\mathtt{id},i} \leftarrow \mathsf{RecursiveHash}(\mathtt{pC}_{\mathtt{id},i}, \mathtt{vc_{id}}, \mathtt{ee}, \hat{\tau}_i)$
4: $\quad \mathtt{key} \leftarrow \mathsf{Base64Encode}(\mathsf{RecursiveHash}(\mathtt{lCC}_{\mathtt{id},i}))$
5: $\quad$ **if** $\mathtt{key} \notin \mathsf{CMtable}$ **then**
6: $\quad\quad$ **return** $\perp$
7: $\quad$ **else**
8: $\quad\quad \mathtt{ctCC}_{\mathtt{id},i,\mathtt{encoded}} \leftarrow \mathsf{CMtable}(\mathtt{key})$ ▷ Retrieve the value from the key-value map
9: $\quad\quad \mathtt{ctCC}_{\mathtt{id},i,\mathtt{combined}} \leftarrow \mathsf{Base64Decode}(\mathtt{ctCC}_{\mathtt{id},i,\mathtt{encoded}})$
10: $\quad\quad \mathtt{length} \leftarrow$ Length in bytes of $\mathtt{ctCC}_{\mathtt{id},i,\mathtt{combined}}$
11: $\quad\quad \mathtt{split} \leftarrow \mathtt{length} - \mathtt{nonceLength}$ ▷ Nonce length in bytes defined by the symmetric algorithm used; see crypto primitives specification
12: $\quad\quad \mathtt{ctCC}_{\mathtt{id},i,\mathtt{ciphertext}} \leftarrow \mathtt{ctCC}_{\mathtt{id},i,\mathtt{combined}}[0 : \mathtt{split})$
13: $\quad\quad \mathtt{ctCC}_{\mathtt{id},i,\mathtt{nonce}} \leftarrow \mathtt{ctCC}_{\mathtt{id},i,\mathtt{combined}}[\mathtt{split} : \mathtt{length})$
14: $\quad\quad \mathtt{skcc}_{\mathtt{id},i} \leftarrow \mathsf{KDF}(\mathtt{lCC}_{\mathtt{id},i}, (), \mathtt{l_{KD}})$
15: $\quad\quad \mathtt{CC}_{\mathtt{id},i,\mathtt{bytes}} \leftarrow \mathsf{GetPlaintextSymmetric}(\mathtt{skcc}_{\mathtt{id},i}, \mathtt{ctCC}_{\mathtt{id},i,\mathtt{ciphertext}}, \mathtt{ctCC}_{\mathtt{id},i,\mathtt{nonce}}, ())$
16: $\quad\quad \mathtt{CC}_{\mathtt{id},i} \leftarrow \mathsf{ByteArrayToString}(\mathtt{CC}_{\mathtt{id},i,\mathtt{bytes}})$
17: $\quad$ **end if**
18: **end for**
19: $\mathbf{CC}_{\mathtt{id}} \leftarrow (\mathtt{CC}_{\mathtt{id},0}, \ldots, \mathtt{CC}_{\mathtt{id},\psi-1})$

Output:

Short Choice Return Codes $\mathbf{CC}_{\mathtt{id}} \in ((\mathbb{A}_{10})^{\mathtt{l_{CC}}})^\psi$ ▷ The algorithm returns the short Choice Return Codes only if *all* of them are extractable.

Copyright 2026 Swiss Post Ltd.

5.2.7 Process Choice Return Codes

In the beginning of the SendVote sub-protocol (see Figure 11), the voting client derives the selected abstentions $\hat{\mathbf{a}}_{\mathtt{id}}$ from the voter's selections and passes it as an input to Algorithm 5.4 CreateVote.
At the end of the SendVote sub-protocol, after the voting server has returned the individual short Choice Return Codes $\mathbf{CC}_{\mathtt{id}}$, the voting client invokes Algorithm 3.31 ProcessAbstentionChoiceReturnCodes with the short Choice Return Codes and the selected abstentions $\hat{\mathbf{a}}_{\mathtt{id}}$. For each abstention group where the voter chose to abstain, Algorithm 3.31 ProcessAbstentionChoiceReturnCodes produces a merged Abstention Choice Return Code $\mathtt{ACC}$; for non-abstaining groups the individual short Choice Return Codes are displayed as usual. The voter can then compare the codes displayed on screen with those printed on the voting card.

5.2.8 Merge Return Codes upon Relogin

If a voter starts the voting process on one device and sends the encrypted vote (see Algorithm 5.4 CreateVote) using that device, but later logs in on a different device, this second device must be able display the Choice Return Codes so that the voter can complete the voting process. However, the second device has no local knowledge of the selected abstentions $\hat{\mathbf{a}}_{\mathtt{id}}$ from the interaction on the first device.
Therefore, as part of Algorithm 5.4 CreateVote, the voting client encrypts the selected abstentions $\hat{\mathbf{a}}_{\mathtt{id}}$ and sends them to the voting server along with the encrypted vote (see Section 3.9.2). Upon a relogin, the voting server returns the stored $\mathtt{ct}_{\mathbf{a},\mathtt{id}}$ together with the Choice Return Codes $\mathbf{CC}_{\mathtt{id}}$. The voting client on the second device re-derives the verification card secret key $\mathtt{k}_{\mathtt{id}}$ using Algorithm 5.3 GetKey, decrypts $\mathtt{ct}_{\mathbf{a},\mathtt{id}}$ to recover the selected abstentions $\hat{\mathbf{a}}_{\mathtt{id}}$ using Algorithm 3.33 GetDecryptedAbstentionVector, and finally invokes Algorithm 3.31 ProcessAbstentionChoiceReturnCodes to retrieve the Abstention Choice Return Code $\mathtt{ACC}$ if necessary.
The voting server never learns the selected abstentions $\hat{\mathbf{a}}_{\mathtt{id}}$, since $\mathtt{ct}_{\mathbf{a},\mathtt{id}}$ is opaque to the voting server.
Figure 12 illustrates a relogin on a second device.

Copyright 2026 Swiss Post Ltd.

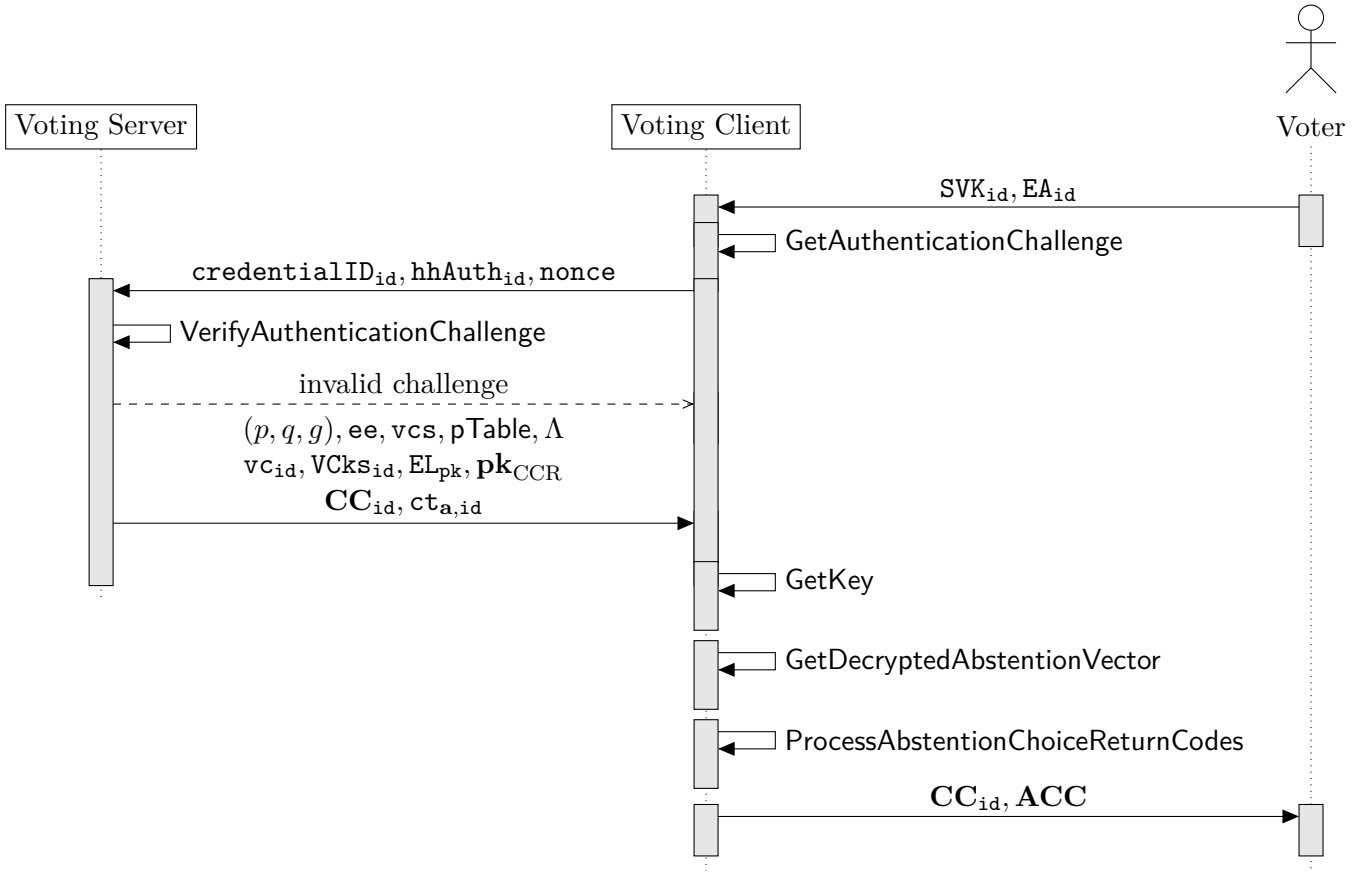


Fig. 12: Sequence diagram of the relogin flow for merging abstention return codes.

Copyright 2026 Swiss Post Ltd.

5.3 ConfirmVote

After the voter successfully checked the short Choice Return Codes and Abstention Choice Return Codes, the ConfirmVote sub-protocol comprises confirming the vote until receiving the Vote Cast Return Code. Figure 13 highlights the algorithms and data flows.

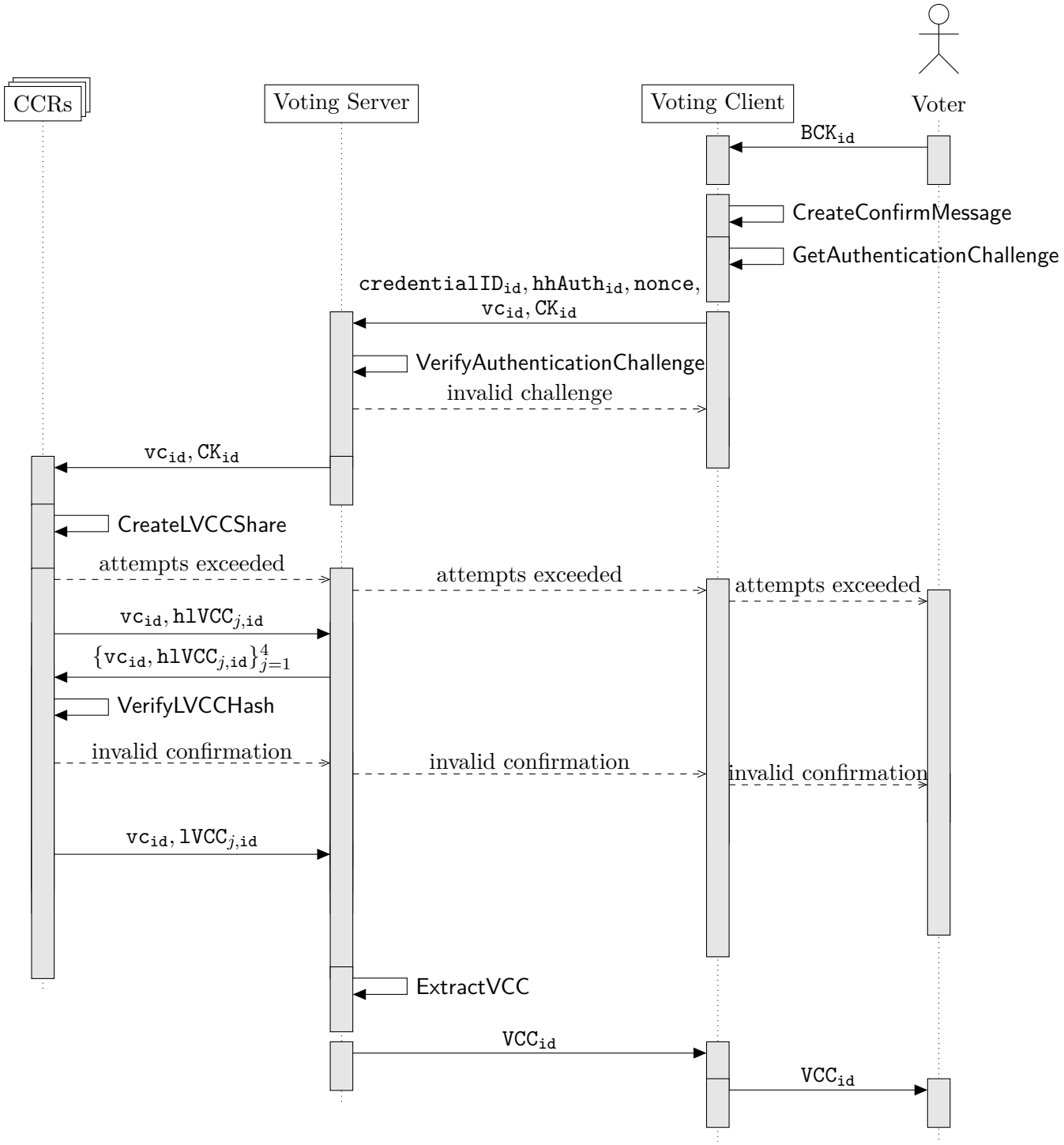


Fig. 13: Sequence diagram of the ConfirmVote sub-protocol.

Copyright 2026 Swiss Post Ltd.

5.3.1 CreateConfirmMessage

The voter enters the Ballot Casting Key $\texttt{BCK}_{\texttt{id}}$ in the voting client, which executes the CreateConfirmMessage algorithm.

Algorithm 5.10 CreateConfirmMessage

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$

Input:

Ballot Casting Key $\texttt{BCK}_{\texttt{id}} \in (\mathbb{A}_{10})^{l_{\texttt{BCK}}}$ ▷ Must contain one non-zero element
Verification Card Secret Key $\texttt{k}_{\texttt{id}} \in \mathbb{Z}_q$

Operation:

1: $\texttt{hBCK}_{\texttt{id}} \leftarrow \mathsf{HashAndSquare}(\mathsf{StringToInteger}(\texttt{BCK}_{\texttt{id}}))$ ▷ See crypto primitives specification
2: $\texttt{CK}_{\texttt{id}} \leftarrow \texttt{hBCK}_{\texttt{id}}^{\texttt{k}_{\texttt{id}}} \bmod p$

Output:

Confirmation Key $\texttt{CK}_{\texttt{id}} \in \mathbb{G}_q$

Copyright 2026 Swiss Post Ltd.

5.3.2 CreateLVCCShare

The control components execute the CreateLVCCShare algorithm. CreateLVCCShare is similar to CreateLCCShare (see section 5.2.5). However, while the control components run CreateLCCShare at most once, the control components allow up to five executions of CreateLVCCShare — in case the voter entered an invalid ballot casting key. Furthermore, the control components derive their share using the Voter Vote Cast Return Code Generation secret key $\mathtt{kc}_{j,\mathtt{id}}$ instead of the Voter Choice Return Code Generation secret key $\mathtt{k}_{j,\mathtt{id}}$ to prevent the adversary from using the CreateLVCCShare algorithm as an oracle to learn Choice Return Codes.

Algorithm 5.11 CreateLVCCShare

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
The CCR's index $j \in [1, 4]$
Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Verification card set ID $\mathtt{vcs} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Verification card ID $\mathtt{vc_{id}} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$

Stateful Lists and Maps:

$\mathtt{CCR_j}$'s list of sent voting cards $\mathsf{L}_{\mathtt{sentVotes},j}$
$\mathtt{CCR_j}$'s key-value map of number of confirmation attempts per verification card $\mathsf{L}_{\mathtt{confirmationAttempts},j}$
$\mathtt{CCR_j}$'s list of confirmed voting cards $\mathsf{L}_{\mathtt{confirmedVotes},j}$

Input:

Confirmation Key $\mathtt{CK_{id}} \in \mathbb{G}_q$
CCR_j Return Codes Generation secret key $\mathtt{k'}_j \in \mathbb{Z}_q$

Require: $\mathtt{vc_{id}} \in \mathsf{L}_{\mathtt{sentVotes},j}$
Require: $\mathtt{vc_{id}} \notin \mathsf{L}_{\mathtt{confirmedVotes},j}$

Operation: ▷ For all algorithms see the crypto primitives specification

1: $\mathtt{attempts_{id}} \leftarrow \mathsf{L}_{\mathtt{confirmationAttempts},j}(\mathtt{vc_{id}})$ ▷ Retrieve the value from the key-value map
2: **if** $\mathtt{attempts_{id}} \geq 5$ **then return** $\perp$ ▷ The voter has only 5 attempts to confirm the vote.
3: **end if**
4: $\mathtt{PRK} \leftarrow \mathsf{IntegerToByteArray}(\mathtt{k'}_j)$
5: $\mathtt{info_{CK}} \leftarrow (\text{“}\mathtt{VoterVoteCastReturnCodeGeneration}\text{”}, \mathtt{ee}, \mathtt{vcs}, \mathtt{vc_{id}})$
6: $\mathtt{kc}_{j,\mathtt{id}} \leftarrow \mathsf{KDFToZq}(\mathtt{PRK}, \mathtt{info_{CK}}, q)$
7: $\mathtt{hCK_{id}} \leftarrow \mathsf{HashAndSquare}(\mathtt{CK_{id}})$
8: $\mathtt{lVCC}_{j,\mathtt{id}} \leftarrow \mathtt{hCK_{id}}^{\mathtt{kc}_{j,\mathtt{id}}} \bmod p$
9: $\mathbf{i}_{\mathtt{aux}} \leftarrow (\text{“}\mathtt{CreateLVCCShare}\text{”}, \mathtt{ee}, \mathtt{vcs}, \mathtt{vc_{id}}, \mathsf{IntegerToString}(j))$
10: $\mathtt{hlVCC}_{j,\mathtt{id}} \leftarrow \mathsf{Base64Encode}(\mathsf{RecursiveHash}(\mathbf{i}_{\mathtt{aux}}, \mathtt{lVCC}_{j,\mathtt{id}}))$
11: $\mathsf{L}_{\mathtt{confirmationAttempts},j}(\mathtt{vc_{id}}) \leftarrow \mathtt{attempts_{id}} + 1$ ▷ Set the value in the key-value map

Output:

$\mathtt{CCR_j}$'s long Vote Cast Return Code share $\mathtt{lVCC}_{j,\mathtt{id}} \in \mathbb{G}_q$
$\mathtt{CCR_j}$'s hashed long Vote Cast Return Code share $\mathtt{hlVCC}_{j,\mathtt{id}} \in \mathbb{A}_{Base64}{}^{\mathtt{l_{HB64}}}$
Confirmation attempt number $\mathtt{attempts_{id}} \in [0, 4]$

Copyright 2026 Swiss Post Ltd.

5.3.3 VerifyLVCCHash

The VerifyLVCCHash algorithm allows the control components to determine if the confirmation attempt was successful—or if the confirmation attempt corresponds to an incorrect Ballot Casting Key $\texttt{BCK}_{\texttt{id}}$.
Only if the $\texttt{CCR}_{\texttt{j}}$'s hashed long Vote Cast Return Code share $\texttt{hlVCC}_{j,\texttt{id}}$ is extractable from the long Vote Cast Return Codes allow list $\mathsf{L}_{\texttt{lVCC}}$, do the control components return their $\texttt{CCR}_{\texttt{j}}$'s long Vote Cast Return Code share $\texttt{lVCC}_{j,\texttt{id}}$ to the voting server.

Algorithm 5.12 VerifyLVCCHash

Context:
Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
The CCR's index $j \in [1, 4]$
The other CCR's indices $\hat{\mathbf{j}} = (\hat{j}_1, \hat{j}_2, \hat{j}_3) = (1, 2, 3, 4) \backslash j$
Election event ID $\texttt{ee} \in (\mathbb{A}_{Base16})^{\texttt{l}_{\texttt{ID}}}$
Verification card set ID $\texttt{vcs} \in (\mathbb{A}_{Base16})^{\texttt{l}_{\texttt{ID}}}$
Verification card ID $\texttt{vc}_{\texttt{id}} \in (\mathbb{A}_{Base16})^{\texttt{l}_{\texttt{ID}}}$
Stateful Lists and Maps:
$\texttt{CCR}_{\texttt{j}}$'s list of sent voting cards $\mathsf{L}_{\texttt{sentVotes},j}$
$\texttt{CCR}_{\texttt{j}}$'s list of confirmed voting cards $\mathsf{L}_{\texttt{confirmedVotes},j}$
Input:
Long Vote Cast Return Codes allow list $\mathsf{L}_{\texttt{lVCC}} \in (\mathbb{A}_{Base64}{}^{\texttt{l}_{\texttt{HB64}}})^{\texttt{N}_{\texttt{E}}}$
$\texttt{CCR}_{\texttt{j}}$'s hashed long Vote Cast Return Code share $\texttt{hlVCC}_{j,\texttt{id}} \in \mathbb{A}_{Base64}{}^{\texttt{l}_{\texttt{HB64}}}$
Other CCR's hashed long Vote Cast Return Code shares $(\texttt{hlVCC}_{\hat{j}_1,\texttt{id}}, \texttt{hlVCC}_{\hat{j}_2,\texttt{id}}, \texttt{hlVCC}_{\hat{j}_3,\texttt{id}}) \in (\mathbb{A}_{Base64}{}^{\texttt{l}_{\texttt{HB64}}})^3$
Require: $\texttt{vc}_{\texttt{id}} \in \mathsf{L}_{\texttt{sentVotes},j}$
Require: $\texttt{vc}_{\texttt{id}} \notin \mathsf{L}_{\texttt{confirmedVotes},j}$

Operation: ▷ For all algorithms see the crypto primitives specification

1: $\mathbf{i}_{\texttt{aux}} \leftarrow$ ("VerifyLVCCHash", ee, vcs, $\texttt{vc}_{\texttt{id}}$)
2: $\texttt{hhlVCC}_{\texttt{id}} \leftarrow \mathsf{Base64Encode}(\mathsf{RecursiveHash}(\mathbf{i}_{\texttt{aux}}, \texttt{hlVCC}_{1,\texttt{id}}, \texttt{hlVCC}_{2,\texttt{id}}, \texttt{hlVCC}_{3,\texttt{id}}, \texttt{hlVCC}_{4,\texttt{id}}))$
3: **if** $\texttt{hhlVCC}_{\texttt{id}} \in \mathsf{L}_{\texttt{lVCC}}$ **then**
4: $\quad \mathsf{L}_{\texttt{confirmedVotes},j} \leftarrow \mathsf{L}_{\texttt{confirmedVotes},j} \,||\, \texttt{vc}_{\texttt{id}}$
5: $\quad$ **return** $\top$
6: **else**
7: $\quad$ **return** $\bot$
8: **end if**

Output:
The result of the verification: $\top$ if the verification is successful and an updated list of $\mathsf{L}_{\texttt{confirmedVotes},j}$, $\bot$ otherwise.

The control component sends the $\texttt{CCR}_{\texttt{j}}$'s long Vote Cast Return Code share $\texttt{lVCC}_{j,\texttt{id}}$ to the voting server only upon a successful confirmation attempt. Otherwise, the control component returns a message that the confirmation attempt was not successful.

Copyright 2026 Swiss Post Ltd.

5.3.4 ExtractVCC

In case that the confirmation attempt was successful, the voting server extracts the short Vote Cast Return Code $\texttt{VCC}_{\texttt{id}}$ from the Return Codes Mapping table CMtable.

Algorithm 5.13 ExtractVCC

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Election event ID $\texttt{ee} \in (\mathbb{A}_{Base16})^{\texttt{l}_{\texttt{ID}}}$
Verification card ID $\texttt{vc}_{\texttt{id}} \in (\mathbb{A}_{Base16})^{\texttt{l}_{\texttt{ID}}}$

Input:

CCR long Vote Cast Return Code shares $(\texttt{lVCC}_{1,\texttt{id}}, \texttt{lVCC}_{2,\texttt{id}}, \texttt{lVCC}_{3,\texttt{id}}, \texttt{lVCC}_{4,\texttt{id}}) \in {\mathbb{G}_q}^4$
Return Codes Mapping table $\text{CMtable} \in ({\mathbb{A}_{Base64}}^{\texttt{l}_{\texttt{HB64}}} \times {\mathbb{A}_{Base64}}^{*})^{\texttt{N}_{\texttt{E}} \cdot (n+1)}$

Operation: ▷ For all algorithms see the crypto primitives specification

1: $\texttt{pVCC}_{\texttt{id}} \leftarrow \prod_{j=1}^{4} \texttt{lVCC}_{j,\texttt{id}} \bmod p$
2: $\texttt{lVCC}_{\texttt{id}} \leftarrow \text{RecursiveHash}(\texttt{pVCC}_{\texttt{id}}, \texttt{vc}_{\texttt{id}}, \texttt{ee})$
3: $\texttt{key} \leftarrow \text{Base64Encode}(\text{RecursiveHash}(\texttt{lVCC}_{\texttt{id}}))$
4: **if** $\texttt{key} \notin \text{CMtable}$ **then**
5: **return** $\perp$
6: **else**
7: $\texttt{ctVCC}_{\texttt{id,encoded}} \leftarrow \text{CMtable}(\texttt{key})$ ▷ Retrieve the value from the key-value map
8: $\texttt{ctVCC}_{\texttt{id,combined}} \leftarrow \text{Base64Decode}(\texttt{ctVCC}_{\texttt{id,encoded}})$
9: $\texttt{length} \leftarrow$ Length in bytes of $\texttt{ctVCC}_{\texttt{id,combined}}$
10: $\texttt{split} \leftarrow \texttt{length} - \texttt{nonceLength}$ ▷ Nonce length in bytes defined by the symmetric algorithm used
11: $\texttt{ctVCC}_{\texttt{id,ciphertext}} \leftarrow \texttt{ctVCC}_{\texttt{id,combined}}[0 : \texttt{split})$
12: $\texttt{ctVCC}_{\texttt{id,nonce}} \leftarrow \texttt{ctVCC}_{\texttt{id,combined}}[\texttt{split} : \texttt{length})$
13: $\texttt{skvcc}_{\texttt{id}} \leftarrow \text{KDF}(\texttt{lVCC}_{\texttt{id}}, (), \texttt{l}_{\texttt{KD}})$
14: $\texttt{VCC}_{\texttt{id,bytes}} \leftarrow \text{GetPlaintextSymmetric}(\texttt{skvcc}_{\texttt{id}}, \texttt{ctVCC}_{\texttt{id,ciphertext}}, \texttt{ctVCC}_{\texttt{id,nonce}}, ())$
15: $\texttt{VCC}_{\texttt{id}} \leftarrow \text{ByteArrayToString}(\texttt{VCC}_{\texttt{id,bytes}})$
16: **end if**

Output:

Short Vote Cast Return Code $\texttt{VCC}_{\texttt{id}} \in (\mathbb{A}_{10})^{\texttt{l}_{\texttt{VCC}}}$

Subsequently, the voting server sends the short Vote Cast Return Code $\texttt{VCC}_{\texttt{id}}$ to the voting client and the voter compares it to the one printed on the voting card.

Copyright 2026 Swiss Post Ltd.

As soon as the voting server successfully performs the ExtractVCC algorithm, the voting card status changes, and the voter can no longer vote by post or in person. Conversely, if the voting server did *not* execute the ExtractVCC algorithm, the voter can still vote by post or in person. The Federal Chancellery's Ordinance [7] and its explanatory report [8] elaborate on that requirement.

> [VEleS Annex 4.11]: As long as the system has not registered confirmation of a definitive electronic vote, the voter may still choose to cast his or her vote via a conventional voting channel.

> [Explanatory Report, Sec 4.2.1]: No 4.11: Voters are required to report to the competent cantonal authority if proofs are incorrectly displayed or if they are unsure about this. Voting by post or in person remains an option if an electronic vote has not yet been received. In order to assess this, the cantons have functionality at their disposal in accordance with Number 11.6.

The voting server allows the competent cantonal authority to query a voter's voting card status.
For this specific functionality, the e-voting system can be considered *trustworthy* as highlighted in the Federal Chancellery's Ordinance [7] and its explanatory report [8].

> [VEleS Annex 11.6]: The system allows the polling card to be used to determine whether someone has cast an electronic vote.

> [Explanatory Report, Sec 4.2.1]: No 11.6: It is not possible to decide whether a vote cast by post or in person is a double or even multiple vote by using only the votes cast electronically as a basis for comparison. Nevertheless, the functionality under Number 11.6 falls within the scope of the OEV. However, it is not necessary to specify the functionality by reference to trust assumptions under Number 2.

When a ballot is cast by post or in person (paper vote), the cantonal or municipal authorities check the status of the associated voting card. The cantonal or municipal authorities will refuse the paper vote if the voter has already cast an electronic vote. If not, the paper vote will be cast, and the electronic channel will be blocked through a change of the voting card status.

Copyright 2026 Swiss Post Ltd.

6 Tally Phase

The tally phase's purpose is for each control component to sequentially shuffle and decrypt the encrypted votes—in a verifiable manner—allowing an auditor using a trustworthy verifier software to check that the election result is *verifiably* correct.
We distinguish between the operations in the online control components (MixOnline) and the offline Tally control component (MixOffline). Table 15 shows the algorithms involved.

Algorithm	Actor	Reference
MixOnline		Figures 14 and 15
GetMixnetInitialCiphertexts	Online Control Component (CCM)	6.1
VerifyMixDecOnline	Online Control Component (CCM)	6.2
MixDecOnline	Online Control Component (CCM)	6.3
MixOffline		Figure 14
VerifyVotingClientProofs	Tally Control Component	6.9
VerifyMixDecOffline	Tally Control Component	6.10
MixDecOffline	Tally Control Component	6.11
ProcessPlaintexts	Tally Control Component	6.12
VerifyTally	Auditors	Verifier Specification [21]

Tab. 15: Overview of the algorithms in the tally phase.

Conceptually, it is important to note that each control component generates a CCM election key pair.

$$(\mathtt{EL}_{\mathtt{pk},j}, \mathtt{EL}_{\mathtt{sk},j}) \leftarrow \mathsf{GenKeyPair}$$
$$\mathtt{EL}_{\mathtt{pk},j} = g^{\mathtt{EL}_{\mathtt{sk},j}} \bmod p$$

The election public key is the product of all CCM election public keys and the electoral board public key $\mathtt{EB}_{\mathtt{pk}}$.

$$\mathtt{EL}_{\mathtt{pk}} = \prod_{j=1}^{4} \mathtt{EL}_{\mathtt{pk},j} \cdot \mathtt{EB}_{\mathtt{pk}} \bmod p = g^{\mathtt{EL}_{\mathtt{sk},1}+\mathtt{EL}_{\mathtt{sk},2}+\mathtt{EL}_{\mathtt{sk},3}+\mathtt{EL}_{\mathtt{sk},4}+\mathtt{EB}_{\mathtt{sk}}} \bmod p$$

Now, imagine that the first control component partially decrypted the ciphertexts. The second control component needs to shuffle the ciphertexts using the *remaining* election public key $\overline{\mathtt{EL}}_{\mathtt{pk}}$. For the second control component, the remaining public key does not contain the first control component's contribution:

$$\overline{\mathtt{EL}}_{\mathtt{pk},2} = \mathtt{EL}_{\mathtt{pk},2} \cdot \mathtt{EL}_{\mathtt{pk},3} \cdot \mathtt{EL}_{\mathtt{pk},4} \cdot \mathtt{EB}_{\mathtt{pk}} \bmod p = \frac{\mathtt{EL}_{\mathtt{pk}}}{\mathtt{EL}_{\mathtt{pk},1}} \bmod p = g^{\mathtt{EL}_{\mathtt{sk},2}+\mathtt{EL}_{\mathtt{sk},3}+\mathtt{EL}_{\mathtt{sk},4}+\mathtt{EB}_{\mathtt{sk}}} \bmod p$$

The same principle holds for the subsequent shuffle and decryption operations.
Figure 14 indicates the data flows and algorithms in the tally phase.

Copyright 2026 Swiss Post Ltd.

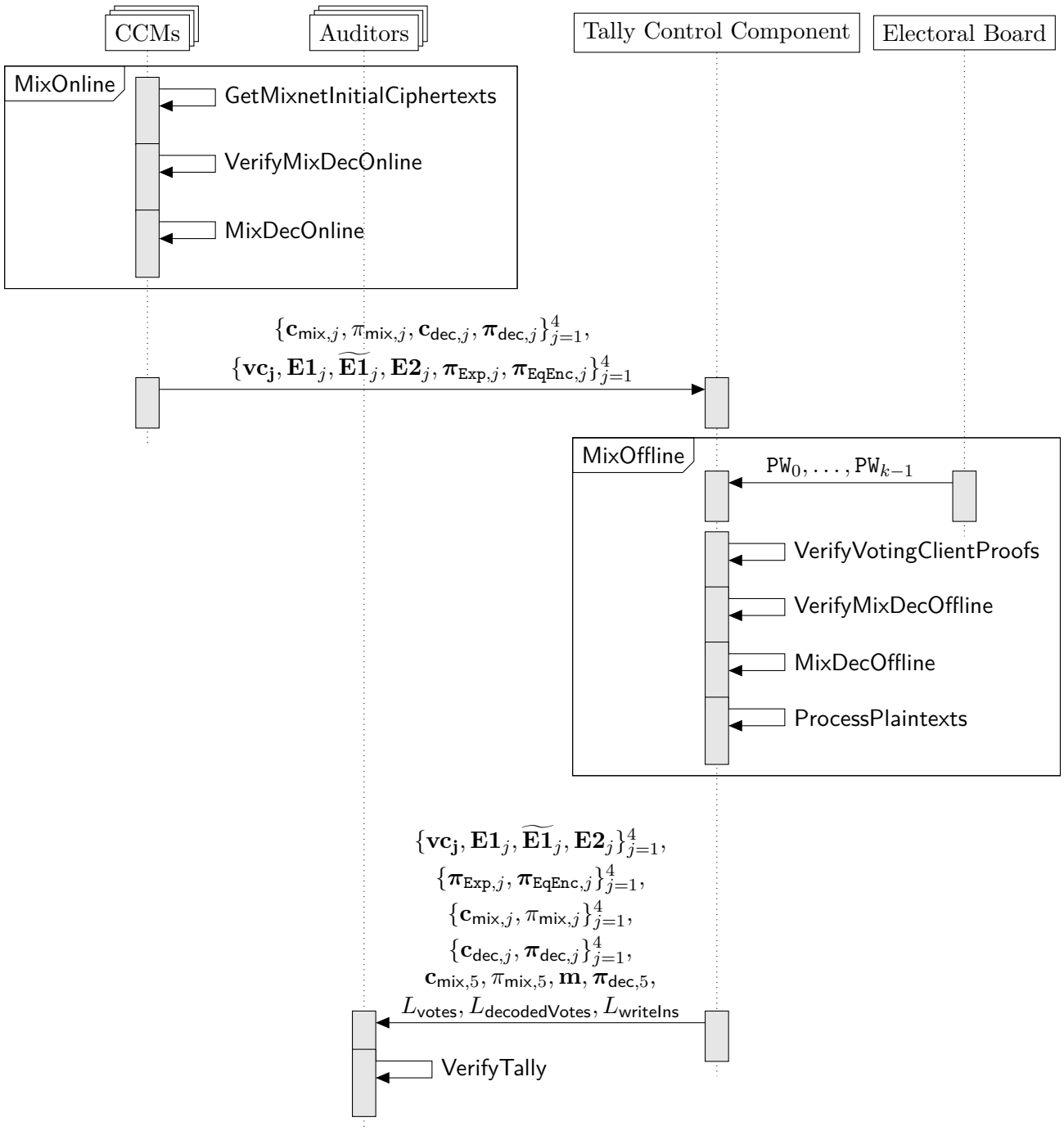


Fig. 14: Sequence diagram of the tally phase showing the data flows and involved algorithms.

Copyright 2026 Swiss Post Ltd.

6.1 MixOnline

The online control components execute the first part of the tally phase. Figure 15 highlights the algorithms and data flows. The CCM_1 does not execute the VerifyMixDecOnline algorithm, since the first mixing control component has nothing to verify.
Each online control component sends back the encrypted confirmed votes registered during the voting phase for later processing by the Tally Control Component and auditors.
The j-th control component's encrypted confirmed votes contains the following data:

- j-th control component's list of confirmed verification card IDs $\mathbf{vc_j} = (\mathtt{vc}_{\mathtt{j},0}, \ldots, \mathtt{vc}_{\mathtt{j},\mathtt{N_C}-1})$
- j-th control component's list of encrypted, confirmed votes $\mathbf{E1}_j = (\mathtt{E1}_{j,0}, \ldots, \mathtt{E1}_{j,\mathtt{N_C}-1})$
- j-th control component's list of exponentiated, encrypted, confirmed votes $\widetilde{\mathbf{E1}}_j = (\widetilde{\mathtt{E1}}_{j,0}, \ldots, \widetilde{\mathtt{E1}}_{j,\mathtt{N_C}-1})$
- j-th control component's list of encrypted, partial Choice Return Codes $\mathbf{E2}_j = (\mathtt{E2}_{j,0}, \ldots, \mathtt{E2}_{j,\mathtt{N_C}-1})$
- j-th control component's list of exponentiation proofs $\boldsymbol{\pi}_{\mathtt{Exp},j} = (\pi_{\mathtt{Exp},j,0}, \ldots, \pi_{\mathtt{Exp},j,\mathtt{N_C}-1})$
- j-th control component's list of plaintext equality proofs $\boldsymbol{\pi}_{\mathtt{EqEnc},j} = (\pi_{\mathtt{EqEnc},j,0}, \ldots, \pi_{\mathtt{EqEnc},j,\mathtt{N_C}-1})$

Moreover, the control components ensure that the ballot box cannot be decrypted before the election event period ends, except for test ballot boxes which can be decrypted at any time (see Section 3.4.3).

Copyright 2026 Swiss Post Ltd.

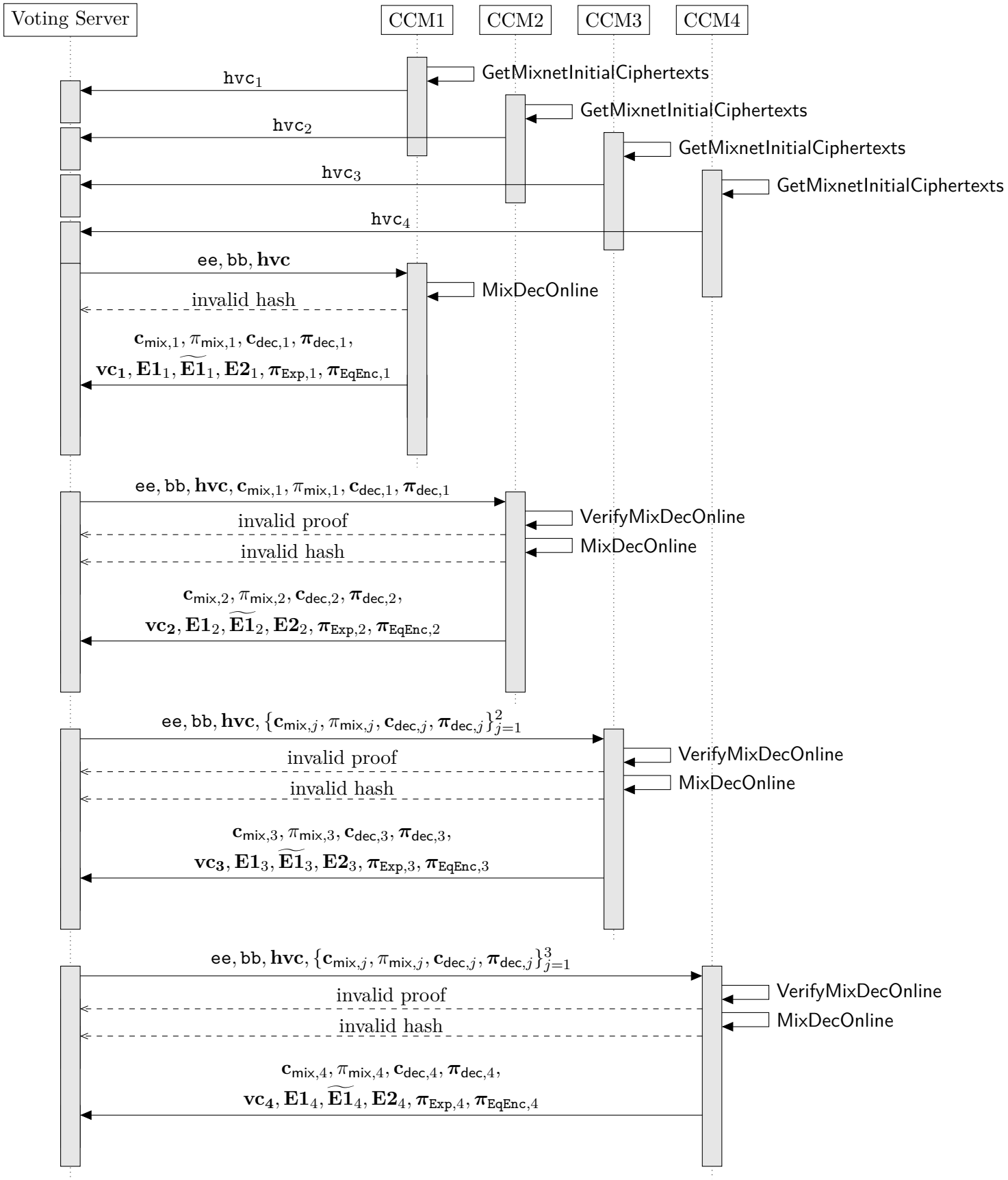


Fig. 15: Sequence diagram of the MixOnline protocol.

Copyright 2026 Swiss Post Ltd.

6.1.1 GetMixnetInitialCiphertexts

The control components retrieve the mixnet's initial ciphertexts from the list of confirmed votes $\mathsf{L}_{\mathtt{confirmedVotes},j}$ that they established during the voting phase. To this end, the control components retrieve a key-value map linking the verification card IDs to the encrypted, confirmed votes $\mathbf{vcMap}_j = \Big((\mathtt{vc}_1, \mathtt{E1}_{j,1}) \dots, (\mathtt{vc}_{\mathtt{N_C}-1}, \mathtt{E1}_{j,\mathtt{N_C}-1})\Big)$ and provide this map to the algorithm GetMixnetInitialCiphertexts. Moreover, shuffling requires at least two votes in the ballot box. Therefore, the algorithm adds two trivial encryptions (containing an encryption of 1) if there are less than two confirmed votes in the ballot box. The tally control component removes the trivial encryptions during the ProcessPlaintexts algorithm. Therefore, the number of actual, mixable votes $\hat{\mathtt{N}}_\mathtt{C}$ is at least two.

Algorithm 6.1 GetMixnetInitialCiphertexts

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Number of eligible voters for this verification card set $\mathtt{N_E} \in \mathbb{N}^+$
Number of allowed write-ins + 1 for this specific ballot box $\delta \in [1, \delta_{\mathtt{sup}}]$ ▷ Can be derived from pTable using algorithm 3.12
Election public key $\mathtt{EL_{pk}} \in \mathbb{G}_q^{\delta_{\max}}$

Input:

Key-value map of verification card IDs to encrypted, confirmed votes $\mathbf{vcMap}_j = \Big((\mathtt{vc}_1, \mathtt{E1}_{j,1}) \dots, (\mathtt{vc}_{\mathtt{N_C}-1}, \mathtt{E1}_{j,\mathtt{N_C}-1})\Big) \in \Big((\mathbb{A}_{Base16})^{\mathtt{l_{ID}}} \times {\mathbb{G}_q}^{\delta+1}\Big)^{\mathtt{N_C}}$

Require: $\mathtt{N_E} \geq \mathtt{N_C}$

Operation:

1: $\mathbf{vcMap}_{j,ordered} \leftarrow \mathsf{Order}(\mathbf{vcMap}_j, 1)$ ▷ Lexicographic ordering by the table's first column (verification card ID)
2: $\mathbf{c}_{\mathsf{init},j} \leftarrow$ Retrieve the encrypted votes from the $\mathbf{vcMap}_{j,ordered}$.
3: **if** $\mathtt{N_C} < 2$ **then** ▷ We add trivial encryptions to the ballot box, if there are less than 2 votes
4: $\quad \vec{1} \leftarrow \underbrace{(1, \dots, 1)}_{\delta \text{ times}}$
5: $\quad \mathtt{E}_{\mathtt{trivial}} \leftarrow \mathsf{GetCiphertext}(\vec{1}, 1, \mathtt{EL_{pk}})$ ▷ See crypto primitives specification
6: $\quad \mathbf{c}_{\mathsf{init},j} \leftarrow (\mathbf{c}_{\mathsf{init},j}, \mathtt{E}_{\mathtt{trivial}}, \mathtt{E}_{\mathtt{trivial}})$ ▷ Adding two trivial encryptions to $\mathbf{c}_{\mathsf{init},j}$
7: **end if**
8: $\mathtt{hvc}_j \leftarrow \mathsf{Base64Encode}(\mathsf{RecursiveHash}(\mathbf{vcMap}_{j,ordered}))$ ▷ See crypto primitives specification

---

Output:

CCM_j hash of the encrypted, confirmed votes $\mathtt{hvc}_j \in {\mathbb{A}_{Base64}}^{\mathtt{l_{HB64}}}$
Mix net initial ciphertexts $\mathbf{c}_{\mathsf{init},j} \in ({\mathbb{G}_q}^{\delta+1})^{\hat{\mathtt{N}}_\mathtt{C}}$

Test values for the algorithm 6.1 are provided in get-mixnet-initial-ciphertexts.json.

Copyright 2026 Swiss Post Ltd.

6.1.2 VerifyMixDecOnline

The online control component verifies the preceding control components' mixing and decryption proofs. The control component bases its verification upon the mix net initial ciphertexts established in the algorithm GetMixnetInitialCiphertexts. The first control component $j = 1$ omits the algorithm VerifyMixDecOnline.

Algorithm 6.2 VerifyMixDecOnline

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Control component index $j \in [2, 4]$
Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Ballot box ID $\mathtt{bb} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Number of allowed write-ins + 1 for this specific ballot box $\delta \in [1, \delta_{\mathsf{sup}}]$ ▷ Can be derived from pTable using algorithm 3.12
Election public key $\mathtt{EL_{pk}} \in \mathbb{G}_q^{\delta_{\max}}$
CCM election public keys $(\mathtt{EL_{pk,1}}, \mathtt{EL_{pk,2}}, \mathtt{EL_{pk,3}}, \mathtt{EL_{pk,4}}) \in \left(\mathbb{G}_q^{\delta_{\max}}\right)^4$
Electoral board public key $\mathtt{EB_{pk}} \in \mathbb{G}_q^{\delta_{\max}}$

Input:

Mix net initial ciphertexts $\mathbf{c}_{\mathsf{init},j} \in (\mathbb{G}_q{}^{\delta+1})^{\hat{\mathtt{N}}_\mathtt{C}}$ ▷ From internal view
Preceding shuffled votes $\{\mathbf{c}_{\mathsf{mix},k}\}_{k=1}^{j-1} \in ((\mathbb{G}_q{}^{\delta+1})^{\hat{\mathtt{N}}_\mathtt{C}})^{j-1}$
Preceding shuffle proofs $\{\pi_{\mathsf{mix},k}\}_{k=1}^{j-1}$ ▷ See the domain of the shuffle argument in the crypto primitives specification
Preceding partially decrypted votes $\{\mathbf{c}_{\mathsf{dec},k}\}_{k=1}^{j-1} \in ((\mathbb{G}_q{}^{\delta+1})^{\hat{\mathtt{N}}_\mathtt{C}})^{j-1}$
Preceding decryption proofs $\{\boldsymbol{\pi}_{\mathsf{dec},k}\}_{k=1}^{j-1} \in ((\mathbb{Z}_q \times \mathbb{Z}_q{}^{\delta})^{\hat{\mathtt{N}}_\mathtt{C}})^{j-1}$

Require: $\hat{\mathtt{N}}_\mathtt{C} \geq 2$ ▷ The algorithm runs with at least two votes

Operation: ▷ For all algorithms see the crypto primitives specification

1: $\mathsf{shuffleVerif}_1 \leftarrow \mathsf{VerifyShuffle}(\mathbf{c}_{\mathsf{init},j}, \mathbf{c}_{\mathsf{mix},1}, \pi_{\mathsf{mix},1}, \mathtt{EL_{pk}})$
2: $\mathbf{i}_{\mathsf{aux},1} \leftarrow (\mathtt{ee}, \mathtt{bb}, \text{“}\mathtt{MixDecOnline}\text{”}, \mathsf{IntegerToString}(1))$
3: $\mathsf{decryptVerif}_1 \leftarrow \mathsf{VerifyDecryptions}(\mathbf{c}_{\mathsf{mix},1}, \mathtt{EL_{pk},1}, \mathbf{c}_{\mathsf{dec},1}, \boldsymbol{\pi}_{\mathsf{dec},1}, \mathbf{i}_{\mathsf{aux},1})$
4: **for** $k \in [2, j)$ **do** ▷ We omit the loop for $j = 2$
5: $\quad\overline{\mathtt{EL}}_{\mathtt{pk}} \leftarrow \mathsf{CombinePublicKeys}\Big((\mathtt{EL_{pk},}k, \ldots, \mathtt{EL_{pk},4}, \mathtt{EB_{pk}})\Big)$
6: $\quad\mathsf{shuffleVerif}_k \leftarrow \mathsf{VerifyShuffle}(\mathbf{c}_{\mathsf{dec},k-1}, \mathbf{c}_{\mathsf{mix},k}, \pi_{\mathsf{mix},k}, \overline{\mathtt{EL}}_{\mathtt{pk}})$
7: $\quad\mathbf{i}_{\mathsf{aux},k} \leftarrow (\mathtt{ee}, \mathtt{bb}, \text{“}\mathtt{MixDecOnline}\text{”}, \mathsf{IntegerToString}(k))$
8: $\quad\mathsf{decryptVerif}_k \leftarrow \mathsf{VerifyDecryptions}(\mathbf{c}_{\mathsf{mix},k}, \mathtt{EL_{pk},}k, \mathbf{c}_{\mathsf{dec},k}, \boldsymbol{\pi}_{\mathsf{dec},k}, \mathbf{i}_{\mathsf{aux},k})$
9: **end for**
10: **if** $(\mathsf{decryptVerif}_k \wedge \mathsf{shuffleVerif}_k)\ \forall\ k \in [1, j)$ **then**
11: $\quad$**return** $\top$
12: **else**
13: $\quad$**return** $\bot$
14: **end if**

Output:

The result of the verification: $\top$ if the verification is successful, $\bot$ otherwise.

Copyright 2026 Swiss Post Ltd.

6.1.3 MixDecOnline

The online Mixing control component CCM shuffle (and re-encrypt) the previous control component's ciphertexts and perform partial decryption. If the control component is the first to mix ($j = 1$), the mixing public key corresponds to the election public key $\texttt{EL}_{\texttt{pk}}$. Each control component ensures that its view of the initial ciphertexts is consistent with the other control components before initiating the mixing process. This is confirmed by validating that the hash of the encrypted, confirmed votes—as computed by each control component in algorithm 6.1—is identical across all control components. In case the hash values are not equal, indicating disagreement among the control components on the list of initial ciphertexts, the process halts. In order to proceed and manage these discrepancies, a third-party could run the dispute resolver, as outlined in section 6.2.

Copyright 2026 Swiss Post Ltd.

Algorithm 6.3 MixDecOnline

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Control component index $j \in [1, 4]$
Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Ballot box ID $\mathtt{bb} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Number of allowed write-ins + 1 for this specific ballot box $\delta \in [1, \delta_{\mathtt{sup}}]$ ▷ Can be derived from pTable using algorithm 3.12
CCM election public keys $(\mathtt{EL_{pk,1}}, \mathtt{EL_{pk,2}}, \mathtt{EL_{pk,3}}, \mathtt{EL_{pk,4}}) \in \left(\mathbb{G}_q^{\delta_{\mathtt{max}}}\right)^4$
Electoral board public key $\mathtt{EB_{pk}} \in \mathbb{G}_q^{\delta_{\mathtt{max}}}$

Stateful Lists and Maps:

List of bb of the shuffled and decrypted ballot boxes $\mathsf{L}_{\mathtt{bb},j}$

Input:

Partially decrypted votes $\mathbf{c}_{\mathsf{dec},j-1} \in (\mathbb{G}_q{}^{\delta+1})^{\hat{\mathtt{N}}_\mathtt{C}}$ ▷ CCM_1 uses $\mathbf{c}_{\mathsf{init},1}$ from internal view
CCM_j election secret key $\mathtt{EL_{sk},}_j \in \mathbb{Z}_q^{\delta_{\mathtt{max}}}$
CCM_j hash of the encrypted, confirmed votes $\mathtt{hvc}_j \in \mathbb{A}_{Base64}{}^{\mathtt{l_{HB64}}}$ ▷ From internal view
CCM hashes of the encrypted, confirmed votes $\mathbf{hvc} = (\mathtt{hvc}_1, \mathtt{hvc}_2, \mathtt{hvc}_3, \mathtt{hvc}_4) \in (\mathbb{A}_{Base64}{}^{\mathtt{l_{HB64}}})^4$

Require: $\mathtt{hvc}_j = \mathtt{hvc}_1 = \mathtt{hvc}_2 = \mathtt{hvc}_3 = \mathtt{hvc}_4$ ▷ The view of the initial ciphertexts must be the same for all CCs before mixing begins
Require: $\hat{\mathtt{N}}_\mathtt{C} \geq 2$ ▷ The algorithm runs with at least two votes
Require: $\mathtt{bb} \notin \mathsf{L}_{\mathtt{bb},j}$

Operation: ▷ For all algorithms see the crypto primitives specification

1: $\overline{\mathtt{EL}}_\mathtt{pk} \leftarrow \mathsf{CombinePublicKeys}\Big((\mathtt{EL_{pk},}_j, \ldots, \mathtt{EL_{pk,4}}, \mathtt{EB_{pk}})\Big)$
2: $\mathbf{i}_\mathsf{aux} \leftarrow (\mathtt{ee}, \mathtt{bb}, \text{“}\mathtt{MixDecOnline}\text{”}, \mathsf{IntegerToString}(j))$
3: $(\mathbf{c}_{\mathsf{mix},j}, \pi_{\mathsf{mix},j}) \leftarrow \mathsf{GenVerifiableShuffle}(\mathbf{c}_{\mathsf{dec},j-1}, \overline{\mathtt{EL}}_\mathtt{pk})$
4: $(\mathbf{c}_{\mathsf{dec},j}, \boldsymbol{\pi}_{\mathsf{dec},j}) \leftarrow \mathsf{GenVerifiableDecryptions}(\mathbf{c}_{\mathsf{mix},j}, (\mathtt{EL_{pk},}_j, \mathtt{EL_{sk},}_j), \mathbf{i}_\mathsf{aux})$
5: $\mathsf{L}_{\mathtt{bb},j} \leftarrow \mathsf{L}_{\mathtt{bb},j} \,||\, \mathtt{bb}$

Output:

Shuffled votes $\mathbf{c}_{\mathsf{mix},j} \in (\mathbb{G}_q{}^{\delta+1})^{\hat{\mathtt{N}}_\mathtt{C}}$
Shuffle proof $\pi_{\mathsf{mix},j}$ ▷ See the domain of the shuffle argument in the crypto primitives specification
Partially decrypted votes $\mathbf{c}_{\mathsf{dec},j} \in (\mathbb{G}_q{}^{\delta+1})^{\hat{\mathtt{N}}_\mathtt{C}}$
Decryption proofs $\boldsymbol{\pi}_{\mathsf{dec},j} \in (\mathbb{Z}_q \times \mathbb{Z}_q{}^{\delta})^{\hat{\mathtt{N}}_\mathtt{C}}$

Copyright 2026 Swiss Post Ltd.

6.2 Handling Inconsistent Views of Confirmed Votes

6.2.1 Overview and Trigger of the Dispute Resolution Process

This section elaborates on the scenario in which the control components do not agree on the list of confirmed votes. The Federal Chancellery's Ordinance and the explanatory report require to anticipate inconsistencies between control components.

> [VEleS Annex 11.11]: The canton anticipates any anomalies and, in consultation with the bodies concerned, draws up an emergency plan specifying the appropriate course of action. It creates transparency towards the public.

> [Explanatory Report, Sec 4.2.1]: As a condition for the successful examination of the proof referred to in Number 2.6, all control components must have recorded the same votes as having been cast in conformity with the system. Cases where the control components show inconsistencies in this respect must be anticipated in accordance with Number 11.11 and the procedure determined in advance.

In this section, we assume a *dispute resolver* (see section 2.10): a party outside the cryptographic protocol who can interact with the control components and can verify the authenticity of the protocol messages (see section 7).
Section 6.1.1 describes how each control component independently retrieves its list of confirmed votes using the GetMixnetInitialCiphertexts algorithm. Before the mixing process begins in the MixDecOnline algorithm, each control component verifies that the list of confirmed votes is consistent across all control components.
In exceptional cases, discrepancies may arise between control components regarding the list of confirmed votes. In such situations, the cantons require the intervention of a *dispute resolver* to resolve the inconsistency.
Figure 16 outlines the sequence of operations that the *dispute resolver* performs to unambiguously identify and resolve inconsistencies.

Copyright 2026 Swiss Post Ltd.

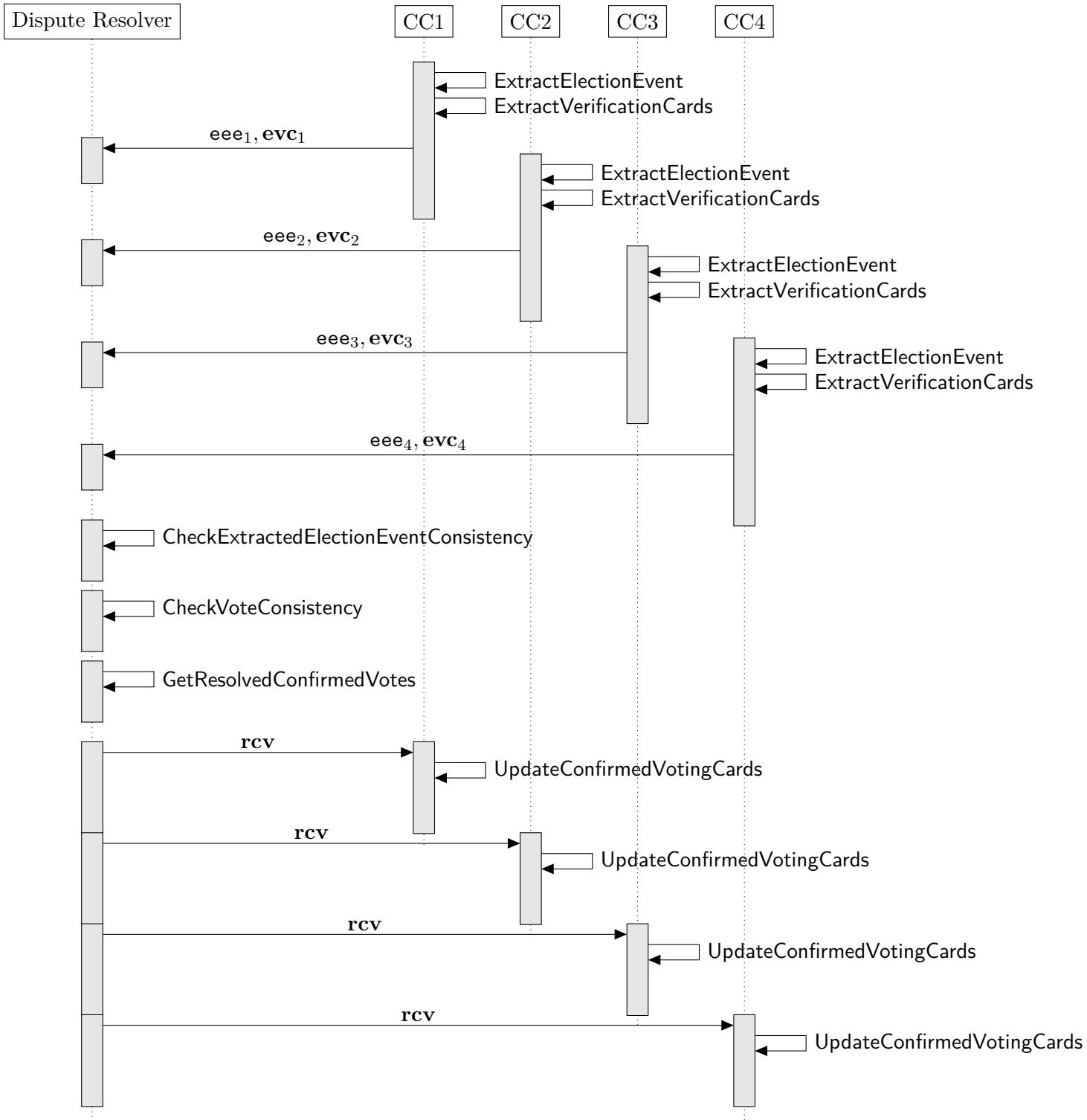


Fig. 16: Sequence diagram of the DisputeResolver protocol.

Copyright 2026 Swiss Post Ltd.

6.2.2 Extraction Phase

The dispute resolution process begins by requesting each control component to invoke the following algorithms:

- 3.16 ExtractElectionEvent
- 3.19 ExtractVerificationCards

Each control component securely transfers its extracted election event and associated verification cards to the dispute resolver via an out-of-band channel (see table 18).

6.2.3 Dispute Resolver's Consistency Checks

The dispute resolver receives, as input, the extracted election event and associated verification cards from each control component.
Before proceeding with the resolution protocol, the dispute resolver checks the authenticity and validity of the messages (see section 7). Moreover, the operator of the dispute resolver manually checks that the received inputs correspond to the expected election event.
The dispute resolver then executes the following algorithms:

- 6.4 CheckExtractedElectionEventConsistency
- 6.5 CheckVoteConsistency
- 6.6 GetResolvedConfirmedVotes

The CheckExtractedElectionEventConsistency algorithm checks that all control components agree on the entire election event-specific data. For each $\texttt{CCR}_\texttt{j}$, the algorithm GetHashExtractedElectionEvent computes the hash of the extracted election event $\texttt{eee}_j$. If all the hashes are identical, the dispute resolver will use this shared context in the subsequent algorithms. If not, the algorithm fails, and the canton must initiate a comprehensive forensic investigation (details are beyond the scope of this document).

Algorithm 6.4 CheckExtractedElectionEventConsistency

Input:

The CCR's Extracted election event $(\texttt{eee}_1, \texttt{eee}_2, \texttt{eee}_3, \texttt{eee}_4)$

▷ $\texttt{eee} = (\texttt{hContext}, (p, q, g), \texttt{ee}, \textbf{evcs})$

Operation:

1: **for** $j \in [1, 4]$ **do**
2: $\quad \texttt{h}_{\texttt{eee}_j} \leftarrow$ GetHashExtractedElectionEvent() ▷ See algorithm 3.18
3: **end for**
4: **if** $\texttt{h}_{\texttt{eee}_1} = \texttt{h}_{\texttt{eee}_2} = \texttt{h}_{\texttt{eee}_3} = \texttt{h}_{\texttt{eee}_4}$ **then**
5: $\quad$ **return** $\top$
6: **else**
7: $\quad$ **return** $\bot$
8: **end if**

Output:

The result of the algorithm: $\top$ if the context is identical, $\bot$ otherwise.

Copyright 2026 Swiss Post Ltd.

Next, the dispute resolver runs the CheckVoteConsistency algorithm to ensure that all control components have the same view of the submitted, encrypted votes. If discrepancies are found, the algorithm fails, and the canton must initiate a comprehensive forensic investigation (details are beyond the scope of this document).

Algorithm 6.5 CheckVoteConsistency

Input:

The CCR's extracted verification cards $(\mathbf{evc}_1, \mathbf{evc}_2, \mathbf{evc}_3, \mathbf{evc}_4)$

▷ $\mathbf{evc} = (\mathtt{evc}_0, \ldots, \mathtt{evc}_{\mathtt{N_S}-1})$, see algorithm 3.19 for the content

Operation: ▷ We use nested structures in this algorithm

1: **for** $j \in [1, 4]$ **do**
2: **for** $i \in [0, \mathtt{N_S})$ **do**
3: $h_{\mathtt{E1}_{j,i}} \leftarrow (\gamma_1, \phi_{1,0}, \ldots, \phi_{1,\delta-1})$ ▷ See algorithm 5.4
4: $h_{\mathtt{evc}_{j,i}} \leftarrow (\mathtt{vc_{id},}_{j,i}, \mathtt{vcs}_{j,i}, h_{\mathtt{E1}_{j,i}})$
5: **end for**
6: $h_j \leftarrow (h_{\mathtt{evc}_{j,0}}, \ldots, h_{\mathtt{evc}_{j,\mathtt{N_S}-1}})$
7: $d_j \leftarrow \mathsf{Base64Encode}(\mathsf{RecursiveHash}(h_j))$ ▷ See crypto primitives specification
8: **end for**
9: **if** $d_1 = d_2 = d_3 = d_4$ **then**
10: **return** $\top$
11: **else**
12: **return** $\bot$
13: **end if**

Output:

The result of the algorithm: $\top$ if the list of sent votes is identical, $\bot$ otherwise.

The Swiss Post Voting System enforces consistency among control components with respect to both the election event context and the encrypted votes throughout the voting phase. All control components must have used the same context and encrypted votes during the voting phase, as they commit to these values in PartialDecryptPCC and verify each other's zero-knowledge proofs in DecryptPCC. The zero-knowledge proofs generated during the voting phase include the ExtractElectionEvent and the hash of the encrypted vote in the auxiliary information, in analogy to what is extracted for the dispute resolver (see section 6.2.2). Thus, the SendVote protocol establishes an explicit agreement on the election event context and the encrypted votes.

However, the Swiss Post Voting System does not provide a similar agreement mechanism for the vote confirmation status during the voting phase. Thus, discrepancies in vote confirmation status may occur, that is, cases where the control components for a given verification card ID registered the same ciphertext, but disagree whether the vote was confirmed or not. Such discrepancies can only be resolved through the dispute resolution process. To this end, the dispute resolver constructs the list of resolved confirmed votes in the GetResolvedConfirmedVotes algorithm. In this exceptional scenario, the control components subsequently update their view of the ballot box in the UpdateConfirmedVotingCards algorithm.

Copyright 2026 Swiss Post Ltd.

Algorithm 6.6 GetResolvedConfirmedVotes

Context:

Extracted election event $\texttt{eee} = (\texttt{hContext}, (p, q, g), \texttt{ee}, \mathbf{evcs})$

Input:

The CCR's extracted verification cards $(\mathbf{evc}_1, \mathbf{evc}_2, \mathbf{evc}_3, \mathbf{evc}_4)$

▷ $\mathbf{evc} = (\texttt{evc}_0, \ldots, \texttt{evc}_{\texttt{N}_\texttt{s}-1})$, see algorithm 3.19 for the content

Operation:

1: $\mathbf{rcv} \leftarrow ()$
2: **for** $j \in [1, 4]$ **do**
3: **for** $i \in [0, \texttt{N}_\texttt{S})$ **do**
4: **if** $\texttt{vc}_{\texttt{id},j,i} \notin \{\texttt{vc}_{\texttt{id},x} \mid \texttt{rcv}_x \in \mathbf{rcv}\} \wedge \texttt{evc}_{j,i} \neq (\texttt{vc}_{\texttt{id},j,i}, \texttt{vcs}_{j,i}, \texttt{E1}_{j,i}, ())$ **then**
5: $\texttt{evc}_{j,i} = (\texttt{vc}_{\texttt{id},j,i}, \texttt{vcs}_{j,i}, \texttt{E1}_{j,i}, (\texttt{hlVCC}_{1,\texttt{id}}, \texttt{hlVCC}_{2,\texttt{id}}, \texttt{hlVCC}_{3,\texttt{id}}, \texttt{hlVCC}_{4,\texttt{id}})_{j,i})$
6: $\mathsf{Verif}_{j,i} \leftarrow \mathsf{ConfirmVoteAgreement}((\texttt{hlVCC}_{1,\texttt{id}}, \texttt{hlVCC}_{2,\texttt{id}}, \texttt{hlVCC}_{3,\texttt{id}}, \texttt{hlVCC}_{4,\texttt{id}})_{j,i})$ ▷ See algorithm 6.7
7: **if** $\mathsf{Verif}_{j,i}$ **then**
8: $\mathbf{rcv} \leftarrow \mathbf{rcv} \,||\, (\texttt{vc}_{\texttt{id},j,i}, \texttt{vcs}_{j,i}, (\texttt{hlVCC}_{1,\texttt{id}}, \texttt{hlVCC}_{2,\texttt{id}}, \texttt{hlVCC}_{3,\texttt{id}}, \texttt{hlVCC}_{4,\texttt{id}})_{j,i})$
9: **end if**
10: **end if**
11: **end for**
12: **end for**
13: $\mathbf{rcv} \leftarrow \mathsf{Order}(\mathbf{rcv})$ ▷ Lexicographic ordering by the verification card id.

Output:

The dispute resolver's resolved confirmed votes $\mathbf{rcv} = (\texttt{rcv}_0, \ldots, \texttt{rcv}_{\texttt{N}_\texttt{c}-1})$

Copyright 2026 Swiss Post Ltd.

Algorithm 6.7 ConfirmVoteAgreement

Context:

Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Verification card set ID $\mathtt{vcs} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Verification card ID $\mathtt{vc_{id}} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Long Vote Cast Return Codes allow list $\mathsf{L}_{\mathtt{lVCC}} \in (\mathbb{A}_{Base64}{}^{\mathtt{l_{HB64}}})^{\mathtt{N_E}}$

Input:

The CCR's hashed long Vote Cast Return Code shares $(\mathtt{hlVCC}_{1,\mathtt{id}}, \mathtt{hlVCC}_{2,\mathtt{id}}, \mathtt{hlVCC}_{3,\mathtt{id}}, \mathtt{hlVCC}_{4,\mathtt{id}}) \in (\mathbb{A}_{Base64}{}^{\mathtt{l_{HB64}}})^4$

Operation:

1: $\mathbf{i}_{\mathtt{aux}} \leftarrow$ ("VerifyLVCCHash", ee, vcs, $\mathtt{vc_{id}}$)
2: $\mathtt{hhlVCC_{id}} \leftarrow \mathsf{Base64Encode}(\mathsf{RecursiveHash}(\mathbf{i}_{\mathtt{aux}}, \mathtt{hlVCC}_{1,\mathtt{id}}, \mathtt{hlVCC}_{2,\mathtt{id}}, \mathtt{hlVCC}_{3,\mathtt{id}}, \mathtt{hlVCC}_{4,\mathtt{id}}))$
▷ See crypto primitives specification
3: **if** $\mathtt{hhlVCC_{id}} \in \mathsf{L}_{\mathtt{lVCC}}$ **then**
4: **return** $\top$
5: **else**
6: **return** $\bot$
7: **end if**

Output:

The result of the algorithm: $\top$ if the vote should be part of the tally, $\bot$ otherwise.

Copyright 2026 Swiss Post Ltd.

6.2.4 Control Component's Update

After the dispute resolver has resolved the inconsistencies, the control components update their list of confirmed votes. To minimize the impact of the dispute resolution process, updates are restricted to changes from *SENT* to *CONFIRMED* status for individual votes. We prevent the control components from modifying or dropping confirmed votes. Operational safeguards ensure that this update can only occur with approval from the cantons.

Algorithm 6.8 UpdateConfirmedVotingCards

Context:

The CCR's index $j \in [1, 4]$

Election event ID $\mathsf{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$

Long Vote Cast Return Codes allow lists $(\mathsf{L}_{\mathtt{lVCC},0}, \ldots, \mathsf{L}_{\mathtt{lVCC},\mathtt{N_{bb}}-1}) \in ((\mathbb{A}_{Base64}{}^{\mathtt{l_{HB64}}})^{\mathtt{N_E}})^{\mathtt{N_{bb}}}$

Stateful Lists and Maps:

$\mathtt{CCR_j}$'s list of sent voting cards $\mathsf{L}_{\mathtt{sentVotes},j}$

$\mathtt{CCR_j}$'s list of confirmed voting cards $\mathsf{L}_{\mathtt{confirmedVotes},j}$

Input:

The dispute resolver's resolved confirmed votes $\mathbf{rcv} = (\mathtt{rcv}_0, \ldots, \mathtt{rcv}_{\mathtt{N_C}-1})$

Operation:

1: **for** $(\mathtt{vc_{id}}, \mathtt{vcs}, (\mathtt{hlVCC}_{1,\mathtt{id}}, \mathtt{hlVCC}_{2,\mathtt{id}}, \mathtt{hlVCC}_{3,\mathtt{id}}, \mathtt{hlVCC}_{4,\mathtt{id}})) \in \mathbf{rcv}$ **do**
2: **if** $\mathtt{vc_{id}} \notin \mathsf{L}_{\mathtt{sentVotes},j}$ **then**
3: **return** $\bot$
4: **end if**
5: **if** $\mathtt{vc_{id}} \notin \mathsf{L}_{\mathtt{confirmedVotes},j}$ **then**
6: Verif $\leftarrow$ ConfirmVoteAgreement$((\mathtt{hlVCC}_{1,\mathtt{id}}, \mathtt{hlVCC}_{2,\mathtt{id}}, \mathtt{hlVCC}_{3,\mathtt{id}}, \mathtt{hlVCC}_{4,\mathtt{id}}))$ ▷ See algorithm 6.7
7: **if** Verif **then**
8: $\mathsf{L}_{\mathtt{confirmedVotes},j} \leftarrow \mathsf{L}_{\mathtt{confirmedVotes},j} \,||\, \mathtt{vc_{id}}$
9: **else**
10: **return** $\bot$
11: **end if**
12: **end if**
13: **end for**
14: **return** $\top$

Output:

The result of the algorithm: $\top$ if the update was successful, $\bot$ otherwise.

After a successful execution of the dispute resolution process, the control components re-initialize the tally phase by re-running the GetMixnetInitialCiphertexts algorithm. The dispute resolution process is executed only once. In the event that the control components maintain inconsistent views, the canton will initiate a comprehensive forensic investigation (details are beyond the scope of this document). All control components must retain all received messages for potential future analysis.

Copyright 2026 Swiss Post Ltd.

6.3 MixOffline

6.3.1 VerifyVotingClientProofs

The Tally control component verifies the consistency of the online control components' encrypted confirmed votes $\{\mathbf{vc_j}, \mathbf{E1}_j, \widetilde{\mathbf{E1}}_j, \mathbf{E2}_j, \boldsymbol{\pi}_{\mathtt{Exp},j}, \boldsymbol{\pi}_{\mathtt{EqEnc},j}\}_{j=1}^{4}$. After this check, the Tally control component proceeds with verifying the voting client's zero-knowledge proofs. By convention, we use the first control component's confirmed encrypted votes.

Algorithm 6.9 VerifyVotingClientProofs

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Verification card set ID $\mathtt{vcs} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Primes mapping table $\mathsf{pTable} \in \left(\mathcal{D}_{\mathrm{v}} \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times {\mathbb{A}_{UCS}}^* \times \mathcal{D}_\tau\right)^n$ ▷ pTable is of the form
$((\mathrm{v}_0, \tilde{\mathrm{p}}_0, \sigma_0, \tau_0), \ldots, (\mathrm{v}_{n-1}, \tilde{\mathrm{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1}))$ ▷ ψ and δ can be derived from pTable using algorithms 3.11 and 3.12
Number of eligible voters for this verification card set $\mathtt{N_E} \in \mathbb{N}^+$
Electoral Model Context $\Lambda = (\psi, \mathbf{DoI}, \mathbf{pg}, \mathbf{ag}) \in \left((\mathbb{N}^+)^t \times (\mathcal{T}_1^{50})^k \times [0, \eta)^t \times \mathbb{Z}_2^\eta\right)$ ▷ See Section 3.4.4
Election public key $\mathtt{EL_{pk}} \in \mathbb{G}_q^{\delta_{\max}}$
Choice Return Codes encryption public key $\mathbf{pk}_{\mathrm{CCR}} \in \mathbb{G}_q^{\psi_{\max}}$

Input:

Control component's list of confirmed verification card IDs $\mathbf{vc_1} = (\mathtt{vc}_{1,0}, \ldots, \mathtt{vc}_{1,\mathtt{N_C}-1}) \in \left((\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}\right)^{\mathtt{N_C}}$
Control component's list of encrypted, confirmed votes $\mathbf{E1}_1 = (\mathtt{E1}_{1,0}, \ldots, \mathtt{E1}_{1,\mathtt{N_C}-1}) \in \left({\mathbb{G}_q}^{\delta+1}\right)^{\mathtt{N_C}}$
Control component's list of exponentiated, encrypted, confirmed votes $\widetilde{\mathbf{E1}}_1 = (\widetilde{\mathtt{E1}}_{1,0}, \ldots, \widetilde{\mathtt{E1}}_{1,\mathtt{N_C}-1}) \in \left({\mathbb{G}_q}^2\right)^{\mathtt{N_C}}$
Control component's list of encrypted, partial Choice Return Codes $\mathbf{E2}_1 = (\mathtt{E2}_{1,0}, \ldots, \mathtt{E2}_{1,\mathtt{N_C}-1}) \in \left({\mathbb{G}_q}^{\psi+1}\right)^{\mathtt{N_C}}$
Control component's list of exponentiation proofs $\boldsymbol{\pi}_{\mathtt{Exp},1} = (\pi_{\mathtt{Exp},1,0}, \ldots, \pi_{\mathtt{Exp},1,\mathtt{N_C}-1}) \in \left(\mathbb{Z}_q \times \mathbb{Z}_q\right)^{\mathtt{N_C}}$
Control component's list of plaintext equality proofs $\boldsymbol{\pi}_{\mathtt{EqEnc},1} = (\pi_{\mathtt{EqEnc},1,0}, \ldots, \pi_{\mathtt{EqEnc},1,\mathtt{N_C}-1}) \in \left(\mathbb{Z}_q \times {\mathbb{Z}_q}^2\right)^{\mathtt{N_C}}$
Key-value map of the verification card public keys $\mathbf{KMap} = \left((\mathtt{vc}_0, \mathtt{K}_0), \ldots, (\mathtt{vc}_{\mathtt{N_E}-1}, \mathtt{K}_{\mathtt{N_E}-1})\right) \in \left((\mathbb{A}_{Base16})^{\mathtt{l_{ID}}} \times \mathbb{G}_q\right)^{\mathtt{N_E}}$

Require: $\mathtt{N_E} \geq \mathtt{N_C} \geq 1$ ▷ The algorithm runs with at least one confirmed vote
Require: $\mathtt{vc}_{1,\mathtt{i}} \neq \mathtt{vc}_{1,\mathtt{k}}, \forall i, k \in \{0, \ldots, (\mathtt{N_C} - 1)\} \wedge i \neq k$ ▷ All verification card IDs must be distinct

Operation: ▷ For all algorithms see the crypto primitives specification

1: $\widetilde{\mathrm{pk}}_{\mathrm{CCR}} \leftarrow \prod_{i=0}^{\psi-1} \mathrm{pk}_{\mathrm{CCR},i} \bmod p$
2: **for** $i \in [0, \mathtt{N_C})$ **do**
3: $\quad \mathtt{K_{id}} \leftarrow \mathbf{KMap}(\mathtt{vc}_{1,\mathtt{i}})$ ▷ Retrieve the value from the key-value map
4: $\quad \mathtt{E1}_{1,i} = (\gamma_1, \phi_{1,0}, \ldots, \phi_{1,\delta-1})$
5: $\quad \widetilde{\mathtt{E1}}_{1,i} = (\gamma_1^{\mathtt{k_{id}}}, \phi_{1,0}^{\mathtt{k_{id}}})$
6: $\quad \mathtt{E2}_{1,i} = (\gamma_2, \phi_{2,0}, \ldots, \phi_{2,\psi-1})$
7: $\quad \widetilde{\mathtt{E2}}_i \leftarrow (\gamma_2, \prod_{k=0}^{\psi-1} \phi_{2,k} \bmod p)$
8: $\quad \mathbf{i}_{\mathtt{aux}} \leftarrow (\text{“}\mathtt{CreateVote}\text{”}, \mathtt{vc}_{1,\mathtt{i}}, \mathsf{GetHashContext}())$ ▷ See algorithm 3.15
9: $\quad \mathbf{i}_{\mathtt{aux}} \leftarrow (\mathbf{i}_{\mathtt{aux}}, \mathsf{IntegerToString}(\gamma_1), \mathsf{IntegerToString}(\phi_{1,0}), \ldots, \mathsf{IntegerToString}(\phi_{1,\delta-1}))$
10: $\quad \mathbf{i}_{\mathtt{aux}} \leftarrow (\mathbf{i}_{\mathtt{aux}}, \mathsf{IntegerToString}(\gamma_2), \mathsf{IntegerToString}(\phi_{2,0}), \ldots, \mathsf{IntegerToString}(\phi_{2,\psi-1}))$
11: $\quad \mathsf{VerifExp}_i \leftarrow \mathsf{VerifyExponentiation}((g, \gamma_1, \phi_{1,0}), (\mathtt{K_{id}}, \gamma_1^{\mathtt{k_{id}}}, \phi_{1,0}^{\mathtt{k_{id}}}), \pi_{\mathtt{Exp},1,i}, \mathbf{i}_{\mathtt{aux}})$
12: $\quad \mathsf{VerifEqEnc}_i \leftarrow \mathsf{VerifyPlaintextEquality}(\widetilde{\mathtt{E1}}_{1,i}, \widetilde{\mathtt{E2}}_i, \mathtt{EL}_{\mathtt{pk},0}, \widetilde{\mathrm{pk}}_{\mathrm{CCR}}, \pi_{\mathtt{EqEnc},1,i}, \mathbf{i}_{\mathtt{aux}})$
13: **end for**
14: **if** $(\mathsf{VerifExp}_i \wedge \mathsf{VerifEqEnc}_i)\ \forall\ i \in [0, \mathtt{N_C})$ **then**
15: $\quad$ **return** $\top$
16: **else**
17: $\quad$ **return** $\bot$
18: **end if**

Output:

The result of the verification: $\top$ if the verification is successful, $\bot$ otherwise.

Copyright 2026 Swiss Post Ltd.

6.3.2 VerifyMixDecOffline

After the verification of the voting client's zero-knowledge proofs, the Tally control component verifies the online control components' shuffle and decryption proofs. Using the algorithm GetMixnetInitialCiphertexts, the Tally control component retrieves the first online control component's mixnet initial ciphertexts $\mathbf{c}_{\mathsf{init},1}$ from its encrypted confirmed votes (the Tally control component verifies the consistency of the control components' confirmed encrypted votes prior to the execution of this algorithm).

Algorithm 6.10 VerifyMixDecOffline

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Election event ID $\mathsf{ee} \in (\mathbb{A}_{Base16})^{l_{\mathrm{ID}}}$
Ballot box ID $\mathsf{bb} \in (\mathbb{A}_{Base16})^{l_{\mathrm{ID}}}$
Number of allowed write-ins + 1 for this specific ballot box $\delta \in [1, \delta_{\mathsf{sup}}]$ ▷ Can be derived from pTable using algorithm 3.12
Election public key $\mathtt{EL}_{\mathtt{pk}} \in \mathbb{G}_q^{\delta_{\max}}$
CCM election public keys $(\mathtt{EL}_{\mathtt{pk},1}, \mathtt{EL}_{\mathtt{pk},2}, \mathtt{EL}_{\mathtt{pk},3}, \mathtt{EL}_{\mathtt{pk},4}) \in \left(\mathbb{G}_q^{\delta_{\max}}\right)^4$
Electoral board public key $\mathtt{EB}_{\mathtt{pk}} \in \mathbb{G}_q^{\delta_{\max}}$

Input:

Mix net initial ciphertexts $\mathbf{c}_{\mathsf{init},1} \in ({\mathbb{G}_q}^{\delta+1})^{\hat{\mathtt{N}}_{\mathtt{C}}}$
Preceding shuffled votes $\{\mathbf{c}_{\mathsf{mix},j}\}_{j=1}^{4} \in (({\mathbb{G}_q}^{\delta+1})^{\hat{\mathtt{N}}_{\mathtt{C}}})^4$
Preceding shuffle proofs $\{\pi_{\mathsf{mix},j}\}_{j=1}^{4}$ ▷ See the domain of the shuffle argument in the crypto primitives specification
Preceding partially decrypted votes $\{\mathbf{c}_{\mathsf{dec},j}\}_{j=1}^{4} \in (({\mathbb{G}_q}^{\delta+1})^{\hat{\mathtt{N}}_{\mathtt{C}}})^4$
Preceding decryption proofs $\{\boldsymbol{\pi}_{\mathsf{dec},j}\}_{j=1}^{4} \in (({\mathbb{Z}_q \times \mathbb{Z}_q}^{\delta})^{\hat{\mathtt{N}}_{\mathtt{C}}})^4$

Require: $\hat{\mathtt{N}}_{\mathtt{C}} \geq 2$ ▷ The algorithm runs with at least two votes

Operation: ▷ For all algorithms see the crypto primitives specification

1: $\mathsf{shuffleVerif}_1 \leftarrow \mathsf{VerifyShuffle}(\mathbf{c}_{\mathsf{init},1}, \mathbf{c}_{\mathsf{mix},1}, \pi_{\mathsf{mix},1}, \mathtt{EL}_{\mathtt{pk}})$
2: $\mathbf{i}_{\mathsf{aux},1} \leftarrow (\mathsf{ee}, \mathsf{bb}, \text{“}\mathtt{MixDecOnline}\text{”}, \mathsf{IntegerToString}(1))$
3: $\mathsf{decryptVerif}_1 \leftarrow \mathsf{VerifyDecryptions}(\mathbf{c}_{\mathsf{mix},1}, \mathtt{EL}_{\mathtt{pk},1}, \mathbf{c}_{\mathsf{dec},1}, \boldsymbol{\pi}_{\mathsf{dec},1}, \mathbf{i}_{\mathsf{aux},1})$
4: **for** $j \in [2, 4]$ **do**
5: $\quad \overline{\mathtt{EL}}_{\mathtt{pk}} \leftarrow \mathsf{CombinePublicKeys}\Big((\mathtt{EL}_{\mathtt{pk},j}, \ldots, \mathtt{EL}_{\mathtt{pk},4}, \mathtt{EB}_{\mathtt{pk}})\Big)$
6: $\quad \mathsf{shuffleVerif}_j \leftarrow \mathsf{VerifyShuffle}(\mathbf{c}_{\mathsf{dec},j-1}, \mathbf{c}_{\mathsf{mix},j}, \pi_{\mathsf{mix},j}, \overline{\mathtt{EL}}_{\mathtt{pk}})$
7: $\quad \mathbf{i}_{\mathsf{aux},j} \leftarrow (\mathsf{ee}, \mathsf{bb}, \text{“}\mathtt{MixDecOnline}\text{”}, \mathsf{IntegerToString}(j))$
8: $\quad \mathsf{decryptVerif}_j \leftarrow \mathsf{VerifyDecryptions}(\mathbf{c}_{\mathsf{mix},j}, \mathtt{EL}_{\mathtt{pk},j}, \mathbf{c}_{\mathsf{dec},j}, \boldsymbol{\pi}_{\mathsf{dec},j}, \mathbf{i}_{\mathsf{aux},j})$
9: **end for**
10: **if** $(\mathsf{decryptVerif}_j \wedge \mathsf{shuffleVerif}_j)\ \forall\ j \in [1, 4]$ **then**
11: $\quad$ **return** $\top$
12: **else**
13: $\quad$ **return** $\bot$
14: **end if**

Output:

The result of the verification: $\top$ if the verification is successful, $\bot$ otherwise.

Copyright 2026 Swiss Post Ltd.

6.3.3 MixDecOffline

If all verifications in the VerifyMixDecOffline algorithm succeed, the Tally control component interacts with the electoral board (see section 2.5) and shuffles (and re-encrypts) the votes and performs the final decryption.

Algorithm 6.11 MixDecOffline

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Election event ID $\mathtt{ee} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Ballot box ID $\mathtt{bb} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$
Number of allowed write-ins + 1 for this specific ballot box $\delta \in [1, \delta_{\mathsf{sup}}]$ ▷ Can be derived from pTable using algorithm 3.12

Stateful Lists and Maps:

List of bb of the shuffled and decrypted ballot boxes $\mathsf{L}_{\mathtt{bb,Tally}}$

Input:

Partially decrypted votes $\mathbf{c}_{\mathsf{dec},4} \in ({\mathbb{G}_q}^{\delta+1})^{\hat{\mathtt{N}}_\mathtt{C}}$
Passwords of k electoral board members $(\mathtt{PW}_0, \ldots, \mathtt{PW}_{k-1}) \in (\mathbb{A}_{UCS}{}^*)^k$

Require: $\hat{\mathtt{N}}_\mathtt{C} \geq 2$ ▷ The algorithm runs with at least two votes
Require: $k \geq 2$ ▷ We require at least 2 electoral board members
Require: $\mathtt{bb} \notin \mathsf{L}_{\mathtt{bb,Tally}}$

Operation: ▷ For all algorithms see the crypto primitives specification

1: **for** $i \in [0, \delta)$ **do**
2: $\quad \mathtt{EB}_{\mathtt{sk},i} \leftarrow \mathsf{RecursiveHashToZq}(q, (\text{“}\mathtt{ElectoralBoardSecretKey}\text{”}, \mathtt{ee}, i, \mathtt{PW}_0, \ldots, \mathtt{PW}_{k-1}))$
3: $\quad \mathtt{EB}_{\mathtt{pk},i} \leftarrow g^{\mathtt{EB}_{\mathtt{sk},i}} \bmod p$
4: **end for**
5: $\mathtt{EB}_{\mathtt{sk}} \leftarrow (\mathtt{EB}_{\mathtt{sk},0}, \ldots, \mathtt{EB}_{\mathtt{sk},\delta-1})$
6: $\mathtt{EB}_{\mathtt{pk}} \leftarrow (\mathtt{EB}_{\mathtt{pk},0}, \ldots, \mathtt{EB}_{\mathtt{pk},\delta-1})$
7: $\mathbf{i}_{\mathsf{aux}} \leftarrow (\mathtt{ee}, \mathtt{bb}, \text{“}\mathtt{MixDecOffline}\text{”})$
8: $(\mathbf{c}_{\mathsf{mix},5}, \pi_{\mathsf{mix},5}) \leftarrow \mathsf{GenVerifiableShuffle}(\mathbf{c}_{\mathsf{dec},4}, \mathtt{EB}_{\mathtt{pk}})$
9: $(\mathbf{c}_{\mathsf{dec},5}, \boldsymbol{\pi}_{\mathsf{dec},5}) \leftarrow \mathsf{GenVerifiableDecryptions}(\mathbf{c}_{\mathsf{mix},5}, (\mathtt{EB}_{\mathtt{pk}}, \mathtt{EB}_{\mathtt{sk}}), \mathbf{i}_{\mathsf{aux}})$
10: **for** $i \in [0, \hat{\mathtt{N}}_\mathtt{C})$ **do**
11: $\quad \mathsf{c}_{\mathsf{dec},5,i} = (\gamma_i, \phi_{i,0}, \ldots, \phi_{i,\delta-1})$
12: $\quad m_i \leftarrow (\phi_{i,0}, \ldots, \phi_{i,\delta-1})$
13: **end for**
14: $\mathbf{m} \leftarrow (m_0, \ldots, m_{\hat{\mathtt{N}}_\mathtt{C}-1})$
15: $\mathsf{L}_{\mathtt{bb,Tally}} \leftarrow \mathsf{L}_{\mathtt{bb,Tally}} \,||\, \mathtt{bb}$

Output:

Shuffled votes $\mathbf{c}_{\mathsf{mix},5} \in ({\mathbb{G}_q}^{\delta+1})^{\hat{\mathtt{N}}_\mathtt{C}}$
Shuffle proof $\pi_{\mathsf{mix},5}$ ▷ See the domain of the shuffle argument in the crypto primitives specification
Decrypted votes $\mathbf{m} = (m_0, \ldots, m_{\hat{\mathtt{N}}_\mathtt{C}-1}) \in ({\mathbb{G}_q}^{\delta})^{\hat{\mathtt{N}}_\mathtt{C}}$
Decryption proofs $\boldsymbol{\pi}_{\mathsf{dec},5} \in (\mathbb{Z}_q \times {\mathbb{Z}_q}^{\delta})^{\hat{\mathtt{N}}_\mathtt{C}}$

Copyright 2026 Swiss Post Ltd.

6.3.4 ProcessPlaintexts

$\mathbf{m}$ contains the list of plaintext votes; each message m consists of the multiplied primes $\hat{\mathbf{p}} = (\hat{\mathrm{p}}_0, \ldots, \hat{\mathrm{p}}_{\psi-1})$ encoding the selected actual voting options $(\hat{\mathrm{v}}_0, \ldots, \hat{\mathrm{v}}_{\psi-1})$. The Tally control component retrieves the list of encoded voting options $\tilde{\mathbf{p}}$ from the primes mapping table pTable, factorizes each plaintext vote to retrieve the voter's encoded voting options and decodes them to the actual voting options. If the original ballot box contained less that two confirmed votes ($\mathtt{N_C} < 2$), the ProcessPlaintexts algorithm strips away the trivial encryptions that the GetMixnetInitialCiphertexts algorithm added. For elections belonging to an abstention group ($\mathsf{ag}_j = 1$), if all ψ_j abstention voting options are selected, the vote is classified as an abstention; otherwise, any selected abstention voting options are treated as blank voting options. Furthermore, if write-ins are present in the encrypted vote, the algorithm decodes the write-in fields, if they are present (see section 3.8.5).

Copyright 2026 Swiss Post Ltd.

Algorithm 6.12 ProcessPlaintexts

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Primes mapping table $\mathsf{pTable} \in (\mathcal{D}_{\mathrm{v}} \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times {\mathbb{A}_{UCS}}^* \times \mathcal{D}_\tau)^n$ ▷ pTable is of the form $((\mathrm{v}_0, \tilde{\mathrm{p}}_0, \sigma_0, \tau_0), \ldots, (\mathrm{v}_{n-1}, \tilde{\mathrm{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1}))$ ▷ ψ, δ, $\tilde{\mathbf{p}}$ and $\tilde{\mathbf{p}}_{\mathrm{w}}$ can be derived from pTable using algorithms 3.11, 3.12, 3.2 and 3.8
Number of selections vector $\boldsymbol{\psi} = (\psi_0, \ldots, \psi_{t-1}) \in (\mathbb{N}^+)^t$
Presentation groups $\mathbf{pg} = (\mathrm{pg}_0, \ldots, \mathrm{pg}_{t-1}) \in [0, \eta)^t$

Input:

List of plaintext votes $\mathbf{m} = (m_0, \ldots m_{\hat{\mathrm{N}}_{\mathrm{C}}-1}) \in ({\mathbb{G}_q}^\delta)^{\hat{\mathrm{N}}_{\mathrm{C}}}$

Require: $\hat{\mathrm{N}}_{\mathrm{C}} \geq 2$ ▷ The algorithm runs with at least two votes

Operation:

1: $L_{\mathsf{votes}} \leftarrow ()$
2: $L_{\mathsf{decodedVotes}} \leftarrow ()$
3: $L_{\mathsf{writeIns}} \leftarrow ()$
4: $\vec{1} \leftarrow (\underbrace{1, \ldots, 1}_{\delta \text{ times}})$
5: $\hat{\boldsymbol{\tau}} \leftarrow \mathsf{GetBlankCorrectnessInformation}()$ ▷ See Algorithm 3.6
6: $k \leftarrow 0$
7: **for** $i \in [0, \hat{\mathrm{N}}_{\mathrm{C}})$ **do**
8: $m_i = (\phi_{i,0}, \ldots, \phi_{i,\delta-1})$
9: **if** $m_i \neq \vec{1}$ **then**
10: $\hat{\mathbf{p}}_k \leftarrow \mathsf{Factorize}(\phi_{i,0})$ ▷ See Algorithm 3.13
11: $\hat{\mathbf{v}}_k \leftarrow \mathsf{GetActualVotingOptions}(\hat{\mathbf{p}}_k)$ ▷ See Algorithm 3.3
12: $\boldsymbol{\tau}' \leftarrow \mathsf{GetCorrectnessInformation}(\hat{\mathbf{v}}_k)$ ▷ See Algorithm 3.5
13: **if** $\hat{\boldsymbol{\tau}} \neq \boldsymbol{\tau}'$ **then**
14: **return** $\perp$ ▷ If the vote contains an invalid combination of voting options (see section 3.4.8), the algorithm aborts and returns nothing
15: **end if**
16: $\mathbf{w}_k \leftarrow (\phi_{i,1}, \ldots, \phi_{i,\delta-1})$ ▷ An empty vector if the election event has no write-ins
17: $\hat{\mathbf{s}}_k \leftarrow \mathsf{DecodeWriteIns}(\hat{\mathbf{p}}_k, \mathbf{w}_k)$ ▷ See Algorithm 3.30
18: $\hat{\mathbf{v}}'_k \leftarrow \mathsf{ProcessInvalidEncoding}(\hat{\mathbf{v}}_k)$ ▷ See Algorithm 6.13
19: $L_{\mathsf{votes}} \leftarrow (L_{\mathsf{votes}}, \hat{\mathbf{p}}_k)$
20: $L_{\mathsf{decodedVotes}} \leftarrow (L_{\mathsf{decodedVotes}}, \hat{\mathbf{v}}'_k)$
21: $L_{\mathsf{writeIns}} \leftarrow (L_{\mathsf{writeIns}}, \hat{\mathbf{s}}_k)$
22: $k \leftarrow k + 1$
23: **end if**
24: **end for**

Output:

List of decrypted votes $L_{\mathsf{votes}} = (\hat{\mathbf{p}}_0, \ldots, \hat{\mathbf{p}}_{\mathrm{N_C}-1}) \in \left(((\mathbb{G}_q \cap \mathbb{P}) \setminus g)^\psi\right)^{\mathrm{N_C}}$

List of decoded votes $L_{\mathsf{decodedVotes}} = (\hat{\mathbf{v}}_0, \ldots, \hat{\mathbf{v}}_{\mathrm{N_C}-1}) \in \left((\mathcal{D}_{\mathrm{v}})^\psi\right)^{\mathrm{N_C}}$

List of decoded write-ins $L_{\mathsf{writeIns}} = (\hat{\mathbf{s}}_0, \ldots, \hat{\mathbf{s}}_{\mathrm{N_C}-1}) \in \left(({\mathbb{A}_{\mathsf{latin}}}^*)^*\right)^{\mathrm{N_C}}$ ▷ The alphabet $\mathbb{A}_{\mathsf{latin}}$ is defined in section 3.8.1

Test values for the algorithm 6.12 are provided in process-plaintexts.json.

Copyright 2026 Swiss Post Ltd.

As explained in Section 3.4.6, the tally must process invalid encodings and enforce the all-or-nothing semantics of abstention voting options. If a vote contains only a strict subset of the abstention options of an abstention group, the tally control component replaces these options with blank voting options.

Algorithm 6.13 ProcessInvalidEncoding

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Primes mapping table $\mathsf{pTable} \in (\mathcal{D}_{\mathrm{v}} \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times {\mathbb{A}_{UCS}}^* \times \mathcal{D}_\tau)^n$ ▷ pTable is of the form $((\mathrm{v}_0, \tilde{\mathrm{p}}_0, \sigma_0, \tau_0), \ldots, (\mathrm{v}_{n-1}, \tilde{\mathrm{p}}_{n-1}, \sigma_{n-1}, \tau_{n-1}))$ ▷ ψ, δ, $\tilde{\mathbf{p}}$ and $\tilde{\mathbf{p}}_{\mathrm{w}}$ can be derived from pTable using algorithms 3.11, 3.12, 3.2 and 3.8
Number of selections vector $\boldsymbol{\psi} = (\psi_0, \ldots, \psi_{t-1}) \in (\mathbb{N}^+)^t$
Presentation groups $\mathbf{pg} = (\mathrm{pg}_0, \ldots, \mathrm{pg}_{t-1}) \in [0, \eta)^t$

Input:

Selected actual voting options $\hat{\mathbf{v}}_k = (\hat{\mathrm{v}}_{k,0}, \ldots, \hat{\mathrm{v}}_{k,\psi-1}) \in \left(\mathcal{D}_{\mathrm{v}}\right)^{\psi}$

Require: $\hat{\mathrm{v}}_i \in \mathsf{pTable}\ \forall\ i \in [0, \psi)$
Require: $\hat{\mathrm{v}}_i \neq \hat{\mathrm{v}}_k, \forall i, k \in [0, \psi) \wedge i \neq k$ ▷ All selected actual voting options must be distinct

Operation:

1: $\hat{\mathbf{v}}'_k \leftarrow \hat{\mathbf{v}}_k$
2: $\mathbf{v}_{\mathrm{b}} \leftarrow \mathsf{GetBlankActualVotingOptions}()$ ▷ See algorithm 3.7
3: **for** $i \in [0, \eta)$ **do**
4: $\quad \mathbf{pg}_{\mathrm{idx}} \leftarrow \mathsf{indices}(i, \mathbf{pg}, \boldsymbol{\psi})$ ▷ See Equation (1)
5: $\quad \mathbf{v}_{\mathrm{a}} \leftarrow \mathsf{GetAbstentionActualVotingOptions}(i)$ ▷ See algorithm 3.9
6: $\quad \hat{\mathbf{v}}'_{k,i} \leftarrow \{\hat{\mathrm{v}}'_{k,j} \mid j \in \mathbf{pg}_{\mathrm{idx}}\}$ ▷ Selected actual voting options for presentation group i
7: $\quad \mathcal{I} \leftarrow \mathbf{v}_{\mathrm{a}} \cap \hat{\mathbf{v}}'_{k,i}$ ▷ In a valid encoding, either all or no abstention voting options are selected
8: $\quad$ **if** $\mathcal{I} \neq \{\} \wedge \mathcal{I} \neq \mathbf{v}_{\mathrm{a}}$ **then**
9: $\quad\quad$ **for** $j \in \mathbf{pg}_{\mathrm{idx}}$ **do**
10: $\quad\quad\quad$ **if** $\hat{\mathrm{v}}'_{k,j} \in \mathcal{I}$ **then**
11: $\quad\quad\quad\quad \hat{\mathrm{v}}'_{k,j} \leftarrow \mathrm{v}_{\mathrm{b},j}$ ▷ Replace abstention option with the blank option
12: $\quad\quad\quad$ **end if**
13: $\quad\quad$ **end for**
14: $\quad$ **end if**
15: **end for**

Output:

Processed actual voting options $\hat{\mathbf{v}}'_k = (\hat{\mathrm{v}}'_{k,0}, \ldots, \hat{\mathrm{v}}'_{k,\psi-1}) \in \left(\mathcal{D}_{\mathrm{v}}\right)^{\psi}$

Copyright 2026 Swiss Post Ltd.

6.3.5 Creating the Tally File

The tally control component tallies the decoded votes and decoded write-ins from the algorithm 6.12 ProcessPlaintexts in one file:

- The eCH-0222 XML standardizes the decrypted ballots, in the format defined under http://www.ech.ch/xmlns/eCH-0222/3/eCH-0222-3-1.xsd,

We assume a function CreateECH0222 that takes as input the election event configuration configuration XML, the key-value map of $L_{\text{decodedVotes}}$ per authorization name $\text{Map}_{\texttt{decodedVotes}}$, and the key-value map of L_{writeIns} per authorization name $\text{Map}_{\texttt{writeIns}}$, and outputs a eCH-0222 XML compliant with the eCH-0222 standard.
In addition, CreateECH0222 normalizes all write-in contents to ensure compliance with the XML schema. Normalization follows the `xs:token` specification [23]: leading and trailing whitespaces are removed and internal sequences of whitespace are collapsed into a single space. If the normalized string is empty, it is replaced with a hyphen ("-").
In particular, the tally control component's eCH-0222 XML must declare votes for party lists without a candidate selection as invalid votes (if the election imposes this rule) as outlined in section 3.4.8.
The details of the generation of the eCH-0222 XML is beyond the scope of this document; we only outline its high-level structure.
The eCH standard is a Swiss e-government standard that defines structured data formats to ensure interoperability in electronic communication between authorities and organizations. One of these standards, eCH-0222, defines the data types and event messages for exchanging raw data from electronic ballot boxes to the downstream systems responsible for processing and determining the results of an election event.
The eCH-0222 XML is primarily composed of three sections: the delivery header, the reporting body, and the raw data. The eCH-0222 XML combines the reporting body and the raw data into a *raw data delivery*. While the delivery header and reporting body primarily contain static information, the raw data section contains the structured, decoded votes of an election event. Figure 17 illustrates the high-level structure of the eCH-0222 XML.

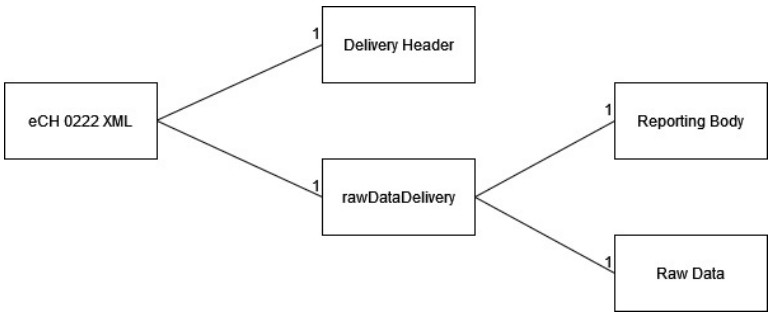


Fig. 17: High-level structure of the eCH-0222 XML.

Copyright 2026 Swiss Post Ltd.

6.3.6 Requesting a Proof of Non-Participation

The Federal Chancellery's Ordinance requires a proof of non-participation after the election period ended [7].

> [VEleS art. 5 para. 2(b)]: A voter who has not cast his or her vote electronically can request proof after the electronic voting system is closed and within the statutory appeal deadlines that the trustworthy part of the system has not registered any vote cast using the client-side authentication credential of the voter.

The explanatory report clarifies that a procedural proof is sufficient to fulfill the requirement.

> [Explanatory Report, Sec 4.2.1]: Regarding '...the attacker has not maliciously cast a vote on the voter's behalf which has subsequently been registered as a vote cast in conformity with the system and counted': such a proof would be of limited use during the ballot, as the attacker would still have time to cast a vote. Therefore, it is sufficient if voters can request this proof after the ballot. For reasons of efficiency, it is sufficient for the competent cantonal office to confirm to the voter that no vote has been cast on their behalf. The assumptions of trustworthiness set out in Number 2.9.1 apply to the examination by the competent body, and the auditors' technical aids may also be considered trustworthy. Furthermore, the requirement breaks the trust model, in that the attacker under Number 2.8 must not be able to access the client-side authentication credentials at all. With regard to the present requirement, the assumption must be made that the attacker has access to the client-side authentication credentials of individual voters.

After the tally phase, the voter can request the competent cantonal office to confirm that no vote has been cast on her behalf. The algorithm RequestProofNonParticipation specifies the procedure by which the competent cantonal office checks whether a confirmed vote corresponds to a specific voting card. Importantly, the algorithm RequestProofNonParticipation relies on a list of confirmed voting cards whose consistency and correctness was validated by the verifier. The voter expects the algorithm RequestProofNonParticipation to return $\perp$ if she did not participate in the election event.

Copyright 2026 Swiss Post Ltd.

Algorithm 6.14 RequestProofNonParticipation

Context:

List of confirmed voting cards $\mathbf{vc_1}$ ▷ Stemming from the first control component that was previously validated by the verifier. The list contains the confirmed voting cards from all ballot boxes of the election event.
Verification card ID $\mathtt{vc_{id}} \in (\mathbb{A}_{Base16})^{\mathtt{l_{ID}}}$

Operation:

1: **if** $\mathtt{vc_{id}} \in \mathbf{vc_1}$ **then**
2: $\quad \mathtt{participated} = \top$
3: **else**
4: $\quad \mathtt{participated} = \bot$
5: **end if**

Output:

$\mathtt{participated} \in \{\top, \bot\}$: $\top$ if the voter successfully cast a vote, $\bot$ otherwise.

Copyright 2026 Swiss Post Ltd.

7 Channel Security

7.1 Digital Signatures for Protocol Messages

The cryptographic protocol ensures the confidentiality of messages using encryption: the adversary cannot learn private information (such as the selected voting options) without controlling trustworthy components. Beyond confidentiality, the computational proof [20] details the requirements on the messages' authenticity in the section `Channel Security`. Digital signatures ensure that the adversary cannot undetectably modify ciphertexts. Furthermore, the sender signs additional information such as the election event ID to prevent replay attacks and allow the receiver to verify the context's correctness. Ultimately, the trustworthy protocol participants aim to achieve *injective agreement* on a protocol run's core messages.
The Swiss Post Voting System uses the signature scheme and key length specified in the crypto primitives specification. Each trustworthy party generates a signature key pair and distributes the corresponding certificate to the other trustworthy parties and the auditors. Each certificate is verified and then imported into the trustworthy party's trust store. All messages exchanged between the trustworthy parties are signed and then verified to be valid and to emanate from the expected party.
The generation of key pairs and corresponding certificates and the import of the certificates are regulated by specific processes, using the algorithm GenKeysAndCert from the crypto primitives specification for the generation and a manual process for the verification before the import.
The tables below identify the messages that are signed during each of the protocol phases. For each message, the signing party is identified, as well as the data that must be signed, along with any additional information required to prevent replay attacks. For each message, the signer calls the GenSignature method defined in the crypto primitives specification, with the parameters given in the corresponding columns.
Each of the recipients listed must verify the validity of the signature with the method VerifySignature defined in the crypto primitives specification, using the certificate previously linked to the expected signing party and duly validated before being imported into the verifying party's trust store. As a side note, since there are no direct communication channels between the control components and the auditors, messages signed by the control components are forwarded to the auditors by either the setup component or the tally control component. The auditors then verify the signatures independently.
Usually, we sign data elements—and not the actual files into which the data elements are serialized. However, we deviate from this rule for XML files that serve as an important interface to upstream and downstream systems. Due to the complexity of these XML files' structure and the existence of widely recognized standards for XML signing, we use two specific algorithms for XML signatures, as outlined in section 7.2.

Copyright 2026 Swiss Post Ltd.

Table 16 highlights the messages that the parties exchange during the configuration phase.

Message Name	**Signer**	**Recipient(s)**	**Message Content**	**Context Data**
ElectionEventContext	Setup Comp.	Online CC_j, Tally CC, Auditors, Voting Server	ElectionEventContextPayload, see table 19	("`election event context`", `ee`)
CantonConfig	Canton	Setup Comp., Auditors, Tally CC	configuration XML[1]	-
ControlComponentPublicKeys	Online CC_j	Setup Comp., Auditors	ControlComponentPublicKeysPayload, see table 19	("`OnlineCC keys`", j, `ee`)
SetupComponentVerificationData	Setup Comp.	Online CC_j	SetupComponentVerificationDataPayload, see table 19	("`verification data`", `ee`, `vcs`)
ControlComponentCodeShares	Online CC_j	Setup Comp.	ControlComponentCodeSharesPayload, see table 19	("`encrypted code shares`", j, `ee`, `vcs`)
SetupComponentLVCCAllowList	Setup Comp.	Online CC_j	$\mathsf{L}_{\mathtt{lVCC}}$	("`lvcc allow list`", `ee`, `vcs`)
SetupComponentEvotingPrint	Setup Comp.	Printing Comp.	evoting print XML[1]	-
SetupComponentCMTable	Setup Comp.	Voting Server	CMtable[2]	("`cm table`", `ee`, `vcs`)
SetupComponentVerificationCardKeyStores	Setup Comp.	Voting Server	**VCks**	("`vc keystore`", `ee`, `vcs`)
SetupComponentVoterAuthenticationData	Setup Comp.	Voting Server	(**credentialID**, **hAuth**)	("`voter authentication`", `ee`, `vcs`)
SetupComponentPublicKeys	Setup Comp.	Online CC_j, Tally CC, Auditors, Voting Server	SetupComponentPublicKeysPayload, see table 19	("`public keys`", "`setup`", `ee`)
SetupComponentTallyData	Setup Comp.	Auditors, Tally CC	SetupComponentTallyDataPayload, see table 19	("`tally data`", `ee`, `vcs`)
SetupComponentElectoralBoardHashes	Setup Comp.	Tally CC	$(\mathtt{hPW}_0, \ldots, \mathtt{hPW}_{k-1})$	("`electoral board hashes`", `ee`)

Tab. 16: Overview of the authenticated messages and their signers and recipients in the configuration phase as displayed in figures 7 and 8.

1 This message is signed using the XML signature algorithm outlined in section 7.2.
2 The CMtable is ordered as per algorithm 4.7, and represented as a vector of `key`, `value` pairs, *i.e.* $((k_0, v_0), \ldots, (k_{\mathtt{N_E}\cdot(n+1)-1}, v_{\mathtt{N_E}\cdot(n+1)-1}))$.

Copyright 2026 Swiss Post Ltd.

The voting phase provides implicit authentication: the voting client can only generate extractable short Choice Return Codes using the correct verification card secret key $\mathtt{k}_{\mathtt{id}}$, and the control components only derive the short Vote Cast Return Code $\mathtt{VCC}_{\mathtt{id}}$ from a valid Ballot Casting Key $\mathtt{BCK}_{\mathtt{id}}$.
However, the elements exchanged between the control components and the voting server should still be authenticated, as shown in table 17. Signing the voting server's messages is not necessary from the trust model's point of view since the voting server is an *untrusted* component. Nevertheless, it provides an additional defense-in-depth mechanism against attackers controlling the network between the voting server and the control components.

Message Name	**Signer**	**Recipient(s)**	**Message Content**	**Context Data**
VotingServerEncryptedVote	Voting server	Online CC_j	$(\mathtt{E1}, \mathtt{E2}, \widetilde{\mathtt{E1}}, \pi_{\mathtt{Exp}}, \pi_{\mathtt{EqEnc}})$	("`encrypted vote`", `ee`, `vcs`, $\mathtt{vc}_{\mathtt{id}}$)
ControlComponentPartialDecrypt	Online CC_j	Online $\mathrm{CC}_{j'}$	$(\mathbf{d_j}, \boldsymbol{\pi}_{\mathtt{decPCC},j})$	("`partial decrypt`", j, `ee`, `vcs`, $\mathtt{vc}_{\mathtt{id}}$)
ControlComponentlCCShare	Online CC_j	Voting Server	ControlComponentlCCSharePayload, see table 19	("`lcc share`", j, `ee`, `vcs`, $\mathtt{vc}_{\mathtt{id}}$)
VotingServerConfirm	Voting server	Online CC_j	$\mathtt{CK}_{\mathtt{id}}$	("`confirmation key`", `ee`, `vcs`, $\mathtt{vc}_{\mathtt{id}}$)
ControlComponenthlVCCShare	Online CC_j	Online $\mathrm{CC}_{j'}$	ControlComponenthlVCCSharePayload, see table 19	("`hlvcc`", j, `ee`, `vcs`, $\mathtt{vc}_{\mathtt{id}}$)
ControlComponentlVCCShare	Online CC_j	Voting Server	ControlComponentlVCCSharePayload, see table 19	("`lvcc share`", j, `ee`, `vcs`, $\mathtt{vc}_{\mathtt{id}}$)

Tab. 17: Overview of the authenticated messages and their signers and recipients in the voting phase as displayed in figures 11 and 13.

Copyright 2026 Swiss Post Ltd.

Table 18 summarizes the messages of the tally phase.

Message Name	Signer	Recipient(s)	Message Content	Context Data
ControlComponentVotesHash	Online CC$_j$	Online CC$_{j'}$	$\texttt{hvc}_j$	("`voteshash`", j, `ee`, `bb`)
ControlComponentBallotBox	Online CC$_j$	Tally CC, Auditors	ControlComponentBallotBoxPayload, see table 19	("`ballotbox`", j, `ee`, `bb`)
ControlComponentShuffle	Online CC$_j$	Online CC$_{j'}$, Tally CC, Auditors	ControlComponentShufflePayload, see table 19	("`shuffle`", j, `ee`, `bb`)
TallyComponentShuffle	Tally CC	Auditors	TallyComponentShufflePayload, see table 19	("`shuffle`", "`offline`", `ee`, `bb`)
TallyComponentVotes	Tally CC	Auditors	TallyComponentVotesPayload, see table 19	("`decoded votes`", `ee`, `bb`)
TallyComponentEch0222	Tally CC	Auditors	eCH-0222 XML[3]	-
ControlComponentExtractedElectionEvent	Online CC$_j$	Dispute Resolver	ControlComponentExtractedElectionEventPayload, see table 19	("`extracted election event`", j, `ee`)
ControlComponentExtractedVerificationCards	Online CC$_j$	Dispute Resolver	ControlComponentExtractedVerificationCardsPayload, see table 19	("`extracted verification cards`", j, `ee`)
DisputeResolverResolvedConfirmedVotes	Dispute Resolver	Online CC$_j$	DisputeResolverResolvedConfirmedVotesPayload, see table 19	("`resolved confirmed votes`", `ee`)

Tab. 18: Overview of the authenticated messages and their signers and recipients in the tally phase as displayed in figure 14.

We refer to the combination of the message content and its digital signature as the *payload* and we accompany each payload with a JSON schema. A JSON schema is a JSON document that defines the structure, constraints, and data types of JSON data, used to describe the message content and its alignment with the system specification. Table 19 provides an overview of the schemas of the JSON payloads.

3 This message is signed using the XML signature algorithm outlined in section 7.2.

Copyright 2026 Swiss Post Ltd.

Payload Name	Referenced schemas
ControlComponentBallotBoxPayload	GqGroup, EncryptedVerifiableVote, ContextIds, ElGamalMultiRecipientCiphertext, ExponentiationProof, PlaintextEqualityProof, CryptoPrimitivesSignature
ControlComponentCodeSharesPayload	GqGroup, ControlComponentCodeShare, ElGamalMultiRecipientCiphertext, ExponentiationProof, CryptoPrimitivesSignature
ControlComponentExtractedElectionEventPayload	ExtractedElectionEvent, GqGroup, ExtractedVerificationCardSet, CryptoPrimitivesSignature
ControlComponentExtractedVerificationCardsPayload	GqGroup, ExtractedVerificationCard, CryptoPrimitivesSignature
ControlComponenthlVCCSharePayload	GqGroup, ConfirmationKey, CryptoPrimitivesSignature
ControlComponentlCCSharePayload	GqGroup, LongChoiceReturnCodeShare, CryptoPrimitivesSignature
ControlComponentlVCCSharePayload	GqGroup, LongVoteCastReturnCodeShare, ConfirmationKey, CryptoPrimitivesSignature
ControlComponentPublicKeysPayload	GqGroup, ControlComponentPublicKeys, SchnorrProof, CryptoPrimitivesSignature
ControlComponentShufflePayload	GqGroup, VerifiableShuffle, ElGamalMultiRecipientCiphertext, ShuffleArgument, ProductArgument, HadamardArgument, ZeroArgument, SingleValueProductArgument, MultiExponentiationArgument, VerifiableDecryptions, CryptoPrimitivesSignature
DisputeResolverResolvedConfirmedVotesPayload	CryptoPrimitivesSignature
ElectionEventContextPayload	GqGroup, ElectionEventContext, PresentationGroup, VerificationCardSetContext, ElectoralModelContext, NumberOfSelectionsVector, PresentationGroupsVector, AbstentionGroupsVector, PrimesMappingTable, PrimesMappingTableEntry, VoteTexts, AbstentionPosition, Ballot, ElectionTexts, ElectionInformation, ListUnion, Election, Candidate, List, EmptyList, WriteInPosition, LanguageMap, CryptoPrimitivesSignature
SetupComponentPublicKeysPayload	GqGroup, SetupComponentPublicKeys, ControlComponentPublicKeys, SchnorrProof, CryptoPrimitivesSignature
SetupComponentTallyDataPayload	GqGroup, CryptoPrimitivesSignature
SetupComponentVerificationDataPayload	GqGroup, SetupComponentVerificationData, ElGamalMultiRecipientCiphertext, CryptoPrimitivesSignature
TallyComponentShufflePayload	GqGroup, VerifiableShuffle, ElGamalMultiRecipientCiphertext, ShuffleArgument, ProductArgument, HadamardArgument, ZeroArgument, SingleValueProductArgument, MultiExponentiationArgument, VerifiablePlaintextDecryption, CryptoPrimitivesSignature
TallyComponentVotesPayload	GqGroup, CryptoPrimitivesSignature

Tab. 19: Overview of the JSON schemas of the payloads exchanged.

The other JSON schema files in alphabetical order are ConfirmationKey, ControlComponentPublicKeys, CryptoPrimitivesSignature, ElGamalMultiRecipientCiphertext, ExponentiationProof, GqGroup, HadamardArgument, MultiExponentiationArgument, ProductArgument, SchnorrProof, SingleValueProductArgument, ShuffleArgument, VerifiableShuffle, ZeroArgument.

Copyright 2026 Swiss Post Ltd.

7.2 Digital Signatures for File Interfaces (XML)

The Swiss Post Voting System communicates with upstream and downstream systems using XML files. Given the complexity of these XML structures, providing detailed pseudo-code describing their structure would be impractical. Instead, we rely on the standardized XML signature syntax and verification methods as defined in [1], which are supported by most programming languages.
Within the Swiss Post Voting System, we serialize the following messages as XML files:

- The message CantonConfig serialized in the file configuration XML as indicated in table 16.
- The message SetupComponentEvotingPrint serialized in the file evoting-print XML as indicated in table 16.
- The message TallyComponentEch0222 serialized in the file eCH-0222 XML as indicated in table 18.

Accordingly, we define Algorithm 7.1 for signing XML files and Algorithm 7.2 for verifying XML signatures.
We assume that the recipient validates the XML messages against the corresponding schema definition (XSD) to ensure that the XML adheres to the expected structure and data constraints.
Subsequently, the signature generation and verification method follows the W3C XML signature standard [1]. Signing an XML file follows a structured series of steps:

1. CreateSignedInfo: The `<SignedInfo>` element is instantiated with the following parameters:
   - Reference URI: An empty string (`""`) to indicate that the entire document is signed.
   - CanonicalizationMethod: The exclusive C14N canonicalization algorithm[4].
   - Transforms: the enveloped-signature transform.
   - DigestMethod: The digest algorithm. We use the `SHA-256` algorihtm.
   - SignatureMethod: The signature algorithm. We use the `RSAPSS` algorithm with 3072-bits key size.
2. Canonicalize: Prior to any cryptographic operation, the XML file is *canonicalized*. This step removes ambiguities such as OS-dependent line breaks, varying whitespace, and attribute ordering differences, ensuring that any semantically equivalent XML produces the same canonical form.
3. ApplyTransforms: The data referenced by the signature undergoes the enveloped-signature transform, which removes the signature element itself from the document prior to digest calculation.
4. Digest: The transformed data is hashed using the `SHA-256` digest algorithm.
5. Update: Update the `<SignedInfo>` with the digest.

Copyright 2026 Swiss Post Ltd.

6. Sign: The canonicalized `<SignedInfo>` element is signed using the Channel Security keystore described in section 7 with the `RSAPSS` algorithm. This signature is stored in the `<SignatureValue>` element.

7. EmbedSignature: The computed signature is embedded into the corresponding XML file. For the XML files in the Swiss Post Voting System, we embed the signature as follows:

   - CantonConfig (configuration XML): Embed the `<Signature>` element as the last child of the root element.
   - SetupComponentEvotingPrint (evoting-print XML): Embed the `<Signature>` element as the last child of the root element.
   - TallyComponentEch0222 (eCH-0222 XML): Embed the `<Signature>` element in the `rawData/extensions` element.

We omit embedding the certificate into the `<SignedInfo>` element since we assume that the recipient knows the sender's public key and ensured its authenticity—as outlined in section 7.1. Following these steps, we instantiate the algorithm GenXMLSignature as follows.

Algorithm 7.1 GenXMLSignature

Input:

XML document D
Secret key of the signer sk

Operation: ▷ The algorithm follows the XML signature standard[1] and uses the parameters defined above.

1: $SI \leftarrow \mathsf{CreateSignedInfo}(D)$
2: $SI_{can} \leftarrow \mathsf{Canonicalize}(SI)$
3: $T \leftarrow \mathsf{ApplyTransforms}(D)$
4: $d \leftarrow \mathsf{Digest}(T, \mathsf{digestAlg})$
5: $SI \leftarrow \mathsf{Update}(SI, d)$
6: $SV \leftarrow \mathsf{Sign}(SI_{can}, sk, \mathsf{sigAlg})$
7: $D_{signed} \leftarrow \mathsf{EmbedSignature}(D, SI, SV)$

Output:

Signed XML document D_{signed}

Copyright 2026 Swiss Post Ltd.

Upon receipt, the signature is verified by applying the same canonicalization and transformation procedures to the XML document, recalculating the digest, and using the public key to validate the `<SignatureValue>`.
The verification algorithm proceeds as follows:

1. ExtractSignedInfo: Extract the `<SignedInfo>` element from the signed XML document. Depending on the XML file type, the location of the corresponding `<Signature>` element varies:
   - CantonConfig (configuration XML): Extract the `<Signature>` element from the last child of the root element.
   - SetupComponentEvotingPrint (evoting-print XML): Extract the `<Signature>` element from the last child of the root element.
   - TallyComponentEch0222 (eCH-0222 XML): Extract the `<Signature>` element from the `rawData/extensions` element.
2. Canonicalize: Prior to any cryptographic operation, the XML file is canonicalized. This step removes ambiguities such as OS-dependent line breaks, varying whitespace, and attribute ordering differences, ensuring that any semantically equivalent XML produces the same canonical form. We use the exclusive C14N canonicalization algorithm[4].
3. ApplyTransforms: The data referenced by the signature undergoes the enveloped-signature transform, which removes the signature element itself from the document prior to digest calculation.
4. Digest: The transformed data is hashed using the `SHA-256` digest algorithm.
5. ExtractDigest: Retrieve the digest value from the `<DigestValue>` element contained within the `<SignedInfo>` section.
6. Check the digest: Compare the digest computed from the transformed data with the extracted digest value. Return false if the digest values do not match.
7. ExtractSignatureValue: Extract the signature value from the `<SignatureValue>` element of the signed XML document.
8. Verify: Use the public key and the specified signature algorithm to verify that the signature is valid for the canonicalized `<SignedInfo>` element. If the signature is valid, the verification returns true; otherwise, it returns false.

Copyright 2026 Swiss Post Ltd.

Algorithm 7.2 VerifyXMLSignature

Input:

Signed XML document $\mathrm{D}_{\mathrm{signed}}$
Public key of the signer pk

Operation: ▷ The algorithm follows the XML signature standard[1] and uses the parameters defined above.

1: $\mathrm{SI} \leftarrow \mathsf{ExtractSignedInfo}(\mathrm{D}_{\mathrm{signed}})$
2: $\mathrm{SI}_{\mathrm{can}} \leftarrow \mathsf{Canonicalize}(\mathrm{SI})$
3: $\mathrm{T} \leftarrow \mathsf{ApplyTransforms}(\mathrm{D}_{\mathrm{signed}})$
4: $\mathrm{d}' \leftarrow \mathsf{Digest}(\mathrm{T}, \mathsf{digestAlg})$
5: $\mathrm{d} \leftarrow \mathsf{ExtractDigest}(\mathrm{SI})$
6: **if** $\mathrm{d}' \neq \mathrm{d}$ **then**
7: **return** $\bot$
8: **end if**
9: $\mathrm{SV} \leftarrow \mathsf{ExtractSignatureValue}(\mathrm{D}_{\mathrm{signed}})$
10: **return** $\mathsf{Verify}(\mathrm{SI}_{\mathrm{can}}, \mathrm{SV}, \mathsf{pk}, \mathsf{sigAlg})$

Output:

$\top$ if the signature is valid, $\bot$ otherwise.

Copyright 2026 Swiss Post Ltd.

7.3 Streamable Symmetric Encryption and Decryption for Dataset Security

Whenever a dataset is exported for transfer to or from an offline component, the dataset is symmetrically encrypted as an additional security mechanism. A dataset contains cryptographic protocol messages serialized in a standard file format such as JSON (JavaScript Object Notation) or XML (Extensible Markup Language). The serialization of the messages is outside the scope of this document.
The files of a dataset are collected into a ZIP archive using the DEFLATE method according to [5, RFC 1951]. The implementation generates the ZIP archive deterministically, including file order and archive metadata, so that auditors can verify and record its hash value.
Moreover, we assume the availability of methods for zipping and unzipping a dataset, along with a unique byte array representation of the resulting ZIP archive. Algorithms 7.3 and 7.4 specify the streamable encryption and decryption of this byte array representation.
We use an authenticated symmetric encryption scheme based on Authenticated Encryption with Associated Data (AEAD) as defined in the crypto primitives specification. AEAD protects the dataset in transit from eavesdropping and tampering by ensuring both the confidentiality and integrity of its content.
We require an encryption and decryption mechanism that can handle datasets too large to fit entirely in memory. To achieve this, we process the byte array representation of the dataset using a streaming approach. Streaming, in this context, means processing the data sequentially using implementation-defined I/O chunks, rather than loading the entire dataset into memory at once.
In this section, the term *chunk* denotes an implementation-defined I/O buffer used for stream processing. It does not refer to an AES block. An AES block has a fixed length of 16 bytes and is an internal detail of AES-GCM. Moreover, a chunk does not denote an independently authenticated unit.
We specify stream processing using the abstract functions readNextChunk and endOfData. The function readNextChunk returns the next implementation-defined I/O chunk of a stream. These functions model the sequential consumption of an input stream of data.
The streaming algorithms specified here use a single AEAD invocation per dataset. We model this invocation using the abstract functions InitAuthenticatedEncryption UpdateAuthenticatedEncryption, and FinalizeAuthenticatedEncryption and similarly InitAuthenticatedDecryption, UpdateAuthenticatedDecryption and FinalizeAuthenticatedDecryption.
The initialization function creates an AEAD instance from the derived key, nonce, and associated data. The update function processes implementation-defined I/O chunks using this initialized AEAD instance. The AEAD instance maintains the internal AES-GCM state, including the counter and authentication state, across chunk updates. The finalization function produces the authentication tag during encryption and verifies it during decryption. For AES-GCM-256, the authenticated ciphertext consists of the encrypted plaintext and the authentication tag. The decryption algorithm is defined such that the tag is consumed as part of the ciphertext stream and processed during finalization.
By default, we instantiate the algorithms in this section with an empty byte array as associated data associated.
The precise definitions of the underlying AEAD encryption and decryption operations are provided in the section *Symmetric Authenticated Encryption* of the crypto primitives specification.

Copyright 2026 Swiss Post Ltd.

As stated in the crypto primitives specification, it is essential that the same key and nonce pair is never reused across invocations of the GenStreamCiphertext algorithm. Therefore, the algorithm must not be used in virtualized environments where an attacker could trigger a rollback, as this may cause key and nonce reuse.

Algorithm 7.3 GenStreamCiphertext

Input:

The plaintext $\mathsf{P} \in \mathcal{B}^*$
A $\mathtt{password} \in \mathbb{A}_{UCS}{}^*$
Associated data $\mathsf{associated} \in \mathcal{B}^*$ ▷ By default, we instantiate the algorithm with an empty byte array as associated data.

Operation:

1: $(\mathsf{derivedKey}, \mathsf{salt}) \leftarrow \mathsf{GenArgon2id}(\mathtt{toBytes}(\mathtt{password}))$ ▷ see crypto primitives specification
2: $\mathsf{nonce} \leftarrow \mathsf{RandomBytes}(12)$ ▷ see crypto primitives specification
3: $\mathsf{enc} \leftarrow \mathsf{InitAuthenticatedEncryption}(\mathsf{derivedKey}, \mathsf{nonce}, \mathsf{associated})$
4: $\mathsf{C} \leftarrow <>$
5: **while** $\neg\mathsf{endOfData}(\mathsf{P})$ **do**
6: $\quad\mathsf{plaintextChunk} \leftarrow \mathsf{readNextChunk}(\mathsf{P})$
7: $\quad\mathsf{ciphertextChunk} \leftarrow \mathsf{UpdateAuthenticatedEncryption}(\mathsf{enc}, \mathsf{plaintextChunk})$
8: $\quad\mathsf{C} \leftarrow \mathsf{C} \,||\, \mathsf{ciphertextChunk}$
9: **end while**
10: $\mathsf{tag} \leftarrow \mathsf{FinalizeAuthenticatedEncryption}(\mathsf{enc})$
11: $\mathsf{C} \leftarrow \mathsf{C} \,||\, \mathsf{tag}$

Output:

The salt, nonce, and authenticated ciphertext $\mathsf{salt} \,||\, \mathsf{nonce} \,||\, \mathsf{C} \in \mathcal{B}^*$
with $\mathsf{salt} \in \mathcal{B}^{16}$, $\mathsf{nonce} \in \mathcal{B}^{12}$, and C including the authentication tag.

Test values for the algorithm 7.3 are provided in gen-stream-ciphertext.json.

Copyright 2026 Swiss Post Ltd.

In stream-based implementations, authentication failure may be raised only when the returned plaintext stream is consumed to completion. The caller must treat the plaintext as authenticated only if the stream is consumed completely and no authentication failure occurs. If this cannot be guaranteed, the implementation must decrypt into temporary storage and expose or commit the plaintext only after authentication succeeds.

Algorithm 7.4 GetStreamPlaintext

Input:

The salt, nonce, and authenticated ciphertext $\mathsf{salt} \,||\, \mathsf{nonce} \,||\, \mathsf{C} \in \mathcal{B}^*$
with $\mathsf{salt} \in \mathcal{B}^{16}$, $\mathsf{nonce} \in \mathcal{B}^{12}$, and C including the authentication tag.
The password $\mathtt{password} \in {\mathbb{A}_{UCS}}^*$
Associated data $\mathsf{associated} \in \mathcal{B}^*$ ▷ By default, we instantiate the algorithm with an empty byte array as associated data.

Operation:

1: derivedKey ← GetArgon2id(toBytes(password), salt) ▷ see crypto primitives specification
2: dec ← InitAuthenticatedDecryption(derivedKey, nonce, associated)
3: P ←<>
4: **while** ¬endOfData(C) **do**
5: ciphertextChunk ← readNextChunk(C)
6: plaintextChunk ← UpdateAuthenticatedDecryption(dec, ciphertextChunk)
7: P ← P || plaintextChunk
8: **end while**
9: finalPlaintextChunk ← FinalizeAuthenticatedDecryption(dec)
10: **if** finalPlaintextChunk = ⊥ **then**
11: **return** ⊥
12: **end if**
13: P ← P || finalPlaintextChunk

Output:

The authenticated plaintext $\mathsf{P} \in \mathcal{B}^*$
Or ⊥ if the authenticated ciphertext does not authenticate.

Test values for the algorithm 7.4 are provided in get-stream-plaintext.json.

Copyright 2026 Swiss Post Ltd.

Acknowledgements

Swiss Post is thankful to all security researchers for their contributions and the opportunity to improve the system's security guarantees. In particular, we want to thank the following experts for their reviews or suggestions reported on our Gitlab repository. We list them here in alphabetical order:

- Veronique Cortier, Pierrick Gaudry, Alexander Debant (Université de Lorraine, CNRS, Inria)
- Aleksander Essex (Western University Canada)
- Rolf Haenni, Reto Koenig, Philipp Locher, Eric Dubuis (Bern University of Applied Sciences)
- Thomas Edmund Haines (Australian National University)
- Johannes Müller
- Olivier Pereira (Universtité catholique Louvain)
- Ruben Santamarta (reversemode)
- Carsten Schürmann (IT University of Copenhagen)
- Vanessa Teague (Thinking Cybersecurity)
- Marc Wyss (ETHZ)

Copyright 2026 Swiss Post Ltd.

References

[1] M. Bartel et al.: *XML Signature Syntax and Processing Version 1.1*. Ed. by D. Eastlake et al. http://www.w3.org/TR/2013/REC-xmldsig-core1-20130411/. W3C Recommendation, 11 April 2013.

[2] D. Bernhard et al.: "Verifiability analysis of CHVote". In: *Cryptology ePrint Archive* (2018).

[3] A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson: *Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications*. RFC 9106. 2021. DOI: 10.17487/RFC9106. URL: https://www.rfc-editor.org/info/rfc9106.

[4] J. Boyer and G. Marcy: *Canonical XML Version 1.1*. http://www.w3.org/TR/2008/REC-xml-c14n11-20080502/. W3C Recommendation, 2 May 2008.

[5] P. Deutsch: *DEFLATE Compressed Data Format Specification*. RFC 1951. 1996. DOI: 10.17487/RFC1951. URL: https://www.rfc-editor.org/info/rfc1951.

[6] Die Schweizerische Bundeskanzlei (BK): *E-voting Catalogue of Measures by the Confederation and Cantons, Approved by the Vote électronique Steering Committee (SC VE): 4 August 2023*. 2023. URL: https://www.bk.admin.ch/dam/bk/en/dokumente/pore/E_Voting/E-voting%20Catalogue%20of%20measures%20by%20the%20Confederation%20and%20cantons,%204%20August%202023.pdf.download.pdf.

[7] Die Schweizerische Bundeskanzlei (BK): *Federal Chancellery Ordinance on Electronic Voting (OEV), 01 July 2022*.

[8] Die Schweizerische Bundeskanzlei (BK): *Partial revision of the Ordinance on Political Rights and total revision of the Federal Chancellery Ordinance on Electronic Voting (Redesign of Trials). Explanatory report for its entry into force on 1 July 2022.*

[9] eCH Association: *eCH-0045 Datenstandard Stimm- und Wahlregister V4.1.0*. eCH E-Government Standards. Version 4.1.0. Approved on June 2, 2022, published on April 19, 2023. 2023. URL: https://www.ech.ch/de/ech/ech-0045/4.1.0.

[10] eCH Association: *eCH-0155 Datenstandard politische Rechte V4.1*. eCH E-Government Standards. Version 4.1. 2021. URL: https://www.ech.ch/de/ech/ech-0155/4.1.

[11] Eidgenössisches Departement für auswärtige Angelegenheiten EDA: *Swiss Political System - Direct Democracy*. https://www.eda.admin.ch/aboutswitzerland/en/home/politik/uebersicht/direkte-demokratie.html/. Retrieved on 2020-07-15.

[12] gfs.bern: *Vorsichtige Offenheit im Bereich digitale Partizipation - Schlussbericht*. 2020.

[13] R. Haenni, R. E. Koenig, P. Locher, and E. Dubuis: *CHVote Protocol Specification, Version 4.3*. Cryptology ePrint Archive, Report 2017/325. https://eprint.iacr.org/2017/325. 2024.

[14] T. Haines, S. J. Lewis, O. Pereira, and V. Teague: *How not to prove your election outcome*. 2020.

[15] S. Josefsson et al.: *The base16, base32, and base64 data encodings*. Tech. rep. RFC 4648, October, 2006.

Copyright 2026 Swiss Post Ltd.

[16] D. M'Raihi, J. Rydell, M. Pei, and S. Machani: *TOTP: Time-Based One-Time Password Algorithm*. RFC 6238. 2011. DOI: 10.17487/RFC6238. URL: https://www.rfc-editor.org/info/rfc6238.

[17] C. Percival: *Stronger key derivation via sequential memory-hard functions*. 2009.

[18] B. Smyth: "A foundation for secret, verifiable elections." In: *IACR Cryptology ePrint Archive* 2018 (2018), p. 225.

[19] Swiss Post: *Cryptographic Primitives of the Swiss Post Voting System. Pseudocode Specification. Version 1.6.0*. https://gitlab.com/swisspost-evoting/crypto-primitives/crypto-primitives. 2026.

[20] Swiss Post: *Protocol of the Swiss Post Voting System. Computational Proof of Complete Verifiability and Privacy. Version 1.5.0*. https://gitlab.com/swisspost-evoting/e-voting/e-voting-documentation/-/tree/master/Protocol. 2026.

[21] Swiss Post: *Swiss Post Voting System. Verifier Specification. Version 1.7.1*. https://gitlab.com/swisspost-evoting/e-voting/e-voting-documentation/-/tree/master/System. 2026.

[22] *W3C XML Schema Definition Language*. https://www.w3.org/TR/xmlschema11-2/#token. 2012.

[23] W3C XML Schema Working Group: *XML Schema Part 2: Datatypes Second Edition*. https://www.w3.org/TR/xmlschema-2/. W3C Recommendation, 2 May 2001 (revised 28 October 2004).

Copyright 2026 Swiss Post Ltd.

List of Algorithms

Copyright 2026 Swiss Post Ltd.

Copyright 2026 Swiss Post Ltd.

List of Figures

List of Tables